

国家“十一五”规划教材配套实验教材

单片机原理与应用实验

栗华 编著

山东大学出版社

内容简介

本书是国家“十一五”规划教材《单片机原理与应用》的配套实验教材。本书是根据《单片机原理与应用》的实验教学需要，结合多年来作者的实验教学经验而编写的。本书设计了软件仿真实验、硬件接口实验和系统综合实验四个模块。实验内容涵盖了 MCS-51 单片机教学大纲中的汇编及 C 语言程序设计、片内外设及中断系统应用、片外存储器扩展、I/O 扩展、人机交互（包括键盘输入、LED 数码管显示、LCD 显示以及 LED 点阵显示）扩展、信号的输入输出通道（包括常用的传感器、A/D、D/A 和开关量的输入输出）扩展，以及综合性系统设计等内容。内容全面，贴近工程实际，注重能力的培养。书中所有实验均提供已通过验证的参考源代码。

本书可作为电子信息、通信工程、计算机、自动化等专业专科生、本科生的单片机原理与应用实验教材，也可作为大学生电子设计大赛的培训教材，或作为从事单片机开发应用工作的工程技术人员自学实训书籍。

前言

单片机这种二十世纪七十年代诞生的专用于小型智能控制领域的计算机是嵌入式计算机的一种，也是到目前为止应用最广泛的一种专用计算机。MCS-51 单片机以其集成度高、体积小、可靠性高、抗干扰性强、控制功能强、可扩展性好、性价比高等特点，使得它不仅成为嵌入式计算机发展历史上的里程碑，而且直到现在仍然是嵌入式计算机的典型代表。学习单片机是电气、电子、自动化领域的学生学习智能控制，智能仪器仪表设计的入门基础。很多高水平的电子设计工程师都是从学习单片机尤其是 MCS-51 开始的。

目前，各种以 MCS-51 为原型的单片机书籍已经非常多。大多数书籍基本上分成两类，第一类主要侧重 MCS-51 单片机原理，把主要精力都放在了单片机芯片内部的介绍，而对于片外扩展的介绍不够详细，对于现在工程设计中常用的一些集成度更高、采用串行总线接口的接口芯片几乎很少介绍。第二类则主要侧重 MCS-51 单片机的接口技术，把主要精力都放在了单片机的外部接口上，对于单片机内部的知识一带而过。接口芯片介绍的比较多，种类也比较丰富。对于第一类书籍，比较适合单片机的初学者学习，但是，所学的知识又很难应用到工程实践中，大多数的单片机的应用工程师最初从课本上所获得的知识非常有限，很多知识都是靠自己多年的工程实践慢慢积累的，而在此过程中也难免走了很多弯路。对于第二类书籍，一般要求读者已经具有单片机的基本知识，甚至有了一定的应用经验，书中的知识只是工程应用中的一种参考资料，不太适合于初学者学习。

为此，我们于 2008 年编写了《单片机原理与应用》一书，该书被评为国家级“十一五”规划教材。本教材将单片机的原理知识和接口知识以及工程应用知识很好的结合在了一起。在原理部分，把概念阐述清楚，把知识体系提取出来，使读者在最短的时间内掌握最核心、最主要的系统理论。在接口部分，密切结合实际，介绍了工程实践中常用的接口芯片以及在单片机应用系统中的应用方法，给出了连接示意电路图和主要接口程序。在系统设计部分，立足工程实践，把常遇到的各种因素详细地介绍给读者。精简而系统的理论知识、实用的接口技术和工程中常用的实战经验是本书相辅相成、缺一不可的三个组成部分。

自 2009 年 1 月以来，《单片机原理与应用》已经过两次印刷，销量在 1 万册以上，被山东大学、山东轻工业学院、山东电力专科学校、山东电子职业技术学院等多家院校选定为专业必修课“单片机原理与应用”的指定教材，使用效果良好。

在本书的使用过程中，用户感到不方便的是现有的实验教学仪器和实验教学计划与本教材无法配套，因而影响了本教材的实际使用效果。导致出现这一现象的原因是现有的实验教

学仪器一般都是 2000 年左右配备的，这些实验教学仪器以及当时的实验教学计划侧重的是 MCS-51 单片机内部的原理性验证，接口实验较少，并且大部分的接口实验都是针对传统的并行接口芯片，这些芯片在现实工程中已很少使用。现实工程中常用的一些串行接口芯片在教材中已有大量篇幅介绍，但由于现有实验教学仪器不支持，所以无法进行实验验证。并且现实工程中用于单片机前向通道的传感器和用于后向通道驱动器在现有的实验教学仪器也支持的很少。系统性的综合实验，虽然原来的教学仪器与教学计划也有所涉及，由于设备陈旧，一般能够开展的创新实验也相对比较简单，一些类似于近年来在全国大学生电子设计大赛中出现的设计题目很难基于现有的实验教学仪器进行。

为此，我们设计了新的“SDU_MCU 系列单片机教学实验箱”，并编写了本实验教材。

本实验教材设计了软件仿真实验、硬件接口实验、综合创新实验三个模块。内容涵盖了整个 MCS-51 单片机教学计划中的汇编及 C 语言程序设计、片内外设及中断系统应用、片外存储器扩展、I/O 扩展、人机交互（包括键盘输入、LED 数码管显示、LCD 显示以及 LED 点阵显示）扩展、信号的输入输出通道（包括常用的传感器、A/D、D/A 和开关量的输入输出）扩展，以及综合性系统设计等内容。整个实验教学计划按 32 到 48 学时设计，具体实验内容如下：

- (1) 纯软件仿真实验（4 个）：单片机软件开发环境 Keil C51 的使用以及以此为基础的纯软件仿真实验共有 4 个，其中 2 个汇编语言程序实验，1 个 C 语言程序实验，1 个 C 与汇编混合编程实验。
- (2) PROTUES 软件仿真实验 12 个。
- (3) 硬件接口实验（26 个）：使用硬件仿真调试器，基于“SDU_MCU 系列单片机教学实验箱”实现的硬件接口实验共有 26 个，分别是：按键声光报警实验 1 个，并口扩展实验 1 个，串并转换实验 1 个，并串转换实验 1 个，8255 接口实验 2 个，7279 键盘/动态 LED 显示实验 1 个，1602 LCD 显示及计算器实验 1 个，12864 LCD 显示实验 1 个，16*16LED 点阵显示实验 1 个，I2C 扩展实验 1 个，RTC 实验 2 个，存储器扩展实验 1 个，AD 实验 2 个，DA 实验 2 个，温度测量实验 1 个，温湿度测量实验 1 个，红外对管发射接收与红外遥控实验 1 个，红外热释电实验 1 个，超声测距实验 1 个，电机驱动实验 3 个。
- (4) 系统综合实验 4 个，分别是：电梯控制系统实验，自动国旗升降系统实验，基于单片机控制的函数发生器系统实验和分布式远程抄表系统实验。

一般情况下要完成这些实验至少需要 48 学时，其中软件仿真实验、大部分硬件接口实

验 1 学时基本上可以完成 1 个实验,而部分较复杂的硬件接口实验可能需要安排 2 个学时或 2 个以上学时来实验,系统综合实验每个实验至少需要 3 个或 4 个学时。由于本实验箱采用模块化结构,便于拆解和携带,对于较为复杂的硬件实验可以将所需要的模块带离实验室进行。当实验学时较少时,也可以根据学时安排选做其中的部分实验。

本书由栗华编写。研究生王增迪、马良、薛树滨和宋玉川同学参与了实验箱硬件电路制板与调测以及本教材中的实验程序调试验证等工作。本书在编写过程中得到山东大学信息科学与工程学院的领导和各位同仁的热情帮助和鼓励,在此一并表示感谢。

感谢山东大学出版社对本书出版给予的支持和帮助。

由于作者水平有限,书中所存在的错误和不完善的地方,恳请读者批评指正。

编者

2012 年 10 月

目 录

第一章 51 单片机开发环境及 Proteus 仿真软件使用指南	7
1.1 集成开发环境 Keil uVision2 应用指南	7
1.1.1 Keil uVision2 软件的启动	7
1.1.2 Keil uVision2 主界面简介	8
1.1.3 Keil uVision2 创建项目实例	9
1.1.4 Keil uVision2 软件调试	24
1.2 系统仿真软件 Proteus 应用指导	30
1.2.1 Proteus 软件的启动	31
1.2.2 Proteus 应用及实例	31
第二章 软件实验	58
2.1 实验 1: 汇编程序实验 1	58
2.1.1 示例实验	58
2.1.2 自我完成实验	59
2.2 实验 2: 汇编程序实验 2	61
2.2.1 示例实验	61
2.2.2 自我完成实验	62
2.3 实验 3: C 语言程序实验	64
2.3.1 示例实验	64
2.3.2 自我完成实验	66
2.4 实验 4: C 与汇编混合编程实验	66
2.4.1 示例实验	66
第三章 Proteus 系统仿真实验	70
3.1 实验 1: 基本并行 I/O 口实验	70
3.1.1 示例实验	70
3.1.2 自我完成实验	73
3.2 实验 2: 扩展并行 I/O 口实验	75
3.2.1 示例实验	75
3.2.2 自我完成实验	77
3.3 实验 3: 静态 LED 显示实验	80
3.3.1 示例实验	80
3.3.2 自我完成实验	85
3.4 实验 4: 矩阵键盘扫描实验	88
3.4.1 示例实验	88
3.4.2 自我完成实验	94
3.5 实验 5: 定时计数器实验	99
3.5.1 示例实验	99
3.5.2 自我完成实验	103
3.6 实验 6: 串口通信实验	107
3.6.1 示例实验	107
3.6.2 自我完成实验	113
3.7 实验 7: LCD1602 显示实验	116

3.7.1 字符型 LCD 1602 简介	116
3.7.2 LCD1602 示例实验	124
3.7.3 LCD1602 自我完成实验	129
3.8 实验 8: ADC0808/9 信号采集实验	132
3.8.1 示例实验	132
3.8.2 自我完成实验	141
3.9 实验 9: DAC0832 应用实验	146
3.9.1 示例实验	146
3.9.2 自我完成实验	156
3.10 实验 10: I2C 扩展实验	161
3.10.1 I2C 总线简介	161
3.10.2 I2C 总线接口器件 PCF8574 说明	165
3.10.3 I2C 总线 E2PROM AT24C02 说明	166
3.10.4 示例实验	168
3.10.5 自我完成实验	181
3.11 实验 11: SPI 扩展实验	184
3.11.1 SPI 总线简介	184
3.11.2 DS1302 简介	185
3.11.3 基于 SPI 器件 DS1302 的万年历实验	187
3.12 实验 12: LCD12864 显示实验	194
3.12.1 点阵型 LCD 12864 简介	194
3.12.2 LCD 12864 显示实验	200
第四章 硬件接口实验	209
4.1 实验 1: 按键声光报警实验	209
4.2 实验 2: 373 并口扩展实验	212
4.3 实验 3: 74LS165 并/串行 IO 口转换实验	214
4.4 实验 4: 74LS164 串/并行 IO 口转换实验	217
4.5 实验 5: 8255 并行 I/O 扩展及交通信号灯控制实验	220
4.6 实验 6: 8255 动态 LED 显示实验	224
4.7 实验 7: 7279 键盘扫描及动态 LED 显示实验	227
4.8 实验 8: 1602 LCD 显示实验	231
4.9 实验 9: 12864 LCD 显示实验	235
4.10 实验 10: 16*16 LED 点阵显示实验	237
4.11 实验 11: ADC0809 并行接口 AD 转换实验	243
4.12 实验 12: DAC0832 并行接口 DA 转换实验	246
4.13 实验 13: DS12887 并行接口 RTC 实验	251
4.14 实验 14: I2C 串行 E2PROM 24C02 读写实验	256
4.15 实验 15: I2C 接口芯片 PCF8574 扩展并口实验	259
4.16 实验 16: I2C 接口芯片 PCF8563 扩展 RTC 实验	262
4.17 实验 17: I2C 接口芯片 AD7810 扩展 AD 实验	266
4.18 实验 18: I2C 接口芯片 TLC5615 扩展 DA 实验	270
4.19 实验 19: 18B20 温度测量实验	274
4.20 实验 20: DHT11 温湿度测量实验	277
4.21 实验 21: 红外对管障碍物检测实验	282

4.22 实验 22: 红外热释电实验.....	286
4.23 实验 23: 超声测距实验.....	289
4.24 实验 24: 步进电机驱动实验.....	293
4.25 实验 25: 直流电机驱动及转速测量实验.....	296
4.26 实验 26: 电机手动调速与程控调速.....	299
第五章 系统综合实验.....	303
5.1 单片机应用系统设计流程简介	303
5.1.1 需求调查.....	304
5.1.2 可行性分析.....	304
5.1.3 总体方案设计.....	304
5.1.4 硬件设计.....	305
5.1.5 软件设计.....	306
5.1.6 仿真调试.....	307
5.2 电梯控制系统实验.....	309
5.3 程控函数发生器系统实验.....	310
5.4 自动国旗升降系统实验.....	312
5.5 自动远程抄表系统实验.....	313
附录.....	314
I2C 总线简介	314
SPI 总线简介	318
FAT 文件系统简介	321

第一章 51 单片机开发环境及 Proteus 仿真软件使用指南

1.1 集成开发环境 Keil uVision2 应用指南

Keil 软件是众多单片机应用开发的优秀软件之一，它集编辑，编译，仿真于一体，支持汇编, PLM 语言和 C 语言的程序设计，界面友好，易学易用。

下面通过图解的方式来介绍 Keil uVision2 软件的使用。学习如何输入源程序 → 新建工程 → 工程详细设置 → 源程序编译得到目标代码文件。

1.1.1 Keil uVision2 软件的启动

首先双击 Keil 的桌面快捷方式（如图 1.1 所示），启动 Keil 集成开发开发软件。



图 1.1 Keil uVision2 桌面图标

启动 Keil uVision2 后，出现如下图 1.2 所示界面：



图 1.2 Keil uVision2 启动界面

几秒钟后出现主界面（如下图 1.3 所示）：

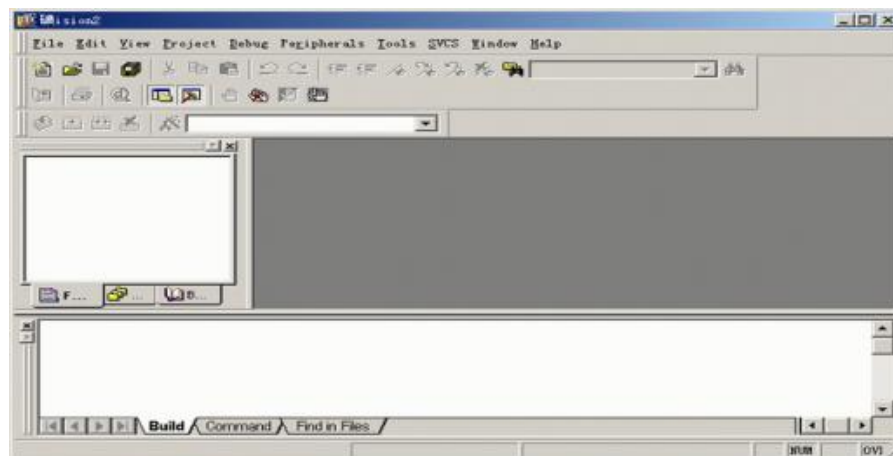


图 1.3 Keil uVision2 主界面

1.1.2 Keil uVision2 主界面简介

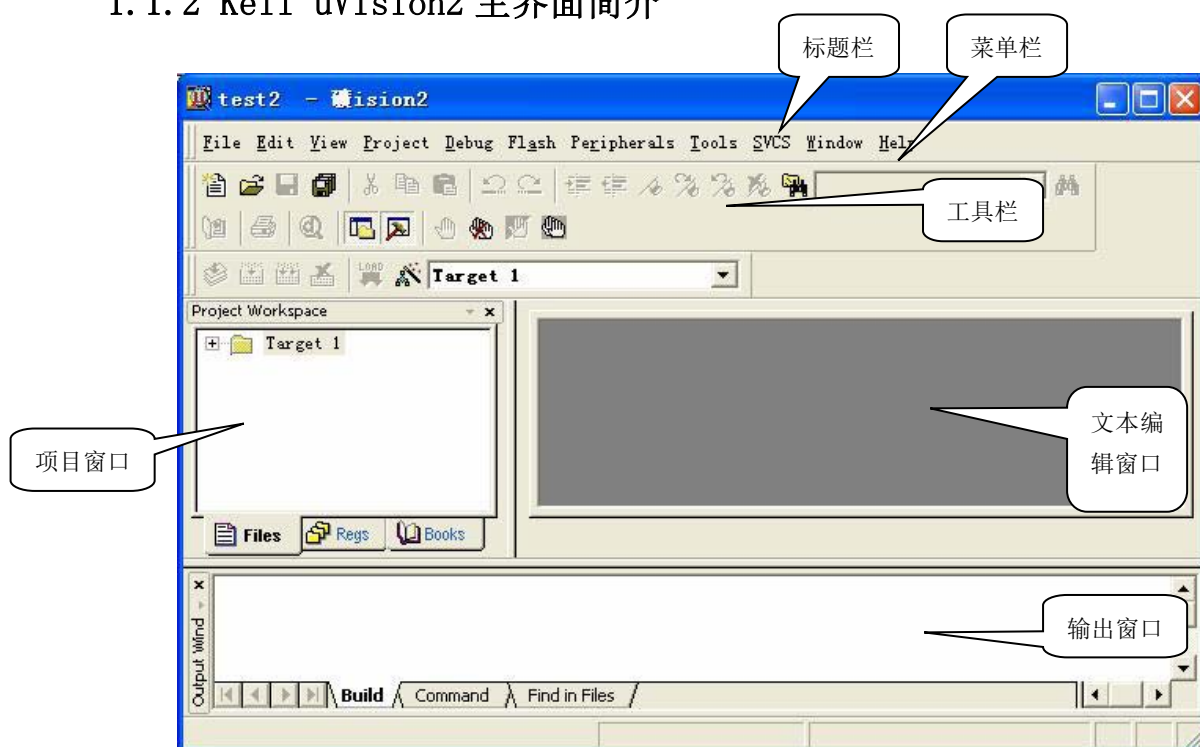


图 1.4 Keil uVision2 主界面窗口

Keil uVision2 的主界面窗口（如图 1.4 所示）分为项目窗口（用于管理项目各组成文件）、文本编辑窗口（用于输入、编辑和察看文件代码）和输出窗口（用于显示文件编译结果），还有一些调试窗口。

Keil uVision2 的主界面窗口也有菜单条和工具栏，大部分的菜单操作和工具栏操作都有对应的快捷键。下面的表格列出了 uVision2 主要菜单命令，工具栏图标，默认的快捷以及他们的描述。

（1）项目菜单 Project 和项目命令 Project（如表 1-1 所示）

表 1-1 项目菜单和项目命令 Project

菜单	工具栏	快捷键	描述
New Project...			创建新项目
Open Project...			打开一个已经存在的项目
Close Project...			关闭当前的项目
Select Device for Target			选择对象的 CPU
Remove ...			从项目中移走一个组或文件
Options ...		Alt+F7	设置对象、组或文件的工具选项
File Extensions			选择不同文件类型的扩展名
Build Target		F7	编译修改过的文件并生成应用
Rebuild Target			重新编译所有的文件并生成应用

Translate ...		Ctrl+F7	编译当前文件
Stop Build			停止生成应用的过程

(2) 调试菜单 Debug 和调试命令 (如表 1-2 所示)

表 1-2 调试菜单和调试命令 Debug

菜单	工具栏	快捷键	描述
Start/Stop Debugging		Ctrl+F5	开始/停止调试模式
Go		F5	运行程序，直到遇到一个中断
Step		F11	单步执行程序，遇到子程序则进入
Step over		F10	单步执行程序，跳过子程序
Step out of		Ctrl+F11	执行到当前函数的结束
Current function stop Runing		Esc	停止程序运行
Breakpoints...			打开断点对话框
Insert/Remove Breakpoint			设置/取消当前行的断点
Enable/Disable Breakpoint			使能/禁止当前行的断点
Disable All Breakpoints			禁止所有的断点
Kill All Breakpoints			取消所有的断点
Memory Map...			打开存储器空间设置对话框
Performance Analyzer...			打开设置性能分析的窗口

1.1.3 Keil uVision2 创建项目实例

uVision2 包括一个项目管理器，它可以使 MCS-51 应用系统的设计变得简单。要创建一个应用，需要按下列步骤进行操作：

- 新建一个项目文件并从器件库中选择一个器件。
- 新建一个源文件并把它加入到项目中。
- 针对目标硬件进行设置选项。
- 编译项目并生成可编程 PROM 的 HEX 文件。

下面将逐步地进行描述，从而指引读者创建一个简单的 uVision2 项目。

(1): 新建文本编辑窗。点击工具栏上的新建文件按钮 ，或者点击“File”→

“New...” 菜单命令，即可在项目窗口的右侧打开一个新的文本编辑窗，如图 1.5 所示。

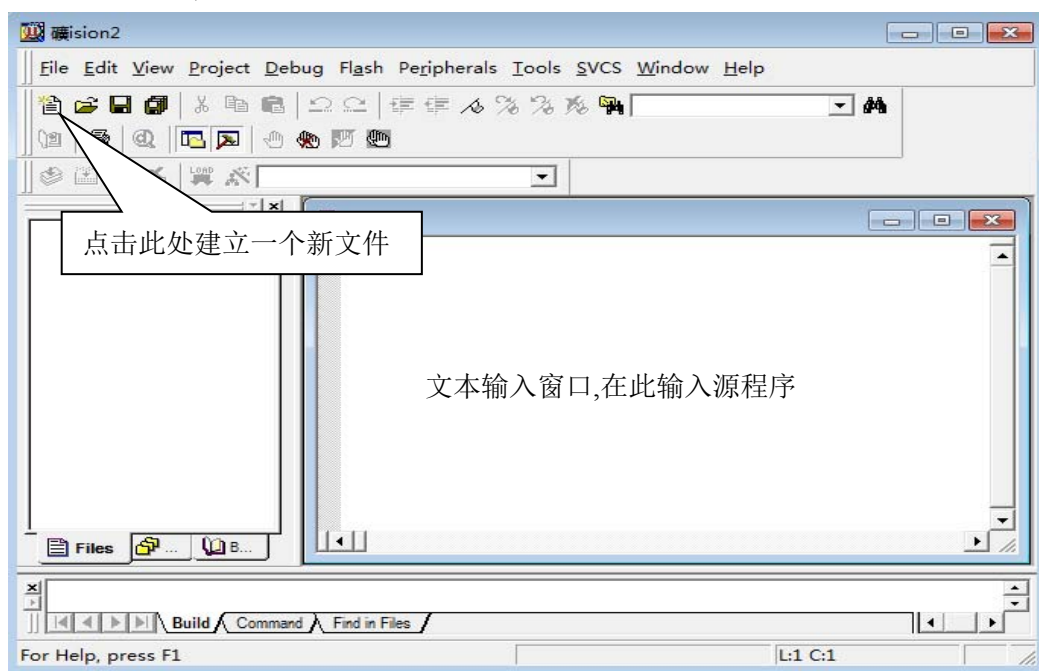


图 1.5 新建文本编辑窗

第三步：输入源程序。在新的文本编辑窗中输入源程序，可以输入 C 语言程序，也可以输入汇编语言程序，如图 1.6 所示。

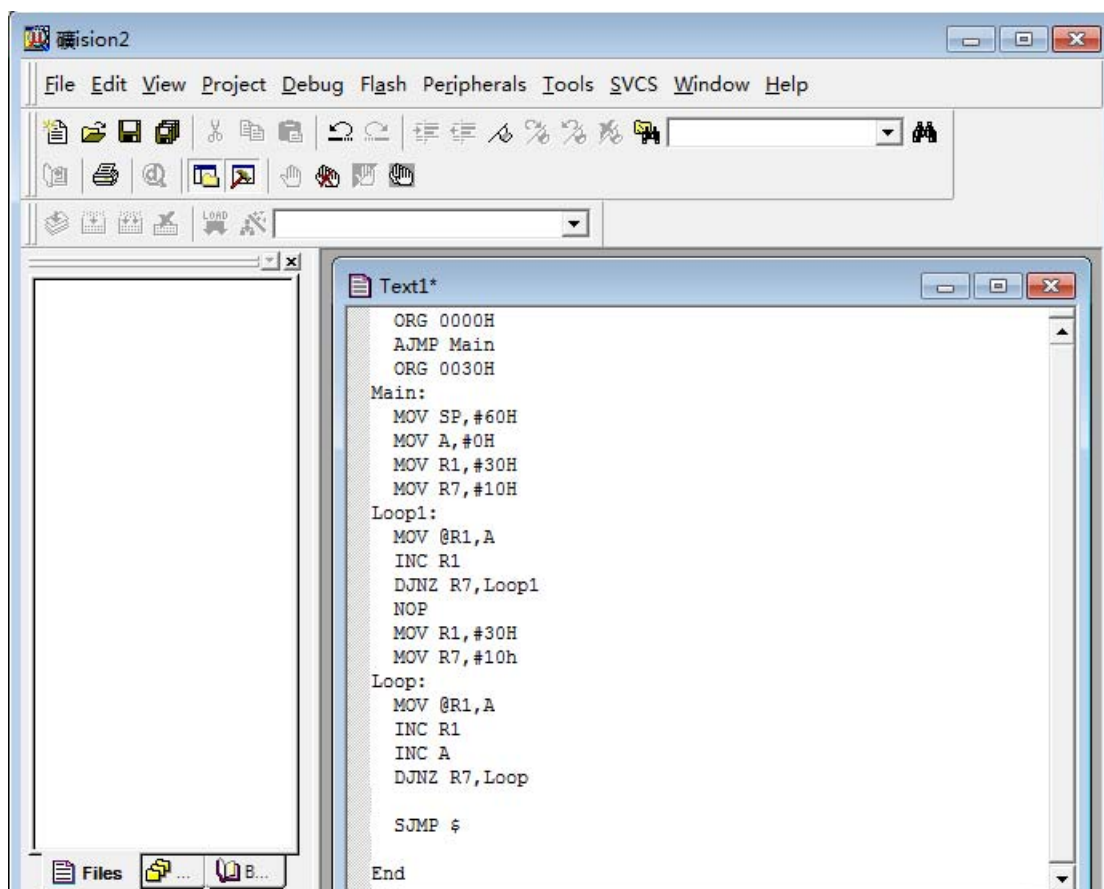


图 1.6 输入源程序

第四步：保存源程序。点击工具栏上的保存文件按钮，或点击 “File” → “Save” 菜单命令保存源程序（如图 1.7 所示），点击后回弹出一个对话框如图 1.8 所示，应在对话框中的“文件名”栏输入要保存的文件名，注意保存文件时必须加上文件的扩展名，如果你使用汇编语言编程，那么保存时文件的扩展名为 “.asm”，如果是 C 语言程序，文件的扩展名使用 “*.C”。注：第三步和第四步之间的顺序可以互换，即可以先输入源程序后保存，也可以先保存后输入源程序。

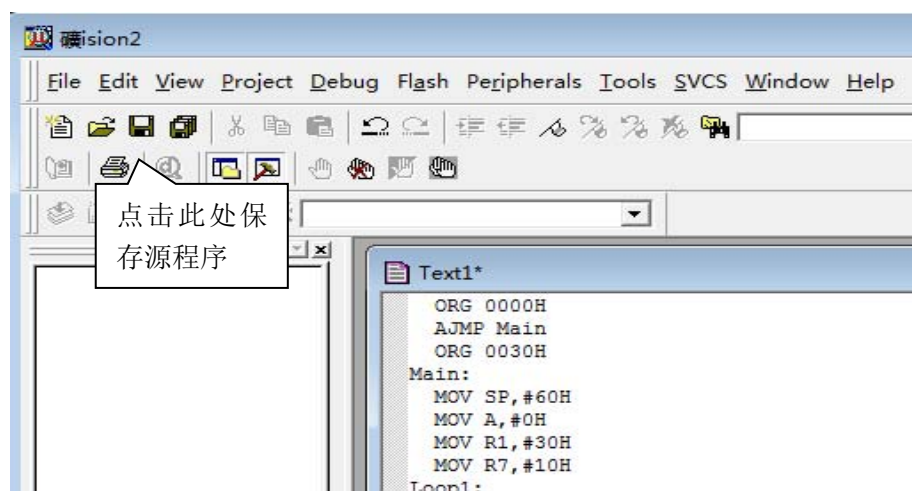


图 1.7 保存源程序

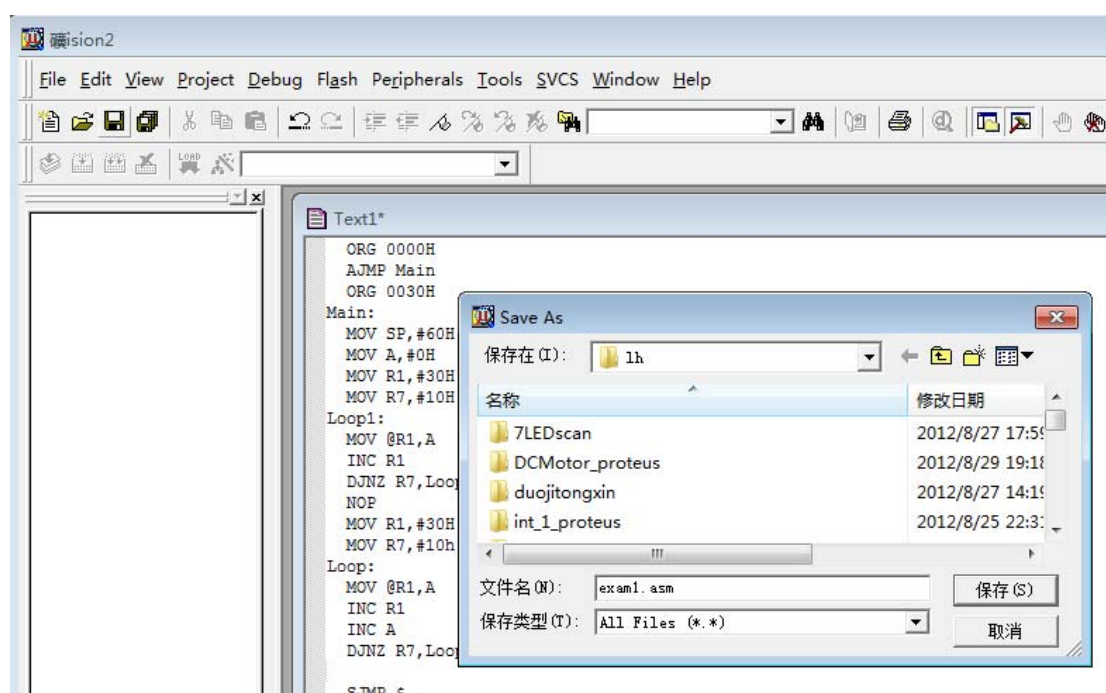


图 1.8 保存源程序对话框

第五步：新建立 Keil 工程。如图 1.9 所示，点击 “Project” → “New Project ...” 菜单命令，将出现如图 1.10 所示对话框。

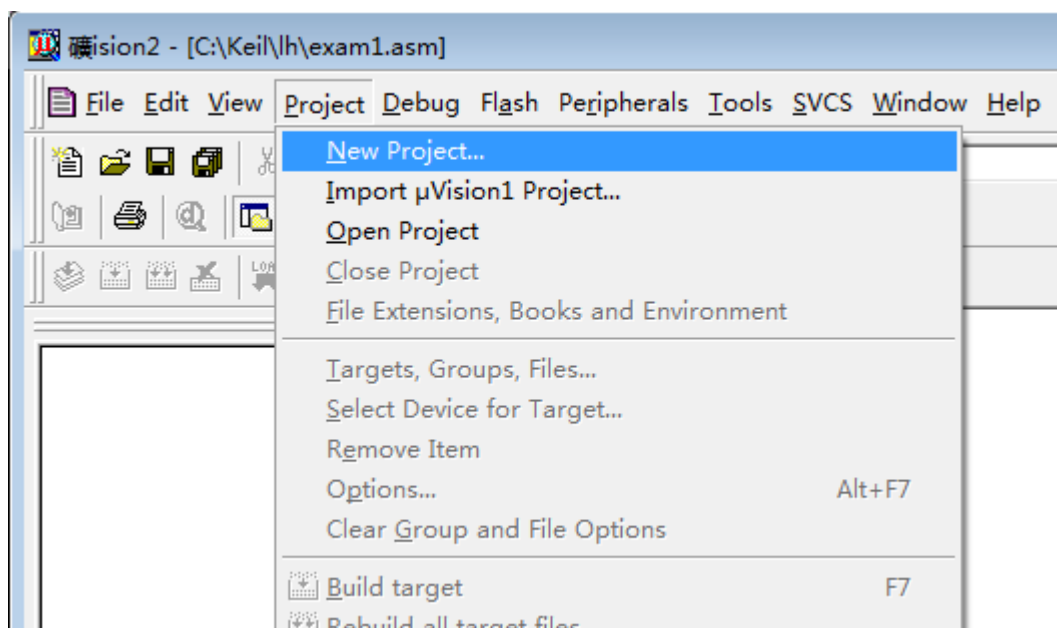


图 1.9 新建工程

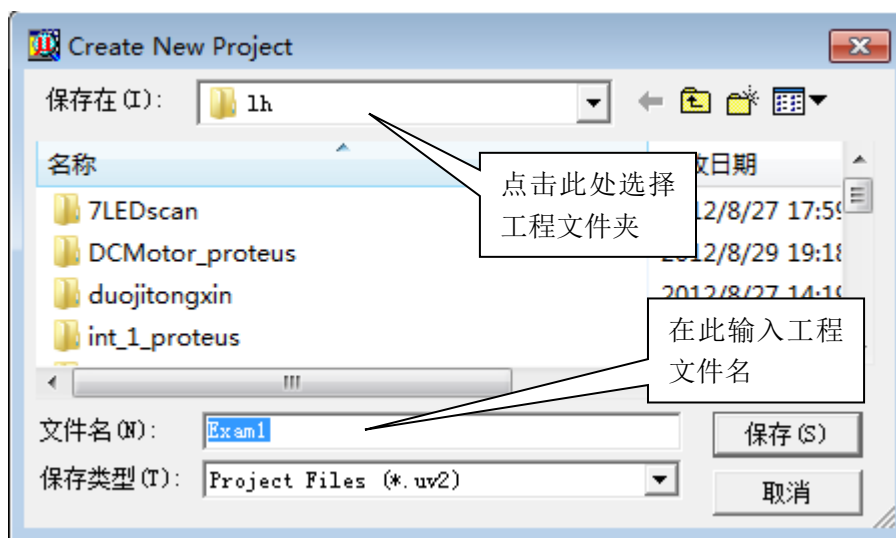


图 1.10 新建工程对话框

在保存工程对话框中输入你的工程的文件名，Keil 工程默认扩展名为“.uv2”，工程名称不用输入扩展名（见图 1.10），一般情况下使工程文件名称和源文件名称相同即可，输入名称后保存，将出现“选择设备”对话框。

第六步：选择 CPU 型号。如图 1.11 所示，为工程选择 CPU 型号，本新建工程选择了 ATMEL 公司的 AT89C51 单片机。

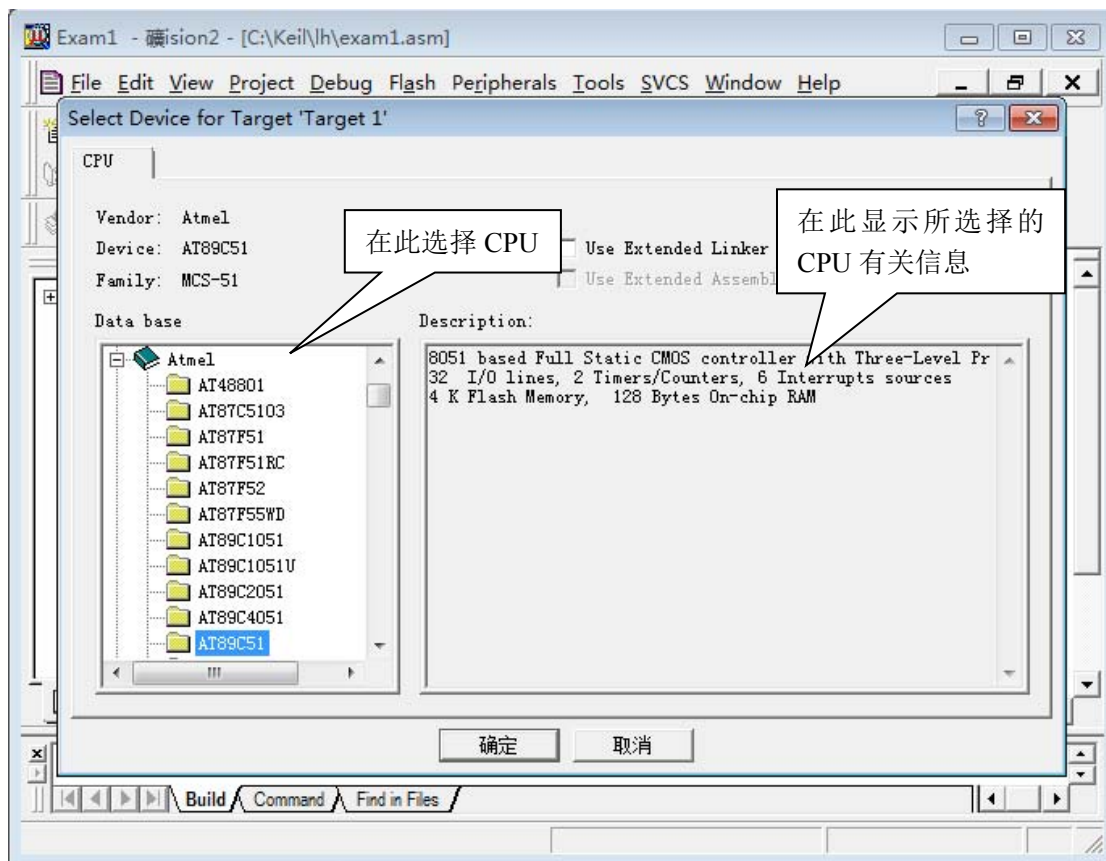


图 1.11 选择 CPU 型号对话框

选定型号，点击确定之后，将会出现一个对话框，询问是否往工程里面添加初始化代码，可以选择“是(Y)”，如图 1.12 所示，这时将在工程管理窗口显示出一个文件“STARTUP. 51”，

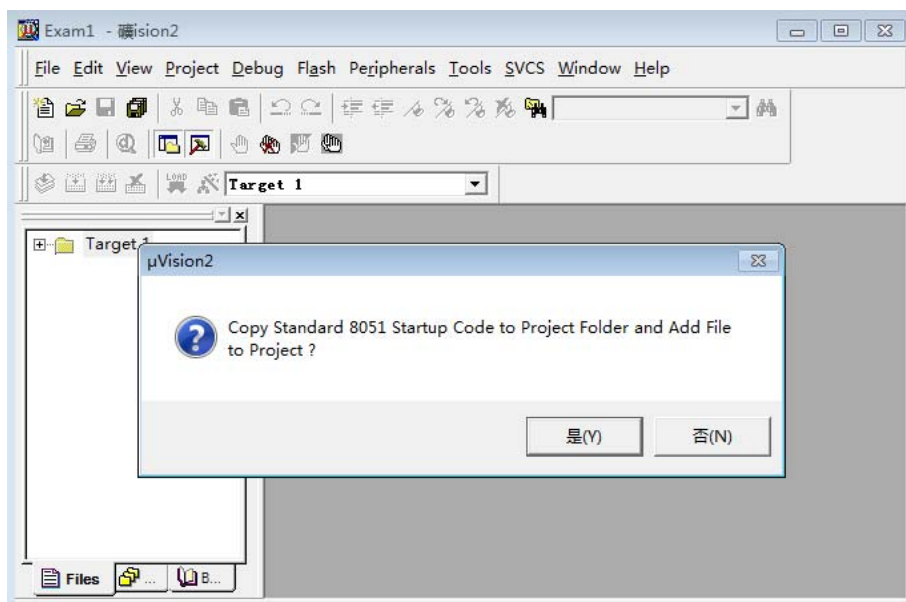


图 1.12 初始化代码对话框

“STARTUP. 51”文件是单片机复位之后执行的初始化程序，该程序将对单片机的内存以及堆栈进行初始化。内存在上电时里面的内容是不确定的，STATUP. A51 文件就是根据内存的大小，把内存初始化为零，并初始化堆栈指针 SP。对于采用汇编语言进行编程的，如果

不需要提前做这些工作，而是由你自己的程序完成这些工作的话，可以不用加入“STARTUP.51”文件（一般情况下，汇编程序不用加入该文件）。而对于采用 C 语言编程的，“STARTUP.51”除了内存及堆栈初始化，还将为 C51 编译器的运行准备初始运行条件，所以要选择加入该文件。

此时可见到工程管理窗中出现“Target 1”，点击“Target 1”前面的“+”号展开出现下一层的“Source Group 1”文件夹，展开“Source Group 1”文件夹前面的“+”，可以看到该文件夹下有一个文件“STARTUP.51”如图 1.13 所示：

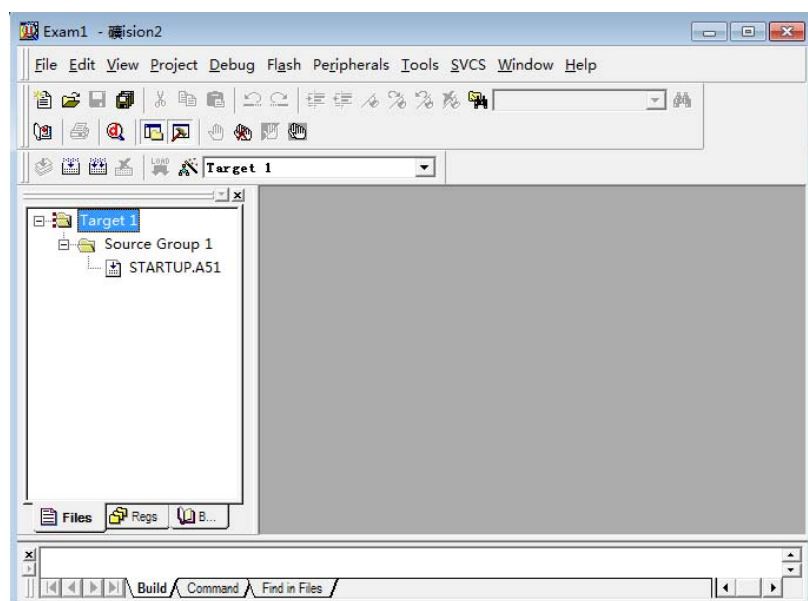


图 1.13 工程管理窗口

第七步：加入源程序到工程中。如图 1.14 所示，右击工程管理窗中的“Source Group 1”，出现下拉菜单，点击“增加文件到组‘Source Group 1’”命令，将出现添加文件对话框（图 1.15），选择刚才建立的源文件，注意文件类型（即扩展名）的选择，在对话框中的文件类型默认为“C 源文件 (*.c)”，如果你要添加到工程中的是汇编语言程序，则在文件类型中必须选中“asm 源文件 (*.s*; *.src; *.a*)”，头文件、库文件和目标文件也有相应的选项，选择不当将看不到你要添加的文件。找到要添加的文件后，选中，点击“Add”按钮，即可将选择的文件添加到工程。

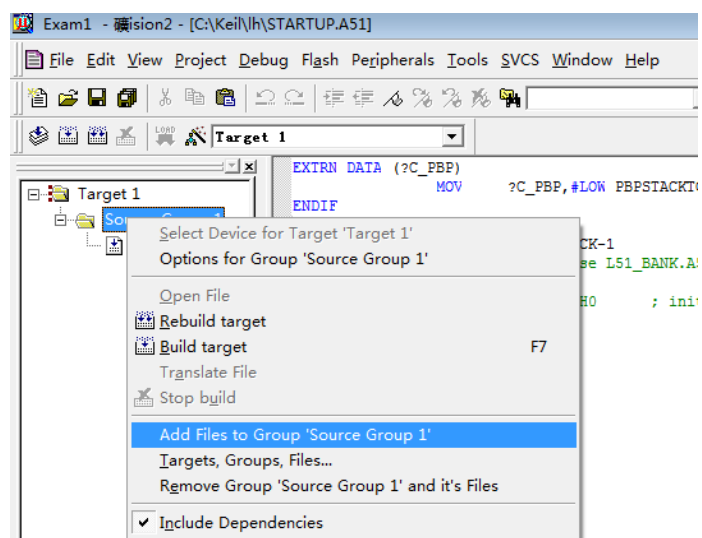


图 1.14 往工程中添加源文件

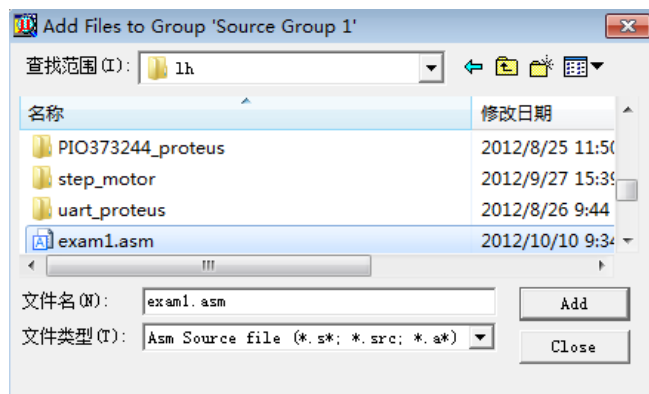


图 1.15 往工程中添加源文件对话框

把文件添加到工程中，此时添加文件对话框并不会自动关闭，而是等待继续添加其他文件，初学者往往以为没有加入成功，再次双击该文件，则会出现图 1.16 对话框，表示该文件不再加入目标。此时应该点击“确定”按钮，返回到前一对话框，再点击“关闭”按钮，返回到主界面。

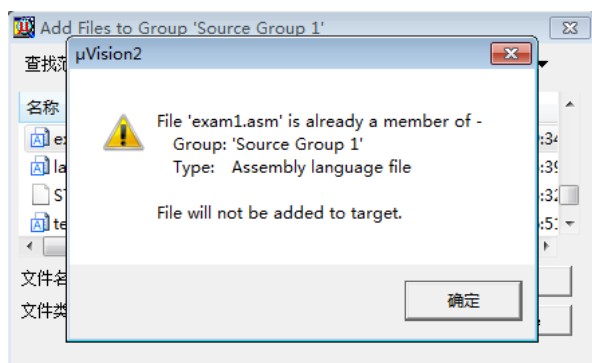


图 1.16 重复加入文件对话框

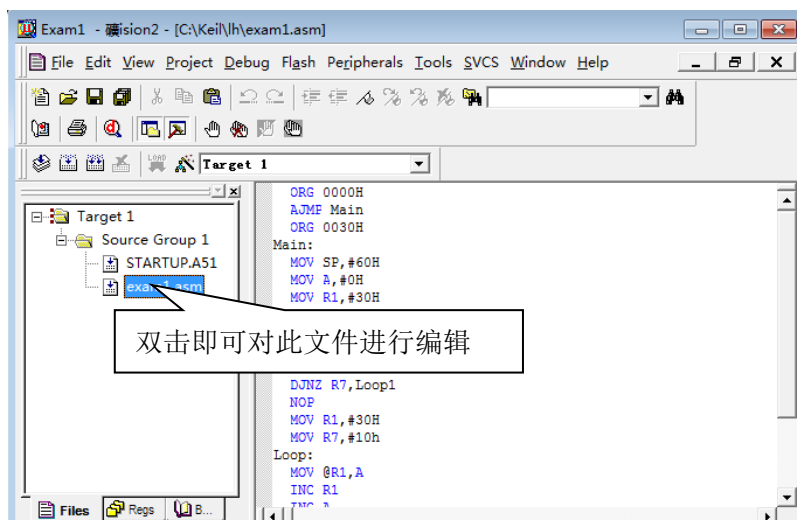


图 1.17 文件成功加入工程

如图 1.17 所示，“Source Group 1”文件夹下具有了“exam1.asm”文件，可见该源文件已成功加入工程，双击即可打开该文件进行编辑修改源程序

第八步：工程目标‘Target 1’属性设置。如下图 1.18 所示，在工程项目管理窗中的“Target 1”文件夹上右击，出现下拉菜单，点击“目标‘Target 1’选项”命令，就进入目标属性设置界面。

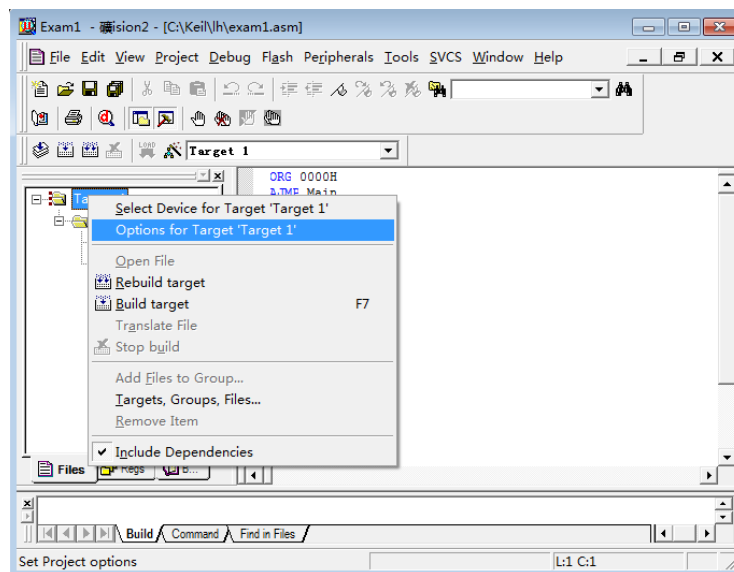


图 1.18 进入工程目标属性设置

工程目标‘Target 1’属性设置对话框（图 1.19）中有 8 个页面，设置的项目繁多复杂，大部分使用默认设置即可，主要设置其中的“目标 Target”、“输出 Output”、“列表 Listing”、“调试 Debug”四个页面，下面对这四个页面的设置进行详细介绍。

（1）、工程目标属性（Target）页面设置

该页面单片机的晶振频率、存储器等，将晶振的频率改为 11.0592，频率设置和所用实验板上的实际晶振频率相同即可（图 1.19）。



图 1.19 工程目标属性设置

本选项页的各选项含义如下：

- Xtal (MHZ)：设置单片机工作的频率，默认是 24.0MHz。
- Use On-chip ROM(0x0-0xFFFF)：表示使用片上的 Flash ROM，At89C51 有 4KB 的可重编程的 Flash ROM，该选项取决于单片机应用系统，如果单片机的 EA 接高电平，则选中这个选项，表示使用内部 ROM，如果单片机的 EA 接低电平，表示使用外部 ROM，则不选中该项。这里选中该项。
- Off-chip Code memory：表示片外 ROM 的开始地址和大小，如果没有外接程序存储器，那么不需要填任何数据。这里假设使用一个片外 ROM，地址从 0x8000 开始，一般填 16 进制的数，Size 为片外 ROM 的大小。假设外接 ROM 的大小为 0x1000 字节，则最多可以外接 3 块 ROM。
- Off-chip Xdata memory：那么可以填上外接 Xdata 外部数据存储器的起始地址和大小，一般的应用是 62256，这里特殊的指定 Xdata 的起始地址为 0x2000，大小为 0x8000。
- Code Banking：是使用 Code Banking 技术。Keil 可以支持程序代码超过 64KB 的情况，最大可以有 2MB 的程序代码。如果代码超过 64KB，那么就要使用 Code Banking 技术，以支持更多的程序空间。Code Banking 支持自动的 Bank 的切换，这在建立一个大型系统时是必需的。例如：在单片机里实现汉字字库，实现汉字输入法，都要用到该技术。
- Memory Model：单击 Memory Model 后面的下拉箭头，会有 3 个选项，如图 1.20 所示。

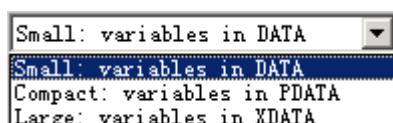


图 1.20 Memory Model 选项

- Small：变量存储在内部 RAM 里。

- Compact: 变量存储在外部 RAM 里, 使用 8 位间接寻址。
- Large: 变量存储在外部 RAM 里, 使用 16 位间接寻址。

一般使用 Small 来存储变量, 此时单片机优先将变量存储在内部 RAM 里, 如果内部 RAM 空间不够, 才会存在外部 RAM 中。Compact 的方式要通过程序来指定页的高位地址, 编程比较复杂, 如果外部 RAM 很少, 只有 256 字节, 那么对该 256 字节的读取就比较快。

如果超过 256 字节, 而且需要不断地进行切换, 就比较麻烦, Compact 模式适用于比较少的外部 RAM 的情况。Large 模式是指变量会优先分配到外部 RAM 里。需要注意的是, 3 种存储方式都支持内部 256 字节和外部 64KB 的 RAM。因为变量存储在内部 RAM 里运算速度比存储在外部 RAM 要快得多, 大部分的应用都是选择 Small 模式。

使用 Small 模式时, 并不说明变量就不可以存储在外部, 只是需要特别指定, 比如:

unsigned char xdata a: 变量 a 存储在内部 RAM。

unsigned char a: 变量存储在内部 RAM。

但是使用 Large 的模式时:

unsigned char xdata a: 变量 a 存储在外部 RAM。

unsigned char a: 变量 a 同样存储在外部 RAM。

这就是它们之间的区别, 可以看出这几个选项只影响没有特别指定变量的存储空间的情况, 默认存储在所选模式的存储空间, 比如上面的变量定义 unsigned char a。

- Code Rom Size: 单击 Code Rom Size 后面的下拉箭头, 将有 3 个选项, 如图 1.21 所示。

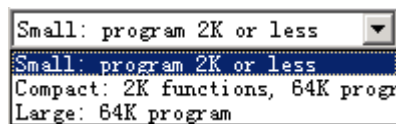


图 1.21 Code Rom Size 选项

- Small: program 2K or less, 适用于 AT89C2051 这些芯片, 2051 只有 2KB 的代码空间, 所以跳转地址只有 2KB, 编译的时候会使用 ACALL AJMP 这些短跳指令, 而不会使用 LCALL, LJMP 指令。如果代码地址跳转超过 2KB, 那么会出错。
- Compact: 2K functions, 64K program, 表示每个子函数的代码大小不超过 2K, 整个项目可以有 64K 的代码。就是说在 main() 里可以使用 LCALL, LJMP 指令, 但在子程序里只会使用 ACALL, AJMP 指令。只有确定每个子程序不会超过 2KB, 才可以使用 Compact 方式。
- Large: 64KB program, 表示程序或子函数代码都可以大到 64KB, 使用 code bank 还可以更大。通常都选用该方式。选择 Large 方式速度不会比 Small 慢很多, 所以一般没有必要选择 Compact 和 Small 方式。这里选择 Large 方式。
- Operating: 单击 Operating 后面的下拉箭头, 会有 3 个选项, 如图 1.22 所示。

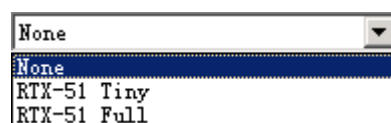


图 1.22 Operating 选项

- None: 表示不使用操作系统。
- RTX-51 Tiny Real-Time OS: 表示使用 Tiny 操作系统。
- RTX-51 Full Real-Time OS: 表示使用 Full 操作系统。

Tiny 是一个多任务操作系统，使用定时器 0 做任务切换。在 11.0592MHz 时，切换任务的速度为 30ms。如果有 10 个任务同时运行，那么切换时间为 300ms。不支持中断系统的任务切换，也没有优先级，因为切换的时间太长，实时性大打折扣。多任务情况下（比如 5 个），轮循一次需要 150ms，即 150ms 才处理一个任务，这连键盘扫描这些事情都实现不了，更不要说串口接收、外部中断了。同时切换需要大概 1000 个机器周期，对 CPU 的浪费很大，对内部 RAM 的占用也很严重。实际上用到多任务操作系统的情况很少。

Keil C51 Full Real-Time OS 是比 Tiny 要好一些的系统（但需要用户使用外部 RAM），支持中断方式的多任务和任务优先级，但是 Keil C51 里不提供该运行库，要另外购买。

这里选择 None。

(2)、工程输出 (Output) 页面设置

该页面设置如下图 1.23 所示。注意：如果要进行单片机写片实验，则一定要把“生成 HEX 文件”选项选中，程序编译后才能生成写单片机需要的 HEX 格式目标文件。（这一步是针对具有编程功能的仿真器或单片机编程器而设置的）

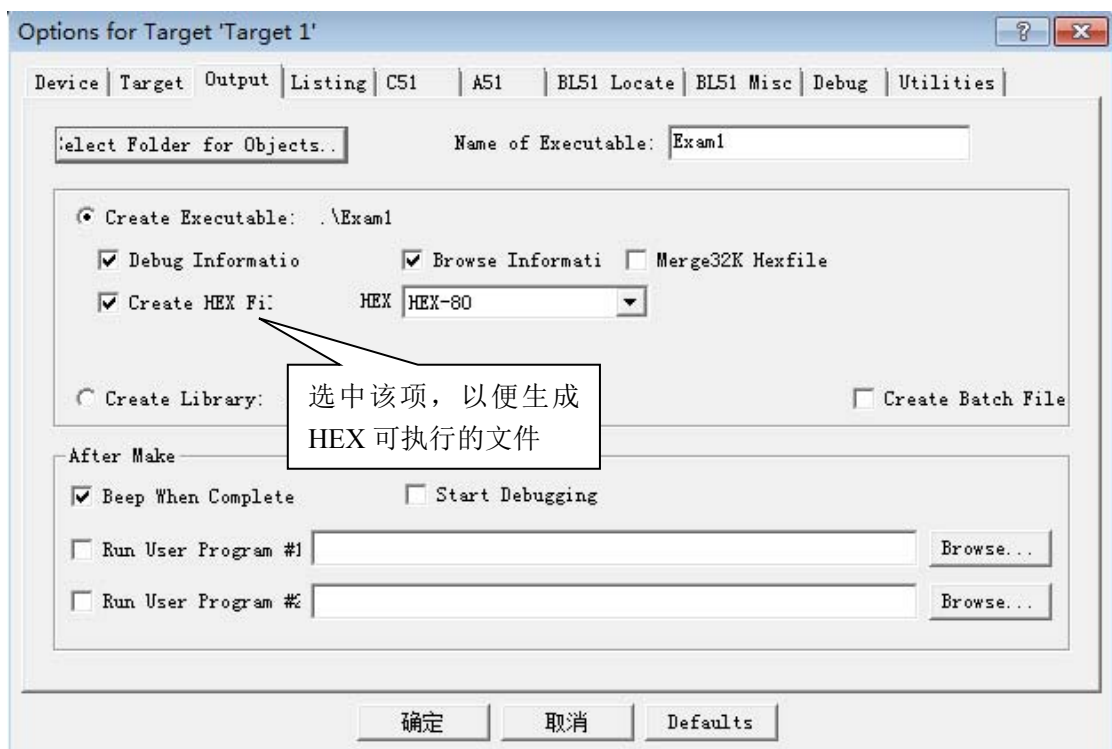


图 1.23 工程输出设置

工程输出设置窗口的其它选项含义如下：

- Select Folder for Objects: 单击该按钮可以选择编译后目标文件的存储目录，如果不设置，就存储在项目文件的目录里。
- Name of Executable: 设置生成的目标文件的名称，缺省情况下和项目的名称一样。目标文件可以生成库或者 obj、HEX 的格式。

- Create Executable: 如果要生成 OMF 以及 HEX 文件，一般选中 Debug Information 和 Browse Information。选中这两项，才有调试所需的详细信息，比如要调试 C 语言程序，如果不选中，调试时将无法看到高级语言写的程序。

- Create HEX File: 要生成 HEX 文件，一定要选中该选项，如果编译之后没有生成 HEX 文件，就是因为这个选项没有被选中。默认是不选中的。

- Create Library: 选中该项时将生成 lib 库文件。根据需要决定是否要生成库文件，一般应用是不生成库文件的。

- After Make: 栏中有以下几个设置。

- Beep when complete: 编译完成之后发出咚的声音。

- Start Debugging: 马上启动调试（软件仿真或硬件仿真），根据需要来设置，一般是不选中。

Run User Program #1, Run User Program #2: 这个选项可以设置编译完之后所要运行的其他应用程序（比如有些用户自己编写了烧写芯片的程序，编译完便执行该程序，将 HEX 文件写入芯片），或者调用外部的仿真器程序。根据自己的需要设置。

(3)、工程列表 (Listing) 页面设置

Keil C51 在编译之后除了生成目标文件之外，还生*.lst、*m51 的文件。这两个文件可以告诉程序员程序中所用的 idata、data、bit、xdata、code、RAM、ROM、stack 等的相关信息，以及程序所需的代码空间。

该页面设置窗口如图 1.24 所示：

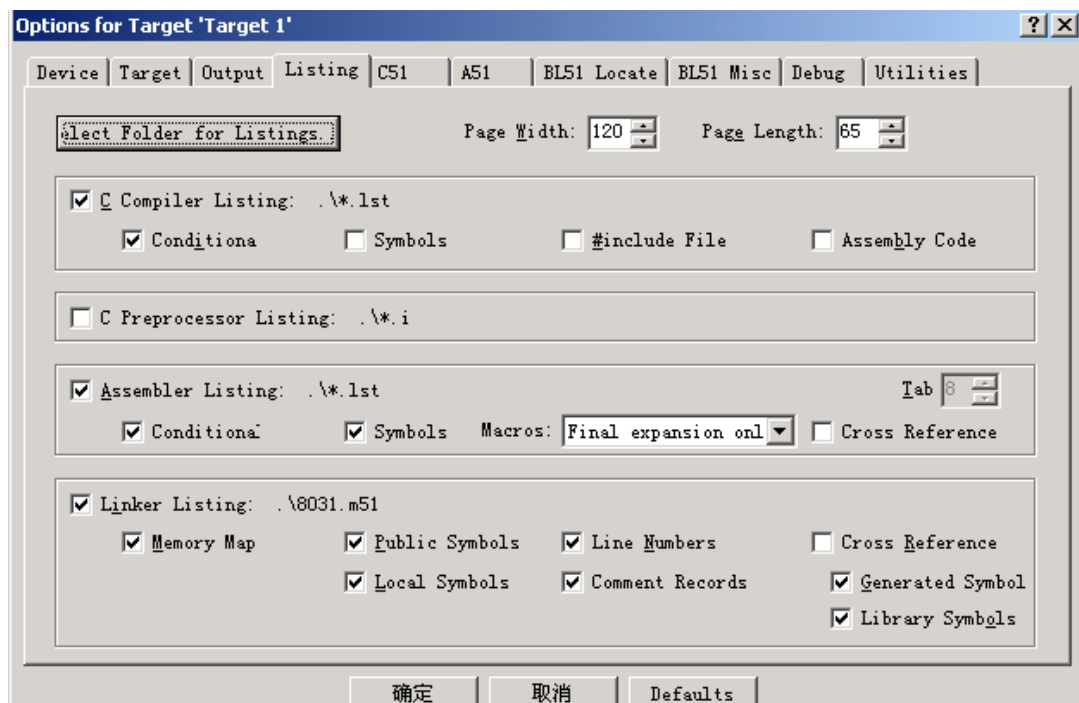


图 1.24 设置 Listing 选项卡

本窗口大部分选用默认设置。但是其中一个 Assembly Code 选项如果选中的话，会生成汇编的代码。这是很有好处的，如果不知道如何用汇编来写一个 long 型数的乘法，那么可以先用 C 语言来写，写完之后编译，就可以得到用汇编实现的代码。对于一个高级的单片机

程序员来说，往往既要熟悉汇编，同时也要熟悉 C 语言，才能更好地编写程序。某些地方用 C 语言无法实现，使用汇编语言却很容易。有些地方用汇编语言，很繁琐，用 C 语言就很方便。

单击 Select Folder for Listings 按钮后，在出现的对话框中可以选择生成的列表文件的存放目录。不做选择时，使用项目文件所在的目录。

(4)、工程调试 (Debug) 页面设置

“调试”页面设置如下图 1.25 所示。



图 1.25 工程调试设置

这里有两类仿真形式可选：Use Simulator) 和 Use: Keil Monitor-51 Driver，前一种是纯软件仿真，后一种是带有 Monitor-51 目标仿真器的仿真。

- Load Application at Start: 选择这项之后，Keil 才会自动装载程序代码。
 - Go till main: 调试 C 语言程序时可以选择这一项，PC 会自动运行到 main 程序处。
- 这里选择 Use Simulator。

如果选择 Use: Keil Monitor-51 Driver，还可以单击图 27 中的 Settings 按钮，打开新的窗口如图 1.26，其中的设置如下。

- Port: 设置串口号，为仿真机的串口连接线 COM_A 所连接的串口。
 - Baudrate: 设置为 9600，仿真机固定使用 9600bit/s 跟 Keil 通信。
 - Serial Interrupt: 允许串行中断，选中它。
 - Cache Options: 可以选也可以不选，推荐选它，这样仿真机会运行得快一点。
- 最后单击 OK 按钮关闭窗口。

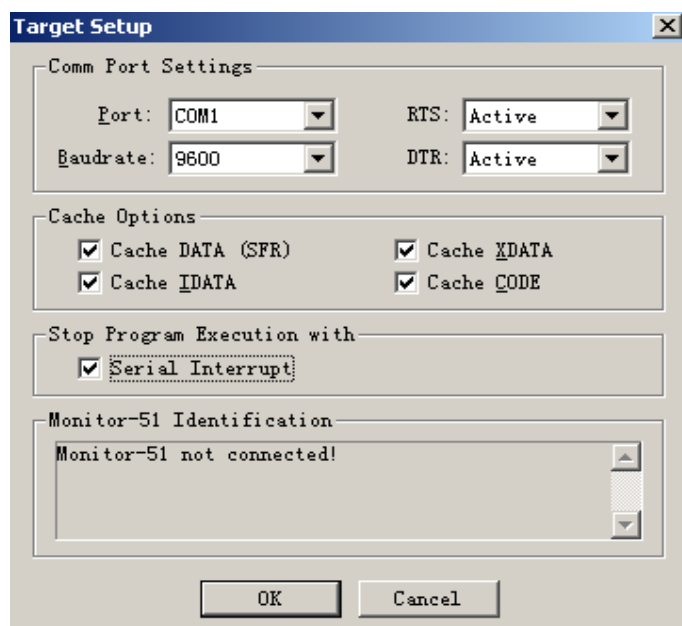


图 1.26 Target 设置

仿真器仿真环境下还可以利用虚拟的仿真器与 Proteus 一起实现联合仿真，在后面 Proteus 介绍章节介绍。

第九步：源程序的编译与目标文件的获得

至此，已经完成了从源程序输入、工程建立、工程详细设置的工作。下面将完成最后的步骤，此时可以在文本编辑窗中继续输入或修改源程序，使程序实现目标，在检查程序无误后保存工程。

下图 1.27 所示，点击“构造目标 Build target”快捷按钮 ，或者选用“Project”→“Build target”菜单命令，甚至使用快捷键“F7”进行源程序的编译连接，源程序编译相关的信息会出现在输出窗口中的“构造”页中。

下图 1.27 显示编译结果为 0 错误，0 警告，同时产生了目标文件 Exam1.hex。如果源程序中有错误，则不能通过编译，错误会在输出窗口中报告出来，双击该错误，就可以定位到源程序的出错行，可以对源程序进行反复修改，再编译，直到没有错误为止。注意：每次修改源程序后一定要保存。

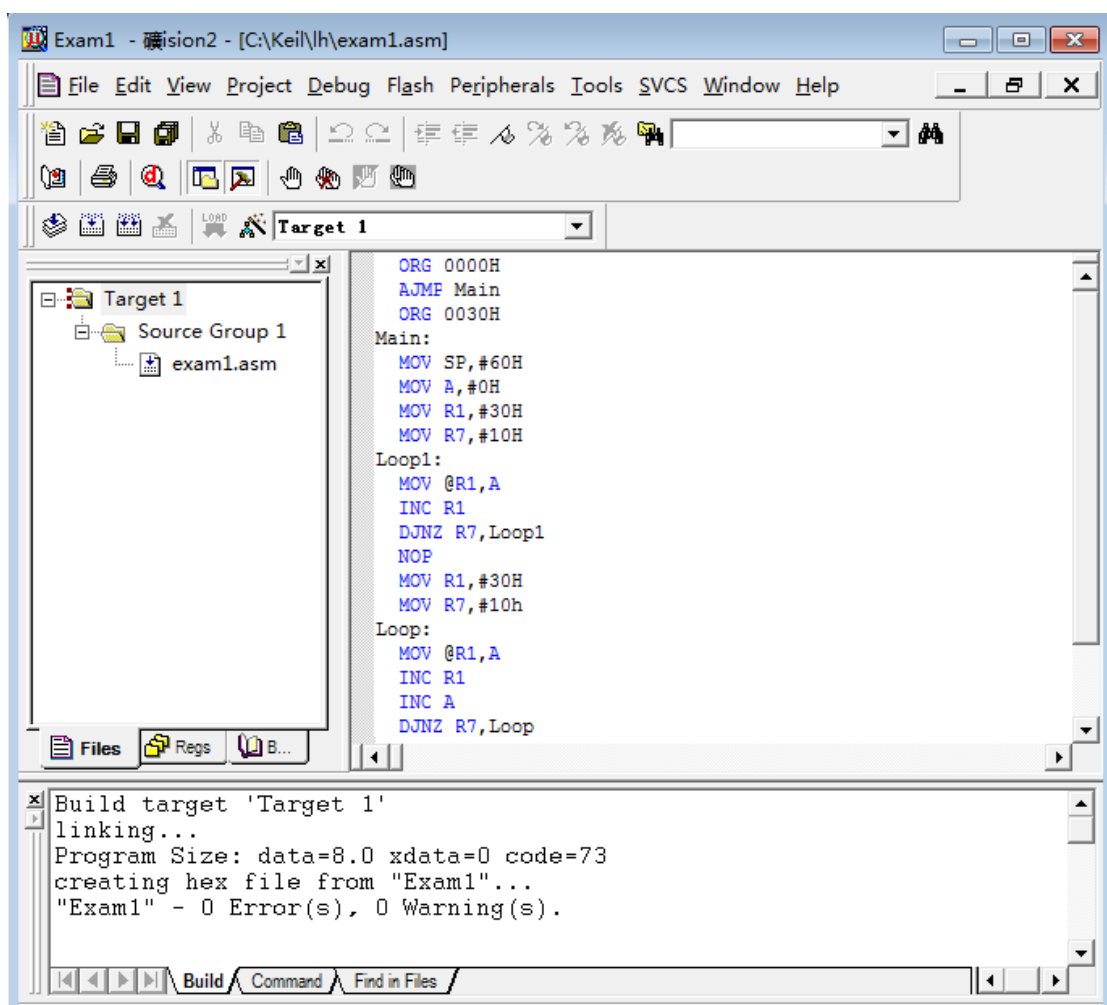


图 1.27 源程序的编译构造

编译通过后，打开工程文件夹（图 1.28），可以看到文件夹中有了“Exam1.hex”，这就是需要的最终目标文件，（此时，可以用编程器（也称烧录器）把该文件写入单片机，单片机就可以实现程序的功能了。

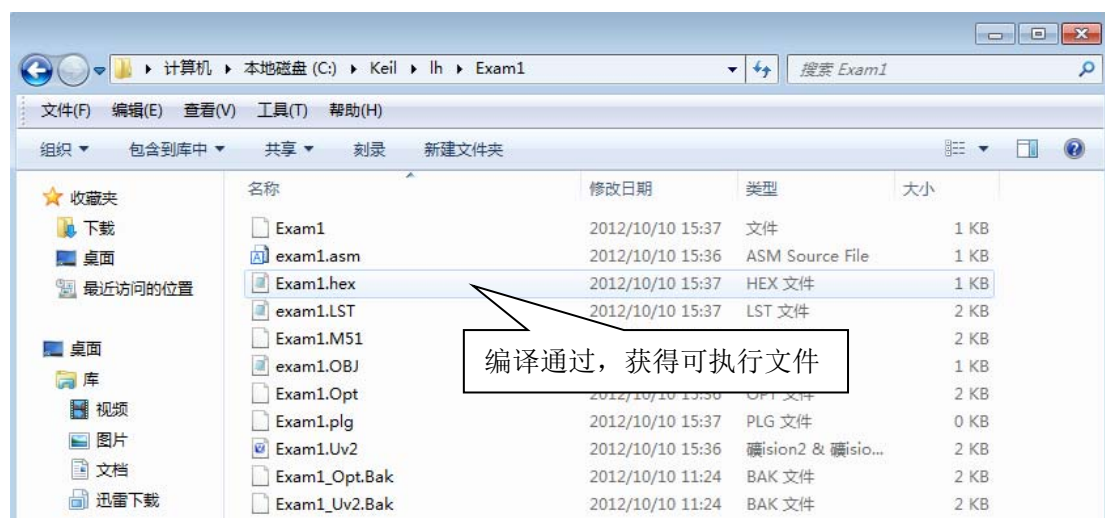


图 1.28 编译成功，获得目标文件 Exam1.hex

1.1.4 Keil uVision2 软件调试

点击“启动/停止调试 Start/Stop Debug Session”快捷按钮，如图 1.29 所示，即可对编译通过的可执行文件进行调试，这时出现图 1.30 所示调试窗口：

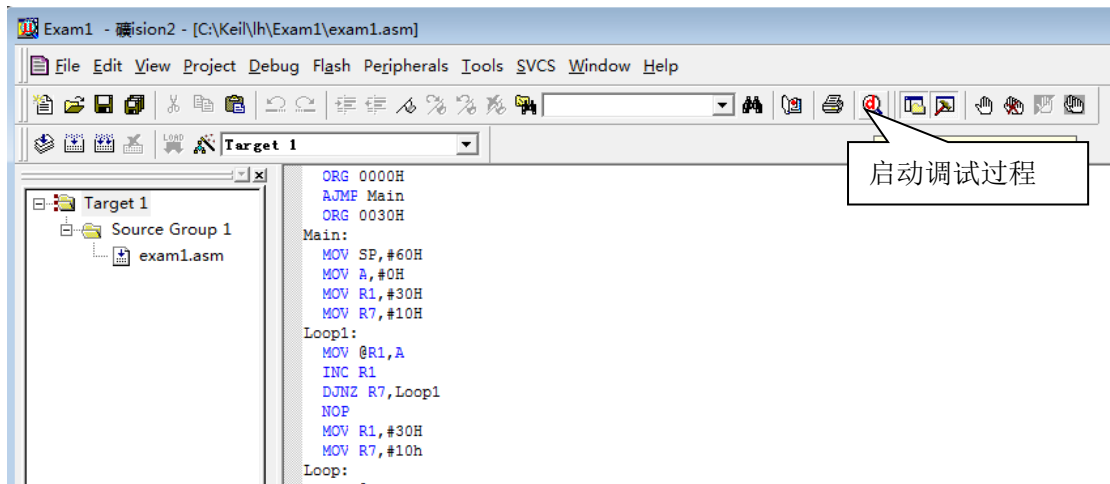


图 1.29 开始调试

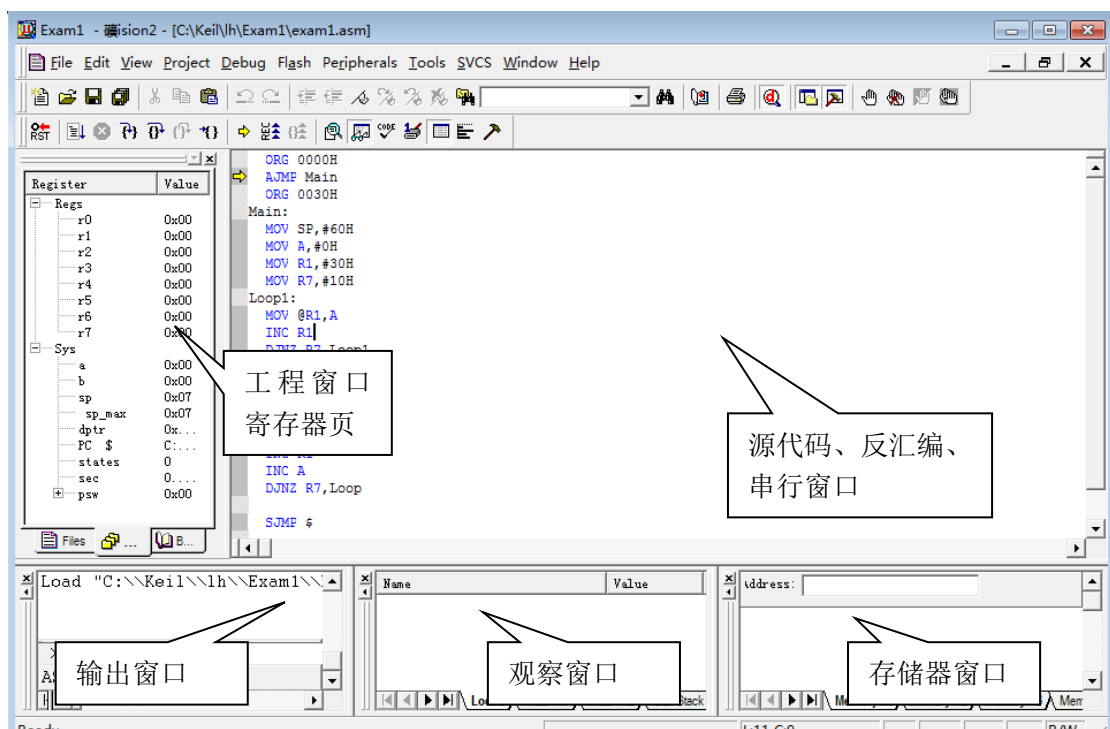


图 1.30 调试窗口

Keil 软件在调试程序时提供了多个窗口，主要包括工程窗口（Project Windows）寄存器（Regs）页、输出窗口（Output Windows）、观察窗口（Watch&Call Stack Windows）、存储器窗口（Memory Window）、反汇编窗口（Disassembly Window）和串行窗口（Serial Window）等，各窗口的大小可以使用鼠标调整。进入调试模式后，可以通过菜单 View 下的相应命令打开或关闭这些窗口，如图 1.31 所示，点击相应菜单便可切换该调试窗口是否显示。

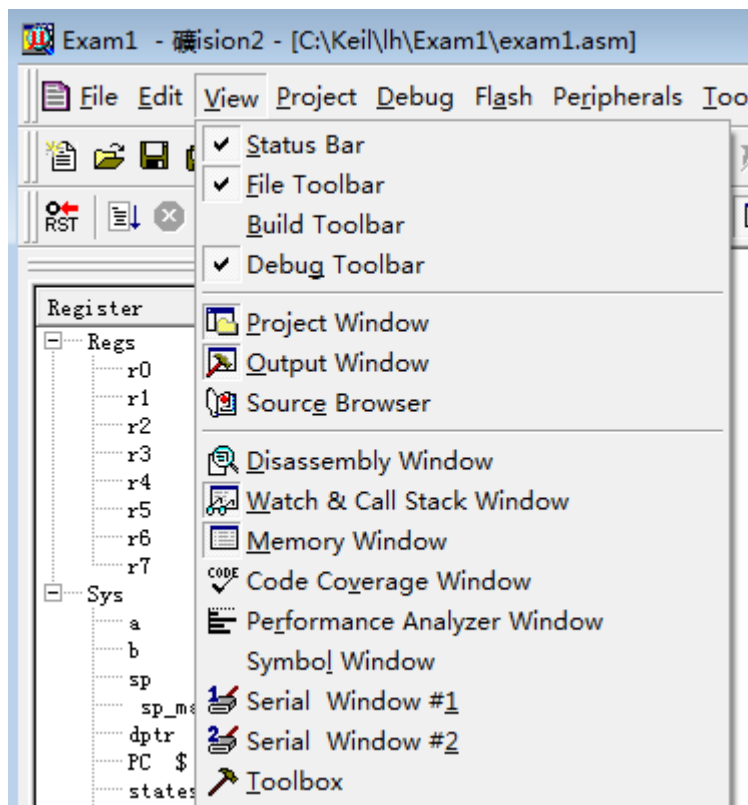


图 1.31 调试窗口显示管理

进入调试程序后，输出窗口自动切换到 Command 页。该页用于输入调试命令和输出调试信息。对于初学者，可以暂不学习调试命令的使用方法。

要想调试程序，应先使程序运行起来。使程序运行起来的方法有三种：全速运行（Run），单步进入（Step Into）和单步越过（Step Over），这些运行过程可以通过相应的工具栏按钮进行如下图 1.32 所示：

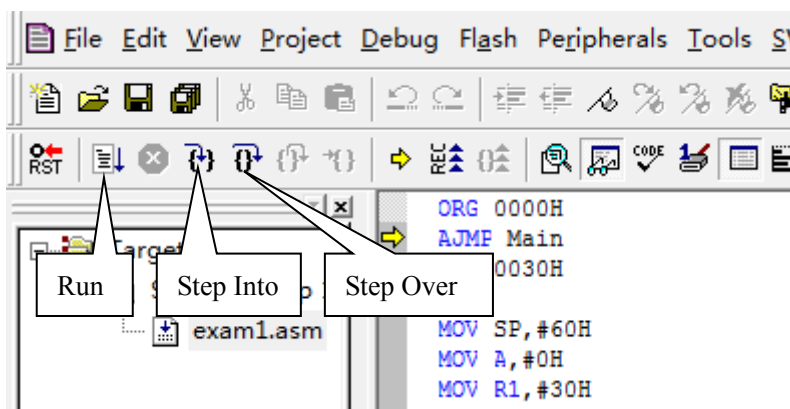


图 1.32 程序运行启动窗口

全速运行（Run）用于验证程序的整体功能，单步进入（Step Into）用于观察每条语句的执行效果（包括子程序内部），单步越过（Step Over）也用于观察每条语句的执行效果，但子程序调用作为一条语句来对待，不进入子程序内部。

刚进入调试窗口时，源程序窗口左侧的黄色箭头指向第一条语句，即程序复位后 PC 所指向的位置，每执行一条单步语句，黄色箭头随着 PC 值的改变而往下调整，并且执行

过的语句左侧都用深绿色标注，如图 1.33 所示：

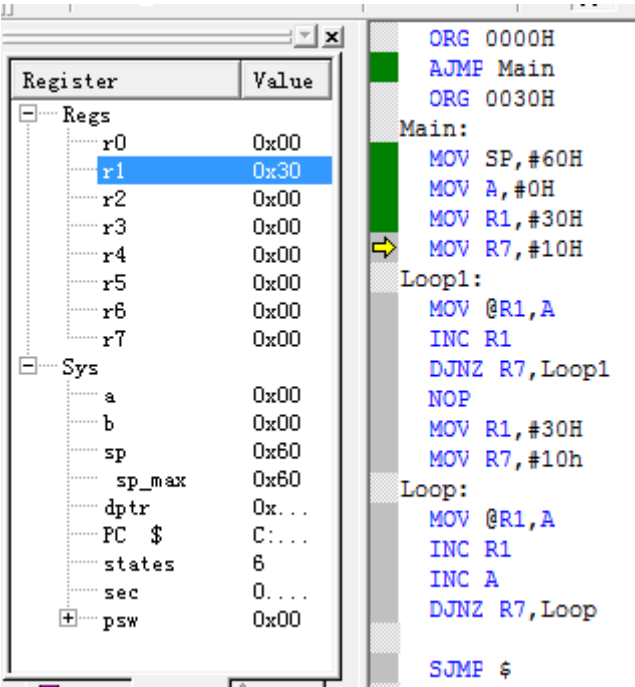


图 1.33 程序单步执行效果

(1)、存储器窗口

存储器窗口中可以显示系统中各种内存中的值，通过在地址（Address）后的编辑框内输入“字母：数字”即可显示相应内存值，其中字母可以是 C、D、I、X，分别代表代码存储空间、直接寻址的片内存储空间、间接寻址的片内存储空间、扩展的外部 RAM 空间，数字代表想要查看的地址。例如输入 D: 0 即可观察到地址 0 开始的片内 RAM 单元值、键入 C: 0 即可显示从 0 开始的 ROM 单元中的值，即查看程序的二进制代码。如图 1.34 所示：

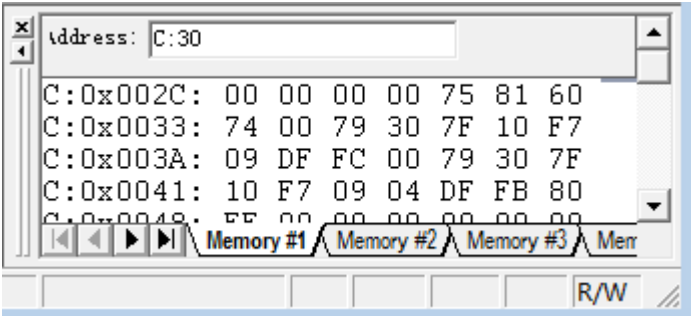


图 1.34 存储器窗口

该窗口的显示值可以以各种形式显示，如十进制、十六进制（默认）、字符型等，改变显示方式的方法是点鼠标右键，在弹出的快捷菜单中选择，如图 1.35 所示：

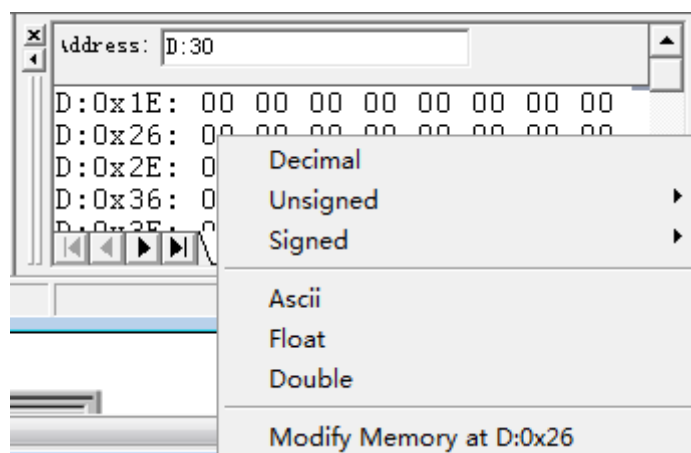


图 1.35 存储器窗口显示格式调整

另外，把鼠标停留在某个存储单元地址，右键点击鼠标时还可以修改该存储单元的内容，如显示的是数据存储单元，将鼠标停留在 26H 单元，右键点击鼠标，出现图 36 所示菜单，选择其中的“Modify Memroy at D:0x26”，即可进入图 1.36 所示的修改窗口，

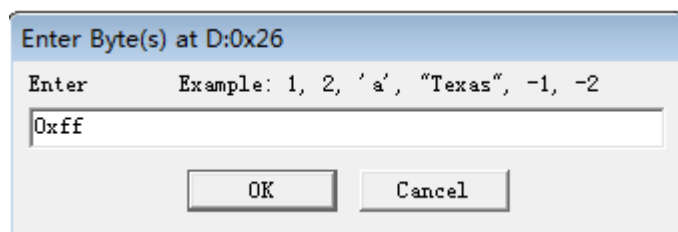


图 1.36 存储器单元值修改对话框

填入相应的修改值（如 0xff），点击“OK”按钮，可以看到 0x26 单元的值已经修改成你所写入的值，如下图 1.37 所示：



图 1.37 存储器单元值已修改

(2)、工程窗口寄存器页

工程窗口寄存器页包括了当前的工作寄存器组和系统寄存器，如图 1.38 所示，系统寄存器组有一些是实际存在的寄存器如 A、B、DPTR、SP、PSW 等，有一些是实际中并不存在或虽然存在却不能对其操作的如 PC、Status 等。

Register	Value
☐ Regs	
r0	0x00
r1	0x00
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x00
☐ Sys	
a	0x00
b	0x00
sp	0x07
sp_max	0x07
dptr	0x...
PC \$	C:...
states	0
sec	0...
☒ psw	0x00

图 1.38 工程窗口寄存器页

每当程序中执行到对某寄存器的操作时，该寄存器会以反色（蓝底白字）显示，用鼠标单击然后按下 F2 键，即可修改该值。如图 1.39 所示。

Register	Value
☐ Regs	
r0	0x00
r1	0x30
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x00
☐ Sys	
a	0x00
b	0x00
sp	0x60
sp_max	0x60
dptr	0x...
PC \$	C:...
states	6
sec	0...
☒ psw	0x00

图 1.39 寄存器修改值

(3)、观察窗口

观察窗口是很重要的一个窗口，工程窗口中仅可以观察到工作寄存器和系统的寄存器如 A、B、DPTR 等，如果需要观察其它的寄存器的值或者在高级语言编程时需要直接观察变量，就要借助于观察窗口了。一般情况下，我们仅在单步执行时才对变量的值的变化感兴趣，全速运行时，变量的值是不变的，只有在程序停下来之后，才会将这些值最新的变化反映出来，

但是，在一些特殊场合下我们也可能需要在全速运行时观察变量的变化，此时可以点击 View->Periodic Window Updata（周期更新窗口），确认该项处于被选中状态，即可在全速运行时动态地观察有关值的变化。但是，选中该项，将会使程序模拟执行的速度变慢。

为了观察某个变量的值，可以在观察窗口中“Name”栏输入要观察的变量名，回车就会看到该变量的值，也可以将光标停留在要观察的变量上（如 R7），然后右键点击鼠标，在弹出对话框中选择“Add ** to Watch Window”，如图 1.40 所示：

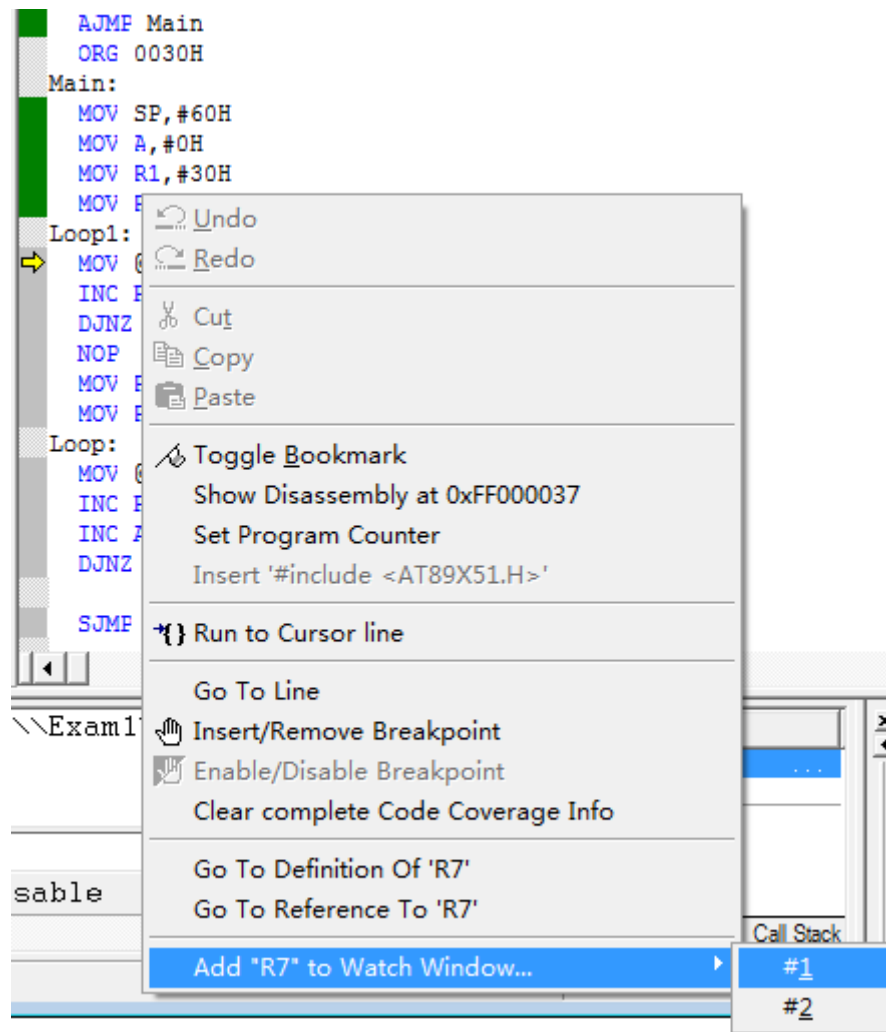


图 1.40 添加观察变量

添加变量后观察窗口将会显示该变量的当前值，如下图 1.41 所示：

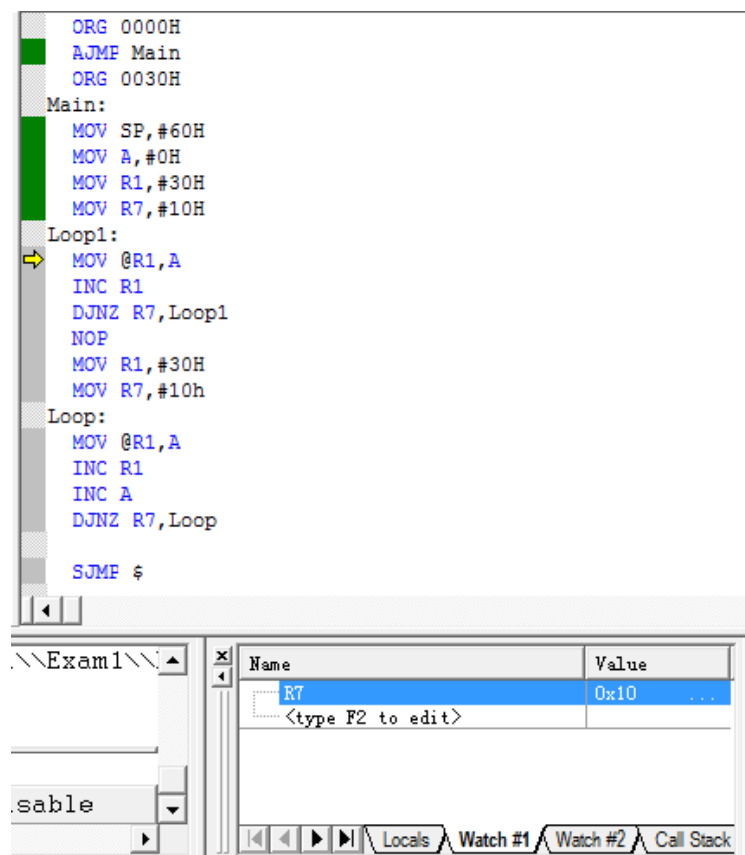


图 1.41 观察变量

如若调试过程中想人工修改所观察变量的值，可以选中该变量，并将鼠标对准该变量的“Value”栏，然后按快捷键“F2”即可。

如果要删除某个观察变量只要在观察窗口选中该变量，然后按“Delete”按键即可。

1.2 系统仿真软件 Proteus 应用指导

Proteus 是英国 Labcenter 公司开发的组合了高级原理设计、混合模式 SPICE 仿真、PCB 系统设计以及自动布线等功能的一个完整的电子系统设计软件。该软件主要有两个应用系统设计构成：ISIS 和 ARES。其中 ISIS 是原理图仿真软件，可以仿真、分析 (SPICE) 各种模拟、数字以及单片机电路系统。ARES 是 PCB 设计软件，该软件具有 32 位数据库、能够实现元件自动布置、自动布线等功能。

这里我们主要介绍 ISIS 原理图仿真软件。该软件的特点是：

- ① 实现了单片机仿真和 SPICE 电路仿真相结合。具有模拟电路仿真、数字电路仿真、单片机及其外围电路组成的系统的仿真、RS232 动态仿真、I2C 调试器、SPI 调试器、键盘和 LCD 系统仿真的功能；
- ② 支持主流单片机系统的仿真。目前支持的单片机类型有：68000 系列、8051 系列、AVR 系列、PIC 系列、以及部分 ARM 系列以及各种常用外围芯片如：存储器、AD/DA、LCD、LED 数码管、步进及直流电机、温度传感器、时钟芯片等。
- ③ 有各种虚拟仪器，如示波器、逻辑分析仪、信号发生器等。

- ④ 提供软件调试功能。支持第三方的软件编译和调试环境，如 Keil C51 uVision 等软件。

利用 ISIS 仿真软件进行单片机应用系统设计，可以实现单片机应用系统的软硬件协同设计，进行硬件设计之前的原理性验证，提高硬件系统设计的成功率，是现代电子系统设计工程师应该掌握的一门设计工具软件和基本技能。

本章主要是以 Proteus7.9 为平台，通过实例讲解 Proteus ISIS 单片机仿真以及与 Keil C51 进行软件联合调试的方法。

1.2.1 Proteus 软件的启动

双击桌面上的 ISIS 7 Professional 图标  或者单击屏幕左下方的“开始”→“程序”→“Proteus 7 Professional” → “ISIS 7 Professional”，出现如图 1.42 所示，开始启动 Proteus。



图 1.42 Proteus 启动时的屏幕

1.2.2 Proteus 应用及实例

Proteus ISIS 的工作界面如图 1.43 所示。包括：标题栏、主菜单、标准工具栏、绘图工具栏、状态栏、对象选择按钮、仿真进程控制按钮、预览窗口、对象选择器窗口、图形编辑窗口。

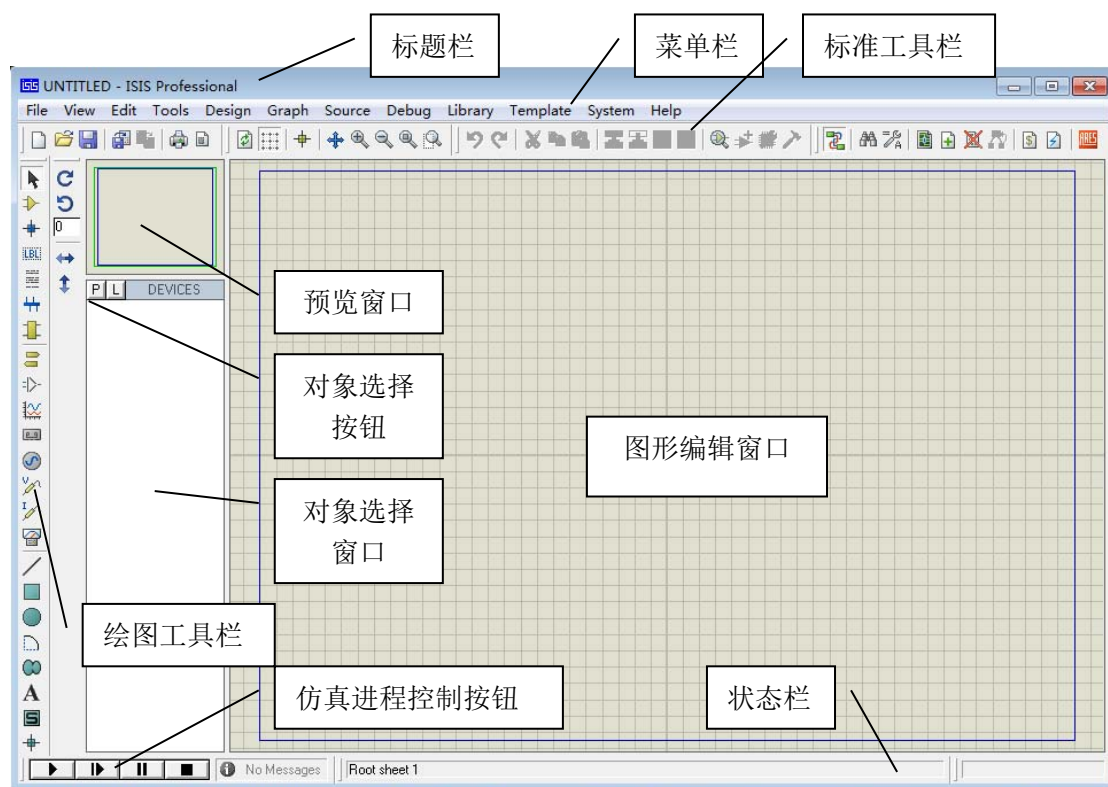


图 1.43 Proteus ISIS 的工作界面

以下分别列出主窗口和四个输出窗口的全部菜单项。对于主窗口，在菜单项旁边同时列出工具条中对应的快捷鼠标按钮。

(1) 图形编辑窗口

在图形编辑窗口内完成电路原理图的编辑和绘制。

◆坐标系统 (COORDINATE SYSTEM)

为了方便作图，图形编辑窗口设置了坐标系统 (COORDINATE SYSTEM)，ISIS 中坐标系统的基本单位是 10nm，主要是为了和 Proteus ARES 保持一致。但坐标系统的识别 (read-out) 单位被限制在 1th ($1th = 1/1000inch = 0.0254mm$)。坐标原点默认在图形编辑区的中间，图形的坐标值能够显示在屏幕的右下角的状态栏中。编辑窗口内有点状的栅格，可以通过 View 菜单的 Grid 命令在打开和关闭间切换。点与点之间的间距由当前捕捉的设置决定。捕捉的尺度可以由 View 菜单的 Snap 命令设置，或者直接使用快捷键 F4、F3、F2 和 CTRL+F1。如图 1.44 所示。如若键入 F3 或者通过 View 菜单的选中 Snap 100th，你会注意到鼠标在图形编辑窗口内移动时，坐标值是以固定的步长 100th 变化，这称为捕捉。如果你想要确切地看到捕捉位置，可以使用 View 菜单的 X-Cursor 命令，选中后将会在捕捉点显示一个小的或大的交叉十字。

View		
	Redraw	R
	Grid	G
	Origin	O
	X-Cursor	X
	Snap 10th	Ctrl+F1
	Snap 50th	F2
	Snap 100th	F3
	Snap 500th	F4
	Pan	F5
	Zoom In	F6
	Zoom Out	F7
	Zoom All	F8
	Zoom to Area	
Toolbars...		

图 1.44 栅格设置

◆实时捕捉 (Real Time Snap)

当鼠标指针指向管脚末端或者导线时，鼠标指针将会被捕捉到这些物体，这种功能被称为实时捕捉，该功能可以使你方便的实现导线和管脚的连接。

◆视图的缩放与移动

视图的缩放可以通过如下几种方式：

- 用鼠标左键点击预览窗口中想要显示的位置，这将使编辑窗口显示以鼠标点击处为中心的内容。也可以用鼠标移动预览窗口中的绿色小框，使其中央部分对准你要显示的内容，然后点击鼠标左键即可。预览窗口中的蓝色框表示了你的图纸边框。
- 在编辑窗口内移动鼠标，按下 SHIFT 键，用鼠标向上下或左右边框移动并碰到边框上，这会使显示向相应方向平移。我们把这称为 Shift-Pan。
- 用鼠标指向编辑窗口并按缩放键或者操作鼠标的滚动键，会以鼠标指针位置为中心重新显示。
- 使用一些快捷键，如：完全显示（或者按“F8”）、放大按钮（或者按“F6”）和缩小按钮（或者按“F7”），拖放、取景、找中心（或者按“F5”）。

（2）元器件放置及基本编辑

通过对象选择按钮，从元件库中选择对象，并置入对象选择器窗口实现器件放置供

（a）器件选择

放置器件的步骤如下

- 单击绘图工具栏中的“”按钮，再单击对象选择按钮“P”（位于预览窗口下方）。弹出图 1.45 所示器件选取窗口，

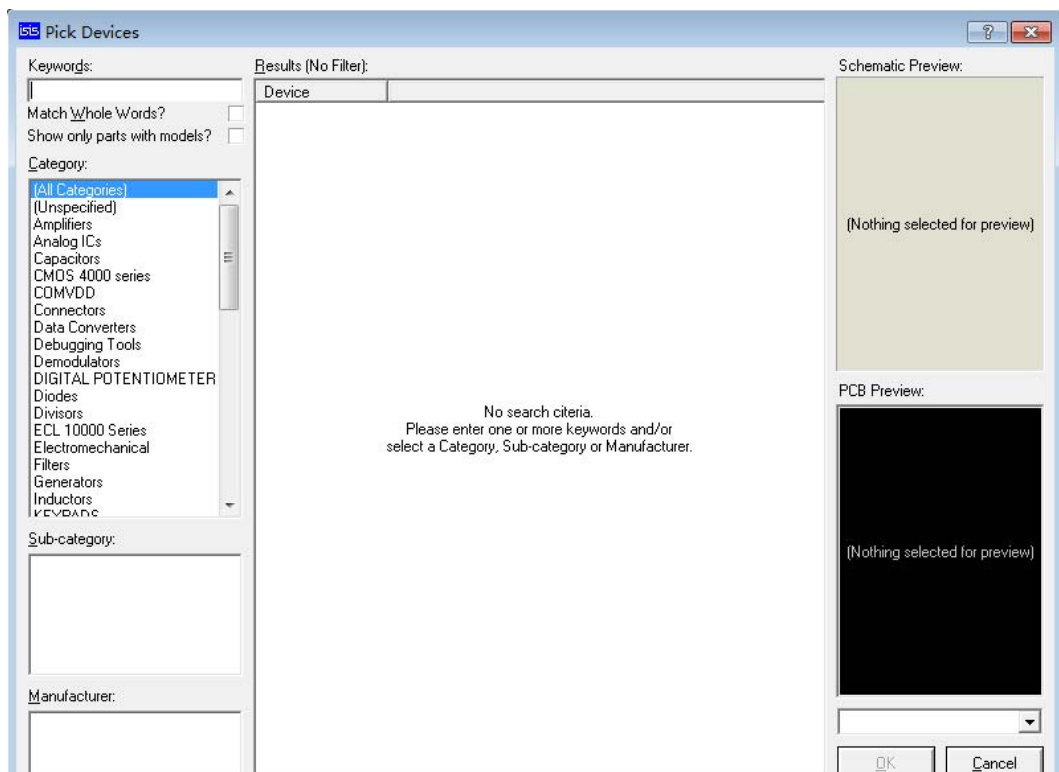


图 1.45 器件选取窗口

- 从器件类型（Category）选择你要选择的器件类型。如“TTL 74LS Series”。
- 根据对象的具体类型选择子模式图标（sub-mode icon）。如“Decoders”。
- 必要时从生产厂家（Manufacturer）栏中做出选择。
- 从结果（Results）栏选择你想要放置的器件。如“74LS138”，选中后右侧将显示该器件的原理图及 PCB 封装预览。如下图 1.46 所示：

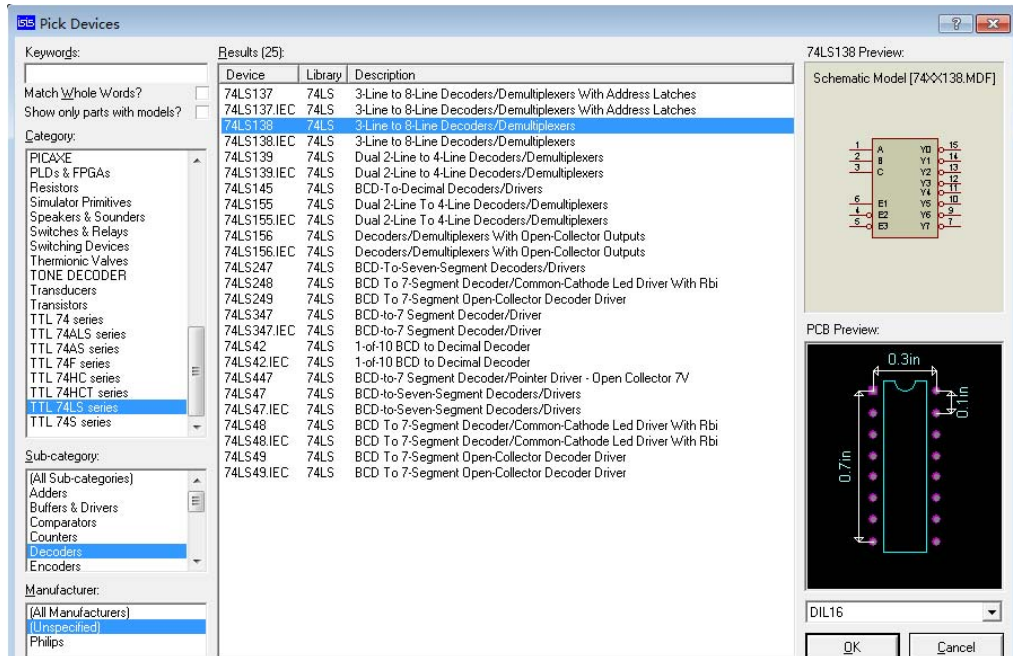


图 1.46 器件选取

- 选定器件后，双击该器件所在栏，所选器件将被加入对象选择窗口。然后还可以继续选择其它器件，将其加入对象选择窗口。待所有器件都选择完毕后，点击器件选择窗口右下侧的“OK”按钮完成器件选择过程。回到主窗口。
- 在主窗口下，从对象选择窗口中选择要加入主编辑窗口的器件，左键点击，预览窗口将出现该器件的原理图预览，如下图 1.47 所示：

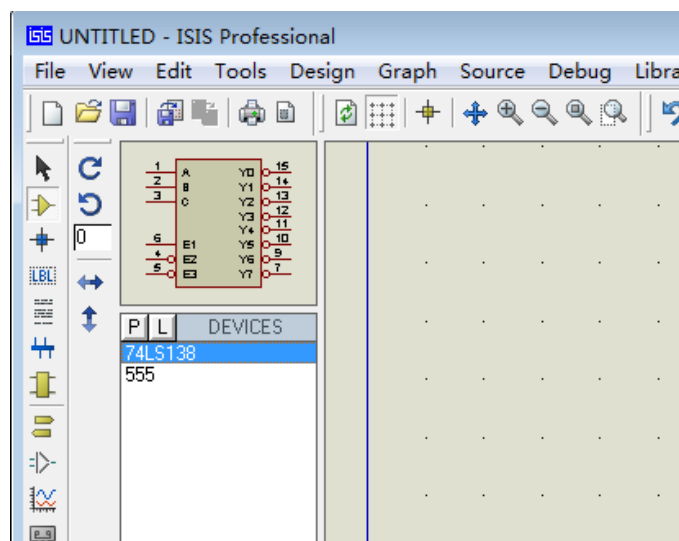


图 1.47 器件选择

- 将鼠标箭头移动到原理图编辑窗口，点击鼠标左键，该器件浮现在原理图编辑窗口如图 1.48 所示，

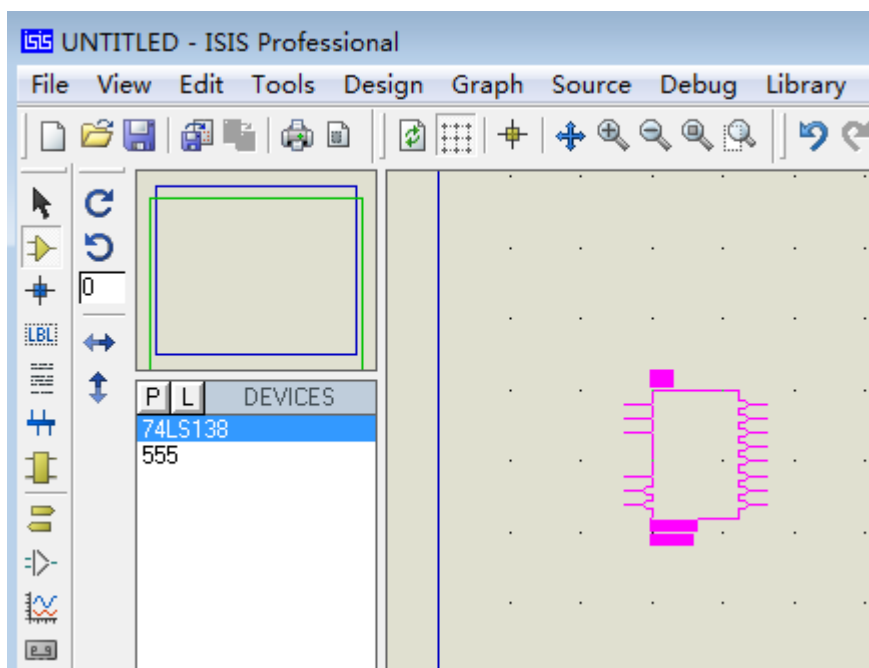


图 1.48 器件浮现

- 将其移动到合适的位置，再次点击鼠标左键，该器件将被放置在原理图中你想放置的位置，如下图 1.49 所示：

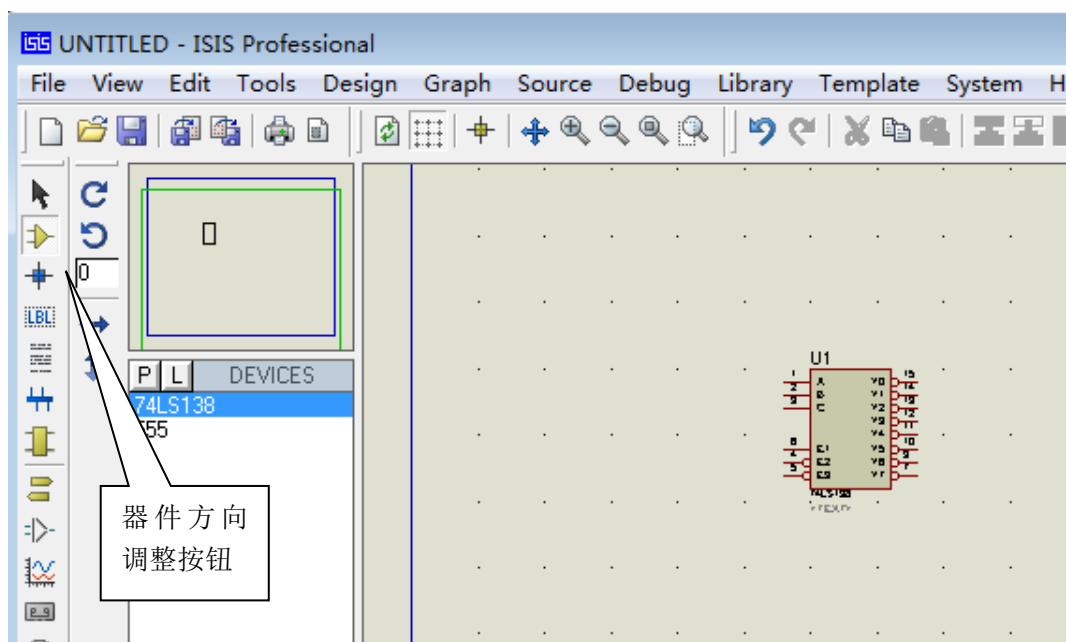


图 1.49 器件放置

(b) 器件选中

器件选中的方法可以用鼠标左键点击需要选中的器件，该器件将变成红色，表示该器件已被选中，你也可以按住“Ctrl”键同时选中多个器件。也可以用在需要选中的器件旁边按下鼠标左键，拖拽形成一个虚线框，使该虚线框覆盖你要选择的器件，该器件也将变红，表

示已被选中，如下图 1.50 所示：

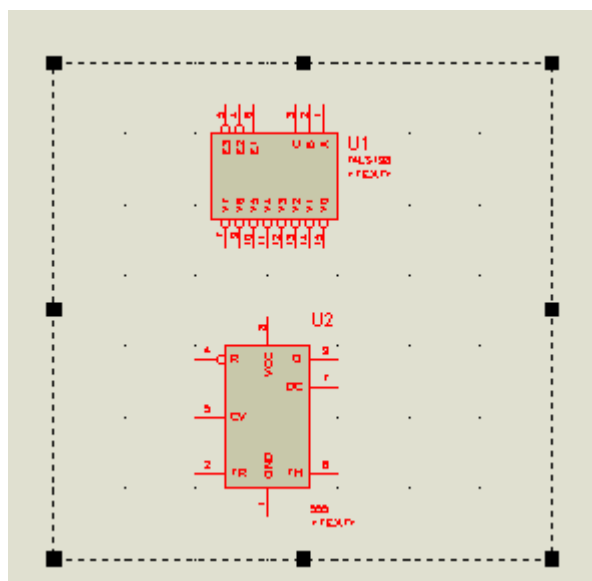


图 1.50 器件选中

对于已有连线的器件，选中该器件后所有与其相连的连线也将同时被选中。

在空白处点击鼠标左键可以取消所有对象的选择。

(c) 器件移动

对于选中的器件，你把鼠标箭头放置其上，按下鼠标左键，对器件进行拖拽，将拖拽出它们的粉红色影子，这表示你正移动器件的位置，移动到指定位置之后，松开鼠标左键，器件将被移动到你指定的位置。

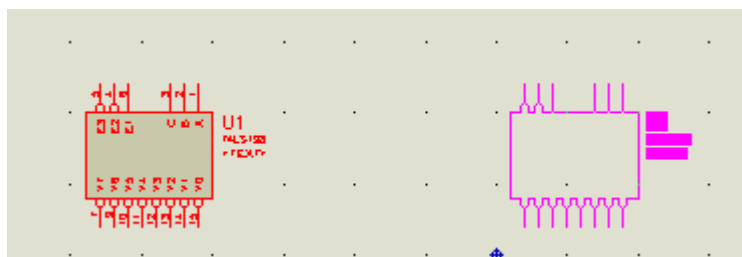


图 1.51 器件移动过程

对于已连线的器件，移动时将影响所连接线。

也可以单独移动器件的各种属性标签。如器件的序号及其描述。这时你需要先选中该器件，然后将鼠标移动到需要移动的器件属性标签上，再点击鼠标左键将其移动。

(d) 器件方向调整

- 器件方向的调整，可以利用器件方向调整按钮（见图 1.49 所示）将顺时针旋转 90°

（），逆时针旋转 90°（），左右镜像（）或上下镜像（）等方式进行

调整。调整后的器件将会在预览窗口显示出来。这种调整将对以后将该器件加入原理图中的器件产生统一影响。如果你不想这样，也可以在器件放入编辑窗口后，选中该器件，右键点击鼠标，弹出器件管理菜单，从中选择“Rotate Clockwise”，“Rotate Anti-Clockwise”，“Rotate 180 degrees”，“X-Mirror”，“Y-Mirror”，

如下图 1.52 所示，进行方向调整：

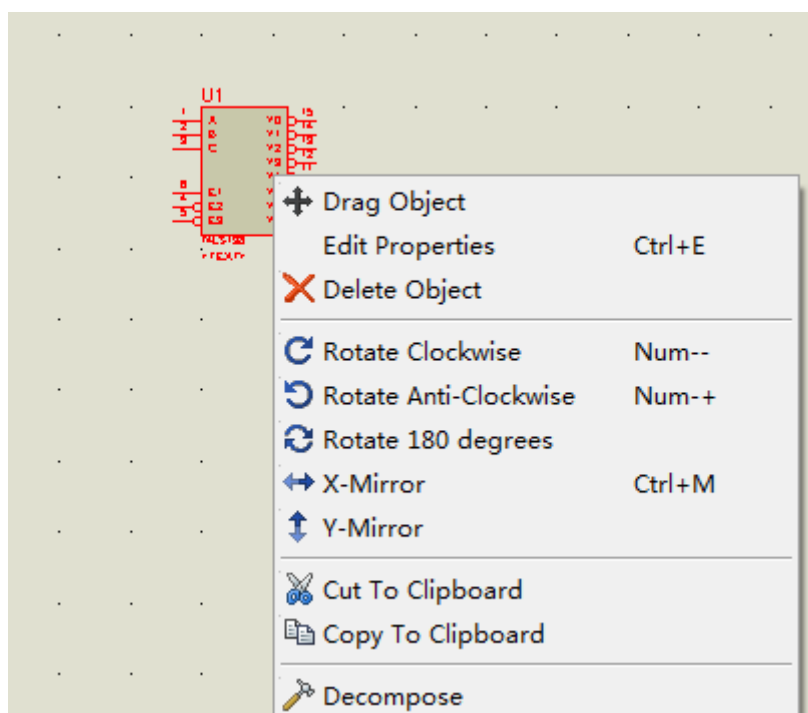


图 1.52 器件方向调整

(e) 器件的删除

为删除器件，你要先用鼠标指向选中的对象，将其选中，然后点击右键，弹出器件管理菜单，选择“Delete Object”菜单（如图 1.52 所示）将其删除，也可以在这时按键盘上的“Delete”键将其删除。删除对象的同时也将删除该对象的所有连线。

(f) 器件的拷贝、剪切与粘贴

为拷贝或剪切器件，你要先用鼠标指向选中的对象，将其选中，然后点击右键，弹出器件管理菜单，选择“Copy To Clipboard”或“Cut To Clipboard”菜单（如图 1.52 所示），也可以在这时按工具栏上的“”或“”按钮对其进行拷贝或剪切。剪切对象的同时也将删除该对象的所有连线。

被拷贝或者剪切到粘贴板（Clipboard）的器件还可以被粘贴回器件编辑窗口，粘贴的方法是点击工具栏上的“”按钮，或者在器件编辑窗口的空白位置，单击鼠标右键，弹出其菜单，并选择“Paste From Clipboard”，选择如下图 1.53 所示：

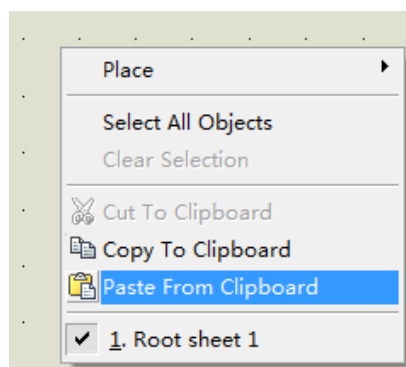


图 1.53 器件粘贴

(g) 器件的编辑

编辑单个器件的步骤是：首先选中所要编辑的对象，用鼠标左键点击选中的要编辑的对象（以上两步也可以一步实现，即将鼠标移动到需要编辑的器件上方，双击需要编辑的器件，或者按下 Ctrl+E 按键），这是会弹出器件编辑对话框，如下图 1.54 所示，在此可以修改

器件编号 (Component Reference)，器件参数值 (如电阻、电容值、封装形式等)，

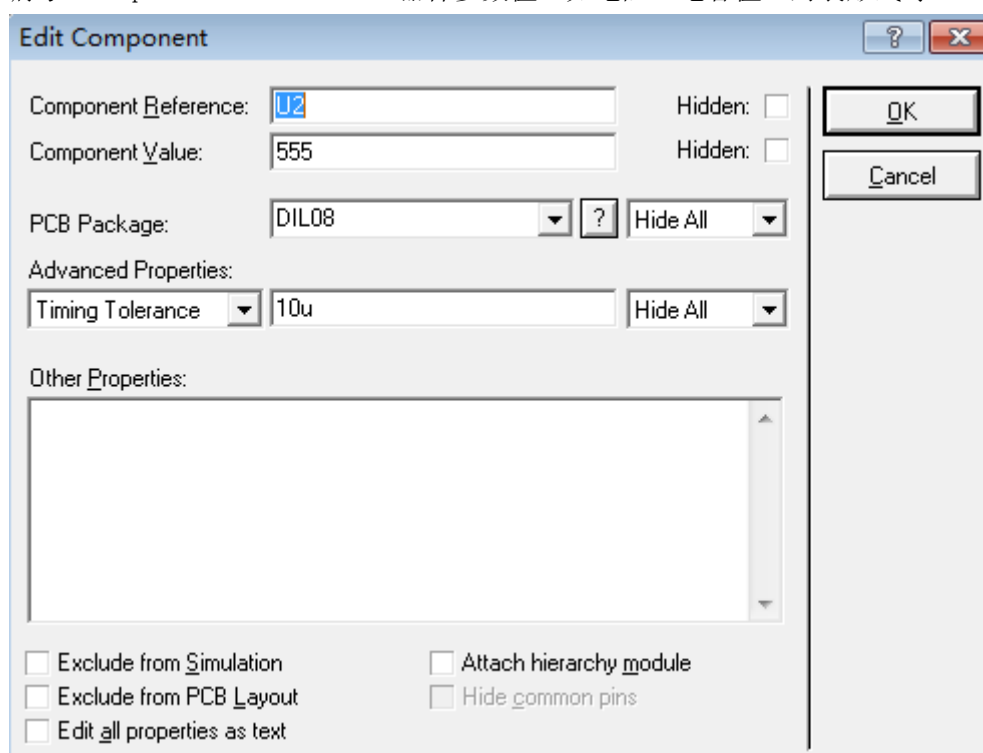


图 1.54 器件编辑窗口

(h) 器件整体拷贝、移动和删除

在选择多个器件后，你可以完成对它们整体的拷贝、移动和删除，使用工具栏中的快捷按钮“Block Copy ”，“Block Move ”和“Block Delete ”。这里的 Block Copy 同时完成了“Copy to Clipboard ”和“Paste From Clipboard ”两步的功能。

(i) 操作的取消与恢复

以上 (c) - (h) 的内容如果操作有误，可以用“Edit” → “Undo”或者工具栏中的快捷按钮“”来实现操作的取消。取消之后还可以用“Edit” → “Redo”或者工具栏中的快捷按钮“”来实现将取消的动作恢复操作

(3) 元器件放置示例

按前述方法放置如下元件：AT89C51，7SEG-COM-AN-GRN，CAP, CAP-ELEC, CRYSTAL, RES, BUTTON。这些元件的库类及子类名如下：

元件	库类名	子类名	生产厂商
AT89C51 单片机	Microprocessor ICs 处理器类	8051 Family 8051 家族	ATMEL
7SEG-COM-AN-GRN 绿色共阳极 7 段数码管	Optoelectronics 光电类	7-Segment Displays 7 段显示	
RES 一般电阻	Resistors 电阻类	Generic 通用	
CAP 非电解电容	Capacitors 电容器类	Generic 通用	
CAP-ELEC 电解电容	Capacitors 电容器类	Generic 通用	
CRYSTAL 晶振	Miscellaneous 杂类		
RX8 双输入*8 排阻	Resistors 电阻类	Resistor Pack 排阻	
BUTTON 按键	Switches&Relays 开关继电器类	Switches 开关	

对于已知元器件详细名称的元器件，也可以用在元器件选择对话框中的“Keywords”栏输入相应元器件名称方法，实现元器件的自动查找，如下图 1.55 所示，

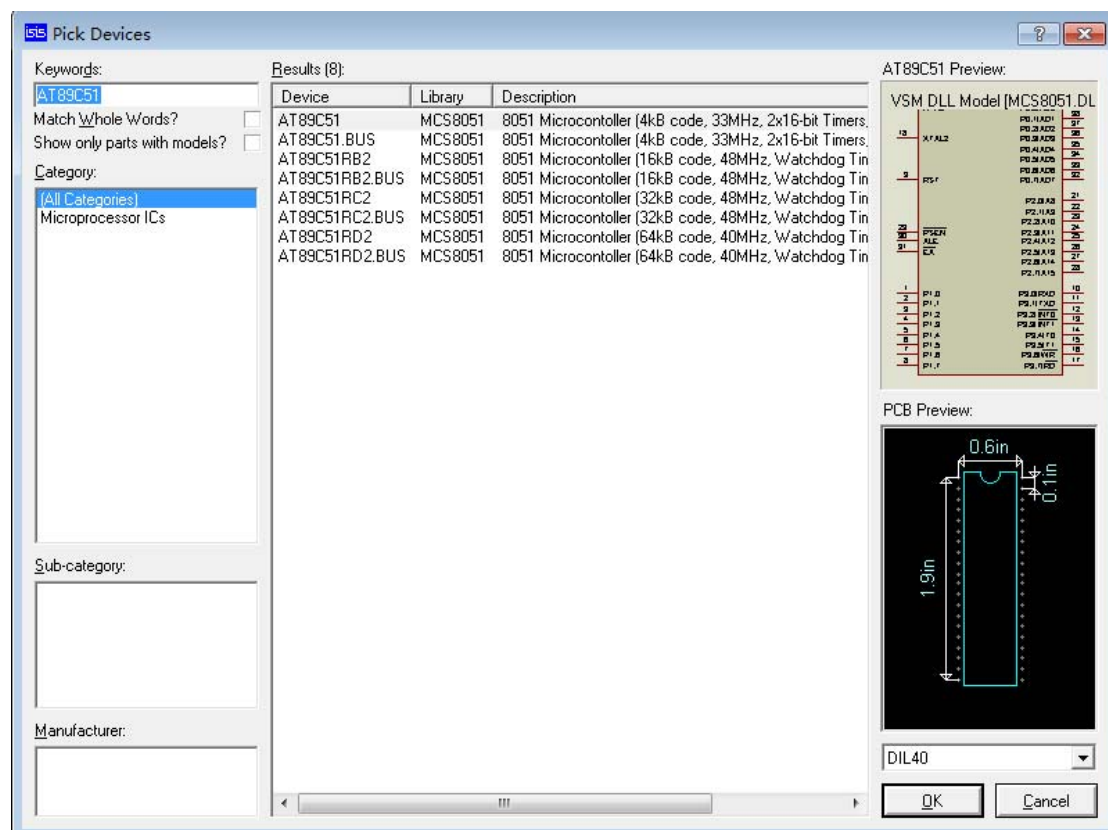


图 1.55 元器件自动查找

这些元器件加入后进行简单布局，并调整其参数，如下图 1.56 所示：

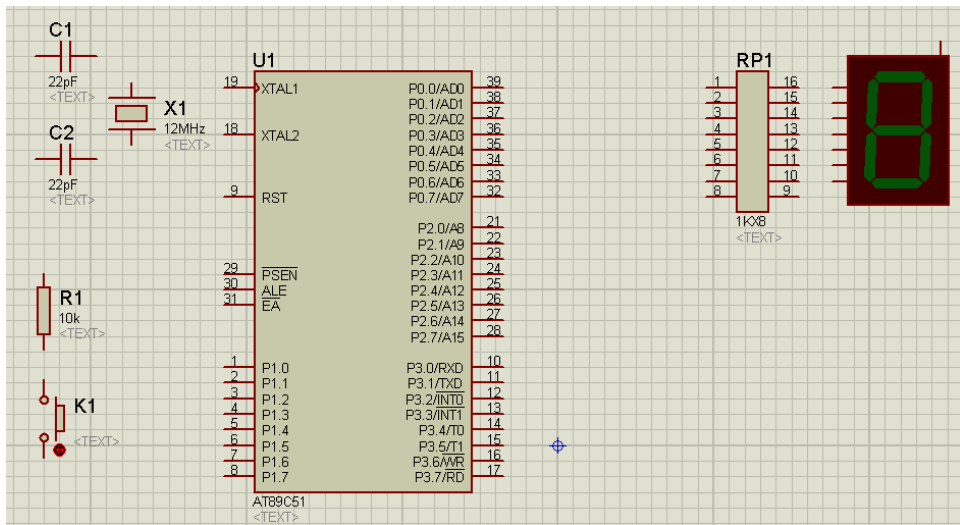


图 1.56 元器件放置示例

继续放置电源和地。单击绘图工具栏中的终端模式按钮“”，从对象选择窗口中，选择“POWER”和“GROUND”，将它们加入图 1.56 中，将出现如下图 1.57 所示窗口，

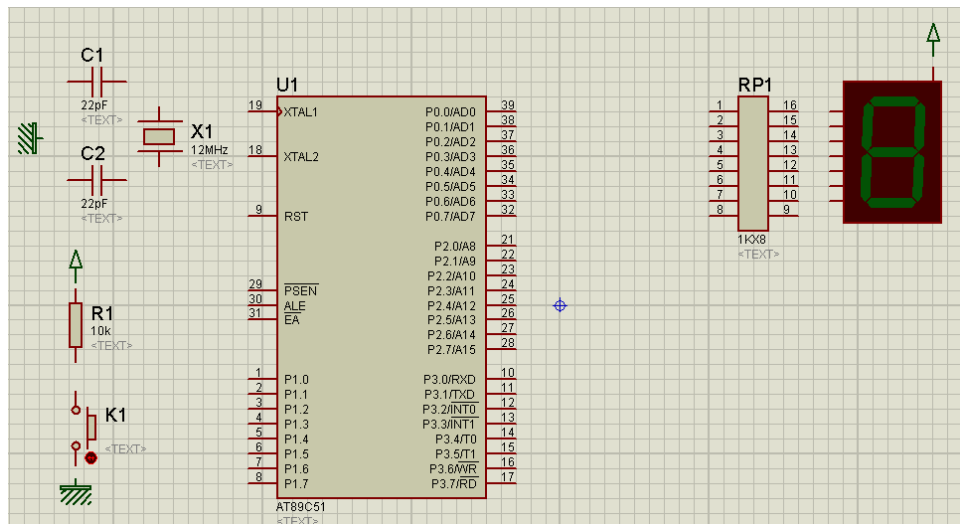


图 1.57 放置电源和地

(4) 元器件连线

元器件之间的连线，分为普通导线连接，总线连接和总线分支连接。

◆导线连接

Proteus 的智能化可以在你想要画线的时候进行自动检测。当鼠标指针靠近一个对象的连接点时，鼠标的指针就会变成一个“×”号，左键点击元器件的连接点，移动鼠标(不用一直按着左键)就出现了粉红色的连接线变成了深绿色。如果你想让软件自动定出线路径，只需左击另一个连接点即可。这就是 Proteus 的线路自动路径功能(Wire Auto-Router 简称 WAR)。如果你只是在两个连接点用鼠标左击，WAR 将选择一个合适的线径。WAR 可通过使用工具栏里的“WAR”命令按钮“”来打开或关闭，也可以在菜单栏的“Tools”下找到这

个图标。如果你想自己决定走线路径，只需在想要拐点处点击鼠标左键即可。在此过程的任何时刻，你都可以按 ESC 或者点击鼠标的右键来放弃画线。

两条导线交叉时，如果没有人工放置电气连接点，则认为两条导线是不相连的。如果需
要将两条交叉导线进行电气连接需要手工进行。方法是点击工具箱的节点放置按钮“”，
当把鼠标指针移到编辑窗口，指向一条导线或者两条导线的交叉点时，会出现一个“×”号，
点击左键就能放置一个电气连接点。

◆画总线

为了简化原理图，我们可以用一条导线代表数条并行的导线，这就是所谓的总线。方法是单击放置工具条中图标“”或执行 Place / Bus 菜单命令，这时工作平面上将出现十字形光标，将十字光标移至要连接的总线分支处单击鼠标左键，系统弹出十字形光标并拖着一条较粗的线，然后将十字光标移至另一个总线分支处，单击鼠标的左键，一条总线就画好了。

◆画总线分支线

总线分支线用来连接总线和元器件管脚，画线的方式类似于前面的导线连接。为了和一般的导线区分，我们一般喜欢画斜线来表示分支线，但是在自动走线（即 WAR 打开）模式下，线都是横竖方向走的，如果你想自己画斜线，只要走线时按下“Ctrl”键，或者临时关闭 WAR 模式即可。

◆添加网络标号

画好分支线我们还需要给分支线起个名字（即增加网络标号 Label），以便将它们进行区别。

增加网络标号的方法有两种。一种是点击画图工具栏中的“”按钮或者执行 Place / Net Label 菜单命令，将鼠标移动到需要增加网络标号的导线上，这时鼠标将呈现“×”号，该导线也有一红色影子出现，这说明在鼠标当前位置增加的网络标号是为该导线添加的。这时，点击鼠标左键，将出现如下图 1.58 所示对话框，

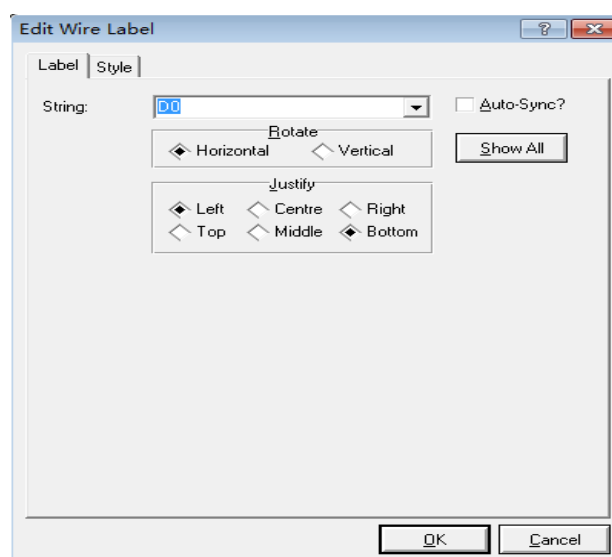


图 1.58 添加网络标号属性

在该对话框的“String”栏写上该网络标号的名字，然后点击“OK”即可。

另一种添加网络标号的方法是用右键点击分支线选中它，这时就会出现如下图 1.59 所示选择菜单，选择其中的“Place Wire Label”菜单项，然后就会出现图 1.58 所示的添加网络标号的属性对话框窗口。

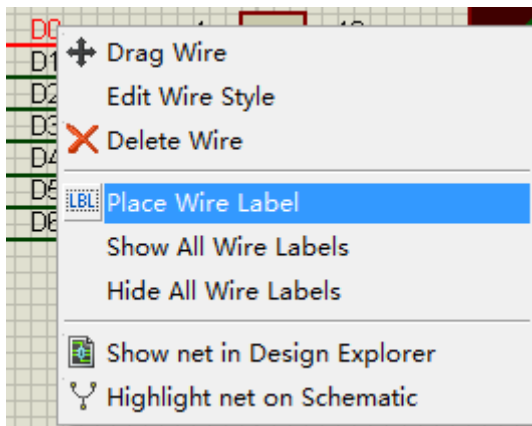


图 1.59 添加网络标号选择菜单

按照前述方法对图 1.57 中所有元件进行连线，并增加网络标号。结果如下图 1.60 所示。

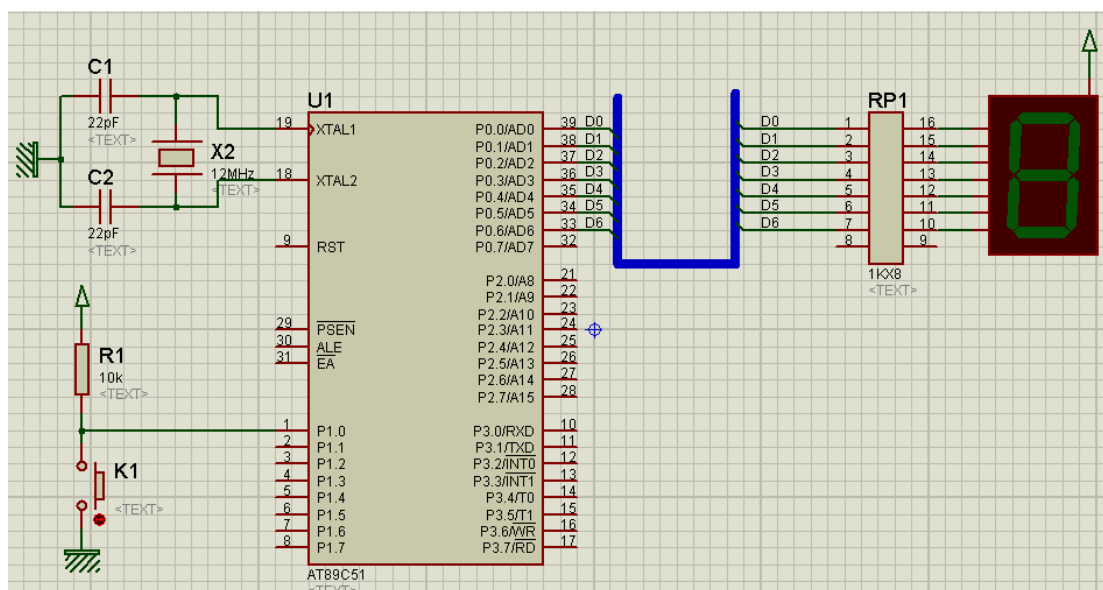


图 1.60 连线结果

(5) 进行仿真调试

◆ 建立源程序

用文本编辑器，如 Proteus 自带的 SRCEDIT.EXE，其位置为“..\Proteus 7 Professional\BIN\SRCEdit.EXE”，这里的 .. 表示 Proteus 的安装目录，也可以使用 Windows 自带的文本编辑器记事本“notepad.exe”，在文本编辑器中输入如下程序：

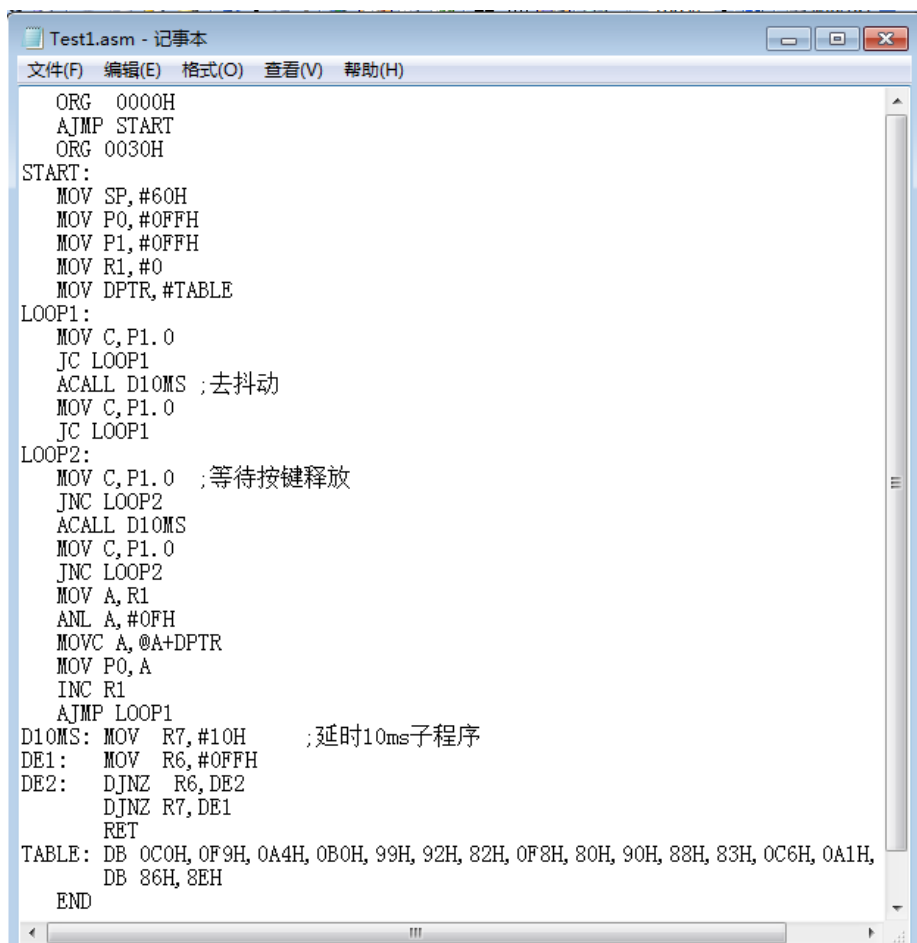
```
ORG 0000H
AJMP START
ORG 0030H
```

```

START:
    MOV SP, #60H
    MOV P0, #0FFH
    MOV P1, #0FFH
    MOV R1, #0
    MOV DPTR, #TABLE
LOOP1:
    MOV C, P1.0
    JC LOOP1
    ACALL D10MS ;去抖动
    MOV C, P1.0
    JC LOOP1
LOOP2:
    MOV C, P1.0 ;等待按键释放
    JNC LOOP2
    ACALL D10MS
    MOV C, P1.0
    JNC LOOP2
    MOV A, R1
    ANL A, #0FH
    MOVC A, @A+DPTR
    MOV P0, A
    INC R1
    AJMP LOOP1
D10MS: MOV R7, #10H ;延时 10ms 子程序
DE1:   MOV R6, #0FFH
DE2:   DJNZ R6, DE2
        DJNZ R7, DE1
        RET
TABLE: DB 0C0H, 0F9H, 0A4H, 0B0H, 99H, 92H, 82H, 0F8H, 80H, 90H, 88H, 83H, 0C6H, 0A1H,
        DB 86H, 8EH
        END

```

并将其保存为汇编程序文件，如“Test1.asm”，如下图 1.61 所示，假设其保存位置为：
“..\Proteus 7 Professional\Test1.asm”



```
Test1.asm - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

ORG 0000H
AJMP START
ORG 0030H
START:
MOV SP, #60H
MOV P0, #0FFH
MOV P1, #0FFH
MOV R1, #0
MOV DPTR, #TABLE
LOOP1:
MOV C, P1.0
JC LOOP1
ACALL D10MS ;去抖动
MOV C, P1.0
JC LOOP1
LOOP2:
MOV C, P1.0 ;等待按键释放
JNC LOOP2
ACALL D10MS
MOV C, P1.0
JNC LOOP2
MOV A, R1
ANL A, #0FH
MOVC A, @A+DPTR
MOV P0, A
INC R1
AJMP LOOP1
D10MS: MOV R7, #10H ;延时10ms子程序
DE1:   MOV R6, #0FFH
DE2:   DJNZ R6, DE2
        DJNZ R7, DE1
        RET
TABLE: DB 0C0H, 0F9H, 0A4H, 0B0H, 99H, 92H, 82H, 0F8H, 80H, 90H, 88H, 83H, 0C6H, 0A1H,
        DB 86H, 8EH
END
```

图 1.61 建立源程序

◆加入源程序

利用鼠标右键选中图 1.60 中的 AT89C51 芯片，弹出菜单，如下图 1.62 所示，点击“Add/Remove Source File”菜单选项，将弹出源程序选择对话框，将刚刚建立的源程序及其所在目录加入，如图 1.63 所示，点击“OK”完成源程序的加入。

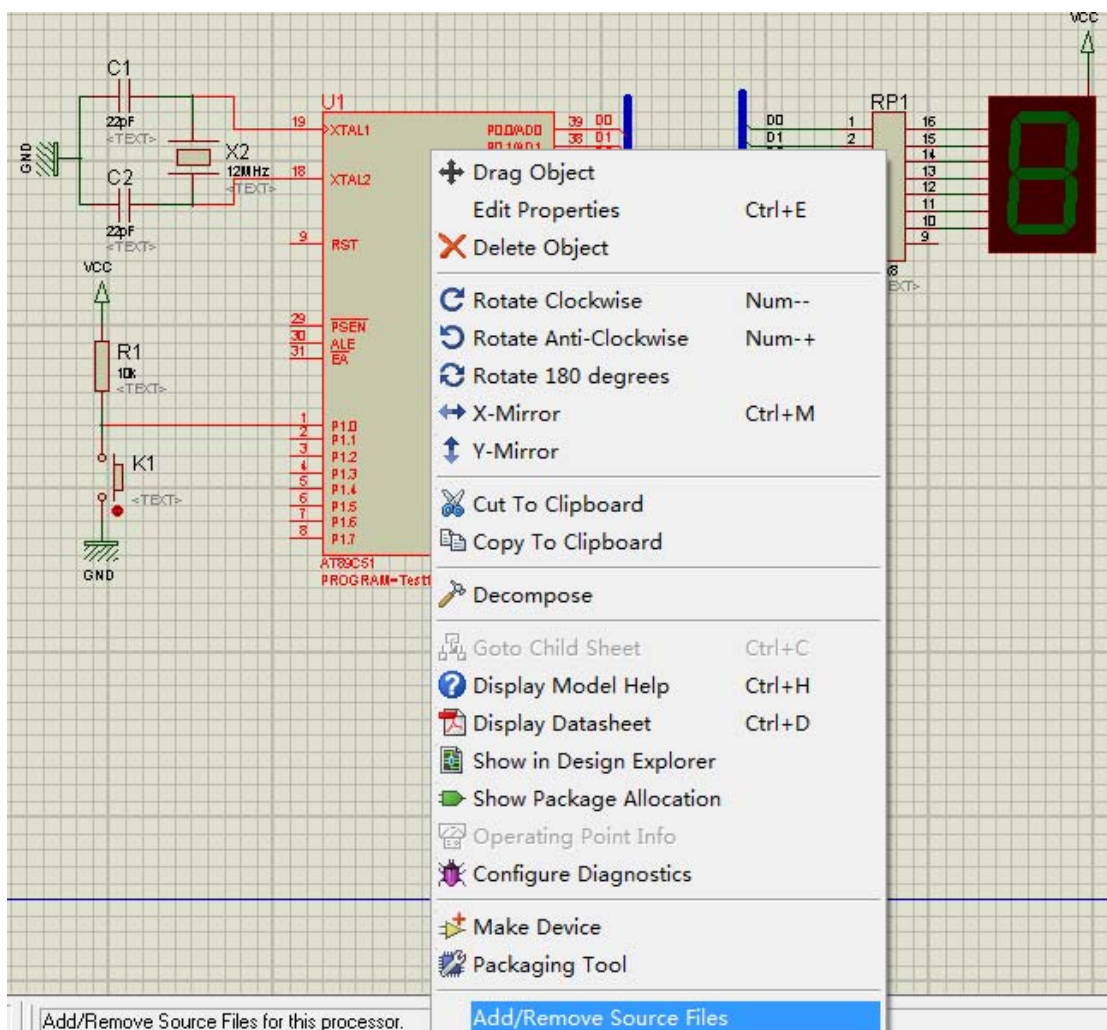


图 1.62 加入源程序

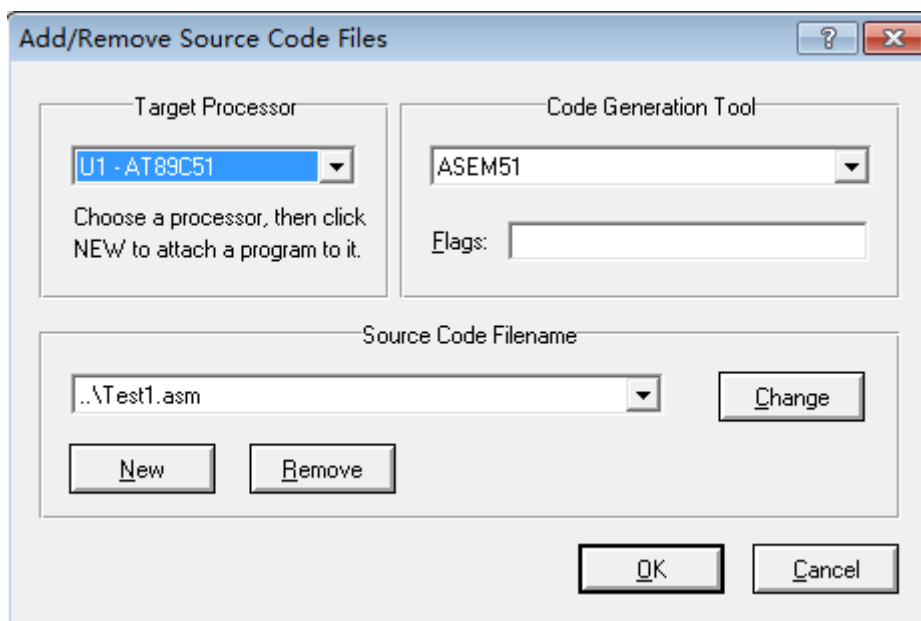


图 1.63 选择源程序

◆编译源程序

利用菜单“Source”→“Build All”将所加入的源程序进行编译，如果源程序没有错

误，将弹出如下图 1.64 所示信息框，

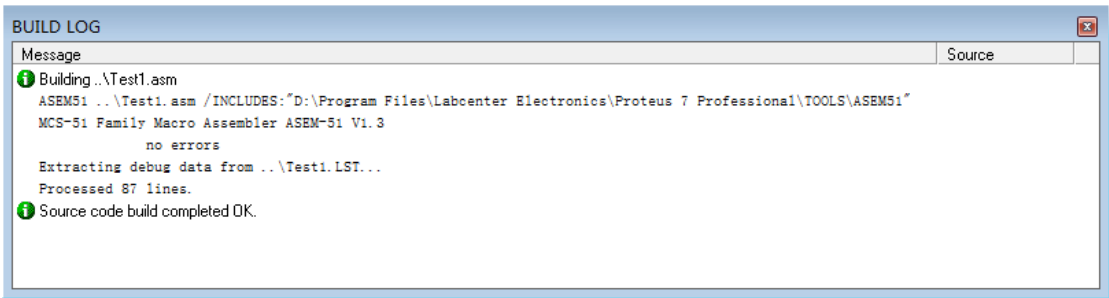


图 1.64 源程序编译信息框

至此，汇编源程序“Test1.asm”已编译成可执行程序“Test1.Hex”。

◆加入可执行程序

双击图 1.60 中的 AT89C51 芯片，弹出对话框，如下图 1.65 所示：

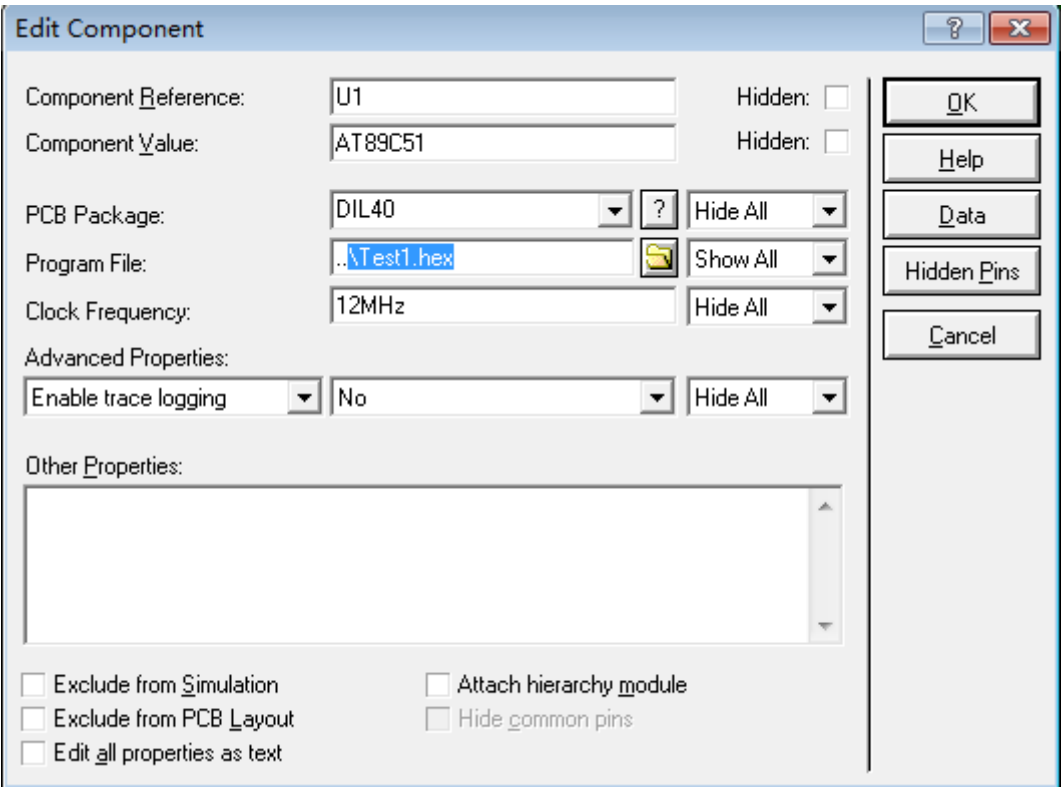


图 1.65 加入可执行程序

在“Program File”栏中写上可执行程序，也可以通过其右侧的“”选择可执行程序及其所在路径。

点击 OK 按钮，完成可执行程序的加入。

◆执行仿真

点击主窗口下方的仿真控制按钮“”，并按图 1.60 中的按键 K1，仿真效果如下图 1.66 所示：

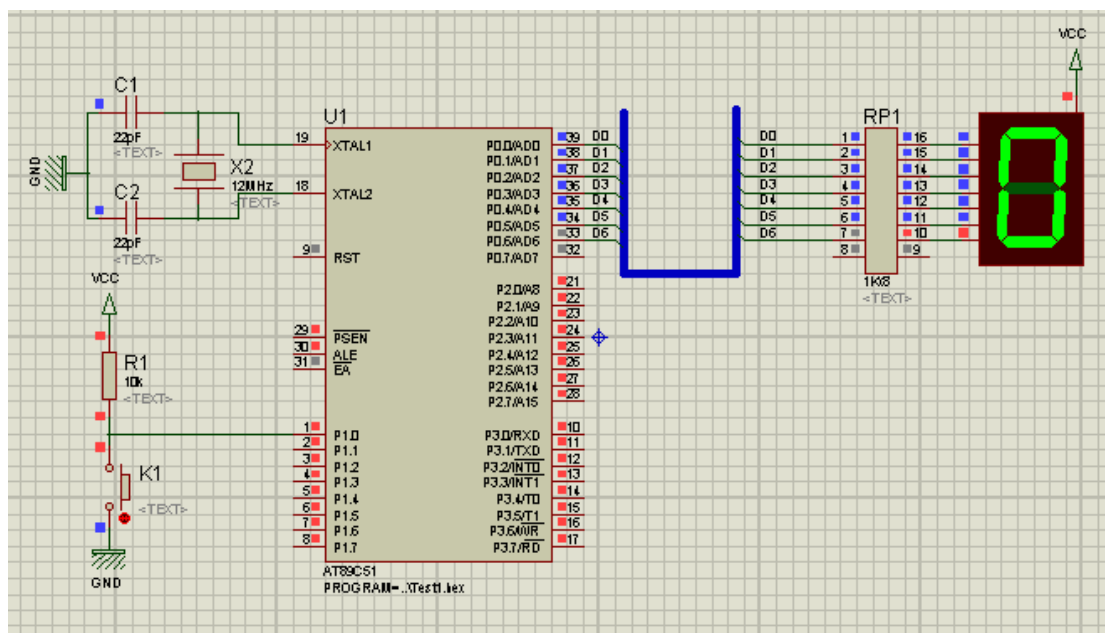


图 1.66 仿真效果

每次按下 K1，则七段数码管显示的字符“加 1”，在“0”到“F”之间循环。

图 1.66 中每个信号端口上的红色方框表示该端口上的电平为高电平，蓝色方框表示该端口上的电平为低电平。

点击仿真控制按钮 “” 可以停止仿真过程。

(6) KeilC 与 Proteus 联合调试

◆ 建立联合调试环境

联合调试环境的建立分三步：

① 首先，假若 Keil C 与 Proteus 均已正确安装在 C:\Program Files 的目录里，把 C:\Program Files\Labcenter Electronics\Proteus 7 Professional\MODELS\VDM51.dll 复制到 C:\Program Files\keil\C51\BIN 目录中。

② 然后，用记事本打开 C:\Program Files\keil\C51\TOOLS.INI 文件，在[C51]栏目下加入：TDRV4=BIN\VDM51.DLL(“PROTEUS VSM MONITOR 51 DRIVER”)，其中“TDRV4”中的“4”要根据实际情况写，只要不和原来的重复就可以。

步骤①和②只需在初次使用设置。

◆ 建立源程序

进入 KeilC μ Vision2 开发集成环境，创建一个新项目(Project)， “C:\Program Files\keil\Test1.uv2”，为该项目选定合适的单片机 CPU 器件(如：Atmel 公司的 AT89C51)，并包含启动文件“STARTUP.A51”。为该项目建立 C 源程序。

源程序如下：

```
#include "reg51.h"

unsigned char code LED_CODES[]= //0-F 的显示码
{ 0xc0, 0xF9, 0xA4, 0xB0, 0x99, //0-4
  0x92, 0x82, 0xF8, 0x80, 0x90, //5-9
```

```

        0x88, 0x83, 0xC6, 0xA1, 0x86, 0x8E} ; //A, b, C, d, E
void main()
{
    unsigned char i=0;
    unsigned char j=0;
    P1=255;    //为输入做准备
    P0=255;    //使输出 7 段数码管不亮
    while(1)
    {
        do{
            i=P1&1;    //按键
        } while(i==1); //等待按键按下
        P0=LED_CODES[j]; //查输出码
        do{
            i=P1&1;
        } while(i==0); //等待按键释放
        j++;
        if (j==16)    //将显示变量限制在 0-15 之间
            j=0;
    }
}

```

将其保存为“C:\Program Files\keil\Test1.c”，并将 Test1.c 加入工程文件 Test1.uv2。

单击“Project 菜单/Options for Target”选项或者点击工具栏的“option for target”

按钮 ，弹出对话框。

将“Target”页中的晶振设置为 12MHz。

在“Output”页中选上“Create Hex File”。

在“Debug”页，选择硬仿真，并选择其中的“PROTEUS VSM MONITOR 51 DRIVER”仿真器，如下图 1.67 所示，

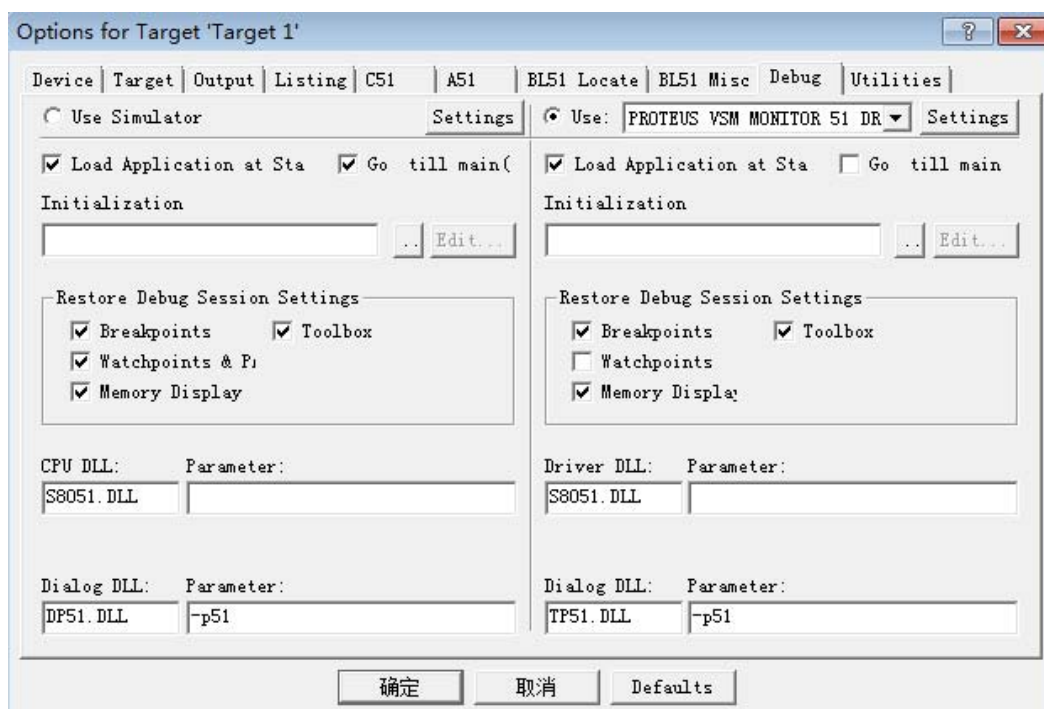


图 1.67 选择 Proteus 虚拟仿真器

再点击“Setting”按钮，设置通信接口，在“Host”后面添上“127.0.0.1”，如果使用的不是同一台电脑，则需要在这里添上另一台电脑的 IP 地址（另一台电脑也应安装 Proteus）。在“Port”后面添加“8000”。设置好的情形如图 1.68 所示，点击“OK”按钮即可。回到图 67 所示界面。

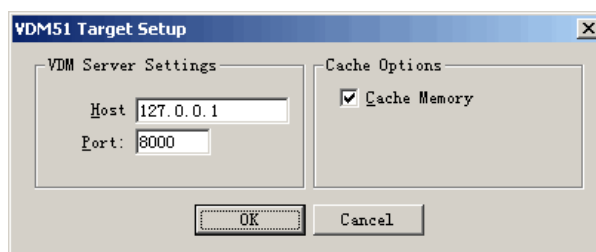


图 1.68 Proteus 虚拟仿真器通信设置

点击“确定”按钮，回到 Keil 主菜单。

构造工程 Test1.uv2, 使其生成可执行程序 Test1.hex, 程序没有错误, 将显示如下图 1.69 所示构造结果。

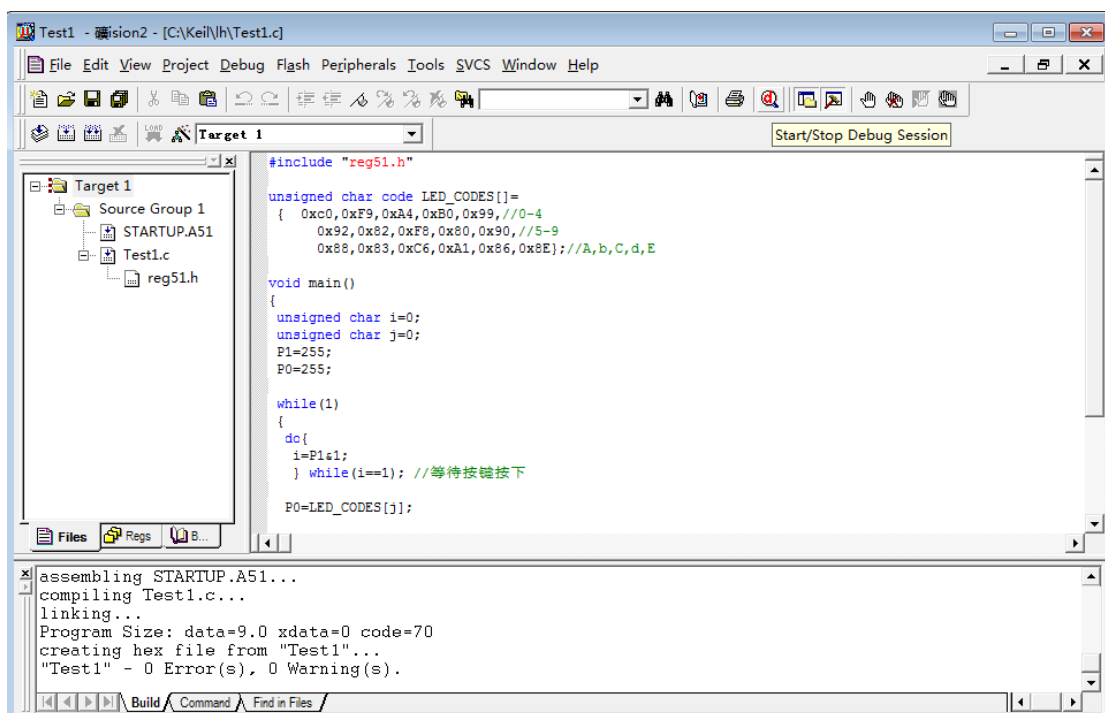


图 1.69 Keil 构造结果

按前面讲述的方法和图 1.65 所示，为 AT89C51 设置可执行程序 Test1.hex。

进入 Proteus 的 ISIS，鼠标左键点击菜单“Debug”，选中“use remote debugger monitor”，如图 1.70 所示。此后，便可实现 KeilC 与 Proteus 连接调试。

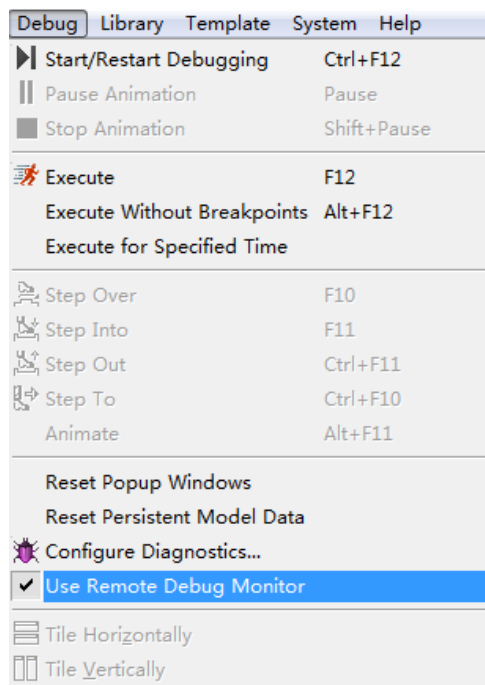


图 1.70 Proteus 远端调试设置

点击图 1.69 中的启动调试按钮“”。Keil 将与 Proteus 连接，并出现如下界面，

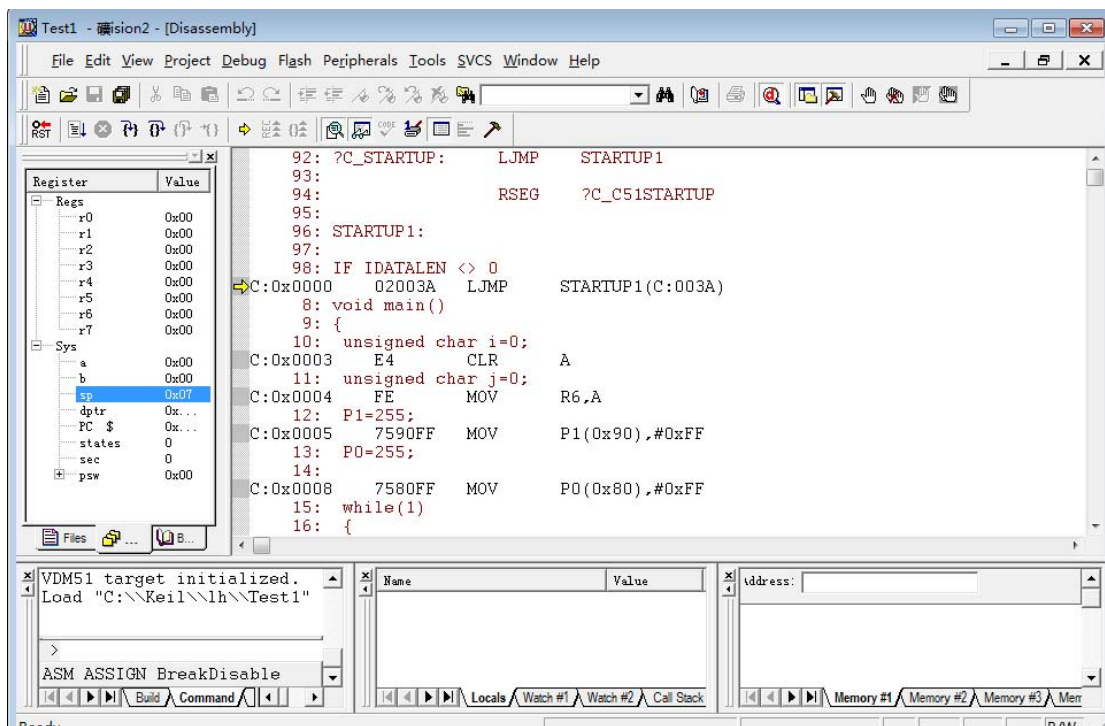


图 1.71 Keil 与 Proteus 连接成功

执行上图 1.71 中的运行程序按钮“”开始调试过程。这时 Proteus 将出现如下图 1.72 所示界面，这表明 Proteus 已开始仿真过程，

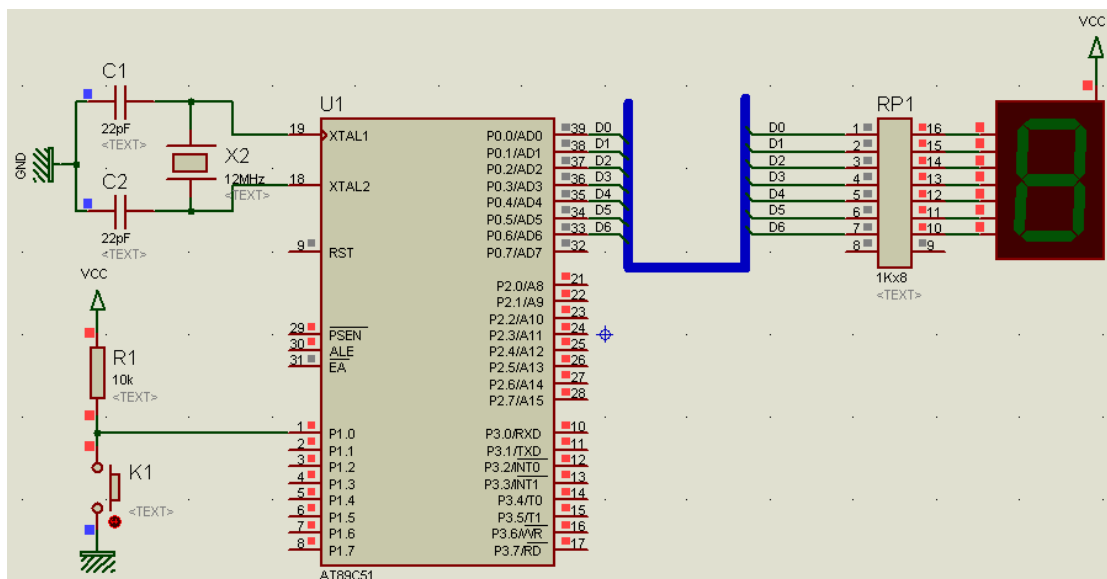


图 1.72 Proteus 仿真界面

按图 1.72 中的 K1 按键一次，将出现图 1.66 所示的仿真结果界面。

再按 K1，每按一次，7 段数码管显示字符将“加 1”，并在“0”到“F”之间循环。

在 Keil 界面图 1.71 中，按下程序停止按钮“”，则仿真程序停止执行，再操作 Proteus

中的 K1 按键 7 段数码管的显示不再变化，再按 Keil 界面中的按钮“”，停止 Keil 仿真，

Proteus 也将停止仿真过程，回到图 1.60 所示的界面。

(7) Proteus 仿真信号测试

这里利用本地仿真（见图 1.61-图 1.66）介绍 Proteus 环境下的信号测试方法。

① 电压探针

Proteus 仿真环境下，除了用红色方块表示端口高电平，用蓝色方块表示端口低电平，还可以用电压探针实测电压大小。

在图 1.60 中的 P1.0 与电阻 R1 和按键 K1 之间的连接线上加入 1 个电压探针。加入的方法是点击绘图工具栏中的“”按钮，将鼠标移动到需要加入电压探针的信号线上，点击鼠标左键，将放置 1 个电压探针，如图 1.73 所示：

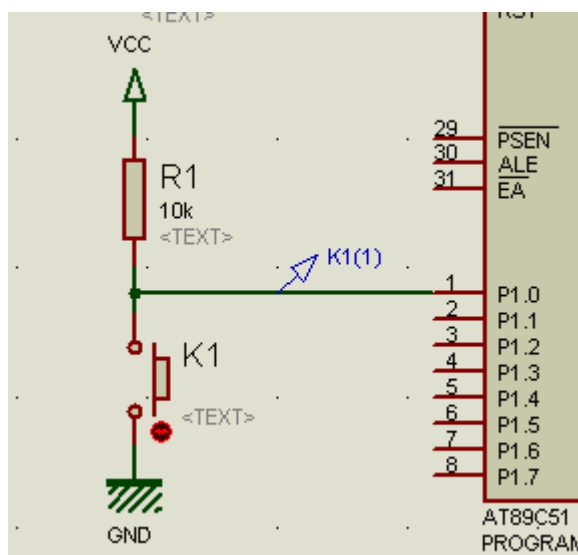


图 1.73 添加电压探针

执行仿真，电压探针将显示当前的电压值，按下 K1，松开 K1，观察电压探针探测的电压如何改变，结果如下图 1.74 所示，

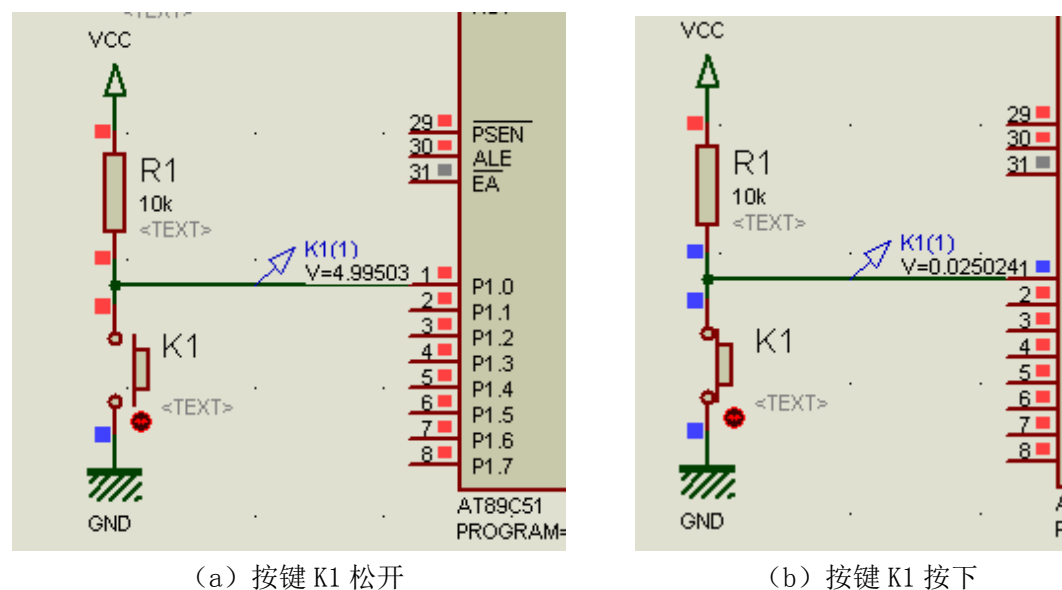


图 1.74 电压探针探测结果

② 虚拟仪器

Proteus 提供了众多的虚拟仪器，如：示波器（OSCILLOSCOPE）、逻辑分析仪（LOGIC ANALYSER）、函数发生器（SIGNAL GENERATOR）、数字信号图案发生器（PATTERN GENERATOR）、时钟计数器（COUNTER TIMER）、虚拟终端（VIRTUAL TERMINAL）以及简单的电压计（DC VOLTMETER 与 DC VOLTMMETER）、电流计（DC AMMETER 与 DC AMMETER）、SPI 协议分析仪（SPI DEBUGGER）、I2C 协议分析仪（I2C DEBUGGER），这些虚拟仪器为仿真调试提供了类似于实际硬件测试环境的测试分析工具，对于电路系统设计具有很好的帮助作用。

这里简单说明常用虚拟示波器和虚拟频率计的使用方法。

在 Proteus 仿真环境下画出如下图 1.75 所示的由 NE555 构成的方波发生器电路，所用元件库来源及参数如下表所示：

选件编号	元件名称	参数	所在元件库类名	子类名
U1	NE555		Analog ICs 模拟集成电路	Timers 定时器
Q1	NPN		Transistors 三极管	Bipolar 双极性
RV1, RV2	POT	100k	Resistors 电阻	Variable 可变的
D1, D2	1N4148		Diodes 二极管	Switching 开关
R1	RES	330	Resistors 电阻	Generic 一般的
R2	RES	10	Resistors 电阻	Generic 一般的
R3	RES	10k	Resistors 电阻	Generic 一般的
C1	CAP	0.1uF	Capacitors 电容	Generic 一般的
C2	CAP	0.01uF	Capacitors 电容	Generic 一般的

其中可变电阻 RV1 用于调节 NE555 3 端信号的占空比和 6 端信号的上升和下降沿，RV2 用于调节信号的周期。

添加这些元件，并按图 1.75 所示添加相应的电源（Power）和地（Ground），连线。

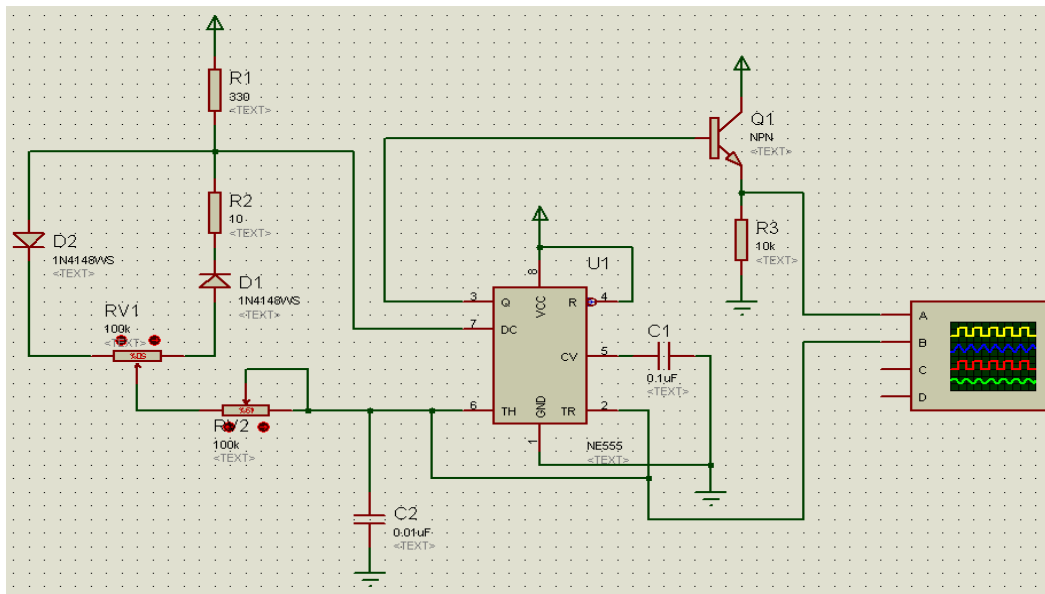


图 1.75 NE555 构成的方波发生器仿真电路

(a) 虚拟示波器

点击 Proteus 仿真工具栏中的虚拟仪器按钮 “”，选择对象选择窗口中的示波器 “OSCILLOSCOPE” 选项，如图 76 所示。对象预览窗口中将显示该虚拟仪器的示图。将鼠标移动到原理图编辑窗口，点击鼠标左键，将该示波器放在原理图中合时的位置，并按图 1.75 所示进行连线。将 Q1 的发射极连接到示波器的 A 输入端，将 U1 的 6 脚连接到示波器的 B 输入端。

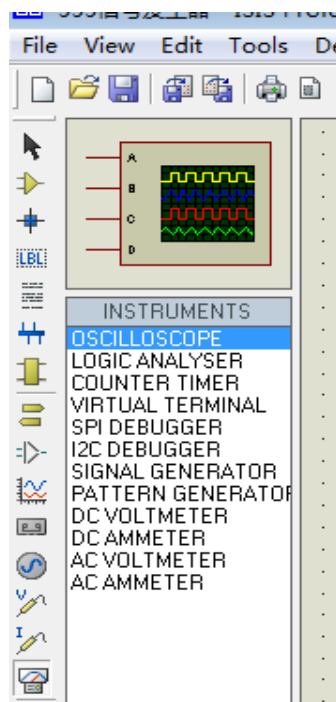


图 1.76 添加虚拟示波器

点击仿真启动按钮 “”，进行仿真，这时将弹出示波器的界面图，并显示 A 端和

B 端的信号波形图，并可用类似于真实示波器的调节方法，调节信号波形的周期和幅度。

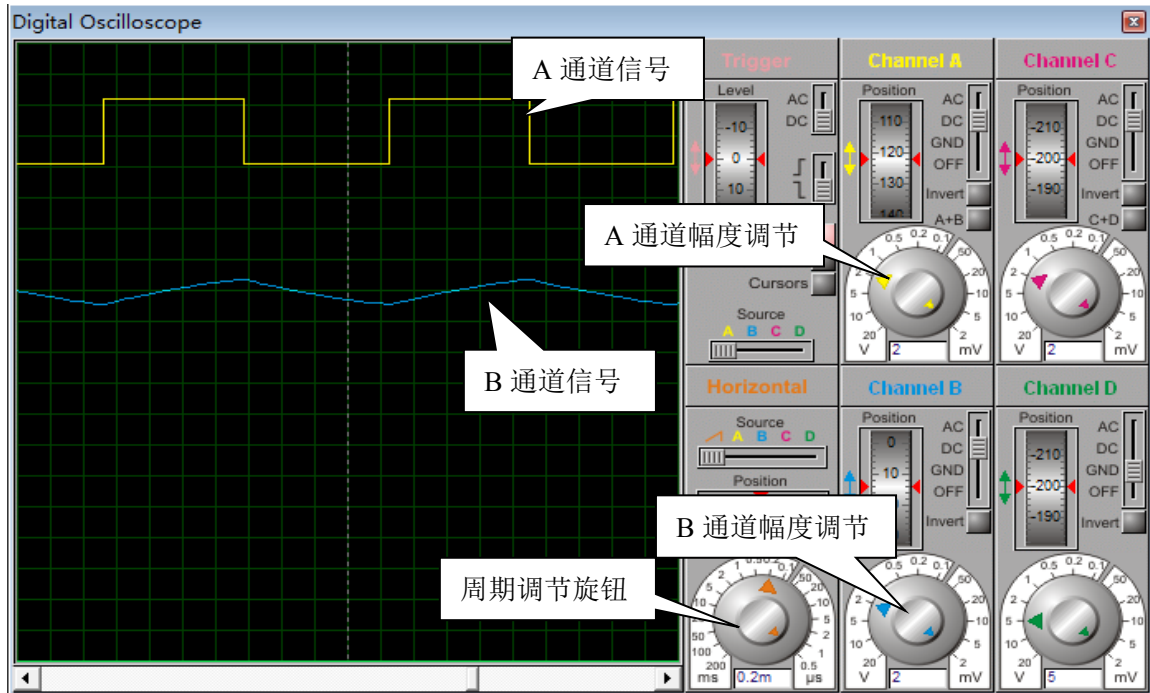


图 1.77 虚拟示波器界面

如果仿真过程中，不慎关掉了示波器的界面，可以用如下方法重新将示波器界面打开。
方法是：用鼠标右键选中图 1.75 中的示波器元件，将弹出如下图 1.78 所示菜单，

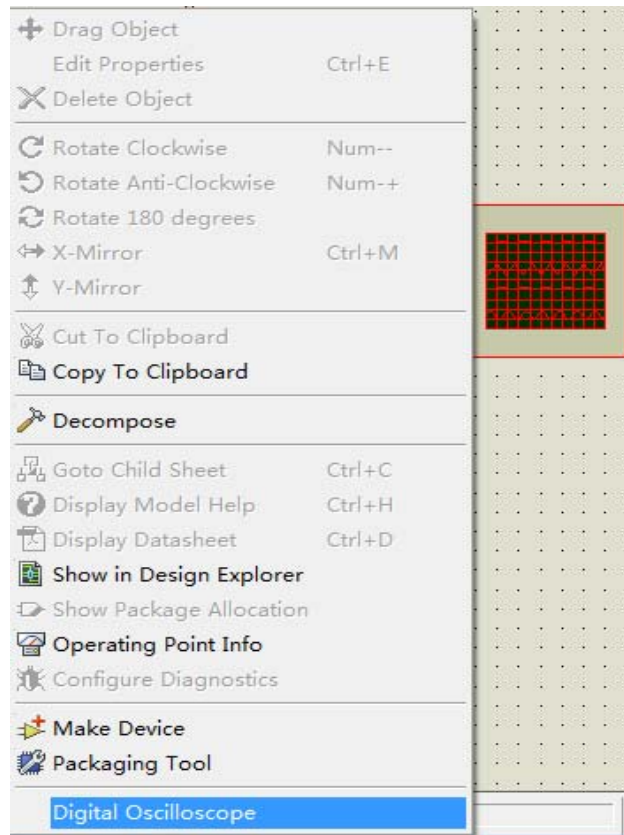


图 1.78 虚拟示波器界面重现菜单

点击其中的“Digital Oscilloscope”菜单项，图 1.77 中的示波器界面将重新出现。

(b) 虚拟时钟计数器

虚拟频率计利用虚拟时钟计数器来实现。具体使用方法如下：

首先，点击 Proteus 仿真工具栏中的虚拟仪器按钮“”，选择对象选择窗口中的示波器“COUNTER TIMER”选项，如图 1.79 所示。

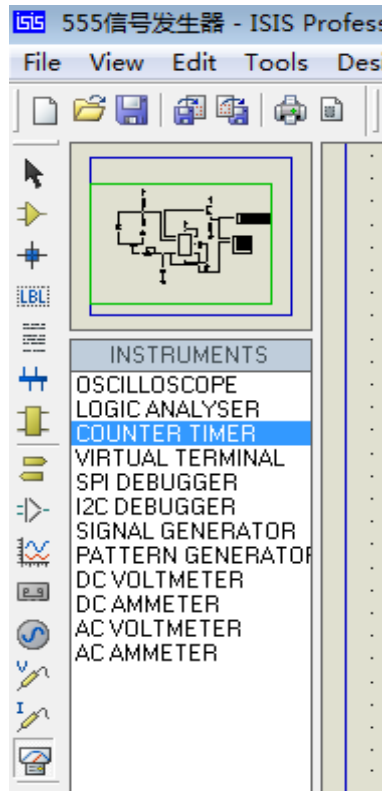


图 1.79 添加虚拟时钟计数器

将鼠标移动到原理图编辑窗口，点击鼠标左键，将该时钟计数器放在原理图中合时的位置，并按图 80 所示进行连线，将 Q1 发射极与时钟计数器的 CLK 进行连接。

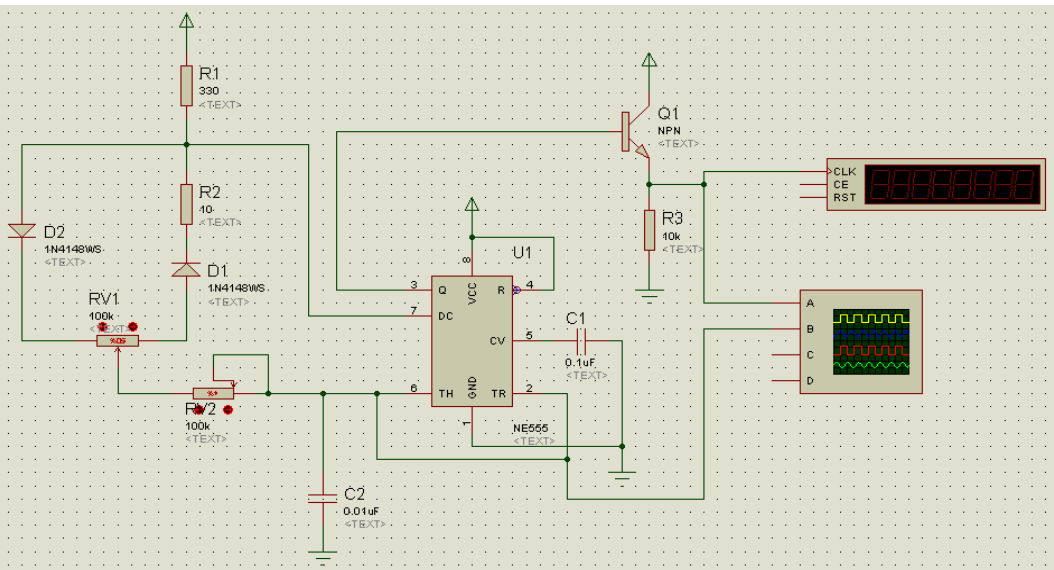


图 1.80 添加虚拟时钟计数器后电路图

点击仿真启动按钮“”，进行仿真，这时时钟计数器开始工作，其计数值在不断变化，用鼠标左键单击该时钟计数器，将显示如图 1.81 所示窗口

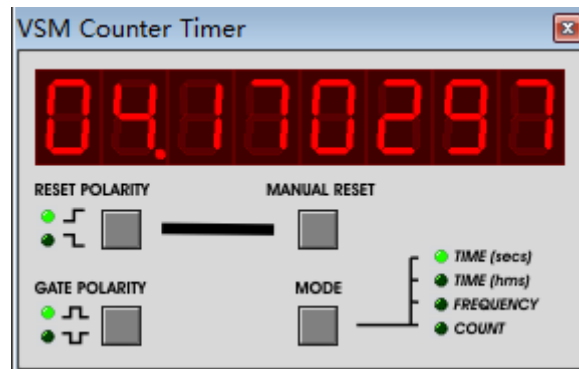


图 1.81 虚拟时钟计数器工作界面

默认方式下为时间测量模式（TIME），单位为秒。

用鼠标左键单击图 1.81 中的“MODE”按钮，将改变该虚拟时钟计数器的工作方式。将其改变为“FREQUENCY”模式就是频率计模式，这时显示的是信号频率。

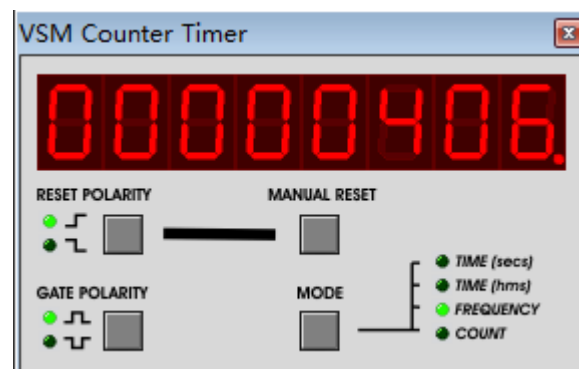


图 1.82 虚拟时钟计数器频率计模式

改变图 1.80 中 VR2 滑动变阻器滑动抽头的位置，可以观察到图 1.82 中频率计显示的信号频率是变化的。

第二章 软件实验

实验前的准备：

(1) 《单片机原理及应用》教材中的单片机“第二章 MCS-51 单片机硬件结构”和“第三章 MCS-51 单片机的软件系统设计”有关内容，了解单片机内部结构及其工作原理，了解汇编及 C 语言程序设计方法，熟悉单片机的汇编语言指令系统和 C51 语法内容。

(2) 第一章 1.1 节关于 Keil uVision2 部分内容，熟悉 Keil uVision2 编程环境及应用方法，能够正确建立工程，并做适当设置，能够利用 Keil 编写、编译、汇编及调试基本的汇编及 C 语言程序。

本部分实验的基本要求：

本部分实验的基本目的是通过四个纯软件仿真实验，初步掌握 Keil uVision2 编成编程环境的应用方法，通过软件实验进一步了解单片机内部结构，了解程序执行对 PSW 的影响，掌握 51 单片机汇编程序设计方法，C51 程序设计方法，以及 C 与汇编的混合编成方法。要求学生能够独立读懂示例程序，并将示例程序运行成功，独立完成类似程序的构思、设计及调试。

2.1 实验 1：汇编程序实验 1

2.1.1 示例实验

(1) 实验内容

在 Keil 环境下建立工程，并将以下程序加入工程，构造工程，并运行可执行程序，记录执行结果。分析程序功能。

(2) 示例程序

```
ORG 0000H
AJMP MAIN
ORG 0030H
MAIN:
MOV SP, #60H
MOV A, #0H
MOV R1, #30H
MOV R7, #10H
LOOP1:
MOV @R1, A
INC R1
DJNZ R7, LOOP1
NOP
MOV R1, #30H
```

```

MOV R7, #10H
LOOP:
MOV @R1, A
INC R1
INC A
DJNZ R7, LOOP
SJMP $
END

```

(3) 实验步骤

① 运行 Keil uVision2 开发环境，按照 1.1.3 节介绍的方法建立工程 esimlab1.uv2，CPU 为 AT89C51，不用包含启动文件 STARTUP.A51。

② 输入源程序，在 Keil uVision2 开发环境中，建立源程序 esimlab1.asm，将上述程序加入该程序文件。并将该文件加入工程 esimlab1.uv2

③ 设置工程 esimlab1.uv2 属性，将其晶振频率设置为 12MHz，选择输出可执行文件，仿真方式为 “Use Simulator”。

④ 构造 (Build) 工程 esimlab1.uv2。如果输入有误进行修改，直至构造正确，生成可执行程序 esimlab1.hex 为止。

⑤ 在上述程序 “NOP” 指令处设置断点，运行程序至断点，并用存储器观察窗口，观察内部 RAM 30H-3FH 单元内的值。

⑥ 单步运行后面的程序，观察寄存器 R1、R7、A、PC、30H-3FH 单元的内容随着指令的执行的变化情况。

(4) 实验作业

- 为示例程序源程序添加注释
- 写出示例程序的功能。
- 记录示例程序运行结果截图。

2.1.2 自我完成实验

(1) 实验内容

将片内 RAM 30H 单元中的 8 位二进制数转换成 10 进制数。希望转换后的结果保存于 31H 和 32H，31H 低 4 位存放个位，高 4 位存放十位，32H 低 4 位存放百位，高 4 位为 0。

(2) 程序流程图

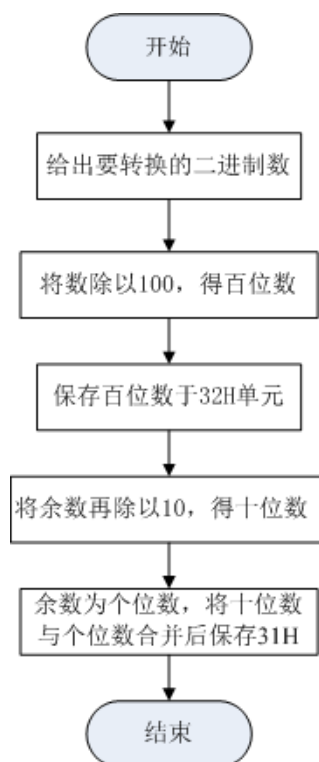


图 2.1 汇编程序实验 1 自我完成实验流程图

(3) 实验步骤

- ① 根据上述实验内容及程序流程图，编写汇编语言源程序，并加上相应注释，注意扩展名为“*.asm”，将其保存。
- ② 运行 Keil uVision2 开发环境，按照 1.1.3 节介绍的方法建立工程 simlab1.uv2，CPU 为 AT89C51，不用包含启动文件 STARTUP.A51。
- ③ 编写好的源程序加入工程 simlab1.uv2，并设置工程 simlab1.uv2 属性，将其晶振频率设置为 12MHz，选择输出可执行文件，仿真方式为“Use Simulator”。
- ④ 构造 (Build) 工程 simlab1.uv2。如果输入有误进行修改，直至构造正确，生成可执行程序 simlab1.hex 为止。
- ⑤ 运行程序，并用存储器观察窗口，观察内部 RAM 30H, 31H, 32H 单元的值。
- ⑥ 更改 30H 单元的值，重新运行程序，观察内部 RAM 30H, 31H, 32H 单元的值。

(4) 实验作业

- 编写源程序并进行注释
- 记录实验过程
- 记录程序运行结果截图

2.2 实验 2：汇编程序实验 2

2.2.1 示例实验

(1) 实验内容

在 Keil 环境下建立工程，并将以下程序加入工程，构造工程，并运行可执行程序，记录执行结果。分析程序功能。

(2) 示例程序

```
ORG 0000H
AJMP MAIN
ORG 0030H
MAIN:
    MOV 30H, #45H
    MOV A, 30H
    ANL A, #0FH
    MOV 31H, A
    MOV A, 30H
    ANL A, #0F0H
    SWAP A
    MOV B, #10
    MUL AB
    ADD A, 31H
    MOV 31H, A
    SJMP $
END
```

(3) 实验步骤

① 运行 Keil uVision2 开发环境，按照 1.1.3 节介绍的方法建立工程 esimlab2.uv2，CPU 为 AT89C51，不用包含启动文件 STARTUP.A51。

② 输入源程序，在 Keil uVision2 开发环境中，建立源程序 esimlab2.asm，将上述程序加入该程序文件。并将该文件加入工程 esimlab2.uv2

③ 设置工程 esimlab2.uv2 属性，将其晶振频率设置为 12MHz，选择输出可执行文件，仿真方式为“Use Simulator”。

④ 构造 (Build) 工程 esimlab2.uv2。如果输入有误进行修改，直至构造正确，生成可执行程序 esimlab2.hex 为止。

⑤ 单步运行程序，观察寄存器 A，B，PSW 以及片内 RAM 30H、31H 单元随程序执行的变化情况，并分析 PSW 位的变化情况。

⑥ 更换“MOV 30H, #45H”语句中“45H”为其它 BCD 码，重新运行程序，并观察片内 RAM 30H、31H 单元中数值间的关系，分析程序功能。

(4) 实验作业

- 为示例程序源程序添加注释

- 写出示例程序的功能。
- 记录示例程序运行结果截图。

2.2.2 自我完成实验

(1) 实验内容

将片内 RAM 30H 开始的 32 个单元中分布着随机的有符号 8 位二进制数，请按从小到大的顺序进行排序，排序后的数据仍然保存到 30H 开始的 32 个单元中（低地址存放小数据）。

(2) 编程思路

首先，在程序存储器中构建一个表格，该表格具有 32 个随机产生的 8 位二进制数，如：

```
TABLE: DB 1, 3, 9, 2, 17, 4, 11, 6  
        DB 5, 20, 100, 64, 21, 14, 79, 35  
        DB 92, 7, 91, 23, 65, 16, 13, 18  
        DB 18, 73, 65, 101, 27, 19, 62, 69
```

然后利用查表指令“MOVC A, @A+DPTR”将它们读取到 30H-4FH 单元中，

然后再利用“冒泡法排序法”将它们排序即可。“冒泡法排序法”的基本原理是：

遍历所有 32 个数据找出其中的最大者，并记下最大数据所在的存储位置，然后将这个最大的数据放置在最后一个单元，同时，将最后一个单元原来的数据保存到这个最大值原来所处的位置，完成第一轮排序。

然后，再遍历除了最后一个单元以外的前面 31 个单元的数据并找出其中最大者，并记下其所在位置，遍历完这一遍后将找到的最大数据保存在倒数第二个单元（对于所有数据来说它是次最大数据，所以保存在倒数第二个单元），并将倒数第二个单元原来的数据保存在刚刚找到的那个最大值原来所在的位置处，完成第二轮的排序。依次类推，用同样的方法把所有数据排好序即可。

(3) 程序流程图

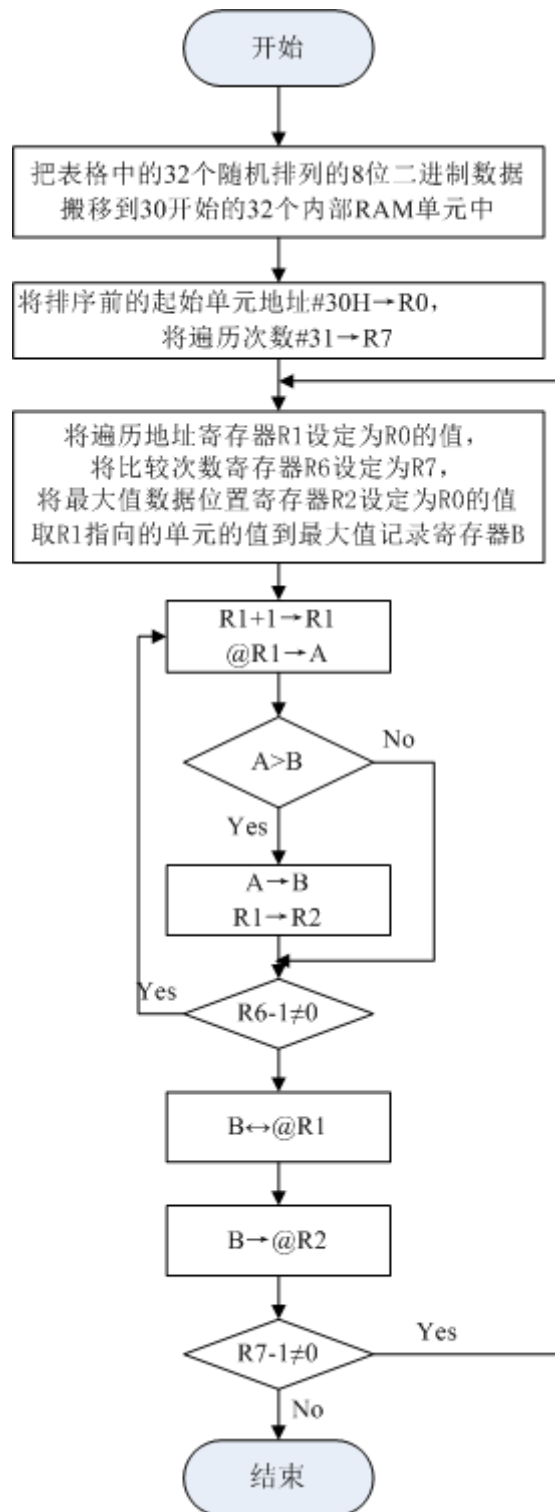


图 2.2 汇编程序实验 2 自我完成实验流程图

(4) 实验步骤

① 根据上述实验内容及程序流程图，编写汇编语言源程序，并加上相应注释，注意扩展名为“*.asm”，将其保存。

② 运行 Keil uVision2 开发环境，按照 1.1.3 节介绍的方法建立工程 simlab2.uv2，CPU 为 AT89C51，不用包含启动文件 STARTUP.A51。

③ 编写好的源程序加入工程 simlab2.uv2，并设置工程 simlab2.uv2 属性，将其晶振频率设置为 12MHz，选择输出可执行文件，仿真方式为“Use Simulator”。

④ 构造 (Build) 工程 simlab2.uv2。如果输入有误进行修改，直至构造正确，生成可执行程序 simlab2.hex 为止。

⑤ 运行程序，并用存储器观察窗口，观察内部 RAM 30H-4FH 单元排序前后的数值。

(5) 实验作业

- 编写源程序并进行注释
- 记录实验过程
- 记录程序运行结果截图

2.3 实验 3：C 语言程序实验

2.3.1 示例实验

(1) 实验内容

在 Keil 环境下建立工程，并将以下程序加入工程，构造工程，并运行可执行程序，记录执行结果。分析程序功能。

(2) 示例程序

```
#include<reg51.h>
#include<stdio.h>
#define uchar unsigned char
#define uint unsigned int

uchar data a[32] _at_ 0x30; //设定数组 a 的起始地址为 30H
uint i _at_ 0x55; //将变量 i 放在地址 55H

//延时程序
void DelayMS(uint ms)
{
    uchar t;
    while(ms-- for(t=0;t<120;t++);
}

//主程序
void main()
```

```

{
    SP=0x60;  //设定堆栈指针位置
    SCON=0x52;
    TMOD=0x20;
    TH1=0xf3;
    TR1=1; //此行及以上 3 行为 PRINTF 函数所必须
    for(i=0;i<32;i++)
    {
        a[i]=i;
        printf("It is : %d now.\n",i); //打印程序执行的信息
        DelayMS(20000);
    }
    while(1);
}

```

(3) 实验步骤

① 运行 Keil uVision2 开发环境，按照 1.1.3 节介绍的方法建立工程 esimlab3.uv2，CPU 为 AT89C51，并包含启动文件 STARTUP.A51。

②输入源程序，在 Keil uVision2 开发环境中，建立源程序 esimlab3.c，将上述程序加入该程序文件。并将该文件加入工程 esimlab3.uv2

③设置工程 esimlab3.uv2 属性，将其晶振频率设置为 12MHz，选择输出可执行文件，仿真方式为“Use Simulator”。

④构造 (Build) 工程 esimlab3.uv2。如果输入有误进行修改，直至构造正确，生成可执行程序 esimlab3.hex 为止。

⑤启动调试过程，并通过“View”→“Serial Window #1”把串行调试窗口 1 显示出来。

⑥运行程序，并用存储器观察窗口，观察内部 RAM 30H 单元内的值是如何变化的，并观察串行调试窗口 1 中显示的内容。

(4) 实验作业

- 为示例程序源程序添加注释
- 写出示例程序的功能。
- 记录示例程序运行结果截图。
- 讨论示例程序中 while(1)的作用。

2.3.2 自我完成实验

(1) 实验内容

同实验 1.2.2。

(2) 编程思路

参考实验 1.2.2。

(3) 程序流程图

参考实验 1.2.2。

(4) 实验步骤

- ① 根据上述实验内容，参考 2.3.1，编写 C 语言源程序，并加上相应注释，注意扩展名为“*.c”，将其保存。
- ② 运行 Keil uVision2 开发环境，按照 1.1.3 节介绍的方法建立工程 simlab3.uv2，CPU 为 AT89C51，包含启动文件 STARTUP.A51。
- ③ 编写好的源程序加入工程 simlab3.uv2，并设置工程 simlab3.uv2 属性，将其晶振频率设置为 12MHz，选择输出可执行文件，仿真方式为“Use Simulator”。
- ④ 构造 (Build) 工程 simlab3.uv2。如果输入有误进行修改，直至构造正确，生成可执行程序 simlab3.hex 为止。
- ⑤ 运行程序，并用存储器观察窗口，观察内部 RAM 30H-4FH 单元排序前后的数值。

(5) 实验作业

- 画流程图
- 编写源程序并进行注释
- 记录实验过程
- 记录程序运行结果截图

2.4 实验 4：C 与汇编混合编程实验

2.4.1 示例实验

(1) 实验内容

在 Keil 环境下建立工程，并将以下程序加入工程，构造工程，并运行可执行程序，记录执行结果。分析程序功能。

(2) 示例程序

```
//该程序保存为 C 语言源程序 esimlab4.c  
  
#include<reg51.h>  
  
#include <math.h> //为使用 sin 函数所以要包含该头文件  
  
  
typedef unsigned char uchar;  
  
typedef unsigned int uint;
```

extern void delay(char n); //在 main 函数调用之前应该先将子函数 void delay()
进行声明。

extern uint add(char c, char d); // 将汇编函数声明为外部函数。

extern float asmsin(float e); //声明一个外部汇编函数

uchar i, j, n;

uint x;

float y, z;

main()

{

//以下为 C 调用有参数传递但是无返回值的汇编函数的示例

n=100;

for(i=0; i<200; i++)

{

for(j=0; j<250; j++)

{

delay(n); //无返回参数的汇编函数

}

}

//以下为 C 调用有参数传递也有返回值的汇编函数的示例

i=150;

j=200;

x=add(i, j); //有参数传递有返回值的汇编函数

//以下 C 先调用汇编，汇编中又调用了 C

y=3.1415926/2;

z=asmsin(y); //汇编函数 asmsin 中调用了 C 的库函数 sin(x)

while(1);

}

=====

;该文件保存名为 delay.asm，在 12MHz 晶振下，本汇编程序的延时长度为 x*0.1ms

;x 表示传递过来的数值，该数值在 0-255 之间

```
?PR?_DELAY SEGMENT CODE; // 作用是在程序存储区中定义段，段名为
                        ; // _DELAY, ?PR? 表示段位于程序存储区内。
```

```
PUBLIC _DELAY; //声明函数为公共函数
```

```
RSEG ?PR?_DELAY; //表示函数可被连接器放置在任何地方，RSEG 是段名的属性。
```

```
_DELAY:
```

```
    MOV A, R7; //只有一个参数，R7
```

```
    MOV R2, A
```

```
DL1:MOV R1, #48
```

```
DL2:DJNZ R1, DL2
```

```
    NOP
```

```
    DJNZ R2, DL1
```

```
RET
```

```
END
```

=====

;该程序保存为 add.asm

```
?PR?_ADD SEGMENT CODE;
```

```
PUBLIC _ADD;
```

```
RSEG ?PR?_ADD;
```

```
_ADD:
```

```
    MOV A, R5
```

```
    CLR C
```

```
    ADD A, R7
```

```
    MOV R7, A
```

```
    MOV A, #0
```

```
    ADDC A, #0
```

```
    MOV R6, A //将函数返回值放在 R6R7 中，返回整型数。
```

```
RET
```

```
END
```

=====

;该程序保存为 asmsin.asm

```
?PR?_ASMSIN SEGMENT CODE; // 在程序存储区中定义段
```

```
PUBLIC _ASMSIN; //声明函数
```

```
EXTRN CODE(_SIN); //声明为外部函数，来自于 C 库函数
```

```
RSEG ?PR?_ASMSIN; //函数可被连接器放置在任何地方
```

`_ASMSIN:`

`LCALL _SIN ; //调用 C 库函数中的正弦函数  $y=\sin(x)$ , 这里的参数是通过 R4-R7 带入, 也是通过 R4 到 R7 带出`

`RET`

`END`

(3) 实验步骤

① 运行 Keil uVision2 开发环境, 按照 1.1.3 节介绍的方法建立工程 esimlab4.uv2, CPU 为 AT89C51, 并包含启动文件 STARTUP.A51。

② 输入源程序, 在 Keil uVision2 开发环境中, 建立上述源程序 esimlab4.c, delay.asm, add.asm, asmsin.asm 将上述这些程序文件加入工程 esimlab4.uv2

③ 设置工程 esimlab4.uv2 属性, 将其晶振频率设置为 12MHz, 选择输出可执行文件, 仿真方式为 “Use Simulator”。

④ 构造 (Build) 工程 esimlab4.uv2。如果输入有误进行修改, 直至构造正确, 生成可执行程序 esimlab4.hex 为止。

⑤ 运行程序, 利用设置断点的方式观察程序每一部分的功能与运行结果。

⑥ 适当调整每一部份的参数, 重新运行程序, 观察运行结果。

(4) 实验作业

编写实验报告, 包含的内容包含两部分:

- 仔细分析每段程序的功能, 并分析 C 调用汇编, 汇编调用 C 的语法格式, 将其总结出来
- 记录示例程序运行结果截图。

第三章 Proteus 系统仿真实验

实验前的准备:

(1) 《单片机原理及应用》教材中的单片机“第二章 MCS-51 单片机硬件结构”和“第三章 MCS-51 单片机的软件系统设计”有关内容,了解单片机内部结构及其工作原理,了解汇编及 C 语言程序设计方法,熟悉单片机的汇编语言指令系统和 C51 语法内容。

(2)复习第一章 Keil uVision2 和 Proteus 软件使用指南,熟悉 Keil uVision2 编程环境及应用方法,能够正确建立工程,并做适当设置,能够利用 Keil 编写、编译、汇编及调试基本的汇编及 C 语言程序,并能利用 Proteus 搭建仿真电路,对 Proteus 进行设置,实现 Proteus 与 Keil 联调。

3.1 实验 1: 基本并行 I/O 口实验

3.1.1 示例实验

(1) 实验内容

在 Proteus 环境下搭建如下图 3.1 所示的电路图:

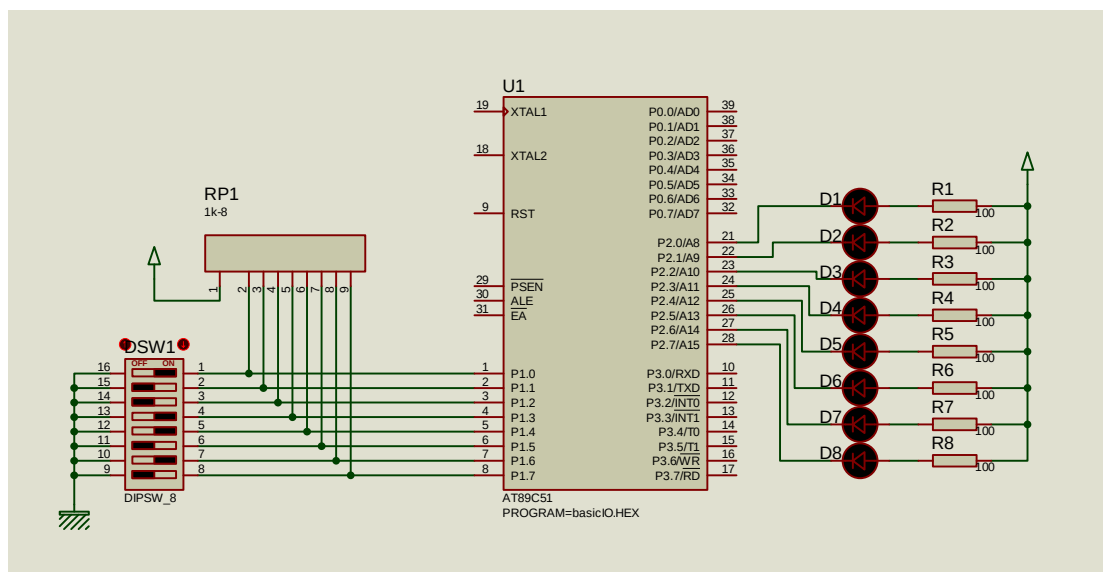


图 3.1 basicIO 原理图

其中所用元器件如下表所示:

选件编号	元件名称	参数	所在元件库类名	子类名	生产厂家
U1	AT89C51		Microprocessor ICs 微处理器	8051 Family 8051 家族	ATMEL
RP1	RESPACK-8	10k*8	Resistors 电阻	Resistor Packs 排阻	
DSW1	DIPSW_8		Switches & Relays 开关与继电器	Switches 开关	
D1-D8	LED-RED		Optoelectronics 光电子器件	LEDs 发光二极管	
R1-R8	RES	100	Resistors 电阻	Generic 一般的	

(2) 示例程序

```

ORG 0000H
AJMP MAIN
ORG 0030H
MAIN:
    MOV SP, #60H
;    MOV P2, #0FFH
LOOP:
    MOV P1, #0FFH
    MOV A, P1
    MOV P2, A
    SJMP LOOP
END

```

(3) 实验步骤

① 按照 1.2.2 节 (1) - (4) 小节所介绍的方法在 Proteus 环境下建立原理图，并保存为 basicI0.DSN 文件。

② 输入源程序，按照 1.2.2 节第 (5) 小节所示方法建立源程序 basicI0.asm，程序内容如上面的“(2) 示例程序”所示。并将源程序加入系统中。

③ 构造 (Build) 源程序。按照 1.2.2 节第 (5) 小节所示方法编译源程序。

④ 执行仿真。点击主窗口下方的仿真控制按钮 “”，并点击图 2.1 中的 DIP 开关 DSW1 中的各开关键，使之在 “ON” 和 “OFF” 之间切换，并观察图中 D1-D8 发光二极管的亮灭情况。

⑤ 程序调试。点击主窗口下的暂停 (Pause) 仿真控制按钮 “”，点击 “Debug” 菜单下的 “3. 8051 CPU Registers - U1” 子菜单，如下图 3.2 所示，将出现图 3.3 所示的寄存器窗口，

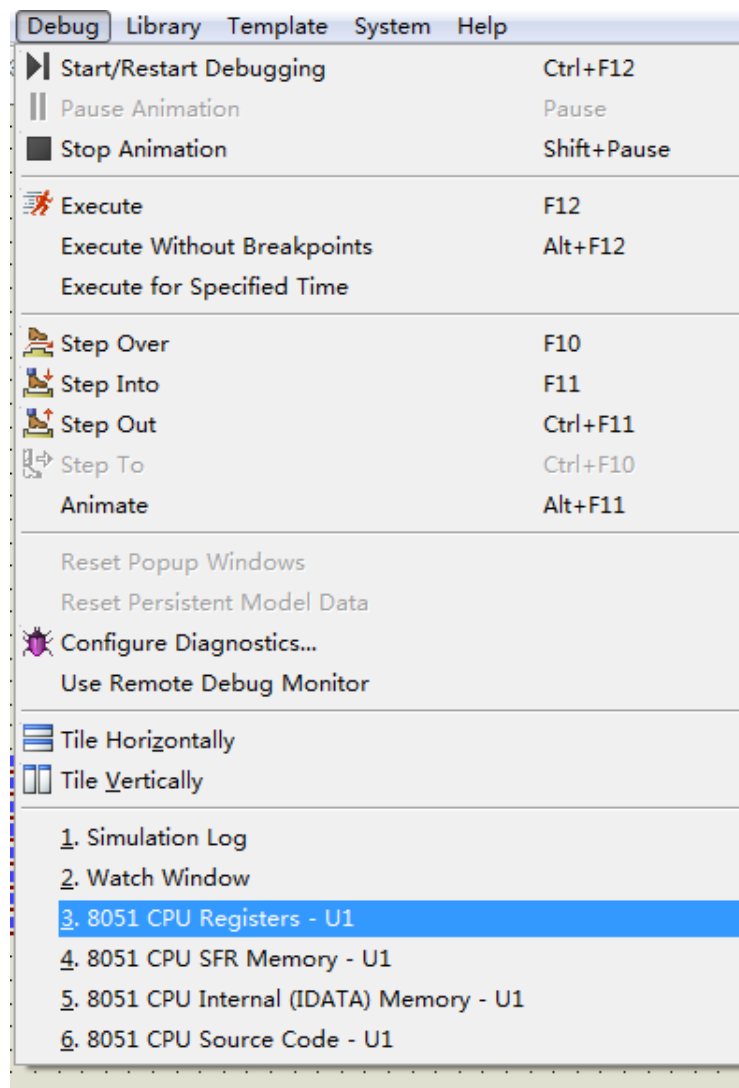


图 3.2 Proteus 中的 Debug 菜单

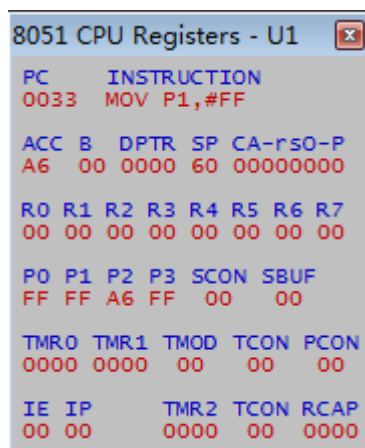


图 3.3 Proteus 中的 CPU 寄存器窗口

然后再点击图 3.2 中 “Debug” 菜单下的 “6. 8051 CPU Source Code - U1 ” 子菜单，将弹出源代码调试窗口，如下图 3.4 所示：

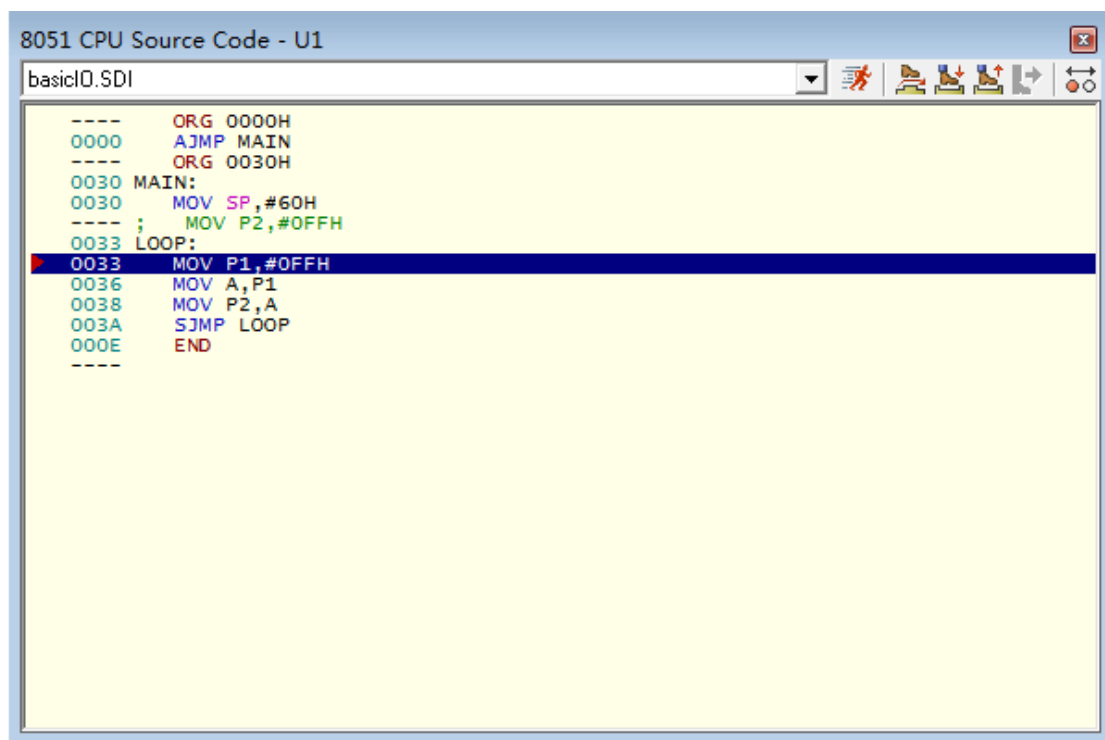


图 3.4 Proteus 中的代码调试窗口

该窗口右上角的“”为“Run Simulation”即全速运行按钮，“”为“Step Over”按钮，“”为“Step Into”按钮，“”为“Step Out”按钮，“”为“Toggle Breakpoint”按钮。

利用这些按钮可以对程序进行调适，并可通过寄存器窗口观察程序运行结果。

(4) 实验作业

- 为示例程序源程序添加注释
- 写出示例程序的功能。
- 记录示例程序运行结果截图。
- 总结 Proteus 单片机系统仿真基本流程。

3.1.2 自我完成实验

(1) 实验内容

在 Proteus 环境下搭建如下图 3.5 所示的电路图：

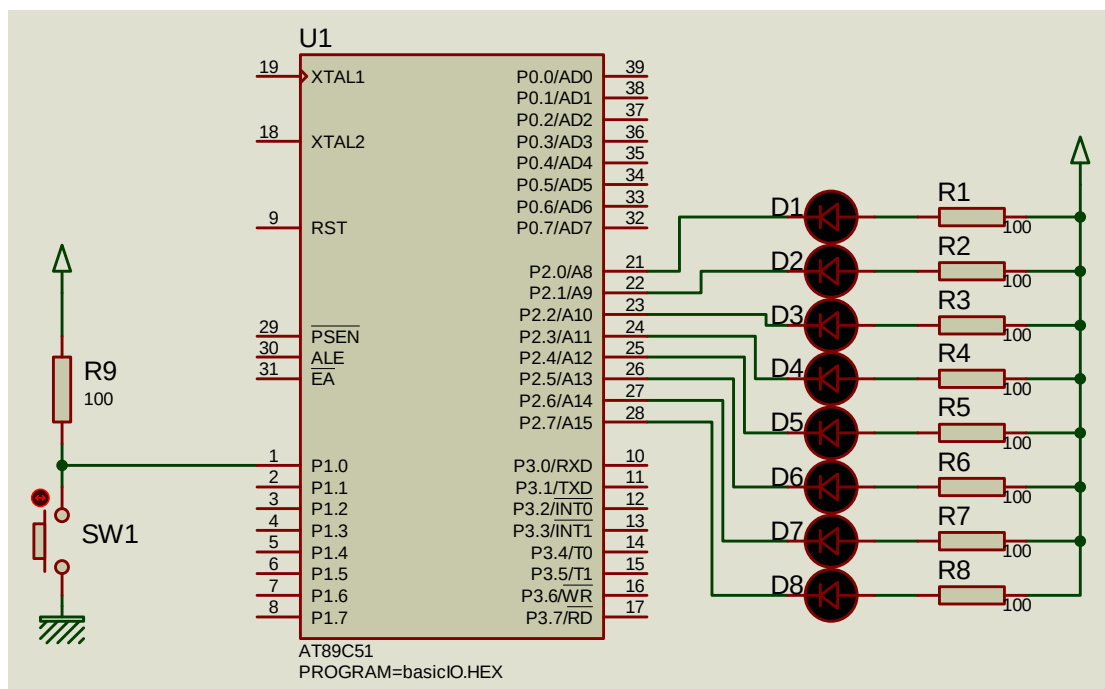


图 3.5 basicIO 原理图 2

其中所用元器件如下表所示：

选件编号	元件名称	参数	所在元件库类名	子类名	生产厂家
U1	AT89C51		Microprocessor ICs 微处理器	8051 Family 8051 家族	ATMEL
SW1	BUTTON		Switches & Relays 开关与继电器	Switches 开关	
D1-D8	LED-RED		Optoelectronics 光电子器件	LEDs 发光二极管	
R1-R9	RES	100	Resistors 电阻	Generic 一般的	

实验功能为：

当按键 SW1 按下之后，D1-D8 轮流点亮，点亮时间为 100ms，当按键停下后，停止轮换，再次按下后继续轮换。

(2) 编程思路

①进行初始化工作，包括设置堆栈指针 SP，将 P2 口所有位设置为 1，使 P2 口所接发光二极管全部熄灭。将显示缓冲单元（设为 20H 单元）初始化为 FEH。

② 从 P1 口读数据，查看 P1.0 位，如果 P1.0 位为 0，则执行如下循环：将显示缓冲单元的值送给 P2 口，调用 100ms 延时程序，将显示缓冲单元的值循环左移 1 位，再送回显示缓冲单元。如果 P1.0 位不为 0 则不执行上述循环。

③重复上面的操作②

(3) 程序流程图

根据上述编程思想，请同学自己画出流程图。

(4) 实验步骤

- ① 根据上述实验内容，参考 1.2.2，在 Proteus 环境下建立图 3.5 所示原理图，并将其保存为 basicIO_self.DSN 文件。
- ② 根据 (2) 和 (3) 编写控制源程序，将其保存为 basicIO_self.asm。
- ③ 将源程序添加到 U1 中，并构造 (build) 该程序。
- ④ 执行仿真过程观察 D1-D8 的指示，查看程序功能是否正确。
- ⑤ 修改延时程序延时参数，重新执行③和④。

(5) 实验作业

- 画流程图
- 编写源程序并进行注释
- 记录实验过程
- 记录程序运行结果截图

3.2 实验 2：扩展并行 I/O 口实验

3.2.1 示例实验

(1) 实验内容

在 Proteus 环境下搭建如下图 3.6 所示的电路图：

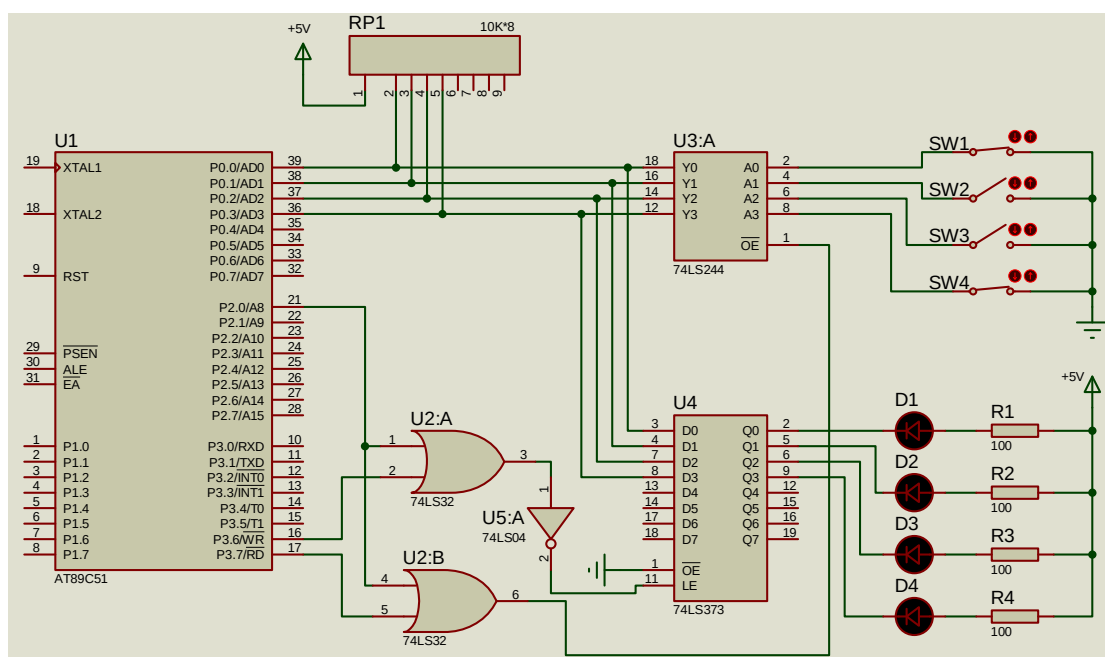


图 3.6 ExpandIO 原理图

其中所用元器件如下表所示：

选件编号	元件名称	参数	所在元件库类名	子类名	生产厂家
U1	AT89C51		Microprocessor ICs 微处理器	8051 Family 8051 家族	ATMEL
U2	74LS32		TTL 74LS series TTL 74LS 系列	Gates& Inverters 门与反相器	
U3	74LS244		TTL 74LS series TTL 74LS 系列	Buffers & Drivers 缓冲与驱动	
U4	74LS373		TTL 74LS series TTL 74LS 系列	Flip-Flops & Latches 触发与锁存	
U5	74LS04		TTL 74LS series TTL 74LS 系列	Gates& Inverters 门与反相器	
RP1	RESPACK-8	10k*8	Resistors 电阻	Resistor Packs 排阻	
SW1-SW4	SWITCH		Switches & Relays 开关与继电器	Switches 开关	
D1-D4	LED-RED		Optoelectronics 光电子器件	LEDs 发光二极管	
R1-R4	RES	100	Resistors 电阻	Generic 一般的	

(2) 示例程序

```

ORG 0000H
AJMP MAIN
ORG 0030H
MAIN:
MOV SP, #60H
MOV DPTR, #0FE00H
LOOP:
MOVX A, @DPTR
MOVX @DPTR, A
SJMP LOOP
END

```

(3) 实验步骤

① 按照 1.2.2 节 (1) – (4) 小节所介绍的方法在 Proteus 环境下建立原理图，并保存为 expandIO.DSN 文件。

②输入源程序，按照 1.2.2 节第（5）小节所示方法建立源程序 expandI0.asm，程序内容如上面的“（2）示例程序”所示。并将源程序加入系统中。

③构造（Build）源程序。按照 1.2.2 节第（5）小节所示方法编译源程序。

④执行仿真。点击主窗口下方的仿真控制按钮“”，并点击图 3.6 中的开关 SW1-SW4 中的各开关，使之在“开”和“关”之间切换，并观察图中 D1-D4 发光二极管的亮灭情况，分析程序功能。

（4）实验作业

- 为示例程序源程序添加注释
- 写出示例程序的功能。
- 记录示例程序运行结果截图。

3.2.2 自我完成实验

（1）实验内容

在 Proteus 环境下搭建如下图 3.7 所示的电路图：

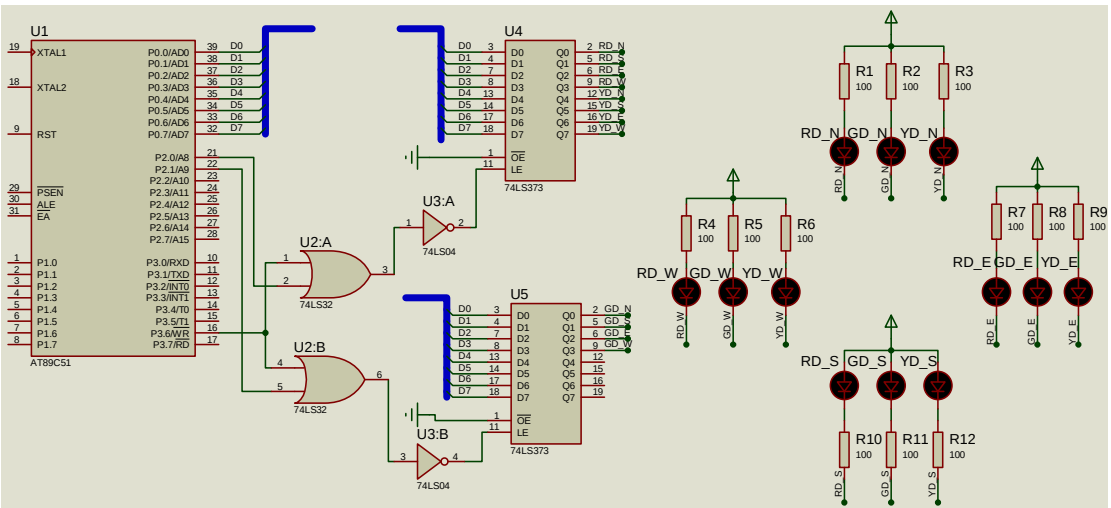


图 3.7 ExpandI02 原理图

其中所用元器件如下表所示：

选件编号	元件名称	参数	所在元件库类名	子类名	生产厂家
U1	AT89C51		Microprocessor ICs 微处理器	8051 Family 8051 家族	ATMEL
U2	74LS32		TTL 74LS series TTL 74LS 系列	Gates& Inverters 门与反相器	
U3	74LS04		TTL 74LS series TTL 74LS 系列	Gates& Inverters 门与反相器	
U4, U5	74LS373		TTL 74LS series TTL 74LS 系列	Flip-Flops & Latches 触发与锁存	
RD_N, RD_S, R	LED-RED		Optoelectronics	LEDs	

D_E, RD_W			光电子器件	发光二极管	
YD_N, YD_S, YD_E, YD_W	LED-YELLOW		Optoelectronics 光电子器件	LEDs 发光二极管	
GD_N, GD_S, GD_E, GD_W	LED-GREEN		Optoelectronics 光电子器件	LEDs 发光二极管	
R1-R12	RES	100	Resistors 电阻	Generic 一般的	

实验功能为：

仿真实现交通信号灯控制功能。

控制顺序为：

- ① 南北绿灯亮，同时东西红灯亮 10s；
- ② 南北黄灯亮，同时东西红灯亮 2s；
- ③ 南北红灯亮，同时东西绿灯亮 10s；
- ④ 东西黄灯亮，同时南北红灯亮 2s；
- ⑤ 重复①~④。

(2) 编程思路

①进行初始化工作，包括设置堆栈指针 SP，将两个 373 的输出口所有位均设置为 1，使所有发光二极管全部熄灭。

②分析两个 373 的地址：

假定所有无关地址均定义为 1，那么 U4 的锁存地址为：#0FE00H，U5 的锁存地址为：#0FD00H。

③分析 4 个状态下两个 373 的输出数据值：

假定“南北绿灯亮，同时东西红灯亮”为状态 1，即：Stat1；“南北黄灯亮，同时东西红灯亮”为状态 2，即：Stat2；“南北红灯亮，同时东西绿灯亮”为状态 3，即：Stat3；“东西黄灯亮，同时南北红灯亮”为状态 4，即：Stat4。

根据图 3.7 所示，

◆状态 1 下 U4 和 U5 的输出应为：

U4								
状态	D7	D6	D5	D4	D3	D2	D1	D0
东西红灯亮	YD_W	YD_E	YD_S	YD_N	RD_W	RD_E	RD_S	RD_N
0F3H	1	1	1	1	0	0	1	1
U5								
状态	D7	D6	D5	D4	D3	D2	D1	D0
南北绿灯亮	X	X	X	X	GD_W	GD_E	GD_S	GD_N
0CH	0	0	0	0	1	1	0	0

备注：表中信号标号“Y”为“Yellow”黄色，“R”为“Red”红色，“G”为“Green”绿色，“W”为“West”西面，“E”为“East”东面，“S”为“South”南面，“N”为“North”北面，“D”为“Diode”二极管。这些信号端低电平时所连接的发光二极管将被点亮。

◆状态 2 下 U4 和 U5 的输出应为：

U4								
状态	D7	D6	D5	D4	D3	D2	D1	D0
南北黄灯亮， 东西红灯亮	YD_W	YD_E	YD_S	YD_N	RD_W	RD_E	RD_S	RD_N
0C3H	1	1	0	0	0	0	1	1
U5								
状态	D7	D6	D5	D4	D3	D2	D1	D0
绿灯全灭	X	X	X	X	GD_W	GD_E	GD_S	GD_N
0FH	0	0	0	0	1	1	1	1

◆状态 3 下 U4 和 U5 的输出应为：

U4								
状态	D7	D6	D5	D4	D3	D2	D1	D0
南北红灯亮	YD_W	YD_E	YD_S	YD_N	RD_W	RD_E	RD_S	RD_N
0FCH	1	1	1	1	1	1	0	0
U5								
状态	D7	D6	D5	D4	D3	D2	D1	D0
东西绿灯亮	X	X	X	X	GD_W	GD_E	GD_S	GD_N
03H	0	0	0	0	0	0	1	1

◆状态 4 下 U4 和 U5 的输出应为：

U4								
状态	D7	D6	D5	D4	D3	D2	D1	D0
东西黄灯亮， 南北红灯亮	YD_W	YD_E	YD_S	YD_N	RD_W	RD_E	RD_S	RD_N
3CH	0	0	1	1	1	1	0	0
U5								

状态	D7	D6	D5	D4	D3	D2	D1	D0
绿灯全灭	X	X	X	X	GD_W	GD_E	GD_S	GD_N
0FH	0	0	0	0	1	1	1	1

④编写延时 1s 的程序，并按功能要求进行状态切换。

(3) 程序流程图

根据上述编程思想，请同学自己画出流程图。

(4) 实验步骤

① 根据上述实验内容，参考 1.2.2，在 Proteus 环境下建立图 3.7 所示原理图，并将其保存为 expandI0_self.DSN 文件。

② 根据 (2) 和 (3) 编写控制源程序，将其保存为 expandI0_self.asm。

③ 将源程序添加到 U1 中，并构造 (build) 该程序。

④ 执行仿真过程观察各个方向的交通信号灯指示，查看程序功能是否正确。

(5) 实验作业

- 画出流程图
- 编写源程序并进行注释
- 实验过程记录
- 记录程序运行结果截图

3.3 实验 3：静态 LED 显示实验

3.3.1 示例实验

(1) 实验内容

在 Proteus 环境下搭建如下图 3.8 所示的电路图：

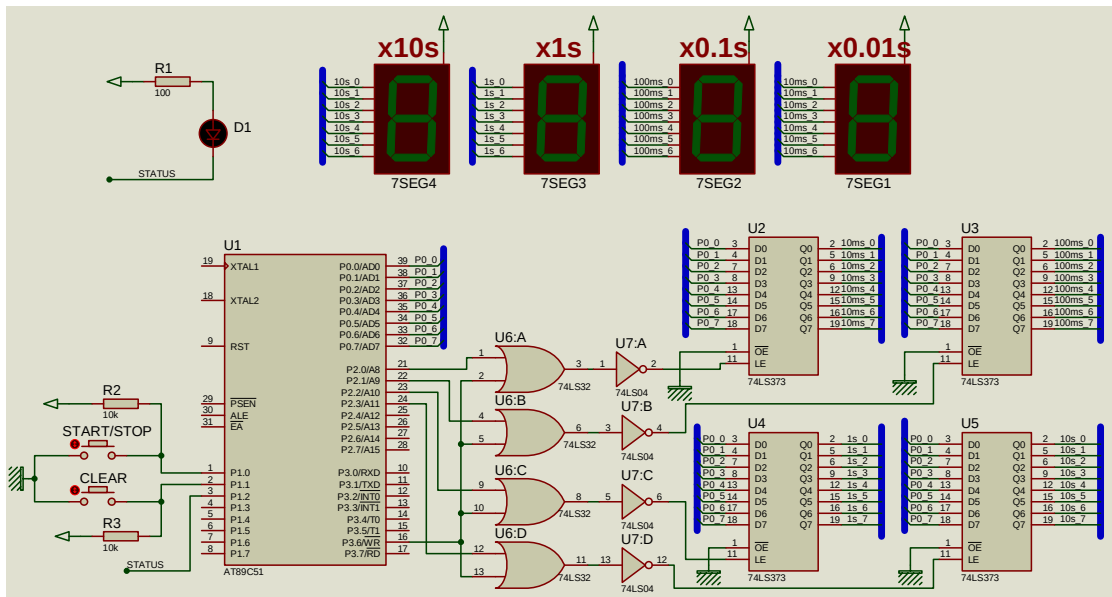


图 3.8 staticLED 原理图

其中所用元器件如下表所示：

选件编号	元件名称	参数	所在元件库类名	子类名	生产厂家
U1	AT89C51		Microprocessor ICs 微处理器	8051 Family 8051 家族	ATMEL
U2-U5	74LS373		TTL 74LS series TTL 74LS 系列	Flip-Flops & Latches 触发与锁存	
U6	74LS32		TTL 74LS series TTL 74LS 系列	Gates& Inverters 门与反相器	
U7	74LS04		TTL 74LS series TTL 74LS 系列	Gates& Inverters 门与反相器	
U8	74LS08		TTL 74LS series TTL 74LS 系列	Gates& Inverters 门与反相器	
7SEG1-7SEG4	7SEG-COM-AN-GRN		Optoelectronics 光电子器件	7-Segment Displays 7 段显示	
D1	LED-GREEN		Optoelectronics 光电子器件	LEDs 发光二极管	
R1	RES	100	Resistors	Generic	

			电阻	一般的	
R2, R3	RES	10k	Resistors 电阻	Generic 一般的	
START/STOP, CLEAR	BUTTON		Switches & Relays 开关与继电器	Switches 开关	

(2) 功能要求

这里要实现一个秒表计时器，最小计时单位为 0.01s。图 3.8 中有 4 个 7 段数码管 7SEG1-7SEG4，其中 7SEG4 用于显示“x10s”，7SEG3 用于显示“x1s”，7SEG2 用于显示“x0.1s”，7SEG1 用于显示“x0.01s”。这四个数码管的显示数据通过 4 个 373 锁存器（U2-U5）提供。两个按键分别是起停按钮“START/STOP”和清零按钮“CLEAR”。当点击起停按钮时用于启动秒表或者暂停秒表。当点击清零按钮时则将秒表清零。图中发光二极管 LED1 用于指示秒表的“起/停”状态，当秒表正在运行时，该指示灯点亮，秒表停止时，该指示灯熄灭。

(3) 示例程序

```

ORG 0000H
AJMP MAIN
ORG 0030H
MAIN:
MOV SP, #60H ;初始化
CLR F0
SETB P1.2
LCALL DISCLR
LOOP:
SETB P1.1
JB P1.1, GOON
WAIT0:
SETB P1.1
JNB P1.1, WAIT0 ;等待 CLEAR 按键释放
LCALL DISCLR
CLR F0
GOON:
SETB P1.0
JB P1.0, NEXT1
WAIT1:
SETB P1.0 ;等待 Start/Stop 按键释放
JNB P1.0, WAIT1
CPL F0
NEXT1:

```

```

JNB F0, NEXT
CLR P1.2    ;执行秒表计时功能
LCALL DISPLAY
LCALL DELAY10ms
LCALL ADJUST
SJMP LOOP
NEXT:
SETB P1.2
SJMP LOOP
DELAY10ms:    ;10ms 延时程序
DL0:MOV R2, #100
DL1:MOV R1, #48
DL2:DJNZ R1, DL2    ;内循环 100us
NOP
DJNZ R2, DL1    ;中循环 10ms
RET
ADJUST:    ;秒表计数器调整子程序
INC 30H
MOV A, 30H
CJNE A, #10, GOTORET
MOV 30H, #0
INC 31H
MOV A, 31H
CJNE A, #10, GOTORET
MOV 31H, #0
INC 32H
MOV A, 32H
CJNE A, #10, GOTORET
MOV 32H, #0
INC 33H
MOV A, 33H
CJNE A, #10, GOTORET
MOV 33H, #0
GOTORET:RET
DISCLR:    ;显示清零
MOV 30H, #0
MOV 31H, #0

```

```

MOV 32H, #0
MOV 33H, #0
CLR A
MOV DPTR, #TABLE
MOVC A, @A+DPTR
MOV DPTR, #D10msADD
MOVX @DPTR, A
MOV DPTR, #D100msADD
MOVX @DPTR, A
MOV DPTR, #D1sADD
MOVX @DPTR, A
MOV DPTR, #D10sADD
MOVX @DPTR, A
SETB P1.2
RET

```

DISPLAY: ;显示子程序

```

MOV A, 30H
MOV DPTR, #TABLE
MOVC A, @A+DPTR
MOV DPTR, #D10msADD
MOVX @DPTR, A
MOV A, 31H
MOV DPTR, #TABLE
MOVC A, @A+DPTR
MOV DPTR, #D100msADD
MOVX @DPTR, A
MOV A, 32H
MOV DPTR, #TABLE
MOVC A, @A+DPTR
MOV DPTR, #D1sADD
MOVX @DPTR, A
MOV A, 33H
MOV DPTR, #TABLE
MOVC A, @A+DPTR
MOV DPTR, #D10sADD
MOVX @DPTR, A
RET

```

TABLE: DB 0C0H, 0F9H, 0A4H, 0B0H, 99H, 92H, 82H, 0F8H, 80H, 90H ;7 段 LED 共阳显示码

D10msADD EQU 0FE00H ;x0.01s 显示锁存地址

D100msADD EQU 0FD00H ;x0.1s 显示锁存地址

D1sADD EQU 0FB00H ;x1s 显示锁存地址

D10sADD EQU 0F700H ;x10s 显示锁存地址

END

(4) 实验步骤

① 按照 1.2.2 节 (1) – (4) 小节所介绍的方法在 Proteus 环境下建立原理图，并保存为 staticLED.DSN 文件。

② 输入源程序，按照 1.2.2 节第 (5) 小节所示方法建立源程序 staticLED.asm，程序内容如上面的“(2) 示例程序”所示。并将源程序加入系统中。

③ 构造 (Build) 源程序。按照 1.2.2 节第 (5) 小节所示方法编译源程序。

④ 执行仿真。点击主窗口下方的仿真控制按钮“”，观察 4 个 7 段数码管的显示情况。点击按下图 3.8 中的按键 START/STOP，观察程序运行中 4 个 7 段数码管的显示及 LED1 的亮灭情况，再次点击按键 KEY_START，观察程序运行中 4 个 7 段数码管的显示及 LED1 的亮灭情况。点击图 3.8 中的按键 CLEAR，并观察图中 4 个 7 段数码管的显示及 LED1 的亮灭情况情况。

(4) 实验作业

- 分析源程序并为源程序添加注释
- 分析示例程序中 30H-33H 单元的功能
- 记录示例程序运行结果截图。
- 总结 51 单片机汇编语言程序设计中的模块化编程方法

3.3.2 自我完成实验

(1) 实验内容

在 Proteus 环境下搭建如下图 3.9 所示的电路图：

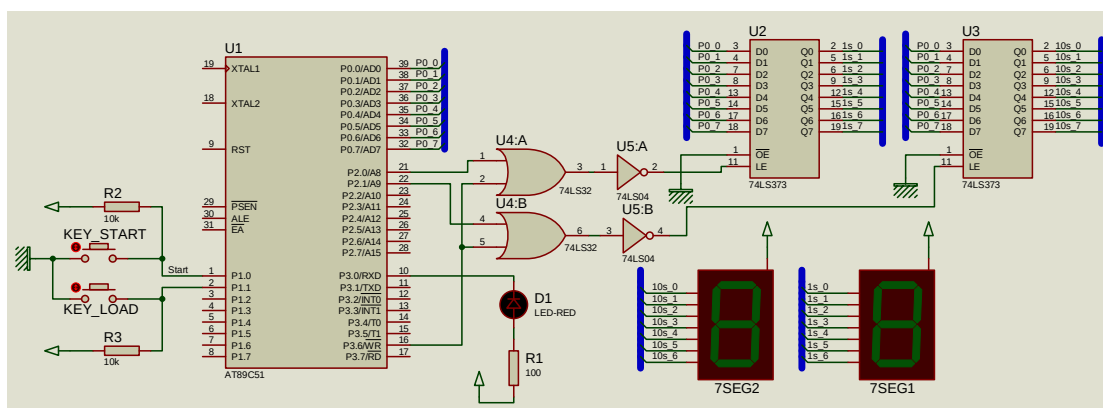


图 3.9 倒计时器原理图

其中所用元器件如下表所示：

选件编号	元件名称	参数	所在元件库类名	子类名	生产厂家
U1	AT89C51		Microprocessor ICs 微处理器	8051 Family 8051 家族	ATMEL
U2, U3	74LS373		TTL 74LS series TTL 74LS 系列	Flip-Flops & Latches 触发与锁存	
U4	74LS32		TTL 74LS series TTL 74LS 系列	Gates& Inverters 门与反相器	
U5	74LS04		TTL 74LS series TTL 74LS 系列	Gates& Inverters 门与反相器	
7SEG1, 7SEG2	7SEG-COM-AN-GRN		Optoelectronics 光电子器件	7-Segment Displays 7 段显示	
D1	LED-RED		Optoelectronics 光电子器件	LEDs 发光二极管	
R1	RES	100	Resistors 电阻	Generic 一般的	
R2, R3	RES	10k	Resistors 电阻	Generic 一般的	
KEY_START, KEY_LOAD	BUTTON		Switches & Relays 开关与继电器	Switches 开关	

(2) 控制要求

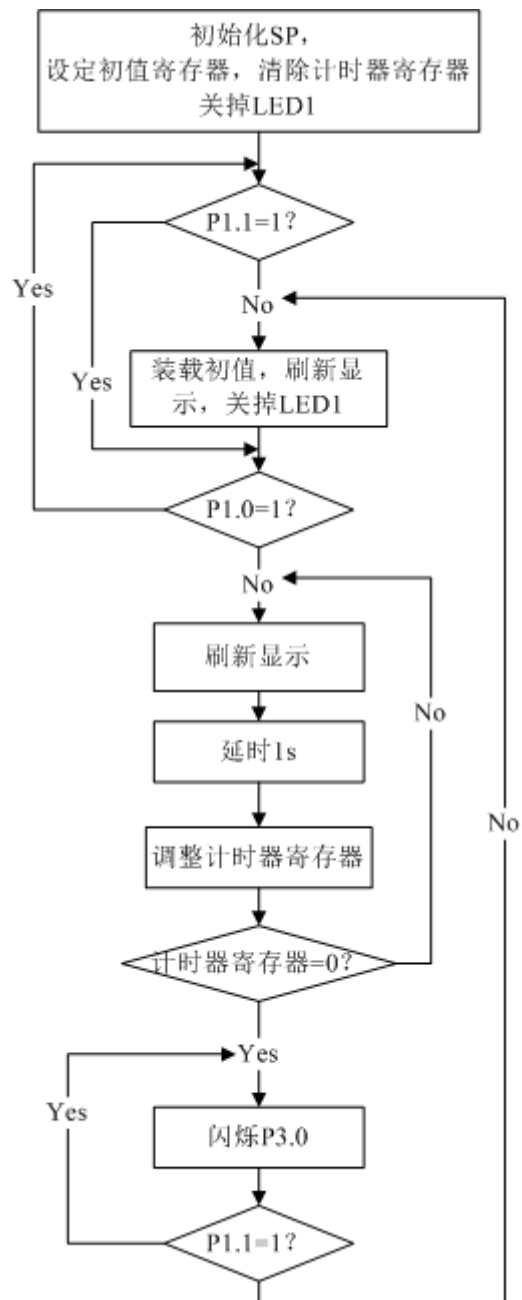
图 3.9 中 7SEG2 为十位显示数码管，7SEG1 为个位显示数码管，KEY_LOAD 为倒计时初值按钮，KEY_START 为倒计时启动按钮。要求实现的功能是：当 KEY_LOAD 按钮按下时加载倒计时初值（如：10s），当按下 KEY_START 按钮时，开始倒计时，每过 1s，计时器减 1，直到减到“00”为止。减到“00”时使 P3.0 引脚上的 LED 按 10Hz 频率进行闪烁，直到再次按下 KEY_LOAD 按钮才重新加载初值，并熄灭 LED。再次按下 KEY_START 按钮又一次开始倒计时，如此反复。

(3) 编程思路

①分析两个 373 的地址：

假定所有无关地址均定义为 1，那么 U2 的锁存地址为：#0FE00H，U3 的锁存地址为：#0FD00H。

②程序流程图：



(4) 实验步骤

- ① 根据上述实验内容,参考 1.2.2,在 Proteus 环境下建立图 3.9 所示原理图,并将其保存为 staticLED_self.DSN 文件。
- ② 根据 (2) 和 (3) 编写控制源程序,将其保存为 staticLED_self.asm。
- ③ 将源程序添加到 U1 中,并构造 (build) 该程序。
- ④ 执行仿真过程观察秒表程序功能是否正确。

② 根据 (2) 和 (3) 编写控制源程序, 将其保存为 staticLED_self.asm。

③ 将源程序添加到 U1 中，并构造 (build) 该程序。

④ 执行仿真过程观察秒表程序功能是否正确。

(5) 实验作业

- 编写源程序并进行注释
- 实验过程记录
- 记录程序运行结果截图

3.4 实验 4：矩阵键盘扫描实验

3.4.1 示例实验

(1) 实验内容

在 Proteus 环境下搭建如下图 3.30 所示的电路图：

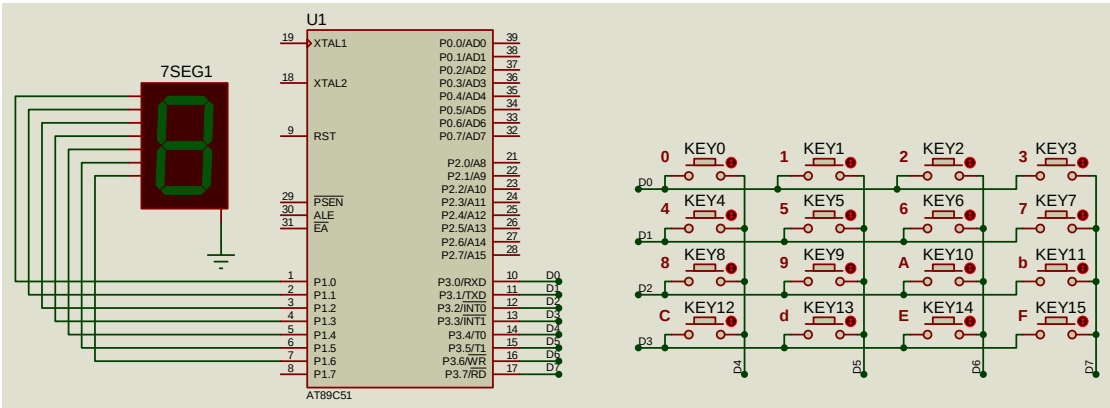


图 3.10 键盘扫描原理图

其中所用元器件如下表所示：

选件编号	元件名称	参数	所在元件库类名	子类名	生产厂家
U1	AT89C51		Microprocessor ICs 微处理器	8051 Family 8051 家族	ATMEL
7SEG1	7SEG-COM-AN-GRN		Optoelectronics 光电子器件	7-Segment Displays 7 段显示	
KEY0-KEY15	BUTTON		Switches & Relays 开关与继电器	Switches 开关	

备注：每个按键旁边的字符“0”～“F”是文本（text）文字，点击左侧绘图工具栏中的“”即可逐个加入，这些文字用于标示按下该键之后数码管将显示的字符。

(2) 功能要求

利用 C 或汇编语言编写程序，实现按键扫描。当点击按键 KEY0 时，7SEG1 数码管将显示字符“0”，当点击按键 KEY1 时，7SEG1 数码管将显示字符“1”，当点击按键 KEY2 时，7SEG1 数码管将显示字符“2”，以此类推，点击哪个按键将显示其旁边的字符。

(3) 示例程序

① C 语言源程序 （保存为 keyscan.c 文件）

```
/******必要的变量定义******/  
  
#include <reg51.h>  
  
#define uint unsigned int  
  
#define uchar unsigned char  
  
uchar code KEY_TABLE[]={ 0x11,0x21,0x41,0x81,0x12,0x22,0x42,0x82,0x14,
```

```

        0x24, 0x44, 0x84, 0x18, 0x28, 0x48, 0x88}; //按键键值表
        //按键值表中前四位与按键所在列有关，后四位与按键所
        //在行有关。四位二进制数中为 1 的那一位所在的位置就
        //是按键所在的行或列的位置。如键值为 0x24，则表示在
        //第 2 行第 1 列；0x82，则表示在第 1 行第 3 列
uchar code TABLE[]={ 0x3f, 0x06, 0x5b, 0x4f, 0x66, 0x6d, 0x7d, 0x07, 0x7f, 0x6f,
        0x77, 0x7c, 0x39, 0x5e, 0x79, 0x71}; //显示码表

/*****长延时子程序，作点亮数码管延时*****/
void delay1()
{
    uint n=50000;while(n--);
}

/*****短延时子程序，作键盘去抖动功能*****/
void delays()
{
    uint n=10000;while(n--);
}

/*****主程序*****/
void main()
{
    uchar temp, key, num, i;
    P1=0x0;
    while(1)
    {
        P3=0xf0;    //行置为 0，列置为 1，读列值
        if(P3!=0xf0)
        {
            delays();    //去抖动
            P3=0xf0;
            if(P3!=0xf0)
            {
                temp=P3;
                P3=0x0f;
                key=temp|P3;
            }
        }
    }
}

```

```

        key=0xff-key; //按位取反
        for (i=0;i<16;i++)
        {
            if(key==KEY_TABLE[i])
            {
                num=i;
                break;
            }
        }
        P1=TABLE[num];
        delay1();
    }
}
}
}

```

② 汇编语言源程序（保存为 keyscan.asm 文件）

```

ORG 0000H
AJMP MAIN
ORG 0030H
MAIN:
    MOV SP, #60H
    MOV P1, #00H
LOOP:
    MOV 30H, #0 ;存放有无按键按下标志
    MOV 31H, #0 ;存放键码
    LCALL KEYSCAN
    MOV A, 30H
    JZ LOOP
    MOV 32H, #0 ;存放按键是否错误，1 表示有错误，0 表示无错误
    MOV 33H, #0 ;存放键号
    LCALL SEARCHKEYNUM
    MOV A, 32H
    JNZ LOOP
    MOV DPTR, #TABLE2
    MOV A, 33H
    MOVC A, @A+DPTR

```

```

MOV P1, A
LJMP LOOP
KEYSCAN:
    MOV P3, #0F0H
    MOV A, P3
    ANL A, #0F0H
    CJNE A, #0F0H, ASKEY
    SJMP KEYSCAN
ASKEY:
    LCALL DELAY10ms
    MOV P3, #0F0H
    MOV A, P3
    ANL A, #0F0H
    CJNE A, #0F0H, ISKEY
    MOV 31H, #0
    MOV 30H, #0
    LJMP TORET1
ISKEY:
    MOV 31H, A
SEARCHLINE0:
    MOV P3, #0FEH
    MOV A, P3
    ANL A, #0F0H
    CLR C
    SUBB A, #0F0H
    JZ SEARCHLINE1
    MOV A, #0EH
    LJMP TOSTORE
SEARCHLINE1:
    MOV P3, #0FDH
    MOV A, P3
    ANL A, #0F0H
    CLR C
    SUBB A, #0F0H
    JZ SEARCHLINE2
    MOV A, #0DH
    LJMP TOSTORE

```

SEARCHLINE2:

```
MOV P3, #0FBH
MOV A, P3
ANL A, #0F0H
CLR C
SUBB A, #0F0H
JZ SEARCHLINE3
MOV A, #0BH
LJMP TOSTORE
```

SEARCHLINE3:

```
MOV P3, #0F7H
MOV A, P3
ANL A, #0F0H
CLR C
SUBB A, #0F0H
JZ WRONGKEY
MOV A, #07H
LJMP TOSTORE
```

WRONGKEY:

```
MOV 31H, #0
MOV 30H, #0
SJMP TORET1
```

TOSTORE:

```
ORL A, 31H
CPL A
MOV 31H, A
MOV 30H, #1
```

TORET1:

```
RET
```

SEARCHKEYNUM:

```
MOV DPTR, #TABLE1
MOV A, 33H
MOVC A, @A+DPTR
CLR C
SUBB A, 31H
JZ ENDSEARCH
INC 33H
```

```

MOV A, 33H
CJNE A, #10H, SEARCHKEYNUM
MOV 32H, #1 ;没有找到相同的键码, 说明是错误按键, 如: 按双键
SJMP T0RET2
ENDSEARCH:
MOV 32H, #0
T0RET2:
RET
DELAY10ms: ;10ms 延时程序
DL0:MOV R2, #100
DL1:MOV R1, #48
DL2:DJNZ R1, DL2 ;内循环 100us
NOP
DJNZ R2, DL1 ;中循环 10ms
RET
TABLE1: DB 11H, 21H, 41H, 81H, 12H, 22H, 42H, 82H, 14H, 24H, 44H, 84H, 18H, 28H, 48H, 88H
TABLE2: DB 3FH, 06H, 5BH, 4FH, 66H, 6DH, 7DH, 07H, 7FH, 6FH, 77H, 7CH, 39H, 5EH, 79H, 71H
END

```

(4) 实验步骤

- ① 根据上述实验内容, 参考 1.2.2, 在 Proteus 环境下建立图 3.9 所示原理图, 并将其保存为 Keyscan.DSN 文件。
- ② 运行 Keil uVision2 开发环境, 按照 1.1.3 节介绍的方法建立工程 keyscan_c.uv2, CPU 为 AT89C51, 包含启动文件 STARTUP.A51。
- ③ 按照 1.2.2 第 (6) 节介绍的方法将前面示例的 C 语言的源程序 keyscan.c 加入工程 keyscan_c.uv2, 并设置工程 keyscan_c.uv2 属性, 将其晶振频率设置为 12MHz, 选择输出可执行文件, 仿真方式为选择硬仿真, 并选择其中的“PROTEUS VSM MONITOR 51 DRIVER”仿真器。
- ④ 构造 (Build) 工程 keyscan_c.uv2。如果输入有误进行修改, 直至构造正确, 生成可执行程序 keyscan_c.hex 为止。
- ⑤ 为 AT89C51 设置可执行程序 keyscan_c.hex。
- ⑥ 运行程序, 点击图 3.10 中各按键, 观察数码管的显示与按键的关系是否符合程序要求。
- ⑦ 重新执行②~⑥, 产生汇编程序的工程和可执行文件。这时新建立的工程文件命名为 keyscan_asm.uv2, 设置后加入的是汇编源程序 keyscan.asm。另外, 不需包含启动文件 STARTUP.A51。为 AT89C51 设置可执行程序是 keyscan_asm.hex。

(5) 实验作业

- 分析源程序并为源程序添加注释

- 分析 C 语言源程序和汇编源程序中分别用了什么方法识别键盘？
- 记录示例程序运行结果截图。

3.4.2 自我完成实验

(1) 实验内容

在 Proteus 环境下搭建如下图 3.11 所示的电路图：

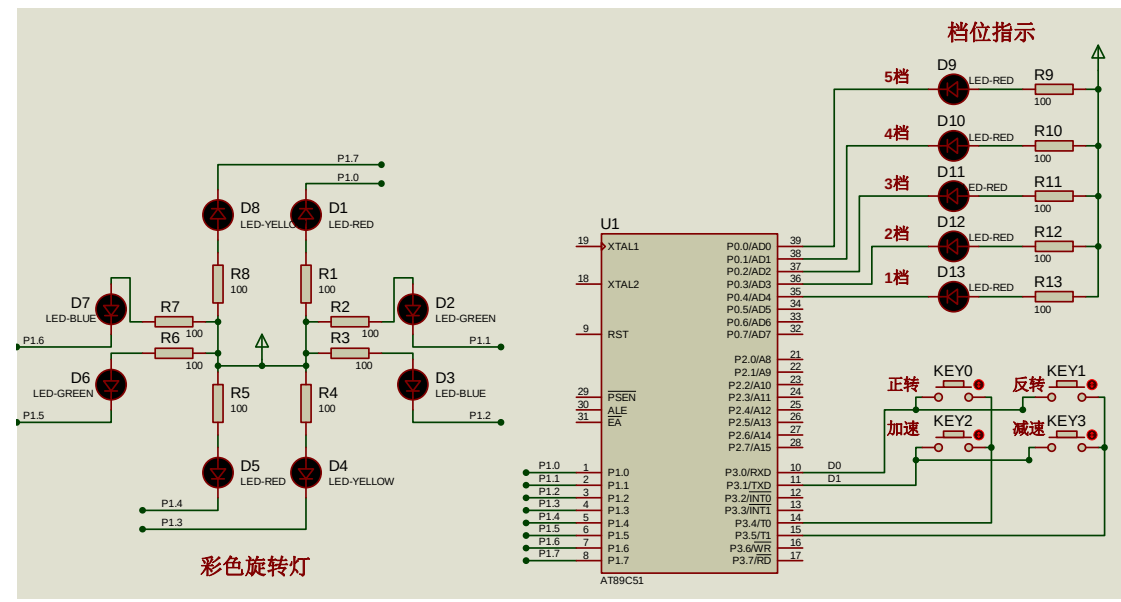


图 3.11 旋转彩灯控制原理图

其中所用元器件如下表所示：

选件编号	元件名称	参数	所在元件库类名	子类名	生产厂家
U1	AT89C51		Microprocessor ICs 微处理器	8051 Family 8051 家族	ATMEL
D1, D5, D9-D13	LED-RED		Optoelectronics 光电子器件	LEDs 发光二极 管	
D2, D6	LED-GREEN		Optoelectronics 光电子器件	LEDs 发光二极 管	
D3, D7	LED-BLUE		Optoelectronics 光电子器件	LEDs 发光二极 管	
D4, D8	LED-YELLOW		Optoelectronics 光电子器件	LEDs 发光二极 管	
R1-R13	RES	100	Resistors 电阻	Generic 一般的	
KEY0-KEY3	BUTTON		Switches & Relays	Switches	

			开关与继电器	开关	
--	--	--	--------	----	--

(2) 控制要求

图 3.11 中 D1~D8 八个发光二极管构成彩色旋转灯，D9~D13 为档位指示灯，一档旋转速度最慢（周期 1s，D13 亮），二档较快（周期 0.8s，D12 亮），三档更快（周期 0.6s，D11 亮），四档再快（周期 0.4s，D10 亮），五档最快（周期 0.2s，D10 亮）。四个按键 KEY0~KEY1 用于设定旋转方向为顺时针旋转或者逆时针旋转，KEY2~KEY3 用于增快或则减慢旋转速度。

(3) 编程思路

按键扫描的方式可以采用前面示例程序中的方法：线反转法或行扫描法。可以用汇编语言实现，也可以用 C 语言实现。建议如前面示例所示，汇编采用行扫描法，C 语言用线反转法。

程序控制流程是：首先初始化设置默认运行参数，然后读取按键，识别键码，并根据键码的不同执行运行参数调整，最后根据当前的运行参数执行发光二极管 D1~D8 的轮流旋转。

按键的识别中的键码可以根据图 3.11 中的连接情况，总结出其键值表，即：正向按钮为：0x22，反向按钮为：0x12，加速按钮为 0x21，减速按钮为 0x11，因此可以定义按键键码表（C 语言方式）：

```
uchar code KEY_TABLE[]={0x22, 0x12, 0x21, 0x11}; //按键键值表
```

速度的控制通过控制调用延时程序的次数来决定，假设延时程序的延时长度为 5ms。延时程序可以按如下方式实现（假设晶振频率为 12MHz）：

```
void delays()
{
    uchar t,ms;
    ms=5; //延时 5ms
    while(ms-->0) for(t=0;t<120;t++);
}
```

或者采用内嵌汇编来实现：

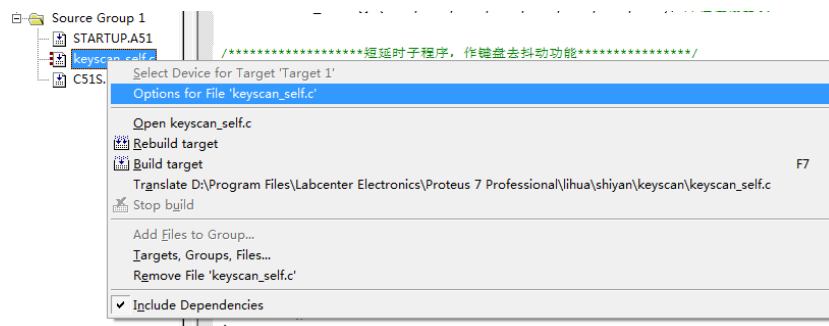
```
void delays()
{
    #pragma asm
        MOV R2, #50 //;5ms 延时程序
    DL1:MOV R1, #48
    DL2:DJNZ R1, DL2 // ;内循环 100us
        NOP
        DJNZ R2, DL1 //;中循环 10ms
    #pragma endasm
}
```

采用内嵌汇编时需要注意的是：

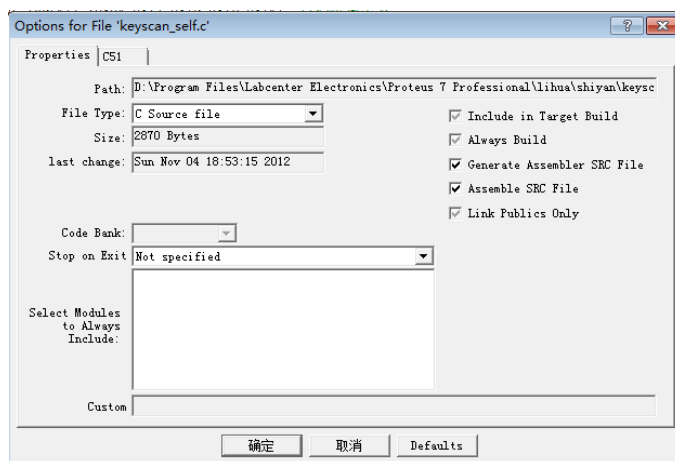
- ① 在工程文件中加入：C51S.lib 库文件（该文件位于：C:\keil\c51\lib 目录下）。

- ② 修改 C 语言选项文件，使其包含两个选项：Generate Assembler SRC File（生成汇编语言 SRC 文件）和 Assemble SRC File（汇编 SRC 文件）。这两个选项通过如下方式加入：

- ◆ 鼠标右键点击 C 语言源程序文件，并点击“Option for File ‘xxx.c’”



- ◆ 在弹出对话框的“Properties”页中将两个选项：Generate Assembler SRC File（生成汇编语言 SRC 文件）和 Assemble SRC File（汇编 SRC 文件）如下图方式选上即可。



调用延时程序次数为 200 次时，周期为 1s；次数为 160 次时，则周期 0.8s；次数为 140 次时，则周期 0.7s；次数为 120 次时，则周期 0.6s；次数为 100 次时，则周期 0.5s；次数为 80 次时，则周期 0.4s；次数为 40 次时，则周期 0.2s。这样周期值表（C 语言方式）为：

```
uchar code T_TABLE[]={200,160,120,80,40}; //周期值表
```

下面给出线反转法 C 语言程序中主程序流程图如下图 3.12 所示：

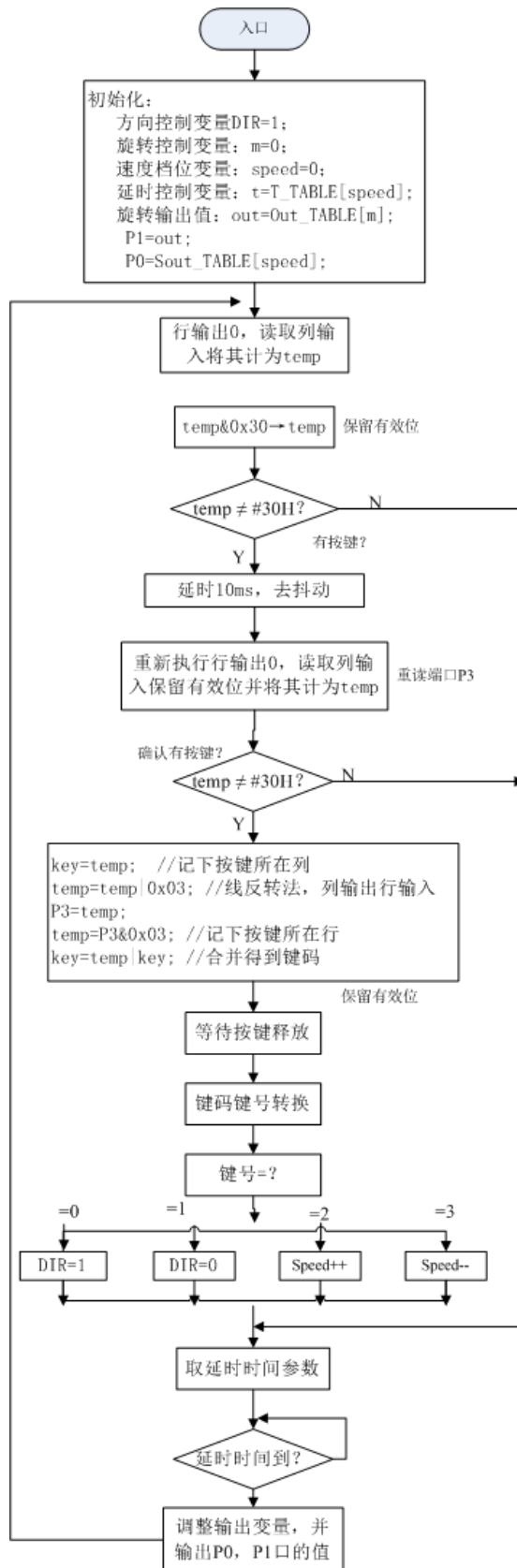
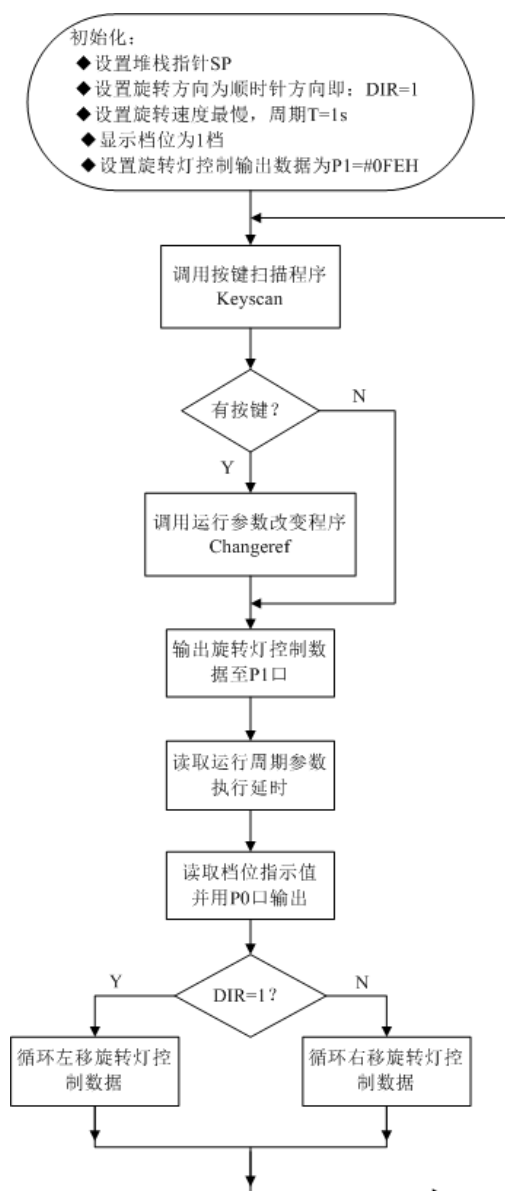
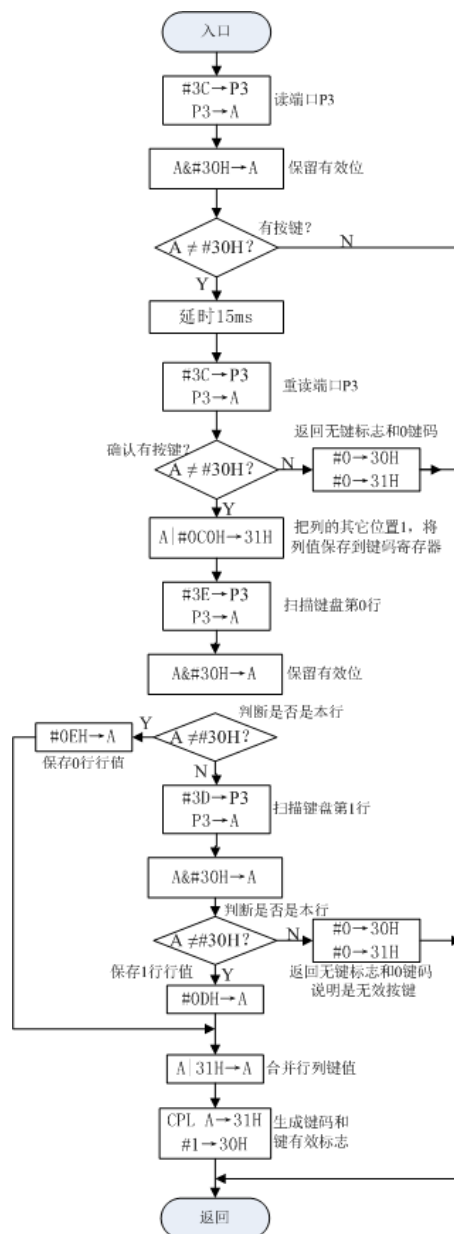


图 3.12 旋转彩灯线反转法 C 语言程序控制流程图

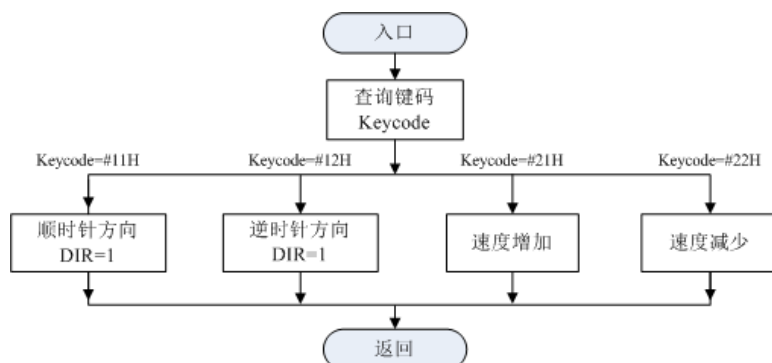
下面给出扫描法汇编程序控制的流程图(如图 3.13 所示)。



(1) 主控流程图



(2) 键盘扫描子程序 Keyscan 流程图



(3) 参数调整子程序 Changeref 流程图

图 3.13 旋转彩灯行扫描法汇编控制流程图

档位输出也可以按查表形式给出，1 档时应给出的档位输出值为 0x0F；2 档时应给出的档位输出值为 0x17；3 档时应给出的档位输出值为 0x1B；4 档时应给出的档位输出值为 0x1D；5 档时应给出的档位输出值为 0x1E。因此档位输出值表为：

```
uchar code Sout_TABLE[]={0x0F, 0x17, 0x1B, 0x1D, 0x1E}; //档位输出表
```

(4) 实验步骤

- ① 根据上述实验内容，参考 1.2.2，在 Proteus 环境下建立图 3.11 所示原理图，并将其保存为 keyscan_self.DSN 文件。
- ② 根据 (2) 和 (3) 编写控制源程序，将其保存为 keyscan_self.asm 或 keyscan_self.c。
- ③ 将源程序添加到 U1 中，并构造 (build) 该程序。
- ④ 执行仿真过程观察秒表程序功能是否正确。

(5) 实验作业

- 编写源程序并进行注释
- 实验过程记录
- 记录程序运行结果截图

3.5 实验 5：定时计数器实验

3.5.1 示例实验

(1) 实验内容

在 Proteus 环境下搭建如图 3.8 所示的电路图。

(2) 功能要求

按键控制功能和实验 3.3.1 一样，但这里的计时功能要用定时器 T0 实现，编程语言采用 C 语言实现。

(3) 示例程序

```
//将如下 C 语言源程序保存为 timer.c
#include<reg51.h>
#define uchar unsigned char
#define uint unsigned int
uchar code LED_Table[]={0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92,
                        0x82, 0xF8, 0x80, 0x90} ;//7 段 LED 共阳显示码
uchar xdata D10ms _at_ 0xfe00; //x0.01s 显示锁存地址
uchar xdata D100ms _at_ 0xfd00; //x0.1s 显示锁存地址
uchar xdata D1s _at_ 0xfb00; //x1s 显示锁存地址
uchar xdata D10s _at_ 0xf700; //x10s 显示锁存地址
```

```

bit start_stop;//起停标志
sbit P10=P1^0;
sbit P11=P1^1;
sbit P12=P1^2;

uchar timerH=0xd8,timerL=0xf0; //12MHz 晶振频率下 10ms 定时，方式 1 下初值

uchar DV10ms; //0.01s 计数值
uchar DV100ms; //0.1s 计数值
uchar DV1s; //1s 计数值
uchar DV10s; //10s 计数值

void clrdisp()
{
    DV10ms=0;
    DV100ms=0;
    DV1s=0;
    DV10s=0;
    D10ms=LED_Table[DV10ms];
    D100ms=LED_Table[DV100ms];
    D1s=LED_Table[DV1s];
    D10s=LED_Table[DV10s];
    //P12=1;
    TH0=timerH;
    TL0=timerL;
}

void s_timer0() interrupt 1 //定时器 0 中断服务程序
{
    DV10ms=DV10ms+1;
    if(DV10ms==10)
    {
        DV10ms=0;
        DV100ms=DV100ms+1;
        if (DV100ms==10)
        {
            DV100ms=0;

```

```

        DV1s=DV1s+1;
        if (DV1s==10)
        {
            DV1s=0;
            DV10s=DV10s+1;
            if (DV10s==10)
            {
                DV10s=0;
            }
        }
    }
    D10ms=LED_Table[DV10ms];
    D100ms=LED_Table[DV100ms];
    D1s=LED_Table[DV1s];
    D10s=LED_Table[DV10s];
    TH0=timerH;
    TL0=timerL;
}

//主程序
void main()
{
    SP=0x60; //设定堆栈指针位置
    TMOD=0x01; //定时/计数器 0 工作于方式 1
    TH0=timerH;
    TL0=timerL;
    start_stop=0;
    P12=1;
    clrdisp();
    while(1)
    {
        P11=1;
        if(!P11)
        {
            while(!P11)
            {

```

```

        P11=1;
    }
    clrdisp();
}

P10=1;
if(!P10)
{
    while(!P10)
    {
        P10=1;
    }
    start_stop=~start_stop;
}
if(start_stop)
{
    P12=0;
    TR0=1;
    EA=1;
    ET0=1;
}
else
{
    P12=1;
    TR0=0;
    EA=0;
    ET0=0;
}
}
}

```

(4) 实验步骤

- ①将 3.3.1 节 staticLED.DSN 文件另保存为 Timer.DSN。
- ② 运行 Keil uVision2 开发环境，按照 1.1.3 节介绍的方法建立工程 timer_c.uv2，CPU 为 AT89C51，包含启动文件 STARTUP.A51。
- ③ 按照 1.2.2 第（6）节介绍的方法将前面示例的 C 语言的源程序 timer.c 加入工程 timer_c.uv2，并设置工程 timer_c.uv2 属性，将其晶振频率设置为 12MHz，选择输出

可执行文件，仿真方式为选择硬仿真，并选择其中的“PROTEUS VSM MONITOR 51 DRIVER”仿真器。

- ④ 构造 (Build) 工程 timer_c.uv2。如果输入有误进行修改，直至构造正确，生成可执行程序 timer_c.hex 为止。
- ⑤ 为 AT89C51 设置可执行程序 timer_c.hex。
- ⑥ 运行程序，点击图 3.8 中各按键，观察数码管的显示与按键的关系是否符合程序要求。

(5) 实验作业

- 分析源程序并为源程序添加注释
- 记录示例程序运行结果截图。
- 分析定时器的控制方法
- 总结 51 单片机 C 语言程序设计中的中断服务程序的实现方法

3.5.2 自我完成实验

(1) 实验内容

在 Proteus 环境下搭建如下图 3.14 所示频率计电路图：

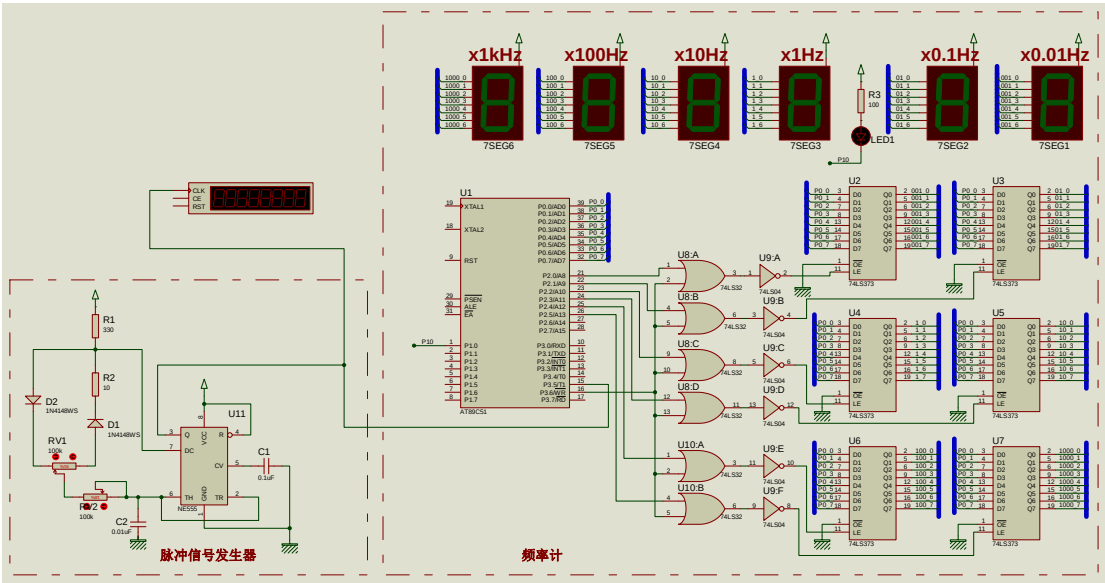


图 3.14 频率计原理图

图中左边是用 NE555 构成的脉冲信号发生器, 右边是用定时/计数器 0 和定时/计数器 1 构成的频率计。

为了检验频率计量是否准确, 用 Proteus 的虚拟频率计来进行测量脉冲信号频率进行对比。

图中所用元器件如下表所示：

选件编号	元件名称	参数	所在元件库类名	子类名	生产厂家
U1	AT89C51		Microprocessor	8051	ATMEL

			ICs 微处理器	Family 8051 家族	
U2-U7	74LS373		TTL 74LS series TTL 74LS 系列	Flip-Flops & Latches 触发与锁存	
U8, U10	74LS32		TTL 74LS series TTL 74LS 系列	Gates& Inverters 门与反相器	
U9	74LS04		TTL 74LS series TTL 74LS 系列	Gates& Inverters 门与反相器	
U11	NE555		Analog ICs 模拟集成电路	Timers 定时器	
7SEG1-7SEG6	7SEG-COM-AN-GRN		Optoelectronics 光电子器件	7-Segment Displays 7 段显示	
LED1	LED-GREEN		Optoelectronics 光电子器件	LEDs 发光二极管	
RV1, RV2	POT	100k	Resistors 电阻	Variable 可变的	
D1, D2	1N4148		Diodes 二极管	Switching 开关	
R1	RES	330	Resistors 电阻	Generic 一般的	
R2	RES	10	Resistors 电阻	Generic 一般的	
R3	RES	10k	Resistors 电阻	Generic 一般的	
C1	CAP	0.1uF	Capacitors 电容	Generic 一般的	
C2	CAP	0.01uF	Capacitors 电容	Generic 一般的	

(2) 控制要求

7SEG1-7SEG6 用于频率计百分位、十分位、个位、十位、百位、千位的显示，单位为 Hz。要求单片机上电运行后作为频率计将一直运行，改变脉冲发生器所产生的脉冲频率，则频率计的显示将跟随变化。

(3) 编程思路

定时/计数器 0 工作在定时器模式方式 1（16 位），定时/计数器 1 工作在计数器模式方式 2（8 位自动重装初值）。

定时/计数器 1 计数 200 个脉冲后（每计数 200 个脉冲产生一次中断）统计这 200 个脉冲总的时间长度，计算出平均每个周期的时间长度。

200 个脉冲所用时间长度的测量是靠定时/计数器 0 来实现的，定时/计数器 0 的初值为 0。当定时/计数器 1 产生中断时，读出定时/计数器 0 当前计数器值，再加上在定时/计数器 0 中断中累积的值即可得到。

为了便于大家编程，这里给出两个定时计数器的中断函数及有关变量的定义如下：

```
#define uchar unsigned char
#define uint unsigned int

uchar code LED_Table[]={0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82,
                        0xF8, 0x80, 0x90} ;//7 段 LED 共阳显示码
uchar xdata DA001 _at_ 0xfe00; //x0.01Hz 显示锁存地址
uchar xdata DA01 _at_ 0xfd00; //x0.1Hz 显示锁存地址
uchar xdata DA1 _at_ 0xfb00; //x1Hz 显示锁存地址
uchar xdata DA10 _at_ 0xf700; //x10Hz 显示锁存地址
uchar xdata DA100 _at_ 0xef00; //x100Hz 显示锁存地址
uchar xdata DA1k _at_ 0xdf00; //x1000Hz 显示锁存地址
uchar DV001, DV01, DV1, DV10, DV100, DV1k;// DV001 百分位、DV01 十分位、DV1 个位、
// DV10 十位、DV100 百位、DV1k 千位数值

long sumtime=0; //总时间

float frequency,ptime; // frequency 为频率,ptime 为单周期时间

void s_timer0() interrupt 1
{
    EA=0;
    sumtime=sumtime+65536;
    TH0=timer0H;
    TL0=timer0L;
    EA=1;
}

void s_timer1() interrupt 3
{
    EA=0;

    sumtime=sumtime+TH0*256+TL0;
    TH0=timer0H;
    TL0=timer0L;
```

```

ptime=(float)sumtime/200; //计算每个周期的时间长度
sumtime=0;

frequency=(float)1000000/ptime; //计算频率
DV1k=(uchar)(frequency/1000); //计算频率千位值
frequency=frequency-DV1k*1000;
DV100=(uchar)(frequency/100); //计算频率百位值
frequency=frequency-DV100*100;
DV10=(uchar)(frequency/10); //计算频率十位值
frequency=frequency-DV10*10;
DV1=(uchar)frequency; //计算频率个位值
frequency=frequency-DV1;
frequency=frequency*10;
DV01=(uchar)frequency; //计算频率十分位值
frequency=frequency-DV01;
frequency=frequency*10;
DV001=(uchar)frequency; //计算频率百分位值

DA001=LED_Table[DV001];
DA01=LED_Table[DV01];
DA1=LED_Table[DV1];
DA10=LED_Table[DV10];
DA100=LED_Table[DV100];
DA1k=LED_Table[DV1k];
EA=1;
}

```

主程序的编写由读者自己完成。

(4) 实验步骤

- ① 根据上述实验内容，参考 1.2.2，在 Proteus 环境下建立图 3.11 所示原理图，并将其保存为 frequencycounter.DSN 文件。
- ② 根据 (2) 和 (3) 编写控制源程序，将其保存为 frequencycounter.c。
- ③ 运行 Keil uVision2 开发环境，按照 1.1.3 节介绍的方法建立工程 frequencycounter.uv2，CPU 为 AT89C51，包含启动文件 STARTUP.A51。
- ④ 按照 1.2.2 第 (6) 节介绍的方法将 C 语言源程序 frequencycounter.c 加入工程 frequencycounter.uv2，并设置工程 frequencycounter.uv2 属性，将其晶振频率设置

为 12MHz，选择输出可执行文件，仿真方式为选择硬仿真，并选择其中的“PROTEUS VSM MONITOR 51 DRIVER”仿真器。

- ⑤ 构造 (Build) 工程 frequencycounter.uv2。如果输入有误进行修改，直至构造正确，生成可执行程序 frequencycounter.hex 为止。
- ⑥ 为 AT89C51 设置可执行程序 frequencycounter.hex。
- ⑦ 运行程序，观察数码管的显示与虚拟频率计是否一致。
- ⑧ 改变 RV2 的值，继续观察频率测量结果，观察数码管的显示与虚拟频率计是否一致。

(5) 实验作业

- 编写源程序并进行注释
- 实验过程记录
- 记录程序运行结果截图

3.6 实验 6：串口通信实验

3.6.1 示例实验

(1) 实验内容

在 Proteus 环境下搭建如下图 3.15 所示的电路图：

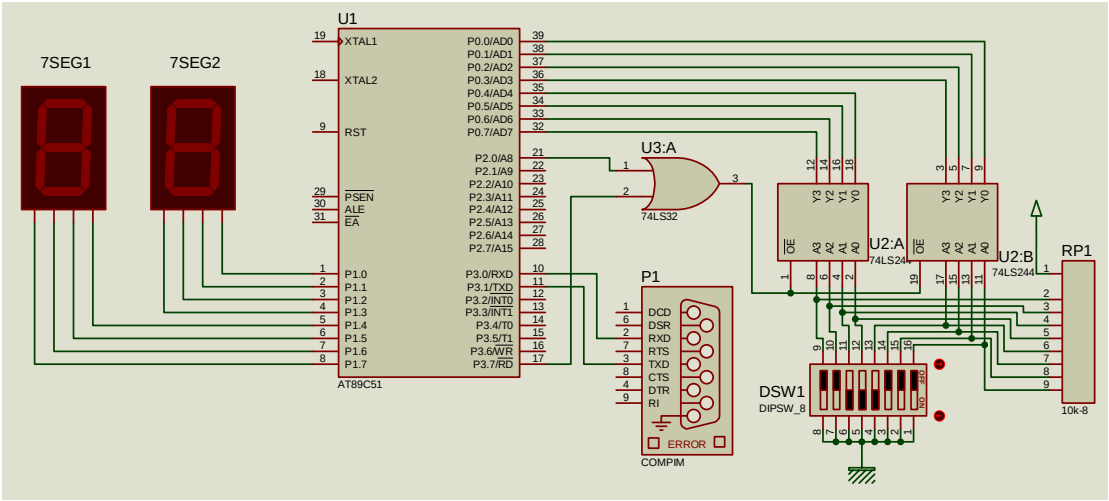


图 3.15 串行通信原理图

其中所用元器件如下表所示：

选件编号	元件名称	参数	所在元件库类名	子类名	生产厂家
U1	AT89C51		Microprocessor ICs 微处理器	8051 Family 8051 家族	ATMEL
U2	74LS244		TTL 74LS series	Buffers&Dribers 缓冲与驱动器	
U3	74LS32		TTL 74LS series	Gates&Inverters 门与反向器	
P1	COMPIM		Miscellaneous 杂类		

7SEG1 7SEG2	7SEG-BCD		Optoelectronics 光电器件	7-Segment Displays 7 段显示	
DSW1	DIPSW_8		Switches&Relays 开关与继电器	Switches 开关	
RP1	RESPACK-8	10k-8	Resistors 电阻	Resistor Packs 排阻	

备注:74LS244 用于扩展并行输入口。COMPIM 是 COM Port Physical Interface model 串口物理端口模型, 用于和计算机虚拟串口进行通信。

(2) 功能要求

利用计算机虚拟串口驱动软件和串口调试工具软件实现计算机串口调试软件与单片机串行口的通信, 要求当计算机发出数字“1”时, 单片机能够采集 244 扩展口的数据, 并将该数据显示在两个 7 段数码管上 (7SEG1 显示高四位, 7SEG2 显示低四位), 并用 0.2 秒的周期闪烁 10 次。当计算机发出数字“1”时, 单片机能够采集 244 扩展口的数据, 并将该数据发送回计算机串口调试软件。编程语言采用 C 语言实现。串口工作在方式 1, 波特率为 9600。

(3) 示例程序

//将如下 C 语言源程序保存为 uart.c

```
#include <reg51.h>

#define uchar unsigned char
#define uint unsigned int

uchar xdata D244 _at_ 0xfe00; //244 地址
uchar rdata, i, j, temp;
uint x;
void delay()
{
    for (i=200;i>0;i--);
}

void longdelay()
{
    for(x=500;x>0;x--)
        for (i=200;i>0;i--);
}

/*****主程序*****/
main()
{
    TMOD=0x20; //定时器 1 设置为方式 2
```

```

SM0=0;
SM1=1;
REN=1; //串口工作方式 1，允许接收控制位 REN=1
PCON=0; //波特率不加倍
TH1=0xFD;
TL1=0xFD; //波特率设定为 9600，晶振频率为 11.0592MHz
TR1=1; //开启定时计数器
rdata=D244;
delay();
P1=rdata;
while(1)
{
    while(!RI);temp=SBUF; RI=0; //接收数据
    rdata=D244;
    delay();
    switch(temp)
    {
        case 1:
            for(j=10;j>0;j--)
            {
                P1=0x88;
                longdelay();
                P1=rdata;
                longdelay();
            }
            break;
        case 2:
            P1=rdata;
            SBUF=rdata;
            while(!TI);
            TI=0; //发送数据
            break;
    }
}
}

```

(4) 实验步骤

- ① 根据上述实验内容，参考 1.2.2，在 Proteus 环境下建立图 3.15 所示原理图，并将其保存为 uart.DSN 文件。
- ② 将上面（3）中控制源程序保存为 uart.c。
- ③ 运行 Keil uVision2 开发环境，按照 1.1.3 节介绍的方法建立工程 uart.uv2，CPU 为 AT89C51，包含启动文件 STARTUP.A51。
- ④ 按照 1.2.2 第（6）节介绍的方法将 C 语言源程序 uart.c 加入工程 uart.uv2，并设置工程 uart.uv2 属性，将其晶振频率设置为 11.0592MHz，选择输出可执行文件，仿真方式为选择硬仿真，并选择其中的“PROTEUS VSM MONITOR 51 DRIVER”仿真器。
- ⑤ 构造（Build）工程 uart.uv2。如果输入有误进行修改，直至构造正确，生成可执行程序 uart.hex 为止。
- ⑥ 为 AT89C51 设置可执行程序 uart.hex。
- ⑦ 从网上下载并安装虚拟串口驱动软件（VSPD 虚拟串口破解版 6.9 汉化版）。运行该软件，将出现如下图 3.16 所示界面：



图 3.16 虚拟串口驱动 VSPD 界面

点击其中的“添加端口”按钮将“COM1”和“COM2”添加为虚拟串口，添加后如下图所示：



图 3.17 VSPD 添加 COM1 和 COM2 后界面

⑧ 从网上下载串口调试助手软件并运行该软件，将出现如下图 3.18 所示界面：

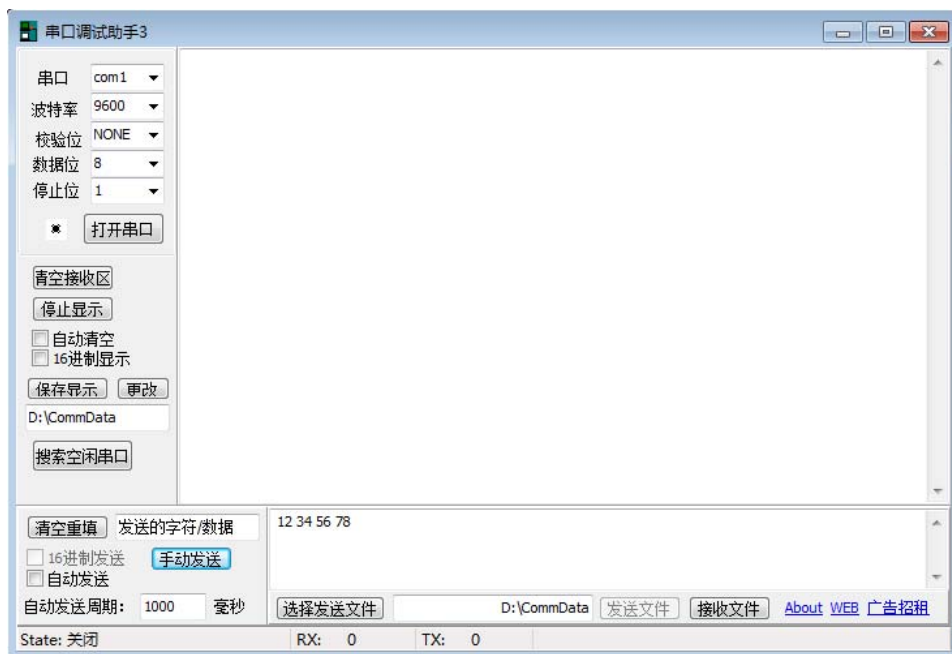


图 3.18 串口调试助手界面

选择串口 com1，波特率为 9600，校验位为“NONE”，数据位为 8，停止位为 1。这些设置和前面单片机程序中设置的串口参数要一致。

⑨ 设置 Proteus 工程 uart.DSN 界面中的 COMPIM 参数，设置方法是鼠标右键点击 P1，选择弹出菜单中的“Edit Properties”选项，如下图 3.19 所示：

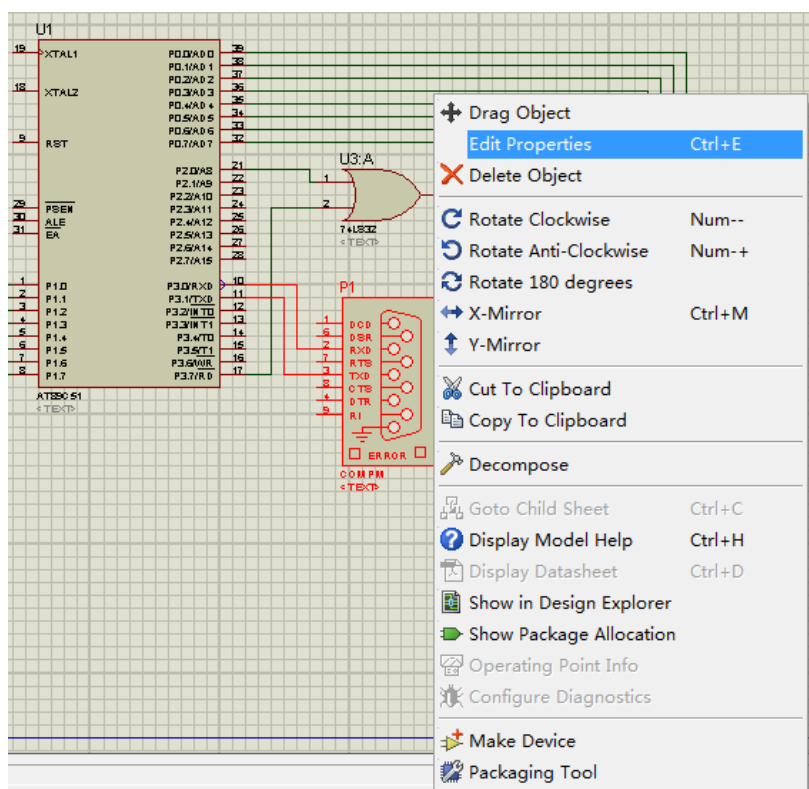


图 3.19 设置 COMPIM 参数

在弹出窗口“Edit Component”中，选择“Physical port”为 COM2（注意前面串口调试助手如果选择 COM1 这里就应该选 COM2，反之，串口调试助手如果选择 COM2 这里就应该选 COM1），其它参数设置如下图 3.20 所示：

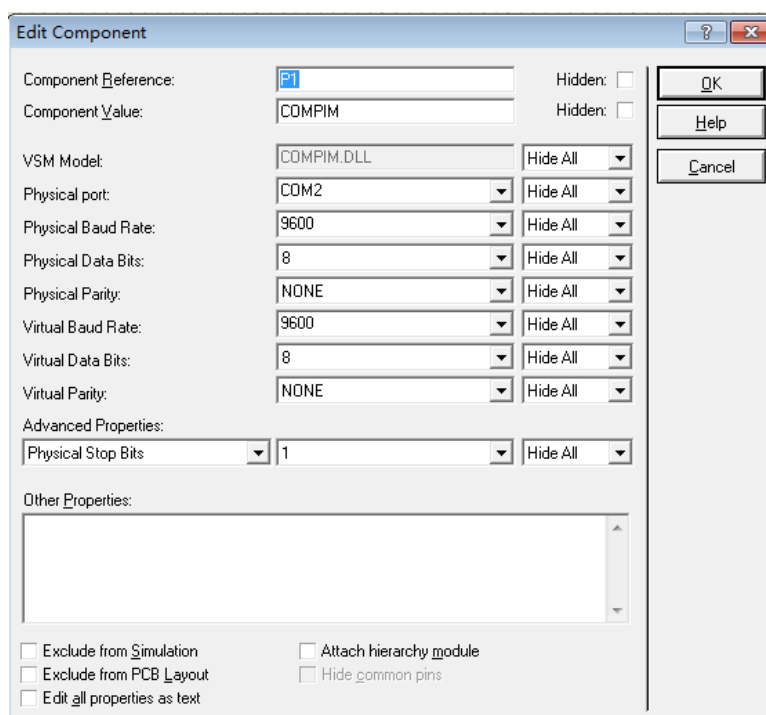


图 3.20 COMPIM 参数设置窗口

⑩点击 Proteus 仿真界面下方的仿真控制按钮“”开始仿真。并且设置图 3.18 中的数据显示及数据发送格式为 16 进制格式，在发送窗口发送数字“2”观察显示窗口中接收的数据与 Proteus 中 7SEG1 和 7SEG2 中显示的是否一致。再发送数字“1”观察程序运行结果是否正确。

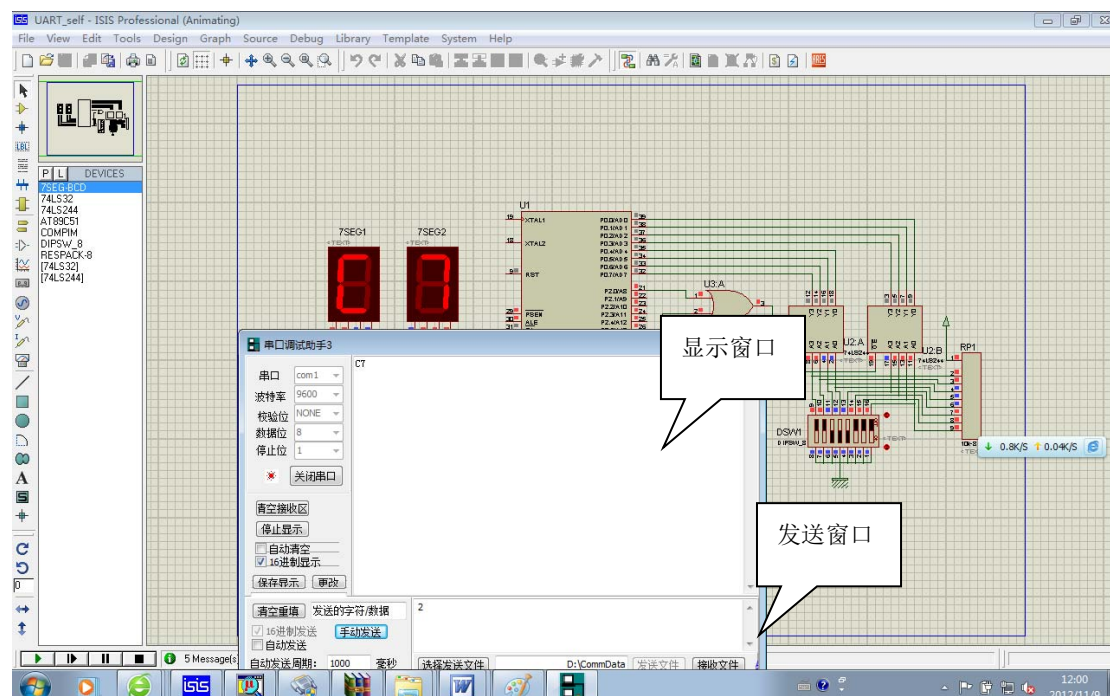


图 3.21 串口通信结果显示窗口

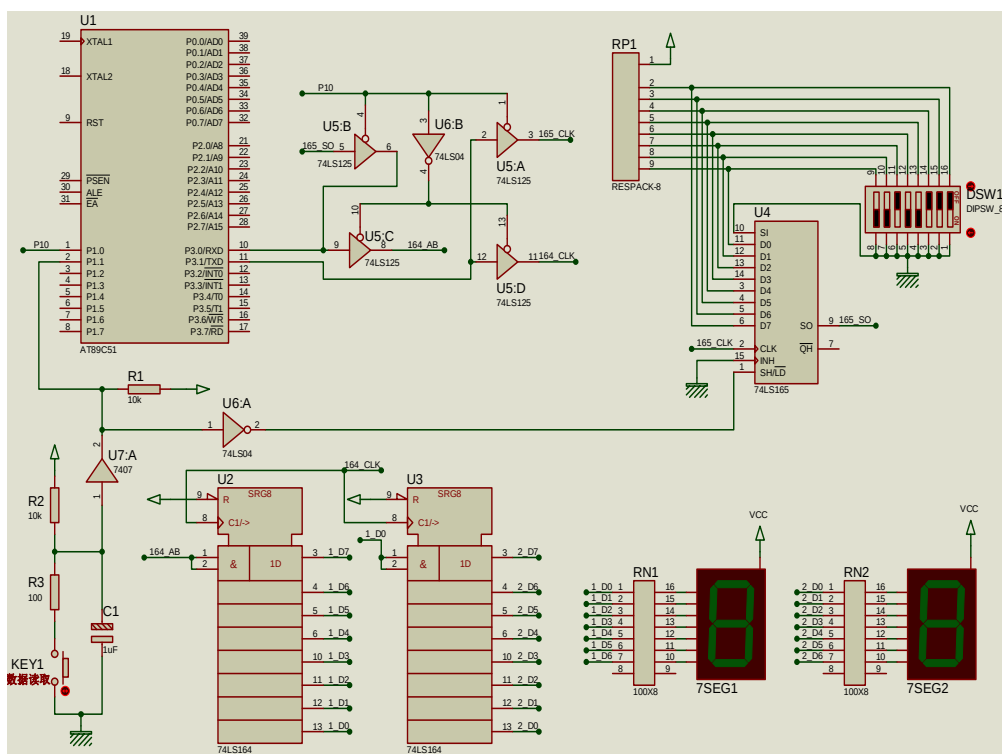
(5) 实验作业

- 仔细分析源程序，了解串口通信编程方法，并为源程序添加注释
- 改变 DIP 开关的设置值，并通过串口通信调试窗口发送数字命令，读取结果，记录程序运行结果截图。
- 总结 51 单片机串口通信虚拟串口调试的方法

3.6.2 自我完成实验

(1) 实验内容

在 Proteus 环境下搭建如下图 3.22 所示 74LS164/165 串口扩展并口的电路图：



3.22 74LS164/165 串口扩展并口的电路图

其中所用元器件如下表所示：

选件编号	元件名称	参数	所在元件库类名	子类名	生产厂家
U1	AT89C51		Microprocessor ICs 微处理器	8051 Family 8051 家族	ATMEL
U2,U3	74LS164		TTL 74LS series	Registers 寄存器	
U4	74LS165		TTL 74LS series	Registers 寄存器	
U5	74LS125		TTL 74LS series	Buffers&Drivers 缓冲器与驱动器	
U6	74LS04		TTL 74LS series	Gates&Inverters 门与反向器	
U7	7407		TTL 74 series	Buffers&Drivers 缓冲器与驱动器	
RN1, RN2	RX8		Resistors 电阻	Resistor Packs 排阻	
RP1	RESPACK-8	10k-8	Resistors 电阻	Resistor Packs 排阻	
DSW1	DIPSW_8		Switches&Relays 开关与继电器	Switches 开关	

7SEG1 7SEG2	7SEG-COM- AN-GRN		Optoelectronics 光电子器件	7-Segment Displays 7 段显示	
R1, R2	RES	10k	Resistors 电阻	Generic 一般的	
R3	RES	100	Resistors 电阻	Generic 一般的	
C1	CAP-ELEC	1uF	Capacitors 电容	Generic 一般的	
KEY1	BUTTON		Switches & Relays 开关与继电器	Switches 开关	

(2) 控制要求

本实验利用 74LS165 实现串口转并行输入端口, 用来采集 DSW1 的开关数据。采集来的数据通过 74LS164 实现的串口转并行输出端口送给两个 7 段 LED 数码管 7SEG1 和 7SEG2。RN1 和 RN2 是两个排阻用于限流。图 3.22 中的 R2, R3, C1 和 KEY1 用于产生低电平脉冲, 其中的 R2, R3 和 C1 用于去除按键抖动。该电平脉冲一方面连接至 P1.1 用于 CPU 探测该脉冲的产生时间, 另一方面经过 U6A 的 74LS04 之后连接至 74LS165 的 1 号引脚 (SH/LD), 该信号在按键 KEY1 未被按下时为低电平, 74LS165 的输入一直被加载至器件内部的所存器, 当 KEY1 被按下时该信号为高电平, 74LS165 加载的数据将在 2 号引脚 CLK 的同步下被串行移出并进入单片机的 RXD 端口。

由于 74LS165 的 9 号引脚需要连接到单片机的 RXD 端口, 而 RXD 又需要连接至 164 的 1、2 号管脚。同时, 单片机的 TXD 引脚不论在读 74LS165 的数据还是向 74LS164 输出数据时都提供时钟。为了防止 74LS165 的 9 号引脚输出影响 RXD 至 164 的 1、2 号引脚的输出, 而且防止 74LS165 的输出被相同的时钟直接移位进入 164 (走近路), 单片机的 TXD 和 RXD 需要分时接入 74LS165 和 74LS164。分时控制的方法就是利用 U5 (74LS125) 这一三态选通缓冲器来实现。TXD 接入 U5A 和 U5D 的输入端, RXD 接入 U5C 的输入和 U5B 的输出。U5A 的输出接 74LS165 的 CLK, U5B 的输入接 74LS165 的 9 号引脚, 当 P1.0=0 时这两个三态缓冲器是接通的, 即 RXD 接至 74LS165 的 9 号引脚, TXD 接至 74LS165 的 CLK, 可以实现单片机对 74LS165 的串行读取。反之, 当 P1.0=1 时, 经过反相器使得 U5C 和 U5D 接通, 使得 TXD 接至 74LS164 的 CLK, RXD 接至 74LS164 的 1、2 号引脚, 可以实现单片机对 164 的串行输出。

最终要求实现的功能是: 当按键按下时, DSW1 的开关数据能够被单片机通过 74LS165 串行读取, 并通过 74LS164 串行输出至两个 7 段数码管 7SEG1 和 7SEG2 显示, 要求显示的数据和 DSW1 的开关数据一致, 没有错误数据被读入或显示。

(3) 编程思路

串行口工作在定时器模式方式 0 模式 (即: SM1=0, SM0=0)。串行口的数据收发利用查询模式实现。只有当查询到 P1.1 有低电平出现时才启动数据收发过程。启动的方法就是先使 P1.0=0 打开接收通道, 再使接收控制 (REN=1) 使能, 这时 TXD 就会自动输出时钟,

由于这时 74LS165 正处于移位模式 (SH/LD 引脚为高电平)，数据会自动串行移入单片机的 RXD 端口。查询接收完成标志 RI，当 RI=1 时 1 个字节接收完成，清掉 RI (即使 RI=0)。然后，使 P1.0=1，打开输出 74LS164 通道，把刚才接收的那个数据分低半字节和高半字节，分别查询出它们共阳七段数码管的显示码，再将显示码 (16 位) 移位送出至 74LS164。两个 74LS164 是按级联方式连接的，先送出的会到达 U3 输出端，后送出的会到达 U2 输出端。发送时通过 TI 标志位查询发送一个字节是否完成。

(4) 实验步骤

- ① 根据上述实验内容，参考 1.2.2，在 Proteus 环境下建立图 3.22 所示原理图，并将其保存为 uart_self.DSN 文件。
- ② 根据 (2) 和 (3) 画出流程图，并编写源程序，将其保存为 uart_self.c。
- ③ 运行 Keil uVision2 开发环境，按照 1.1.3 节介绍的方法建立工程 uart_self.uv2，CPU 为 AT89C51，包含启动文件 STARTUP.A51。
- ④ 按照 1.2.2 第 (6) 节介绍的方法将 C 语言源程序 uart_self.c 加入工程 uart_self.uv2，并设置工程 uart_self.uv2 属性，将其晶振频率设置为 12MHz，选择输出可执行文件，仿真方式为选择硬仿真，并选择其中的“PROTEUS VSM MONITOR 51 DRIVER”仿真器。
- ⑤ 构造 (Build) 工程 uart_self.uv2。如果输入有误进行修改，直至构造正确，生成可执行程序 uart_self.hex 为止。
- ⑥ 为 AT89C51 设置可执行程序 uart_self.hex。
- ⑦ 运行程序，点击 KEY1 观察数码管的显示是否与 DSW1 的设置一致。
- ⑧ 改变 DSW1 的设置，再次点击 KEY1 观察数码管的显示是否与 DSW1 的设置一致。重复本条多次观察程序运行效果。

(5) 实验作业

- 画出流程图，编写源程序并进行注释
- 实验过程记录
- 记录程序运行结果截图
- 仔细分析并总结原理图中 74LS164 和 74LS165 的连接方法，以及控制两种器件分时接入 51 单片机串行通信口的实现方法。

3.7 实验 7: LCD1602 显示实验

3.7.1 字符型 LCD 1602 简介

字符型液晶显示 (LCD) 模块是一种专门用于显示字母、数字、符号等内容的点阵式 LCD，目前常用的有 16*1，16*2，20*2 和 40*2 行等的不同类型。其中 1602 字符型液晶显示器的性能指标为：

- ◆ 显示容量:16×2 个字符
- ◆ 芯片工作电压:4.5—5.5V

- ◆ 工作电流:2.0mA (5.0V)
- ◆ 模块最佳工作电压:5.0V
- ◆ 字符尺寸:2.95×4.35(W×H)mm

1、LCD1602 的外形及引脚分布

LCD1602 的外形如图 3.23 所示， Proteus 中的 LCD1602 仿真模型如图 3.24 所示：



图 3.23 LCD1602 实物图

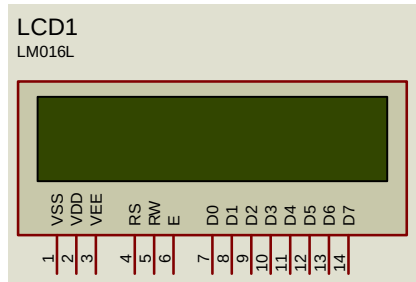


图 3.24 Proteus 中的 LCD1602 仿真模型

LCD1602 分为带背光和不带背光两种，基控制器大部分为 HD44780，带背光的比不带背光的厚，同时多两个引脚（分别是：15 号引脚-背光源正极 BLA 和 16 号引脚-背光源负极 BLK）。在 Proteus 中的 1602 模型不带背光源。

1602LCD 采用标准的 14 脚（无背光）或 16 脚（带背光）接口，各引脚接口说明如下表 3.1 所示：

表 3.1 LCD1602 引脚接口说明表

编号	符号	引脚说明	编号	符号	引脚说明
1	VSS	电源地	9	D2	数据
2	VDD	电源正极，接+5V	10	D3	数据
3	VL	液晶显示偏压	11	D4	数据
4	RS	数据/命令选择	12	D5	数据
5	R/W	读/写选择	13	D6	数据
6	E	使能信号	14	D7	数据
7	D0	数据	15	BLA	背光源正极
8	D1	数据	16	BLK	背光源负极

其中：

- ◆ 第 3 脚：VL 为液晶显示器对比度调整端，接正电源时对比度最弱，接地时对比度最高，对比度过高时会产生“重影”，使用时可以通过一个 10K 的电位器调整对比度。
- ◆ 第 4 脚：RS 为寄存器选择，高电平时选择数据寄存器、低电平时选择指令寄存器。

- ◆ 第 5 脚：R/W 为读写信号线，高电平时进行读操作，低电平时进行写操作。当 RS 和 R/W 共同为低电平时可以写入指令或者显示地址，当 RS 为低电平 R/W 为高电平时可以读忙信号，当 RS 为高电平 R/W 为低电平时可以写入数据。
- ◆ 第 6 脚：E 端为使能端，当 E 端由高电平跳变成低电平时，液晶模块执行命令。
- ◆ 第 7~14 脚：D0~D7 为 8 位双向数据线。

2、LCD1602 的存储器地址映射及标准字库表

LCD1602 是一种字符型点阵 LCD，要显示的字符应该提前保存在相应的存储器里面。LCD1602 模块里面的存储器有三种：CGROM、CGRAM、DDRAM。其中 CGROM 保存了厂家生产时固化在 LCM（LCD 控制模块）中的点阵型显示数据，CGRAM 是留给用户自己定义点阵型显示数据的。而 DDRAM 内的内容正是要立即显示的内容。

1602 液晶模块内部的字符发生存储器（CGROM）已经存储了 160 个不同的点阵字符图形，每一个字符都有一个固定的代码，这就是字符代码。这些字符代码和字符图形的对应关系如下表 3.2 所示：

代码高 4 位 和 CGRAM 中字符代码与字符图形对应关系表

	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
代码低 4 位	CG RAM (1)			0	1	A	Q	a	q				-	9	3	p
xxxx0000																
xxxx0001	(2)		!	1	A	Q	a	q					。	7	4	ä
xxxx0010	(3)		"	2	B	R	b	r					「	イ	ツ	θ
xxxx0011	(4)		#	3	C	S	c	s					」	ウ	テ	ε
xxxx0100	(5)		\$	4	D	T	d	t					、	エ	ト	μ
xxxx0101	(6)		%	5	E	U	e	u					・	オ	ナ	1
xxxx0110	(7)		&	6	F	V	f	v					ヲ	カ	ニ	ヨ
xxxx0111	(8)		'	7	G	W	g	w					ア	キ	ヌ	ラ
xxxx1000	(1)		(	8	H	X	h	x					イ	ク	ネ	リ
xxxx1001	(2)		)	9	I	Y	i	y					ウ	ケ	ノ	ル
xxxx1010	(3)		*	:	J	Z	j	z					エ	コ	ハ	レ
xxxx1011	(4)		+	;	K	[	k	[					オ	サ	ヒ	ロ
xxxx1100	(5)		,	<	L	¥	l	¥					ハ	シ	フ	ワ
xxxx1101	(6)		-	=	M	]	m	]					ユ	ズ	ヘ	ン
xxxx1110	(7)		.	>	N	^	n	^					ヨ	セ	ホ	ベ
xxxx1111	(8)		/	?	O	_	o	_					ッ	リ	マ	°

从上表可以看出，“A”字的对应上面高位代码为 0100，对应左边低位代码为 0001，合起来就是 01000001，也就是 41H。可见它的代码与我们 PC 中的字符代码是基

本一致的。因此我们在向 DDRAM 写 C51 字符代码程序时甚至可以直接用 P1='A' 这样的方法。PC 在编译时就把“A”先转为 41H 代码了。

字符代码 0x00~0x0F 为用户自定义的字符图形 RAM(对于 5X8 点阵的字符,可以存放 8 组, 5X10 点阵的字符, 存放 4 组), 就是 CGRAM 了。

0x20~0x7F 为标准的 ASCII 码, 0xA0~0xFF 为日文字符和希腊文字符, 其余字符码(0x10~0x1F 及 0x80~0x9F)没有定义。

为显示字符, 需要先把需要显示的字符放到 DDRAM 中相应的位置。LCD1602 的 DDRAM 有 80 字节, 而显示屏上只有 2 行×16 列, 共 32 个字符, 所以两者不完全一一对应。默认情况下, 显示屏上第一行的内容对应 DDRAM 中 00H 到 0FH 的内容, 第二行的内容对应 DDRAM 中 40H 到 4FH 的内容。如下图 3.25 所示:



图 3.25 LCD1602 内部显示地址

DDRAM 中 10H 到 27H、50H 到 67H 的内容是不显示在显示屏上的, 但是在滚动屏幕的情况下, 这些内容就可能被滚动显示出来了。

要显示字符时要先输入显示字符地址, 也就是告诉模块在哪里显示什么样的字符, 另外还要告诉控制器如何移动光标, 如何移动字符, 查询控制器是否忙等信息。这些都要靠对控制器发送不同的命令来实现。

3、LCD1602 的控制指令

1602 液晶模块的读写操作, 屏幕和光标的操作都是通过指令编程来实现的。由于液晶显示模块是一个慢显示器件, 所以这些指令都有一定的执行时间要求。因此, 在执行每条指令之前一定要确认模块的忙标志为 0, 表示不忙, 若不满足该条件则指令失效, 指令执行之后要延迟一定的时间。

1602 液晶模块内部的控制器共有 11 条控制指令, 它们是:

(1) 清屏指令

指令功能	指令编码										执行时间 /ms
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
清屏	0	0	0	0	0	0	0	0	0	1	1.64

功能: <1> 清除液晶显示器, 即将 DDRAM 的内容全部填入“空白”的 ASCII 码 20H;

<2> 光标归位, 即将光标撤回液晶显示屏的左上方;

<3> 将地址计数器(AC)的值设为 0。

(2) 光标归位指令

指令功能	指令编码										执行时间 /ms
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
光标归位	0	0	0	0	0	0	0	0	1	X	1.64

功能：〈1〉 DDRAM 所有单元的内容不变，光标移至左上角；

〈2〉 把地址计数器 (AC) 的值设置为 0；

(3) 进入模式设置指令

指令功能	指令编码										执行时间 /us
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
进入模式设置	0	0	0	0	0	0	0	1	I/D	S	40

功能：设定每次定入 1 位数据后光标的移位方向，并且设定每次写入的一个字符是否移动。参数设定的情况为：

位名	设置
I/D	0=写入新数据后地址寄存器 AC 的内容加 1，1=写入新数据后地址寄存器 AC 的内容减 1
S	0=写入新数据后显示屏不移动，1=写入新数据后显示屏整体右移 1 个字符

(4) 显示开关控制指令

指令功能	指令编码										执行时间 /us
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
显示开关控制	0	0	0	0	0	0	1	D	C	B	40

功能：控制显示器开/关、光标显示/关闭以及光标是否闪烁。参数设定的情况如下：

位名	设置
D	0=显示功能关，1=显示功能开
C	0=无光标，1=有光标
B	0=光标闪烁，1=光标不闪烁

(5) 设定显示屏或光标移动方向指令

指令功能	指令编码										执行时间 /us
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
设定显示屏或光标移动方向	0	0	0	0	0	1	S/C	R/L	X	X	40

功能：使光标移位或使整个显示屏幕移位。参数设定的情况如下：

S/C	R/L	设定情况
0	0	光标左移 1 格，且 AC 值减 1
0	1	光标右移 1 格，且 AC 值加 1
1	0	显示器上字符全部左移一格，但光标不动
1	1	显示器上字符全部右移一格，但光标不动

(6) 功能设定指令

指令功能	指令编码										执行时间 / μ s
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
功能设定	0	0	0	0	1	DL	N	F	X	X	40

功能：设定数据总线位数、显示的行数及字型。参数设定的情况如下：

位名	设置
DL	0=数据总线为 4 位，1=数据总线为 8 位
N	0=显示 1 行，1=显示 2 行
F	0=5 $\times$ 7 点阵/每字符，1=5 $\times$ 10 点阵/每字符

(7) 设定 CGRAM 地址指令

指令功能	指令编码										执行时间 / μ s
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
设定 CGRAM 地址	0	0	0	1	CGRAM 的地址 (6 位)						40

功能：设定下一个要存入数据的 CGRAM 的地址。

(8) 设定 DDRAM 地址指令

指令功能	指令编码										执行时间 / μ s
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
设定 DDRAM 地址	0	0	1	CGRAM 的地址 (7 位)							40

功能：设定下一个要存入数据的 DDRAM 的地址，注意其最高位为 1。如第二行第一个字符的地址是 40H，那么是否直接写入 40H 就可以将光标定位在第二行第一个字符的位置

呢？这样不行，因为写入显示地址时要求最高位 D7 恒定为高电平 1，所以实际写入的数据应该是 01000000B (40H) + 10000000B (80H) = 11000000B (C0H)。

(9) 读取忙碌信号或 AC 地址指令

指令功能	指令编码										执行时间 / μ s
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
读取忙碌信号或 AC 地址	0	1	FB	AC 内容 (7 位)							40

功能：<1> 读取忙碌信号 BF 的内容，BF=1 表示液晶显示器忙，暂时无法接收单片机送来的数据或指令；当 BF=0 时，液晶显示器可以接收单片机送来的数据或指令；

<2> 读取地址计数器 (AC) 的内容。

(10) 数据写入 DDRAM 或 CGRAM 指令一览

指令功能	指令编码										执行时间 / μ s
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
数据写入到 DDRAM 或 CGRAM	1	0	要写入的数据 D7~D0								40

功能：<1> 将字符码写入 DDRAM，以使液晶显示屏显示出相对应的字符；

<2> 将使用者自己设计的图形存入 CGRAM。

(11) 从 CGRAM 或 DDRAM 读出数据的指令一览

指令功能	指令编码										执行时间 / μ s
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
从 CGRAM 或 DDRAM 读出数据	1	1	要读出的数据 D7~D0								40

功能：读取 DDRAM 或 CGRAM 中的内容。

4、LCD1602 的控制时需

LCD1602 读操作时序如图 3.26 所示

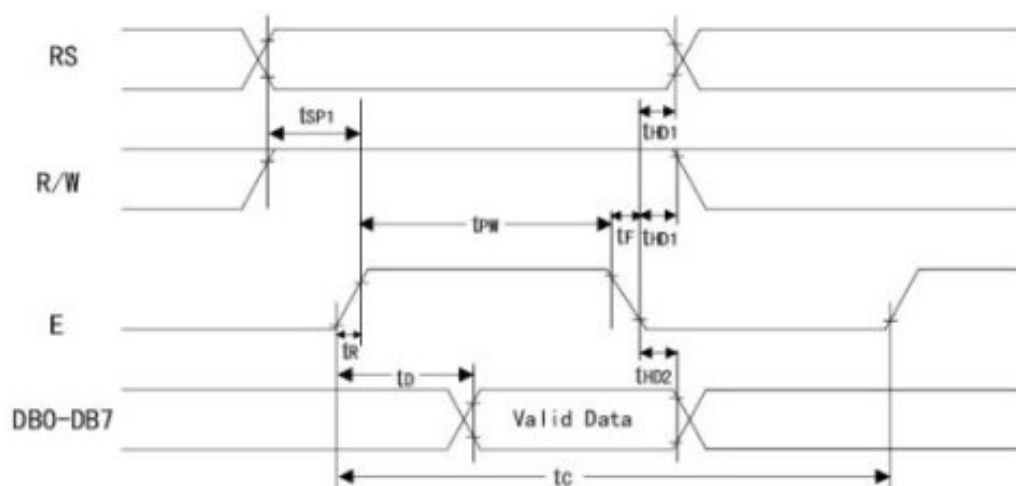


图 3.26 LCD1602 读操作时序

➤ 读状态

输入：RS=L，RW=H，E=H；输出：DB0～DB7=状态字

➤ 读数据

输入：RS=H，RW=H，E=H；输出：DB0～DB7=数据

LCD1602 写操作时序如图 3.27 所示

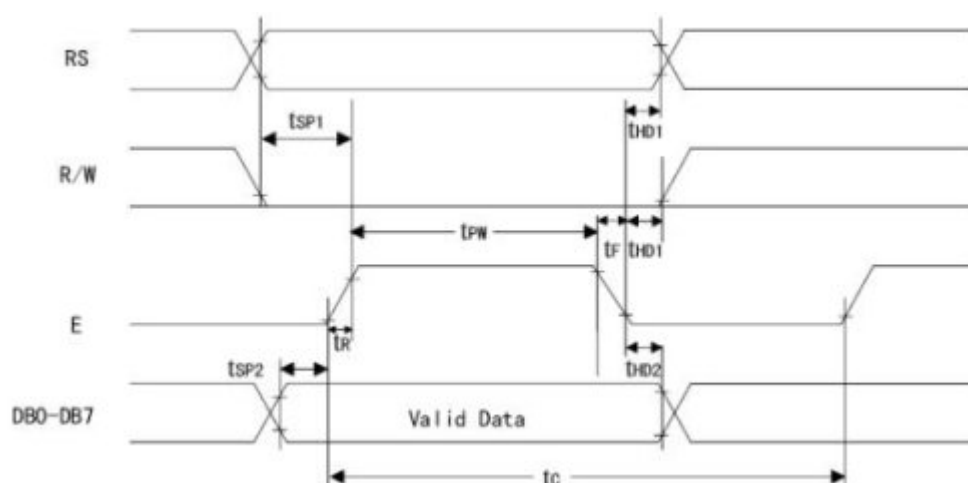


图 3.27 LCD1602 写操作时序

➤ 写指令

输入：RS=L，RW=L，E=下降沿脉冲，DB0～DB7=指令码；输出：无

➤ 写数据

输入：RS=H，RW=L，E=下降沿脉冲，DB0~DB7=数据；输出：无

3.7.2 LCD1602 示例实验

(1) 实验内容

在 Proteus 环境下搭建如下图 3.28 所示的电路图：

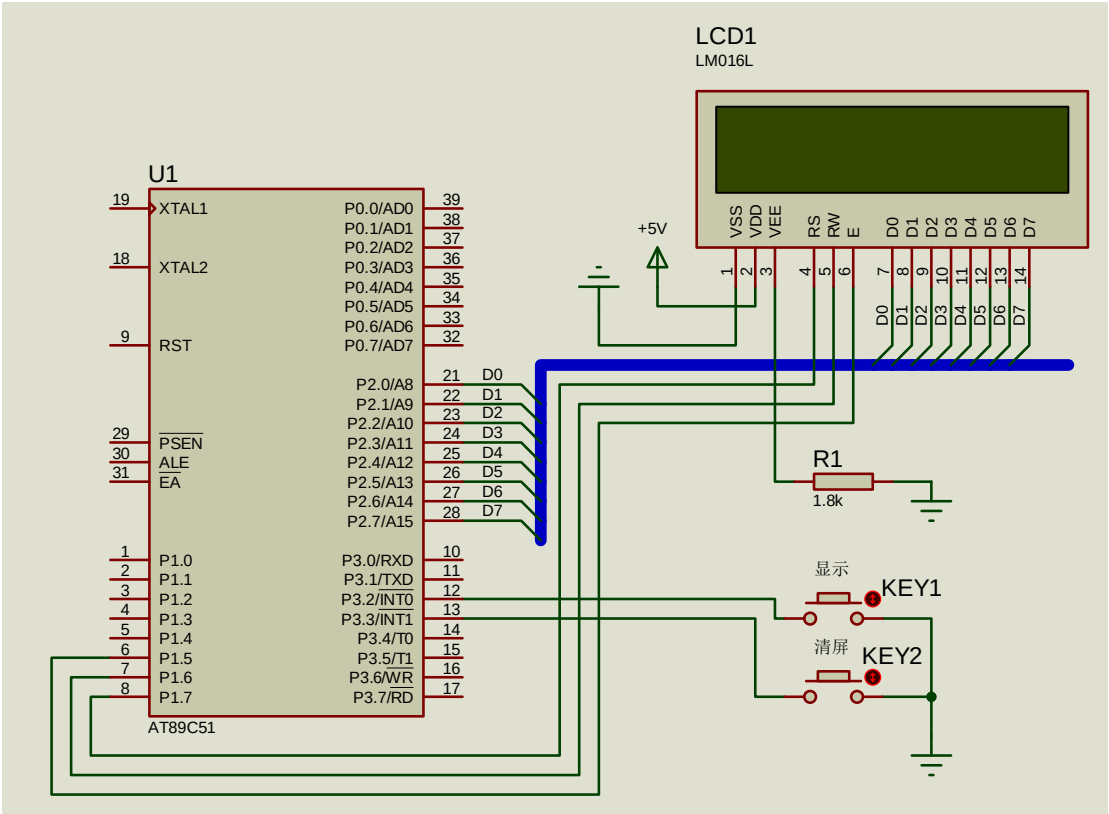


图 3.28 LCD1602 应用示例

其中所用元器件如下表所示：

选件编号	元件名称	参数	所在元件库类名	子类名	生产厂家
U1	AT89C51		Microprocessor ICs 微处理器	8051 Family 8051 家族	ATMEL
LCD1	LM016L		Optoelectronic 光电器件	Alpha Numeric LCDs 字符数字 LCD	
R1	RES	1.8k	Resistors 电阻	Generic 一般的	
KEY1 KEY2	BUTTON		Switches & Relays 开关与继电器	Switches 开关	

(2) 功能要求

要求点击按键 KEY1 可以在 LCD1602 (LM016L) 第一行上显示 “Hello Everyone”，在第二行上显示 “Welcome to SDU”，并闪烁 3 次。点击按键 KEY2 可以将 LCD1602 清屏。

(3) 示例程序

//将如下 C 语言源程序保存为 LCD1602.c

```
/******必要的变量定义******/
#include <reg51.h>
#include <intrins.h> //包含_nop_()函数
#define uint unsigned int
#define uchar unsigned char
uchar code line1_data[] = "Hello Everyone"; //要显示的第一行字符
uchar code line2_data[] = "Welcome to SDU!"; //要显示的第二行字符
sbit LCD_RS=P1^7; //定义控制引脚
sbit LCD_RW=P1^6;
sbit LCD_EN=P1^5;
uchar flag=0;

/******延时子程序******/
void delay_ms(uint xms)
{
    uint i, j;
    for(i=xms; i>0; i--)
        for(j=110; j>0; j--);
}

/******LCD 忙检查子程序******/
bit lcd_busy()
{
    bit result;
    LCD_RS=0;
    LCD_RW=1;
    LCD_EN=1;
    _nop_();
    _nop_();
    _nop_();
    _nop_();
    result=(bit) (P2&0x80); //返回数据最高位 FB 代表是否忙
    LCD_EN =0;
    return result;
}
```

```

/*****写命令子程序*****/
void lcd_wcmd(uchar cmd)
{
    while(lcd_busy());
    LCD_RS=0;
    LCD_RW=0;
    LCD_EN=0;
    _nop_();
    _nop_();
    P2=cmd;
    _nop_();
    _nop_();
    _nop_();
    _nop_();
    LCD_EN=1;
    _nop_();
    _nop_();
    _nop_();
    _nop_();
    LCD_EN=0;
}

/*****LCD 清屏子程序*****/
void lcd_clr()
{
    lcd_wcmd(0x01); //清屏指令 DB7-DB0 部分为 01H
    delay_ms(2); //清屏指令需要 1.64ms 以上
}

/*****写数据子程序*****/
void lcd_wdat(uchar dat)
{
    while(lcd_busy());
    LCD_RS=1;
    LCD_RW=0;
    LCD_EN=0;
    _nop_();
    _nop_();
    P2=dat;
    _nop_();
    _nop_();
    _nop_();
    _nop_();
}

```



```

LCD_EN=1;
_nop_();
_nop_();
_nop_();
_nop_();
LCD_EN=0;
}

/*****初始化子程序*****/
void lcd_init()
{
    delay_ms(15);    //等待 LCD 电源稳定
    lcd_wcmd(0x38);  //功能设定指令中 DL=1, N=1, F=0, 8 位数据宽度, 16*2 显示,
                    //5*7 点阵字符

    delay_ms(5);
    lcd_wcmd(0x0c);  //显示开关控制指令中 D=1, C=0, B=0, 显示开, 关光标, 不闪烁
    delay_ms(5);
    lcd_wcmd(0x06);  //进入模式设置指令中 I/D=1, S=0, 地址自动增加
    delay_ms(5);
    lcd_wcmd(0x01);  //清除 LCD 显示内容, 清屏指令 DB7-DB0 部分为 01H
    delay_ms(5);
}

/*****闪烁子程序*****/
void flash()
{
    delay_ms(1000);
    lcd_wcmd(0x08);  //显示开关控制指令中 D=0, C=0, B=0, 显示关, 关光标, 不闪烁
    delay_ms(500);
    lcd_wcmd(0x0c);  // 显示开, 关光标, 不闪烁
    delay_ms(500);
    lcd_wcmd(0x08);
    delay_ms(500);
    lcd_wcmd(0x0c);
    delay_ms(500);
    lcd_wcmd(0x08);
    delay_ms(500);
    lcd_wcmd(0x0c);
    delay_ms(500);
}

/*****主程序*****/
void main()
{
    uchar i;//, j;
    EA=1;        //打开中断总开关

```

```

EX0=1;      //打开外部中断 0
IT0=1;      //设置中断触发方式为下降沿触发方式
EX1=1;
IT1=1;
delay_ms(10);
lcd_init(); //初始化
lcd_clr();  //清屏
delay_ms(5);
while(1)
{
    if(flag==1)
    {
        lcd_wcmd(0x00|0x80); //DDRAM 地址设置指令, 显示位置为第 1 行第 0 列
        i=0;
        while(line1_data[i]!='\0')
        {
            lcd_wdat(line1_data[i]); //加载第 1 行字符
            delay_ms(100);
            i++;
        }
        lcd_wcmd(0x40|0x80); //设置显示位置为第 2 行第 0 列
        i=0;
        while(line2_data[i]!='\0')
        {
            lcd_wdat(line2_data[i]); //加载第 2 行字符
            delay_ms(100);
            i++;
        }
        flash();
        flag=0;
    }
    if(flag==2)
    {
        lcd_clr(); //清屏
        delay_ms(5);
        flag=0;
    }
}

/*****外部中断 0 子程序
*****/
void INT_0() interrupt 0
{

```

```

        flag=1;
    }
    /*****外部中断 1 子程序
    *****/
    void INT_1() interrupt 2
    {
        flag=2;
    }

```

(4) 实验步骤

- ① 根据上述实验内容，参考 1.2.2，在 Proteus 环境下建立图 3.28 所示原理图，并将其保存为 lcd1602.DSN 文件。
- ② 将上面 (3) 中控制源程序保存为 lcd1602.c。
- ③ 运行 Keil uVision2 开发环境，按照 1.1.3 节介绍的方法建立工程 lcd1602.uv2，CPU 为 AT89C51，包含启动文件 STARTUP.A51。
- ④ 按照 1.2.2 第 (6) 节介绍的方法将 C 语言源程序 lcd1602.c 加入工程 lcd1602.uv2，并设置工程 lcd1602.uv2 属性，将其晶振频率设置为 11.0592MHz，选择输出可执行文件，仿真方式为选择硬仿真，并选择其中的“PROTEUS VSM MONITOR 51 DRIVER”仿真器。
- ⑤ 构造 (Build) 工程 lcd1602.uv2。如果输入有误进行修改，直至构造正确，生成可执行程序 lcd1602.hex 为止。
- ⑥ 为 AT89C51 设置可执行程序 lcd1602.hex。
- ⑦ 运行程序，点击图 2.28 中各按键，观察 1602 的显示是否符合程序要求。

(5) 实验作业

- 分析源程序并为源程序添加注释
- 记录示例程序运行结果截图。
- 总结 c51 控制 LCD1602 的实现方法，理解有关控制指令

3.7.3 LCD1602 自我完成实验

(1) 实验内容

在 Proteus 环境下搭建如下图 3.29 所示的电路图：



图 3.30 LCD1602 实现的十进制计算器显示示例

(3) 编程思路

按键的扫描与识别可以参考“3.4.1 示例实验”中的方法。LCD1602 的显示控制可以参考“3.7.2 LCD1602 示例实验”。

编成时要有一个状态变量，该变量用于记录当前是输入的哪个变量。输入第一个变量，遇到输入运算符时结束第一个变量的输入。输入第二个变量，遇到“=”号时结束第二个变量的输入，并且开始计算结果。

计算结果由于是 16 进制的，要将其转换成十进制，并将该十进制的数转换成字符串后逐位显示出来。减法时要注意结果是否为负，除法时要注意除数是否为 0，结果是否带有余数。

另外，按键要注意去抖动处理。

(4) 实验步骤

- ① 根据上述实验内容，参考 1.2.2，在 Proteus 环境下建立图 3.22 所示原理图，并将其保存为 LCD1602_self.DSN 文件。
- ② 根据 (2) 和 (3) 画出流程图，并编写源程序，将其保存为 LCD1602_self.c。

- ③ 运行 Keil uVision2 开发环境，按照 1.1.3 节介绍的方法建立工程 LCD1602_self.uv2，CPU 为 AT89C51，包含启动文件 STARTUP.A51。
- ④ 按照 1.2.2 第（6）节介绍的方法将 C 语言源程序 LCD1602_self.c 加入工程 LCD1602_self.uv2，并设置工程 LCD1602_self.uv2 属性，将其晶振频率设置为 12MHz，选择输出可执行文件，仿真方式为选择硬仿真，并选择其中的“PROTEUS VSM MONITOR 51 DRIVER” 仿真器。
- ⑤ 构造（Build）工程 LCD1602_self.uv2。如果输入有误进行修改，直至构造正确，生成可执行程序 LCD1602_self.hex 为止。
- ⑥ 为 AT89C51 设置可执行程序 LCD1602_self.hex。
- ⑦ 运行程序，点击按键输入数据与运算符，计算，观察计算结果，并验证其是否正确。
- ⑧ 输入过程中，按“清除按键”观察结果，重新输入数据计算并验证。

(5) 实验作业

- 画出流程图，编写源程序并进行注释
- 实验过程记录
- 记录程序运行结果截图
- 分析该实验还有什么可以进一步优化的地方，并尝试去实现。

3.8 实验 8：ADC0808/9 信号采集实验

3.8.1 示例实验

(1) 实验内容

在 Proteus 环境下搭建如下图 3.31 所示的电路图：

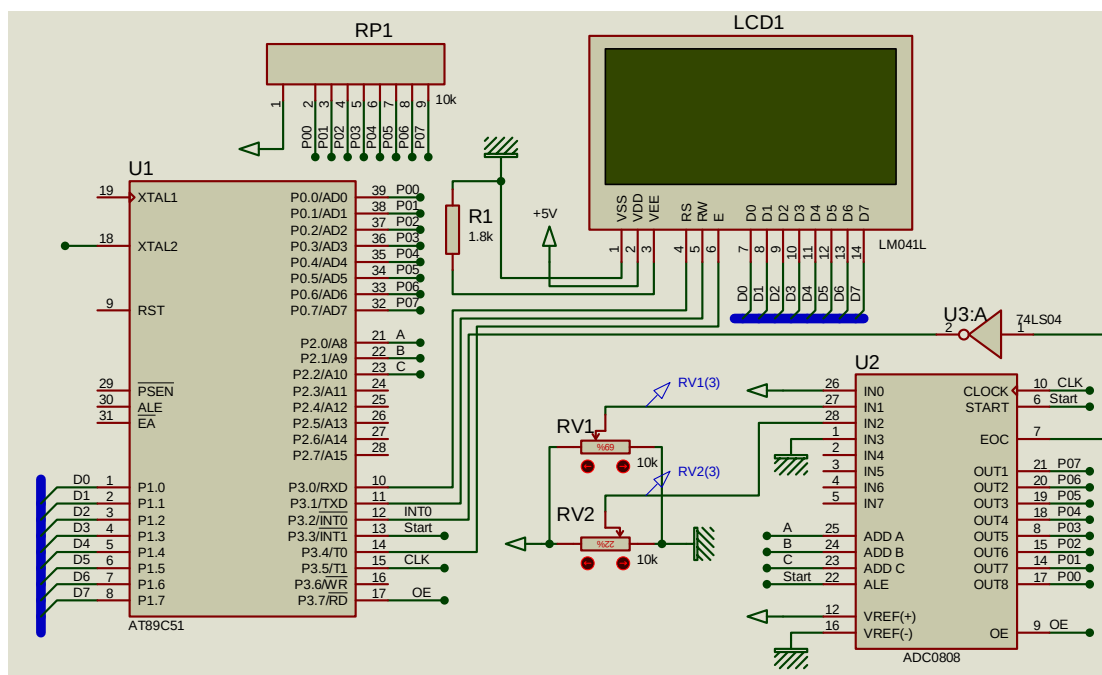


图 3.31 ADC0808/9 数据采集示例

其中所用元器件如下表所示：

选件编号	元件名称	参数	所在元件库类名	子类名	生产厂家
U1	AT89C51		Microprocessor ICs 微处理器	8051 Family 8051 家族	ATMEL
LCD1	LM041L		Optoelectronic 光电器件	Alpha Numeric LCDs 字符数字 LCD	
U2	ADC0808		Data Converters 数据转换器	A/D Converters A/D 转换器	
U3	74LS04		TTL 74LS series	Gates&Inverters 门与反向器	
RP1	RESPACK-8	10k-8	Resistors 电阻	Resistor Packs 排阻	
RV1, RV2	POT-HG	10k	Resistors 电阻	Variable 可变的	
R1	RES	1.8k	Resistors 电阻	Generic 一般的	

原理图说明：

(1) Proteus 中没有 ADC0809 的仿真模型，但是有 ADC0808 的仿真模型，二者功能和结构及连接线上都完全一致相似，区别只是转换精度稍有不同，前者为 $\pm 1\text{LSB}$ ，后者为 $\pm 1/2\text{LSB}$ 。

(2) LCD1604(LM041L)为 4*16 字符的 LCD 显示器，和 LCD1602 的区别是多了两行，各行字符地址为：

行号 \ 列号	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	80	81	82	83	84	85	86	87	88	89	8A	8B	8C	8D	8E	8F
2	C0	C1	C2	C3	C4	C5	C6	C7	C8	C9	CA	CB	CC	CD	CE	CF
3	90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F
4	D0	D1	D2	D3	D4	D5	D6	D7	D8	D9	DA	DB	DC	DD	DE	DF

LCD1604 和 LCD1602 的控制命令一样，时序要求也一样，编程时可以参考 LCD1602 的控制方法。

(3) 定时器 T1 产生 10KHz 的 CLK 方波信号。

(2) 功能要求

要求：程序运行后自动轮询采集 ADC0808 四路输入 In0、In1、In2、In3 的电压，并在 LCD1604 上显示出来。

(3) 示例程序

//将如下 C 语言源程序保存为 ADC0809.c

```
/******必要的变量定义*****  
*****/  
#include <reg51.h>  
#include <intrins.h> //包含_nop_()函数  
#define uint unsigned int  
#define uchar unsigned char  
uchar data line_data[16]; //要显示的一行字符  
uchar code numchar[]={ '0','1','2','3','4','5','6','7','8','9'};  
sbit LCD_RS=P3^0;  
sbit LCD_RW=P3^1;  
sbit LCD_EN=P3^4;  
sbit AD_CLK=P3^5;  
sbit Start=P3^3;  
sbit OE=P3^7;  
uint var,tmpint;  
long tmplong;  
uchar channel,samp_data,tmpchar,n;  
bit end_samp;  
  
/*****延时子程序*****/  
void delay_ms(uint xms)  
{  
    uint i,j;  
    for(i=xms;i>0;i--)  
        for(j=110;j>0;j--);  
}  
  
/*****LCD 忙检查子程序*****  
*****/  
bit lcd_busy()  
{  
    bit flag;  
    LCD_RS=0;  
    LCD_RW=1;
```



```

LCD_EN=1;
_nop_();
_nop_();
_nop_();
_nop_();
flag=(bit) (P1&0x80);
LCD_EN =0;
return flag;
}

```

/******写命令子程序******/

```

void lcd_wcmd(uchar cmd)
{
    while(lcd_busy());
    LCD_RS=0;
    LCD_RW=0;
    LCD_EN=0;
    _nop_();
    _nop_();
    P1=cmd;
    _nop_();
    _nop_();
    _nop_();
    _nop_();
    LCD_EN=1;
    _nop_();
    _nop_();
    _nop_();
    _nop_();
    LCD_EN=0;
}

```

/******LCD 清屏子程序******/

```

void lcd_clr()
{
    lcd_wcmd(0x01);
    delay_ms(2);
}

```

```
}
```

```
/******写数据子程序******/
```

```
void lcd_wdat(uchar dat)
```

```
{
    while(lcd_busy());
    LCD_RS=1;
    LCD_RW=0;
    LCD_EN=0;
    _nop_();
    _nop_();
    P1=dat;
    _nop_();
    _nop_();
    _nop_();
    _nop_();
    LCD_EN=1;
    _nop_();
    _nop_();
    _nop_();
    _nop_();
    LCD_EN=0;
}
```

```
/******初始化子程序******/
```

```
void lcd_init()
```

```
{
    delay_ms(5);
    lcd_wcmd(0x01); //清除 LCD 显示内容
    delay_ms(5);
    lcd_wcmd(0x06); //移动光标
    delay_ms(5);
    lcd_wcmd(0x0c); //显示开，关光标
    delay_ms(5);
    lcd_wcmd(0x38); //16*4 显示，5*7 点阵
    delay_ms(15); //等待 LCD 电源稳定
}
```

```

/*****字符显示子程序*****/
void showstring(uchar m)
{
    uchar i;
    switch(m)
    {
        case 0:
            lcd_wcmd(0x80); //设置显示位置为第 1 行第 0 列
            break;
        case 1:
            lcd_wcmd(0xC0); //设置显示位置为第 2 行第 0 列
            break;
        case 2:
            lcd_wcmd(0x90); //设置显示位置为第 3 行第 0 列
            break;
        case 3:
            lcd_wcmd(0xD0); //设置显示位置为第 4 行第 0 列
    }
    i=0;
    while(line_data[i]!='\0')
    {
        lcd_wdat(line_data[i]); //加载显示字符
        delay_ms(5);
        i++;
    }
}

/*****外部中断 0 服务子程序*****/
void s_int0() interrupt 0 //ADC0808 采集完毕
{
    P0=0xff;
    delay_ms(1);
    OE=1; //打开 ADC0808 的输出使能
    samp_data=P0;
    OE=0; //关闭 ADC0808 的输出使能
    end_samp=1;
    EX0=0; //关闭外部中断 0
}

```

```

/*****外部中断 0 服务子程序*****/
void s_timer1() interrupt 3 //ADC0808 采集完毕
{
    AD_CLK=~AD_CLK ;
}

/*****ADC0808 数据采集触发子程序*****/
void sample(uchar ch)
{
    while(!end_samp); //等待转换完成
    tmplong=(long) samp_data*5*1000;
    var=(uint) (tmplong/255);
    if(var==0)
    {
        n=0;
        line_data[n++]='C' ;
        line_data[n++]='h' ;
        line_data[n++]='0'+ch;
        line_data[n++]=':' ;
        line_data[n++]='0' ;
        line_data[n++]='V' ;
    }
    else
    {
        n=0;
        line_data[n++]='C' ;
        line_data[n++]='h' ;
        line_data[n++]='0'+ch;
        line_data[n++]=':' ;
        tmpint=var;

        tmpchar=(uchar) (tmpint/1000);
        tmpint=tmpint-(uint) tmpchar*1000;
        line_data[n++]=numchar[tmpchar];
    }
}

```

```

        line_data[n++]='.';

        tmpchar=(uchar) (tmpint/100);
        tmpint=tmpint-(uint) tmpchar*100;
        line_data[n++]=numchar[tmpchar];

        tmpchar=(uchar) (tmpint/10);
        tmpint=tmpint-(uint) tmpchar*10;
        line_data[n++]=numchar[tmpchar];

        tmpchar=(uchar) (tmpint);
        line_data[n++]=numchar[tmpchar];
        line_data[n++]='V';
    }
    line_data[n]='\0';
}

/*****主程序*****/
void main()
{
    delay_ms(10);
    lcd_init(); //初始化
    lcd_clr(); //清屏
    delay_ms(2);
    channel=0; //计算状态为状态 0
    var=0;
    line_data[n]='\0';
    AD_CLK=0;
    OE=0;

    TMOD=0x20; //T1 工作在方式 2
    TH1=0xE7; //10KHz
    TL1=0xE7;
    EA=1;      //打开中断总开关
    EX0=1;     //打开外部中断 0
    IT0=1;     //设置中断触发方式为下降沿触发方式

```

```

ET1=1;
TR1=1;
while(1)
{
    end_samp=0;
    EX0=1;    //打开外部中断允许位
    P2=channel;
    Start=1;  //启动 AD
    delay_ms(2);
    Start=0;  //启动信号结束
    sample(channel); //采集
    showstring(channel); //输出
    channel++; //通道变更
    if(channel==4)
        channel=0;
}
}

```

(4) 实验步骤

- ① 根据上述实验内容，参考 1.2.2，在 Proteus 环境下建立图 3.31 所示原理图，并将其保存为 ADC0808.DSN 文件。
- ② 将上面 (3) 中控制源程序保存为 ADC0808.c。
- ③ 运行 Keil uVision2 开发环境，按照 1.1.3 节介绍的方法建立工程 ADC0808.uv2，CPU 为 AT89C51，包含启动文件 STARTUP.A51。
- ④ 按照 1.2.2 第(6)节介绍的方法将 C 语言源程序 ADC0808.c 加入工程 ADC0808.uv2，并设置工程 ADC0808.uv2 属性，将其晶振频率设置为 12MHz，选择输出可执行文件，仿真方式为选择硬仿真，并选择其中的“PROTEUS VSM MONITOR 51 DRIVER”仿真器。
- ⑤ 构造 (Build) 工程 ADC0808.uv2。如果输入有误进行修改，直至构造正确，生成可执行程序 ADC0808.hex 为止。
- ⑥ 为 AT89C51 设置可执行程序 ADC0808.hex。
- ⑦ 运行程序，点击图 3.31 中的可变电位器 RV1 和 RV2 各按键，观察 1604 的显示变化情况。
- ⑧ 在图 3.31 中的可变电位器 RV1 和 RV2 中间抽头(即：ADC0808 的 IN2 和 IN3 输入口处)添加电压探针，比较电压探针探测结果 LCD1604 的显示结果是否近似一致。

(5) 实验作业

- 分析源程序并为源程序添加注释
- 记录示例程序运行结果截图。
- 总结 c51 控制 ADC0808 的实现方法，理解有关控制时序。

3.8.2 自我完成实验

(1) 实验内容

在 Proteus 环境下搭建如下图 3.32 所示的电路图:

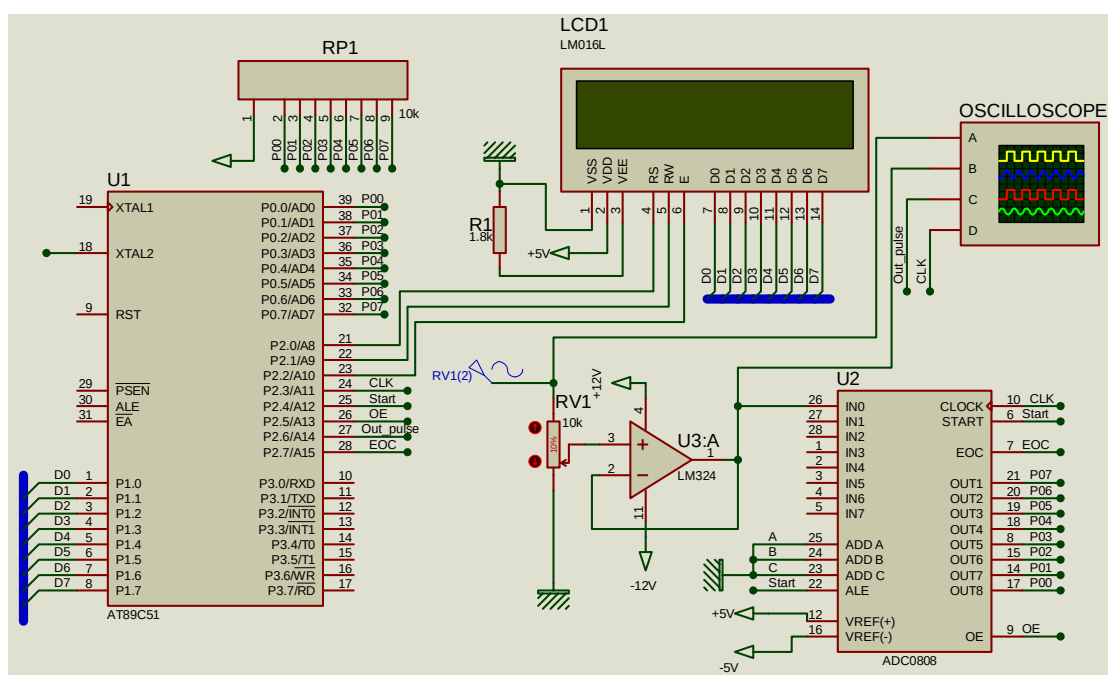


图 3.32 交流信号频率与过零点检测

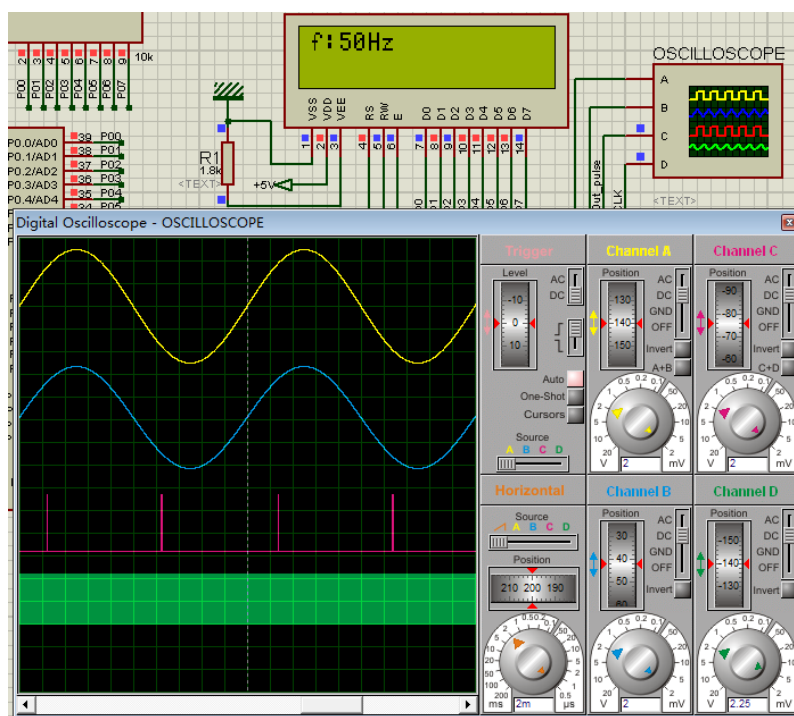
其中所用元器件如下表所示:

选件编号	元件名称	参数	所在元件库类名	子类名	生产厂家
U1	AT89C51		Microprocessor ICs 微处理器	8051 Family 8051 家族	ATMEL
LCD1	LM016L		Optoelectronic 光电器件	Alpha Numeric LCDs 字符数字 LCD	
U2	ADC0808		Data Converters 数据转换器	A/D Converters A/D 转换器	
U3	LM324		Operational Amplifiers 运算放大器	Quad 四	

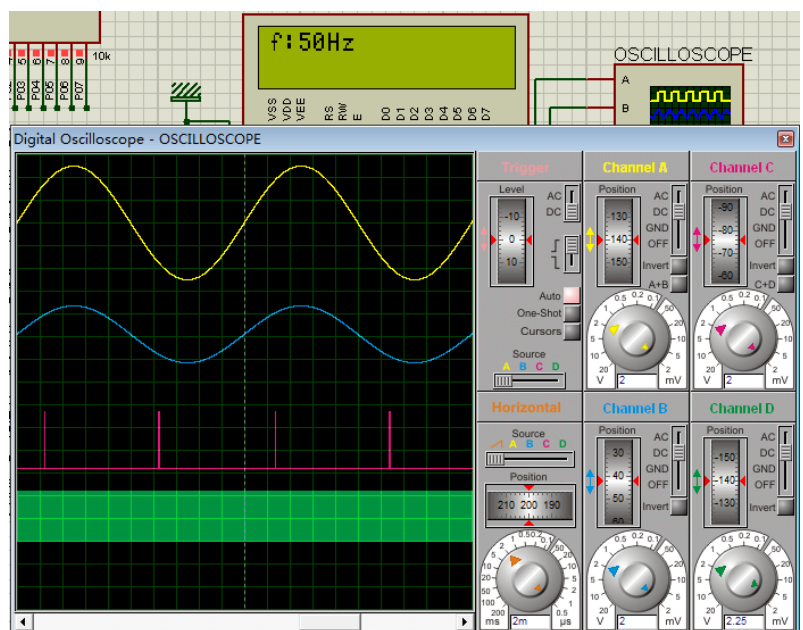
RP1	RESPACK-8	10k-8	Resistors 电阻	Resistor Packs 排阻	
RV1	POT-HG	10k	Resistors 电阻	Variable 可变的	
R1	RES	1.8k	Resistors 电阻	Generic 一般的	

(2) 控制要求

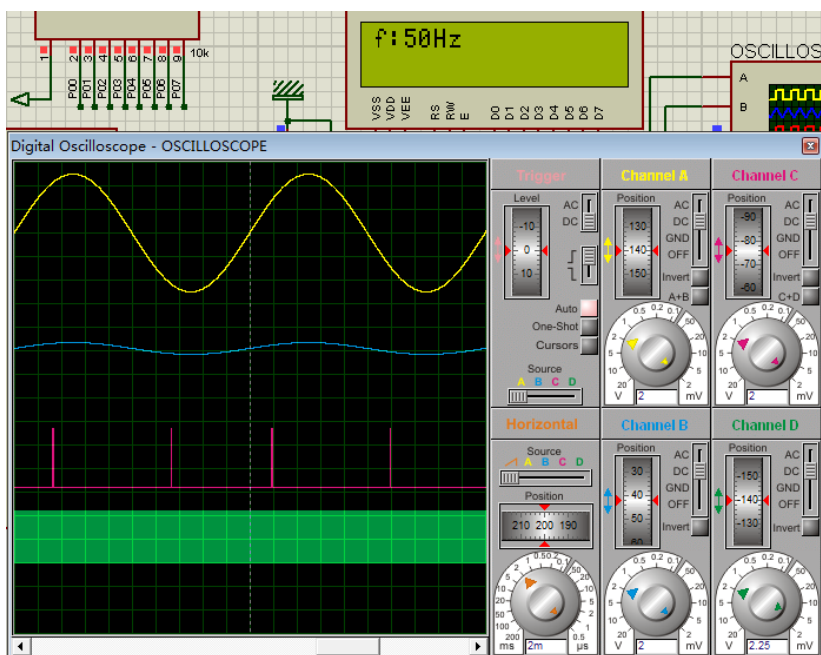
本实验利用 LCD1602 和 AD0808 实现简单的交流信号过零检测与频率分析。要求信号幅度变化时(满量程的 5%-95%), 不影响检测的结果。频率检测的结果通过 LCD1602 的第一行显示出来, 信号过零时, 能够通过 P2.6 输出一个脉冲宽度为 5us 的脉冲信号。检测结果如下图 3.33 所示:



(a) 信号幅值为满量程的 90%



(b) 信号幅值为满量程的 50%



(c) 信号幅值为满量程的 10%

图 3.33 交流信号频率检测结果

图 3.32 中电位器 RV1 用于改变交流信号的幅值。图中的交流信号通过点击窗口左侧绘图工具窗口中的虚拟信号发生器  按钮,然后在器件选择窗口中选择正弦信号(SINE)。如下图 3.33 所示:

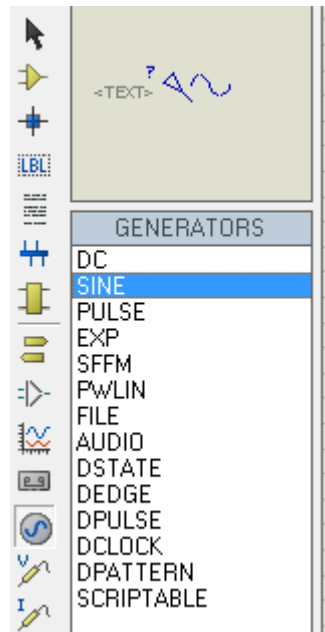


图 3.33 交流信号发生器选择

将信号发生器连入图 3.32 中的电位器 RV1 的上端，RV1 的中间抽头连入 AD0808 的 0 通道输入 IN0。

交流信号发生器加入之后就要设置它的属性，方法是双击所加入的信号发生器, 进入其属性设置页面，如下图 3.34 所示：

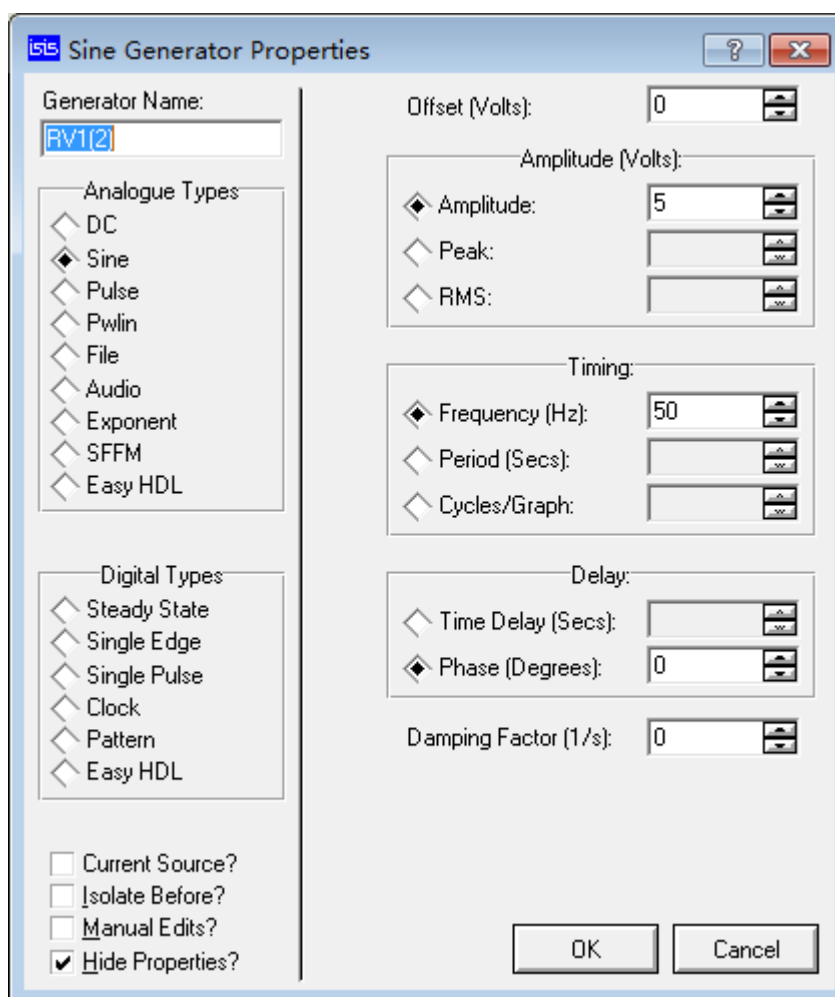


图 3.34 交流信号发生器设置窗口

其中信号的幅值和频率按图 3.34 所示设置。

图 3.32 中的虚拟示波器的加入是为了测量各种信号的波形。

(3) 编程思路

LCD1602 的控制方法按 3.7 节所示方法进行 ADC0808 的控制方法按 3.8.1 所示方法进行。这里主要是过零点的检测方法如何实现。不能采用判断所采集到的数据是否为 0 的方法来实现，因为你的采集时刻不一定能够严格对准过零时刻。但是，我们注意到在 0 点的两边信号的极性是发生变化的，我们可以利用这一特点来实现过零检测。正弦波每个周期有两个过零点，因此，1s 内过零次数除以 2 就是信号的频率。

因此，在程序中可以这样实现。当每次采集到一个新的数据之后都要看一下这个数据是正数还是负数。当这个数大于 128 时是正数，当它小于 128 时是负数。判断当前数据的正负极性和上一个数据的正负极性是否一致，如果不一致，则说明经过了一次过零点，将其记录入次数计数器。

ADC0808 的 CLK 仍然用定时器 T1 来实现，可以将其设置为 50kHz（硬件实现时可以更高，软件仿真再高将难以实现）。利用定时器 T0 实现 50ms 定时，并配合软件实现 1s 钟定时。采用 12M 晶振时，T0 采用方式 1，则处置应为（TH0=0x3C，TL0=0xB0）。

但是，由于中断处理函数需要一定的响应时间，因此这个参数只是理论计算结果，要根据实测情况稍作调整。

同样 T1 理论计算值和实际输出值可能也会有一定的差距，也要进行调整。

(4) 实验步骤

- ① 根据上述实验内容，参考 1.2.2，在 Proteus 环境下建立图 3.22 所示原理图，并将其保存为 ADC0808_self.DSN 文件。
- ② 根据（2）和（3）画出流程图，并编写源程序，将其保存为 ADC0808_self.c。
- ③ 运行 Keil uVision2 开发环境，按照 1.1.3 节介绍的方法建立工程 ADC0808_self.uv2，CPU 为 AT89C51，包含启动文件 STARTUP.A51。
- ④ 按照 1.2.2 第（6）节介绍的方法将 C 语言源程序 ADC0808_self.c 加入工程 ADC0808_self.uv2，并设置工程 ADC0808_self.uv2 属性，将其晶振频率设置为 12MHz，选择输出可执行文件，仿真方式选择硬仿真，并选择其中的“PROTEUS VSM MONITOR 51 DRIVER”仿真器。
- ⑤ 构造（Build）工程 ADC0808_self.uv2。如果输入有误进行修改，直至构造正确，生成可执行程序 ADC0808_self.hex 为止。
- ⑥ 为 AT89C51 设置可执行程序 ADC0808_self.hex。
- ⑦ 运行程序，观察计算结果，并验证其是否正确。
- ⑧ 改变 RV1 的抽头位置，从而改变输入信号的幅值，观察计算结果是否正确。
- ⑨ 更改信号发射器的频率，再次验证其功能是否正确。（注意：因为是软仿真，所以信号采集的速度受到限制，因此所输入的交流信号频率也不能太高，可以在 200Hz 以内尝试）。

(5) 实验作业

- 画出流程图，编写源程序并进行注释
- 实验过程记录
- 记录程序运行结果截图

3.9 实验 9：DAC0832 应用实验

3.9.1 示例实验

(1) 实验内容

在 Proteus 环境下搭建如下图 3.35 所示的电路图：

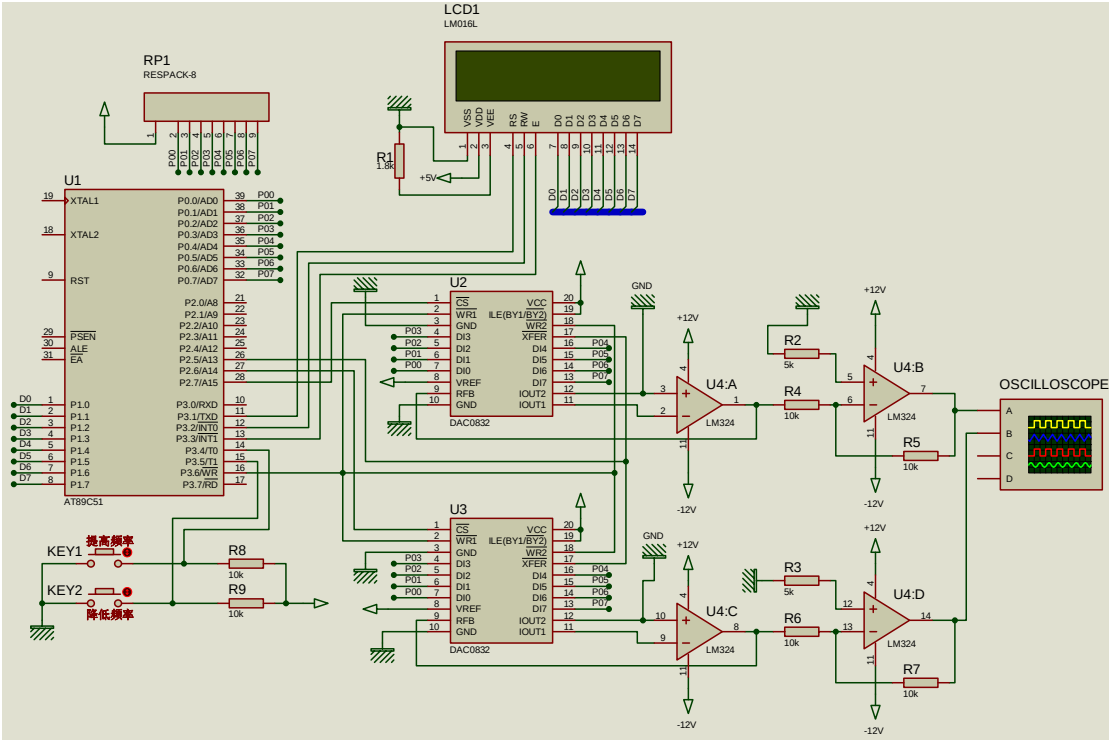


图 3.35 DAC0832 应用实验

其中所用元器件如下表所示：

选件编号	元件名称	参数	所在元件库类名	子类名	生产厂家
U1	AT89C51		Microprocessor ICs 微处理器	8051 Family 8051 家族	ATMEL
LCD1	LM041L		Optoelectronic 光电器件	Alpha Numeric LCDs 字符数字 LCD	
U2,U3	DAC0832		Data Converters 数据转换器	D/A Converters D/A 转换器	
U4	LM324		Operational Amplifier 运算放大器	Quad	
KEY1, KEY2	BUTTON		Switches & Relays 开关与继电器	Switches 开关	
RP1	RESPACK-8	10k- 8	Resistors 电阻	Resistor Packs 排阻	
R1	RES	1.8k	Resistors 电阻	Variable 可变的	
R2, R3	RES	5k	Resistors 电阻	Generic 一般的	
R4-R9	RES	10k	Resistors 电阻	Generic 一般的	

(2) 功能要求

要求：在程序运行后在 U4B 的输出即：7 号引脚输出正弦波，U4D 的输出即：14 引脚输出锯齿波。两路输出周期相同，并且信号同步。信号的频率有 6 种：10Hz，50Hz，100Hz，200Hz，500Hz，1kHz。程序运行后的初始频率为 10Hz，每按一次 KEY1 按键频率增加为下一较高频率即：50Hz，再按一次 KEY1 再增加为 100Hz，依次类推，直到增加到 1kHz 为止。反之，每按一次 KEY2 按键，则频率降低 1 级，直到降低到 10Hz 为止。

(3) 示例程序

```
/******必要的变量定义******/
#include <reg51.h>
#include <intrins.h> //包含_nop_()函数
#include <math.h>
#define uint unsigned int
#define uchar unsigned char
uchar data line_data1[16]; //要显示的第 1 行字符
uchar data result_char[16];
uchar code numchar[]={'0','1','2','3','4','5','6','7','8','9'};
uint code fN[]={10, 50, 100, 200, 500, 1000}; //信号频率为 10Hz, 50Hz,
//100Hz, 200Hz, 500Hz, 1kHz
uint code SN[]={500, 200, 100, 100, 50, 50}; //每周周期采样点数为：500,
//200, 100, 100, 50, 50
uchar code ST[]={56, 156, 156, 206, 216, 236}; //定时器初值，对应的采样点间隔
时
//间为：200us, 100us, 100us, 50us,
//40us, 20us
float code sincoef[]={0.0126, 0.0314, 0.0628, 0.0628, 0.1257, 0.1257};
//x=sin(2*pi*f/t)=sin(2*pi*n/NT),
//coef=2*pi/NT, n=0...NT-1

sbit LCD_RS=P3^1;
sbit LCD_RW=P3^2;
sbit LCD_EN=P3^3;
sbit AD_CLK=P2^3;
sbit UpKey=P3^4;
sbit DownKey=P3^5;
uchar xdata SinAdd _at_ 0x7f00;
```

```

uchar xdata SawAdd _at_ 0xbf00;
uchar xdata StartTranAdd _at_ 0xdf00;
uint tmpint, NT, n, frequency, x_saw;
float x_sin, y_sin;
uchar sinDA_data, sawDA_data, tmpchar, speed;
bit newdata;
/*****延时子程序*****/
void delay_ms(uint xms)
{
    uint i, j;
    for(i=xms; i>0; i--)
        for(j=110; j>0; j--);
}
/*****LCD 忙检查子程序*****/
*****/
bit lcd_busy()
{
    bit flag;
    LCD_RS=0;
    LCD_RW=1;
    LCD_EN=1;
    _nop_();
    _nop_();
    _nop_();
    _nop_();
    flag=(bit) (P1&0x80);
    LCD_EN =0;
    return flag;
}
/*****写命令子程序*****/
void lcd_wcmd(uchar cmd)
{
    while(lcd_busy());
    LCD_RS=0;
    LCD_RW=0;
    LCD_EN=0;
    _nop_();

```

```

    _nop_();
    P1=cmd;
    _nop_();
    _nop_();
    _nop_();
    _nop_();
    LCD_EN=1;
    _nop_();
    _nop_();
    _nop_();
    _nop_();
    LCD_EN=0;
}

/*****LCD 清屏子程序*****/
void lcd_clr()
{
    lcd_wcmd(0x01);
    _nop_();
    _nop_();
    _nop_();
    _nop_();
}

/*****写数据子程序*****/
void lcd_wdat(uchar dat)
{
    while(lcd_busy());
    LCD_RS=1;
    LCD_RW=0;
    LCD_EN=0;
    _nop_();
    _nop_();
    P1=dat;
    _nop_();
    _nop_();
    _nop_();
    _nop_();
    LCD_EN=1;

```



```

    _nop_();
    _nop_();
    _nop_();
    _nop_();
    LCD_EN=0;
}

/*****初始化子程序*****/
void lcd_init()
{
    delay_ms(5);
    lcd_wcmd(0x01); //清除 LCD 显示内容
    delay_ms(5);
    lcd_wcmd(0x06); //移动光标
    delay_ms(5);
    lcd_wcmd(0x0c); //显示开, 关光标
    delay_ms(5);
    lcd_wcmd(0x38); //16*2 显示, 5*7 点阵
    delay_ms(15); //等待 LCD 电源稳定
}

/*****字符显示子程序*****/
void showstring()
{
    uchar i;
    lcd_wcmd(0x80); //设置显示位置为第 1 行第 16 列
    i=0;
    while(line_data1[i]!='\0')
    {
        lcd_wdat(line_data1[i]); //加载第 1 行字符
        i++;
    }
}

/*****DAC0832 数据转换子程序*****/
void DA()
{
    x_sin=sincoef[speed]*n;
    y_sin=sin(x_sin)/2+0.5;
    sinDA_data=(uchar)(y_sin*255);
}

```

```

    SinAdd=sinDA_data;
    x_saw=n*255/NT;
    sawDA_data=(uchar) (x_saw);
    SawAdd=sawDA_data;
    _nop_();
    StartTranAdd=0x0;
    _nop_();
}

/*****定时器 1 服务子程序*****/
void s_timer1() interrupt 3    //DAC0832 输出控制定时器
{
    ET1=0;
    newdata=1;
    ET1=1;
}

/*****频率字符串转换子程序*****/
void Change_f_to_str()
{
    uchar i,j;
    uint result;
    if(frequency==0)
    {
        n=0;
        line_data1[n++]='f';
        line_data1[n++]=':';
        line_data1[n++]='0';
        line_data1[n++]='H';
        line_data1[n++]='z';
    }
    else
    {
        n=0;
        line_data1[n++]='f';
        line_data1[n++]=':';
        result=frequency;
        i=0;

```

```

while(result!=0) //完成结果转换成字符串，但从低位到高位逆序排列
{
    tmpchar=(uchar)(result%10);
    result=result/10;
    result_char[i++]=numchar[tmpchar];
}
i--;
for(j=0;j<=i;j++) //将结果按正序添加到第 1 行显示字符串中
{
    line_data1[n++]=result_char[i-j];
}
line_data1[n++]='H';
line_data1[n++]='z';
}
line_data1[n]='\0';
}

/*****主程序*****/

void main()
{
    delay_ms(10);
    lcd_init(); //初始化
    lcd_clr(); //清屏
    delay_ms(2);
    speed=0;
    newdata=0;
    n=0;
    NT=SN[speed];
    frequency=fN[speed];
    line_data1[n]='\0';
    Change_f_to_str();
    showstring();
    TMOD=0x20; //T1 工作在方式 2
    TH1=ST[speed];
    TL1=ST[speed];
    ET1=1;
    TR1=1;
    EA=1; //打开中断总开关

```

```

while(1)
{
    while(!newdata);
    if(n==NT)
    {
        n=0;
    }
    DA();
    n++;
    newdata=0;
UpKey=1;
if(!UpKey) //如果有加速按键按下，则调整相应参数
{
    delay_ms(10);
    if(!UpKey)
    {
        if(speed<5)
            speed++;
        NT=SN[speed];
        frequency=fN[speed];
        Change_f_to_str();
        showstring();
        TH1=ST[speed];
        TL1=ST[speed];
        while(!UpKey) //等待加速按键释放
        {
            UpKey=1;
        }
    }
}
DownKey=1;
if(!DownKey) //如果有减速按键按下，则调整相应参数
{
    delay_ms(10);
    if(!DownKey)
    {
        if(speed>0)

```


- 记录示例程序运行结果截图。
- 总结 c51 控制 DAC0832 的实现方法
- 自己尝试将 U4 14 脚输出变为与 U4 7 脚输出同步的三角波。

3.9.2 自我完成实验

(1) 实验内容

在 Proteus 环境下搭建如下图 3.36 所示的电路图：

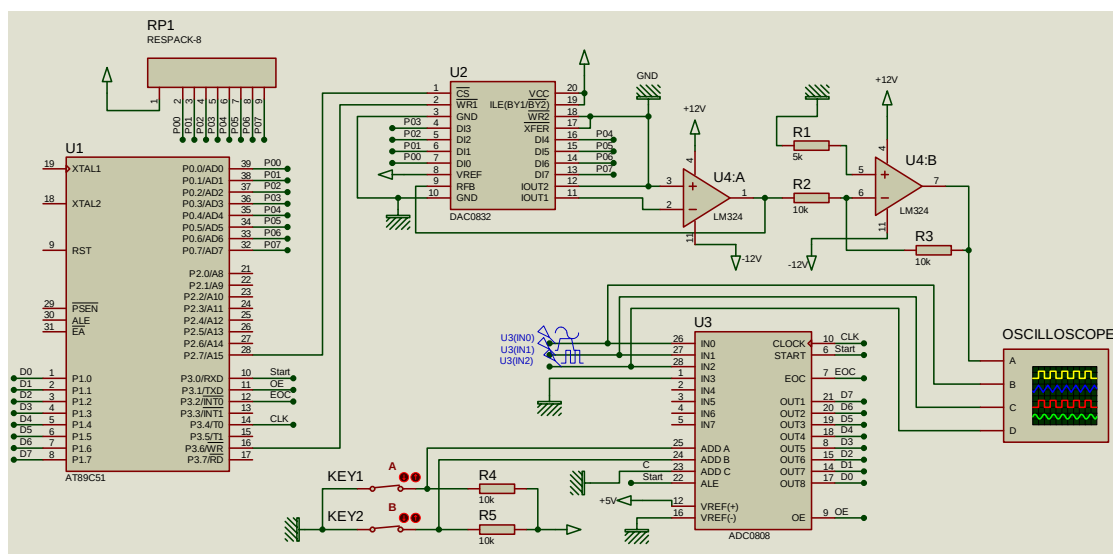


图 3.36 信号采集与输出实验

其中所用元器件如下表所示：

选件编号	元件名称	参数	所在元件库类名	子类名	生产厂家
U1	AT89C51		Microprocessor ICs 微处理器	8051 Family 8051 家族	ATMEL
U2	ADC0832		Data Converters 数据转换器	D/A Converters D/A 转换器	
U3	ADC0808		Data Converters 数据转换器	A/D Converters A/D 转换器	
U4	LM324		Operational Amplifiers 运算放大器	Quad 四	
RP1	RESPACK-8	10k-8	Resistors 电阻	Resistor Packs 排阻	
R1	RES	5k	Resistors 电阻	Generic 一般的	
R2-R5	RES	10k	Resistors 电阻	Generic 一般的	

(2) 控制要求

本实验利用 ADC0808 和 DAC0832 实现简单的信号采集与输出。ADC0808 的 4 路输入分别为：正弦波 In0, 三角波 In1, 方波 In2, 接地 In3。其中正弦波的设置如下图 3. 37-图 3. 39 所示：

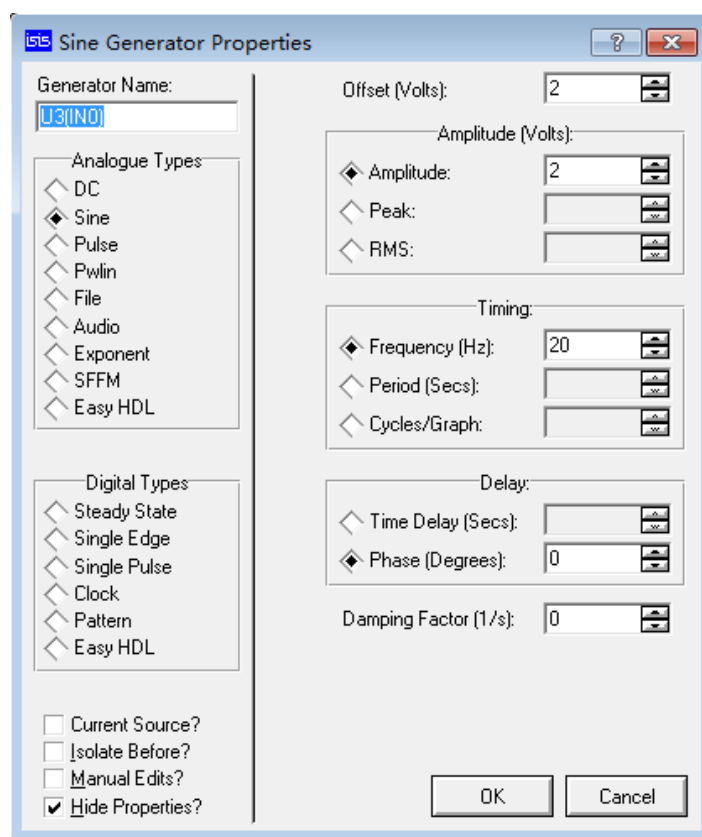


图 3. 37 ADC0808 In0 通道正弦波(sine)属性设置

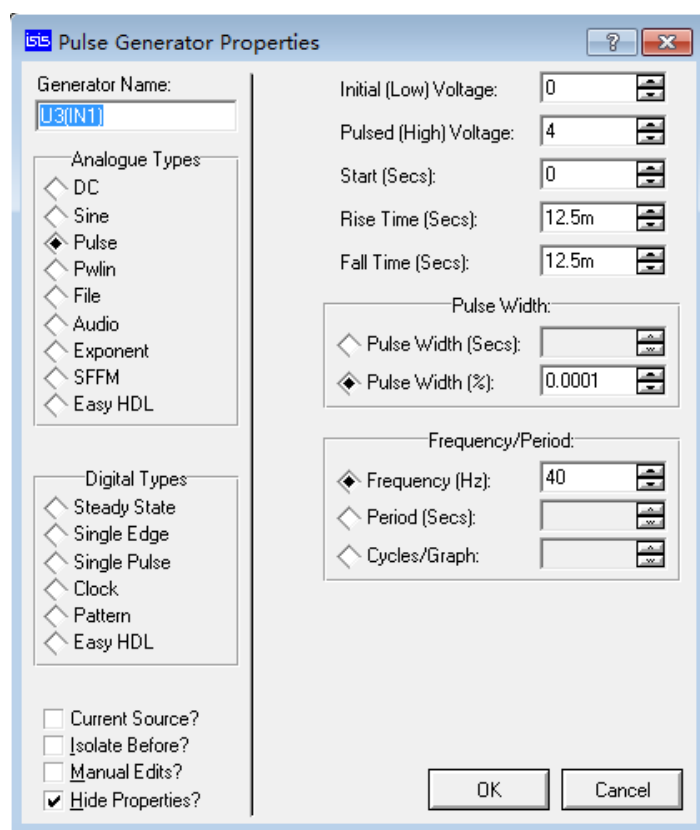


图 3.38 ADC0808 In1 通道三角波 (Pulse) 属性设置

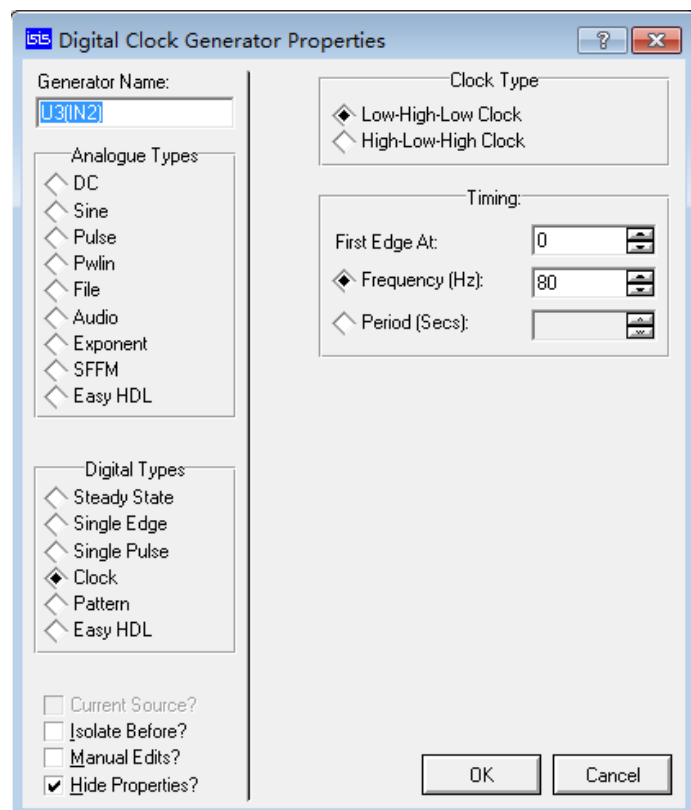


图 3.39 ADC0808 In2 通道方波 (Digital Clock) 属性设置

KEY1, KEY2 用于切换通道, 当开关打开时其所连接的位为 1, 闭合时所连接位为 0。当 KEY2 KEY1 的组合为 00 时选择 In0 通道, 为 01 时选择 In1 通道, 为 10 时选择 In2 通道, 为 11 时选择 In3 通道。

ADC0808 的采样值通过 DAC0832 送出, 采样输出结果如下图 3.40~3.43 所示:

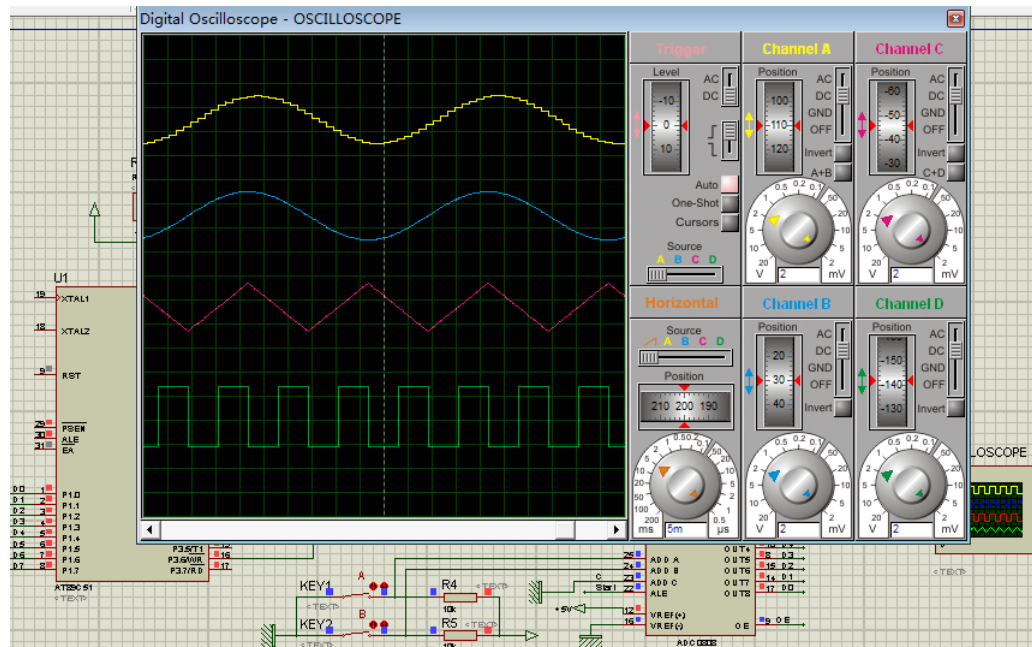


图 3.40 In0 通道采样输出

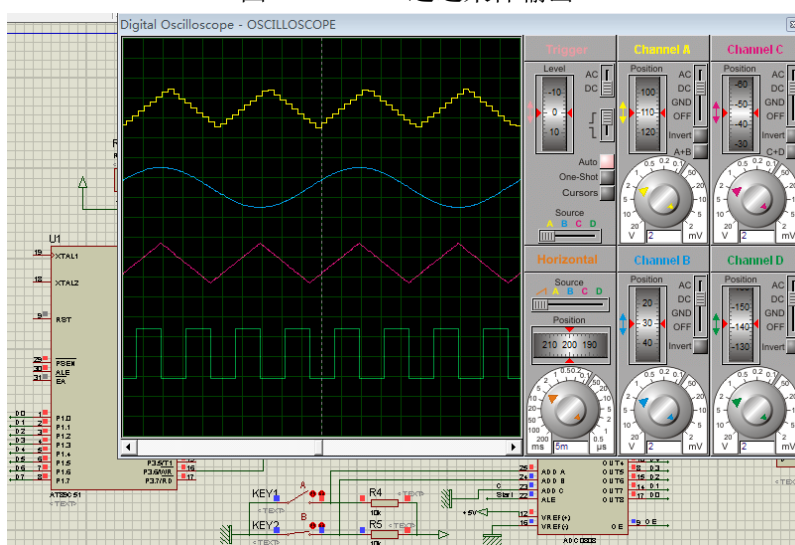


图 3.40 In1 通道采样输出

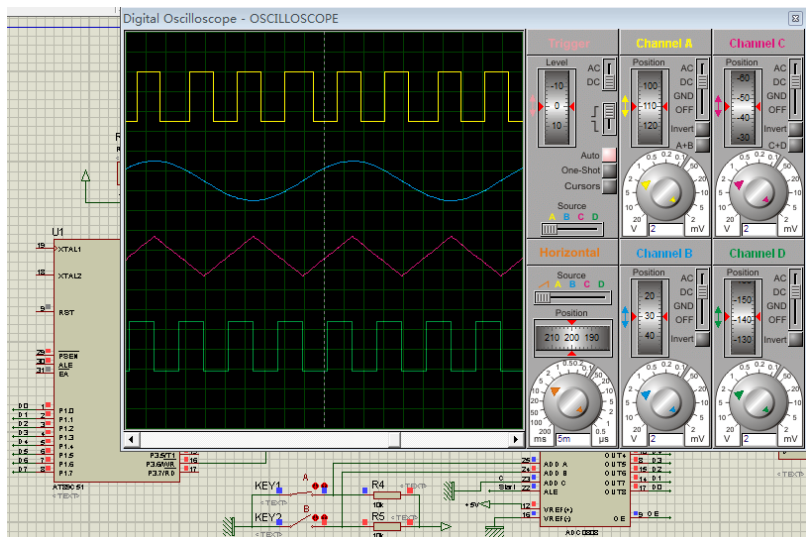


图 3.41 In2 通道采样输出

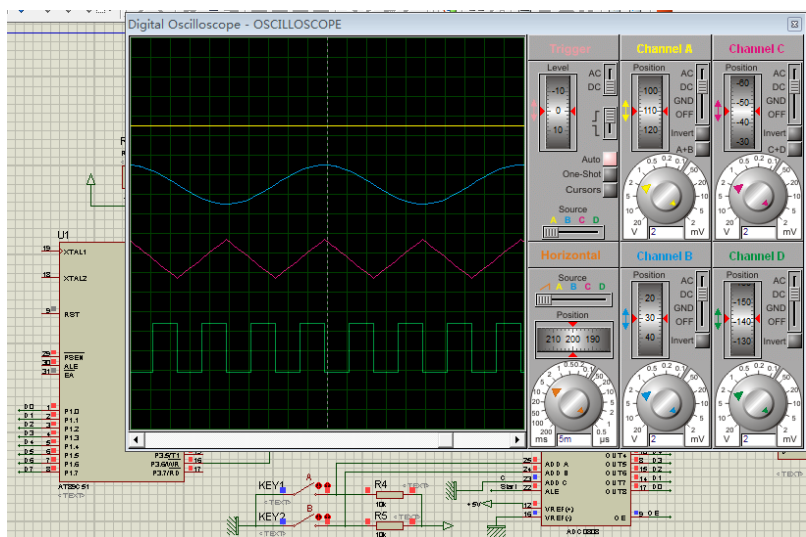


图 3.42 In3 通道采样输出

(3) 编程思路

结合实验 3.8 和实验 3.9.1 来实现。

(4) 实验步骤

- ① 根据上述实验内容，参考 1.2.2，在 Proteus 环境下建立图 3.22 所示原理图，并将其保存为 DAC0832_self.DSN 文件。
- ② 根据 (2) 和 (3) 画出流程图，并编写源程序，将其保存为 DAC0832_self.c。
- ③ 运行 Keil uVision2 开发环境，按照 1.1.3 节介绍的方法建立工程 DAC0832_self.uv2，CPU 为 AT89C51，包含启动文件 STARTUP.A51。
- ④ 按照 1.2.2 第 (6) 节介绍的方法将 C 语言源程序 DAC0832_self.c 加入工程 DAC0832_self.uv2，并设置工程 DAC0832_self.uv2 属性，将其晶振频率设置为 12MHz，选择输出可执行文件，仿真方式为选择硬仿真，并选择其中的“PROTEUS VSM MONITOR 51 DRIVER”仿真器。

- ⑤ 构造 (Build) 工程 DAC0832_self.uv2。如果输入有误进行修改，直至构造正确，生成可执行程序 DAC0832_self.hex 为止。
- ⑥ 为 AT89C51 设置可执行程序 DAC0832_self.hex。
- ⑦ 运行程序，观察运行结果，并验证其是否正确。
- ⑧ 更改信号发射器的频率，再次验证其功能是否正确。（注意：因为是软仿真，所以信号采集的速度受到限制，因此所输入的交流信号频率也不能太高，可以在 200Hz 以内尝试）。

(5) 实验作业

- 画出流程图，编写源程序并进行注释
- 实验过程记录
- 记录程序运行结果截图

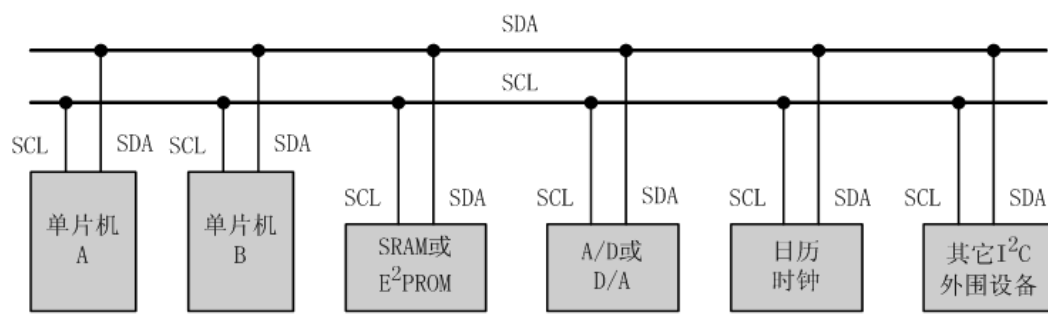
3.10 实验 10：I2C 扩展实验

3.10.1 I2C 总线简介

常用的串行扩展总线有：I2C (Inter IC BUS) 总线、单总线 (1-WIRE BUS)、SPI (Serial Peripheral Interface) 总线及 Microwire/PLUS 等。本节仅研究 I2C 串行总线及其应用扩展。

1、I2C 总线结构

I2C 总线是 PHILIPS 公司推出的一种串行总线。I2C 总线可以具备多个主机与多个从机，是一种高性能的同步串行总线。I2C 总线有两根双向信号线，分别是：数据线 SDA 和时钟信号线 SCL。其通信系统结构如下图 3.43 所示：



在 MCS-51 单片机应用系统 I2C 总线扩展中，经常以单片机为主机，其它接口器件为从机。

每个接接到 I2C 总线上的器件都有自己的地址。主机与这些器件之间进行数据通信时，将先发送需要与之通信的器件地址，所有器件收到地址帧之后进行判断，只有地址相符的器件才会做出响应。

2、I2C 总线时序

(1) 数据有效性规定

I2C 总线进行数据传送时，在时钟信号高电平期间，数据线上的数据必须保持稳定。只有在时钟线上的信号为低电平期间，数据线上的电平状态才允许变化，这就是 I2C 总线的数据有效性规定。如图 3.44 所示：

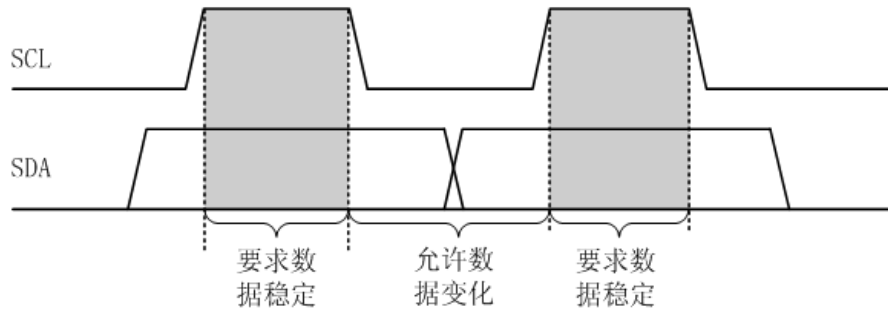


图 3.44 I2C 总线数据有效性规定

(2) 起始和终止信号规定

SCL 线为高电平期间，SDA 线由高电平向低电平的变化表示起始信号；SCL 线为高电平期间，SDA 线由低电平向高电平的变化表示终止信号。如图 3.45 所示：

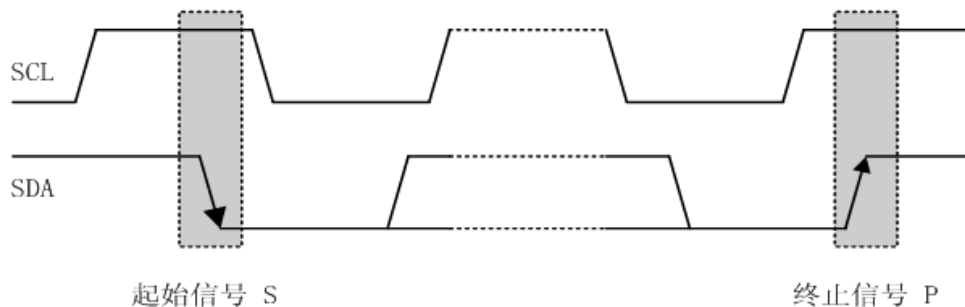


图 3.45 I2C 总线起始与终止信号规定

起始和终止信号只能由主机发出。在起始信号产生之后，总线处于被占用状态；在终止信号产生后，总线处于空闲状态。

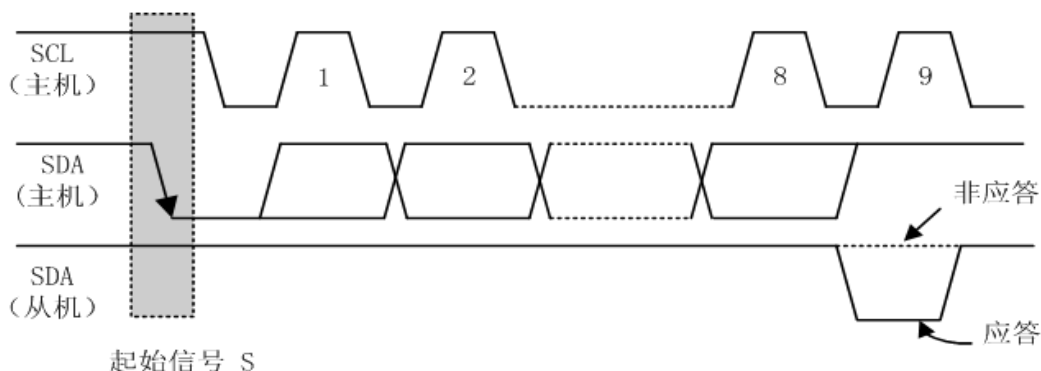
具有 I2C 总线硬件接口的器件很容易检测到起始和终止信号。不具备 I2C 总线硬件接口器件（如单片机）仿真实现 I2C 总线时，为了检测起始和终止信号，必须在每个 SCL 时钟周期内对数据线 SDA 采样两次。

接收器件收到一个完整的数据字节后，有可能需要完成一些其它工作（如保存所接收的数据并做出适当处理等），无法立刻接收下一个字节，这时接收器件可以将 SCL 线拉成低电平，从而使主机处于等待状态。直到接收器件准备好接收下一个字节时，再释放 SCL 线使之成为高电平，从而使数据传送可以继续进行。

(3) 数据传送格式

① 字节传送与应答

每一个字节必须保证是 8 位长度。数据传送时，先传送最高位（MSB），每一个被传送的字节后面都必须跟随一位应答位（即一帧共有 9 位）。



由于某种原因从机不对主机寻址信号应答时（如从机正在进行实时处理工作而无法接收总线上的数据），从机必须将数据线置于高电平，从而使主机产生一个终止信号以结束总线数据传送。

如果从机对主机进行了应答，但在数据传送一段时间后无法继续接收更多的数据时，从机可以通过对无法接收的第一个数据字节的“非应答”通知主机，主机则应发出终止信号以结束数据的继续传送。

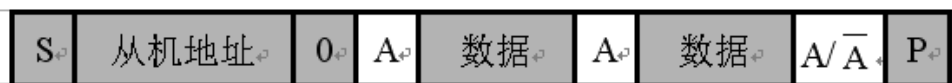
当主机接收数据时，它收到最后一个数据字节后，必须向从机发出一个结束传送的信号。这个信号是由对从机的“非应答”来实现的。然后，从机释放 SDA 线，以允许主机产生终止信号。

② 数据帧格式

I2C 总线上在起始信号后必须传送一个从机的地址（7 位），第 8 位是数据的传送方向位（R/W），用“0”表示主机发送数据（W），“1”表示主机接收数据（R）。每次数据传送总是由主机产生的终止信号结束。但是，若主机希望继续占用总线进行新的数据传送，则可以产生终止信号，马上再次发出起始信号对另一从机进行寻址。

在总线的一次数据传送过程中，可以有以下几种组合方式：

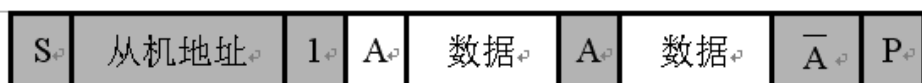
（a）主机向从机发送数据（主机→从机）



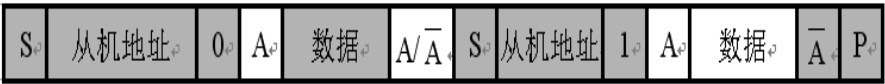
注：有阴影部分表示数据由主机向从机传送，无阴影部分则表示数据由从机向主机传送。

A 表示应答， $\bar{A}$ 表示非应答（高电平），S 表示起始信号，P 表示终止信号。

（b）主机在第一个字节后，立即由从机发送数据，主机接收数据（从机→主机）



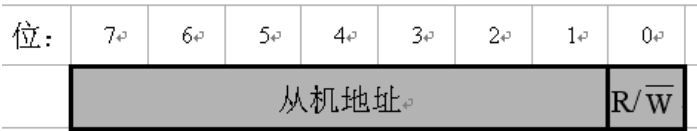
（c）、在传送过程中，当需要改变传送方向时（先主机→从机，再从机→主机）



④ 总线的寻址

I2C 总线明确规定：采用 7 位的寻址字节（寻址字节是起始信号后的第一个字节）。

(a) 寻址字节的位定义



D7～D1 位组成从机的地址。D0 位是数据传送方向位，为“0”时表示主机向从机写数据，为“1”时表示主机由从机读数据。

主机发送地址时，总线上的每个从机都将这 7 位地址码与自己的地址进行比较，如果相同，则认为自己正被主机寻址，根据 R/W 位将自己确定为发送器或接收器。

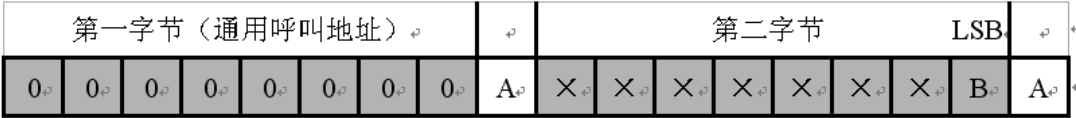
从机的地址由 4 位固定部分和 3 位可编程部分组成。固定地址由硬件生产厂家出厂时直接固定。可编程部分决定了可接入总线的该类器件的最大数目。3 位是可编程位，决定了可以有 8 个同样的器件接入到同一 I2C 总线系统中。

(b) 特殊地址

固定地址编号 0000 和 1111 已被保留作为特殊用途。

地 址 位							R/ $\overline{W}$	意 义
0	0	0	0	0	0	0	0	通用呼叫地址
0	0	0	0	0	0	0	1	起始字节
0	0	0	0	0	0	1	×	CBUS 地址
0	0	0	0	0	1	0	×	为不同总线的保留地址
0	0	0	0	0	1	1	×	保留
0	0	0	0	1	×	×	×	
1	1	1	1	1	×	×	×	
1	1	1	1	0	×	×	×	十位从机地址

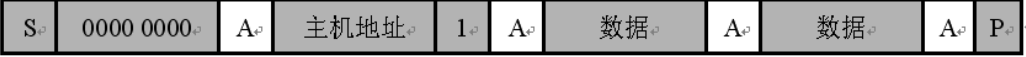
起始信号后的第一字节的 8 位为“0000 0000”时，称为通用呼叫地址。通用呼叫地址的用意在第二字节中加以说明。格式为



第二字节为 06H 时，所有能响应通用呼叫地址的从机器件复位，并由硬件装入从机地址的可编程部分。

第二字节为 04H 时，所有能响应通用呼叫地址并通过硬件来定义其可编程地址的从机器件将锁定地址中的可编程位，但不进行复位。

主机可以利用通用呼叫地址中的第二字节高 7 位说明自己的地址。这样当其它设备作主机时可以与其通信。



3、MCS-51 单片机的 I2C 总线仿真

不带 I2C 总线接口的单片机，如 MCS-51 单片机，可以利用软件结合 PIO 口仿真实现 I2C 总线。

为了保证数据传送的可靠性，标准的 I2C 总线的数据传送有严格的时序要求。I2C 总线的起始信号、终止信号、发送“0”及发送“1”的模拟时序，如下图 3.47 所示：

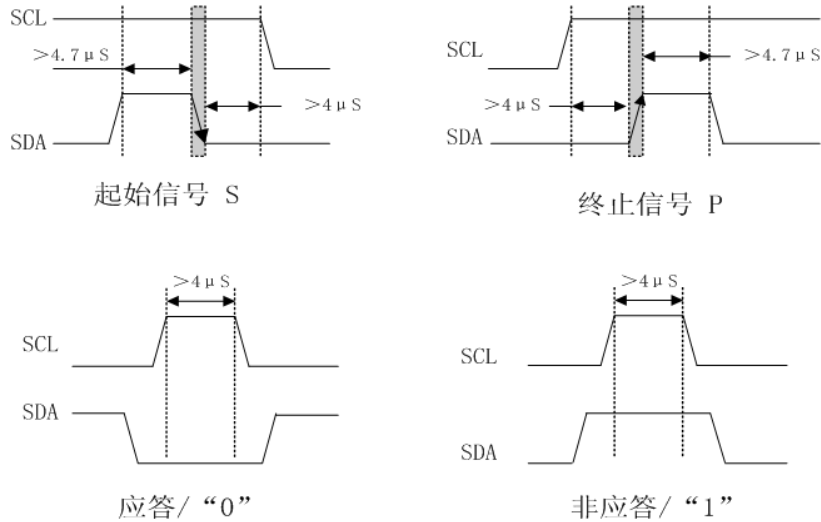


图 3.47 MCS-51 单片机仿真实现 I2C 总线时序要求

3.10.2 I2C 总线接口器件 PCF8574 说明

PCF8574 是 PHILIPS 公司推出的一款带 I2C 总线，可使 MCU 实现远程 I/O 口扩展的串并转换器件。该器件包含一个 8 位准双向口和一个 I2C 总线接口。PCF8574 电流消耗很低且口输出锁存具有大电流驱动能力（可直接驱动 LED）。它还带有一条中断接线 INT 可与 MCU 的中断逻辑相连，当并口数据发生变化时引起 MCS-51 单片机中断。在对 PCF8574 进行一次读写操作后，INT 端撤销中断请求，复位为高电平。

PCF8574 有两种型号：PCF8574 和 PCF8574A，它们的唯一区别仅在于器件地址不相同，其中 PCF8574 的固定地址部分为：0100，而 PCF8574A 的固定地址部分为：0111，如下图 3.48 所示。使用时必须注意这一区别。



图 3.48 PCF8574 器件地址

PCF8574 采用 DIP16，S016 或 SSOP20 形式封装，其外部引脚排列如下图 3.49 所示：

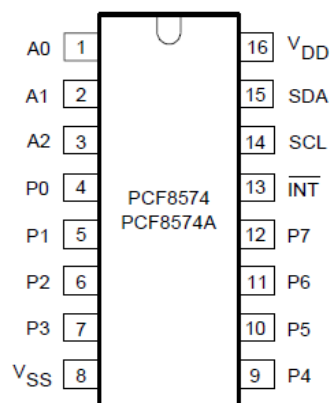


图 3.49 PCF8574 引脚排列

引脚功能为：

标号	管脚	描述
	S016	
A0	1	地址输入 0
A1	2	地址输入 1
A2	3	地址输入 2
P0	4	准双向 I/O 口 0
P1	5	准双向 I/O 口 1
P2	6	准双向 I/O 口 2
P3	7	准双向 I/O 口 3
V _{SS}	8	地
P4	9	准双向 I/O 口 4
P5	10	准双向 I/O 口 5
P6	11	准双向 I/O 口 6
P7	12	准双向 I/O 口 7
INT	13	中断输入（低电平有效）
SCL	14	串行时钟线
SDA	15	串行数据线
V _{DD}	16	电源

3.10.3 I2C 总线 E2PROM AT24C02 说明

AT24C02 是由 ATMEL 公司提供的，I2C 总线串行 EEPROM，其容量为 1KB，工作电压在 1.8V~5.5V 之间，生产工艺是 CMOS 工艺。其引脚排列为：

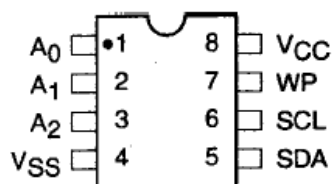


图 3.50 AT24C02 引脚排列

引脚功能为：

管脚名称	功能
A0, A1, A2	器件地址选择
SDA	串行数据/地址
SCL	串行时钟
WP	写保护
Vcc	+1.8V-5.5V 电源电压
Vss	地

AT24C02 的器件地址高 4 位固定为 1010，如下图所示：

24WC01/02	1	0	1	0	A2	A1	A0	R/W
-----------	---	---	---	---	----	----	----	-----

其中 A2, A1, A0 与器件响应引脚上的电平相对应，通过这三个引脚接不通的高低电平编码，可以使得同一条 I2C 总线上最多可以扩展 8 个这样的器件。

1、数据写入过程

AT24C 系列 E2PROM 芯片地址的固定部分为 1010，A2、A1、A0 引脚接高、低电平后得到确定的 3 位编码。形成的 7 位编码即为该器件的地址码。

单片机进行写操作时，首先发送该器件的 7 位地址码和写方向位“0”（共 8 位，即一个字节），发送完后释放 SDA 线并在 SCL 线上产生第 9 个时钟信号。被选中的存储器器件在确认是自己的地址后，在 SDA 线上产生一个应答信号作为相应，单片机收到应答后就可以传送数据了。

传送数据时，单片机首先发送一个字节的被写入器件的存储区的首地址，收到存储器器件的应答后，单片机就逐个发送各数据字节，但每发送一个字节后都要等待应答。

AT24C 系列器件片内地址在接收到每一个数据字节地址后自动加 1，在芯片的“一次装载字节数”（不同芯片字节数不同）限度内，只需输入首地址。装载字节数超过芯片的“一次装载字节数”时，数据地址将“上卷”，前面的数据将被覆盖。

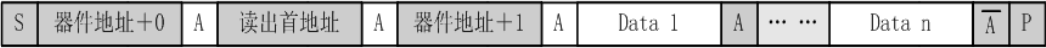
当要写入的数据传送完后，单片机应发出终止信号以结束写入操作。写入 n 个字节的数据格式：

S	器件地址+0	A	写入首地址	A	Data 1	A	...	...	Data n	A	P
---	--------	---	-------	---	--------	---	-----	-----	--------	---	---

2、数据读出过程

单片机先发送该器件的 7 位地址码和写方向位“0”（“伪写”），发送完后释放 SDA 线并在 SCL 线上产生第 9 个时钟信号。被选中的存储器器件在确认是自己的地址后，在 SDA 线上产生一个应答信号作为相应。

然后，再发一个字节的要读出器件的存储区的首地址，收到应答后，单片机要重复一次起始信号并发送器件地址和读方向位（“1”），收到器件应答后就可以读出数据字节，每读出一个字节，单片机都要回复应答信号。当最后一个字节数据读完后，单片机应返回以“非应答”（高电平），并发送终止信号以结束读出操作。



3.10.4 示例实验

(1) 实验内容

在 Proteus 环境下搭建如下图 2.51 所示的电路图：

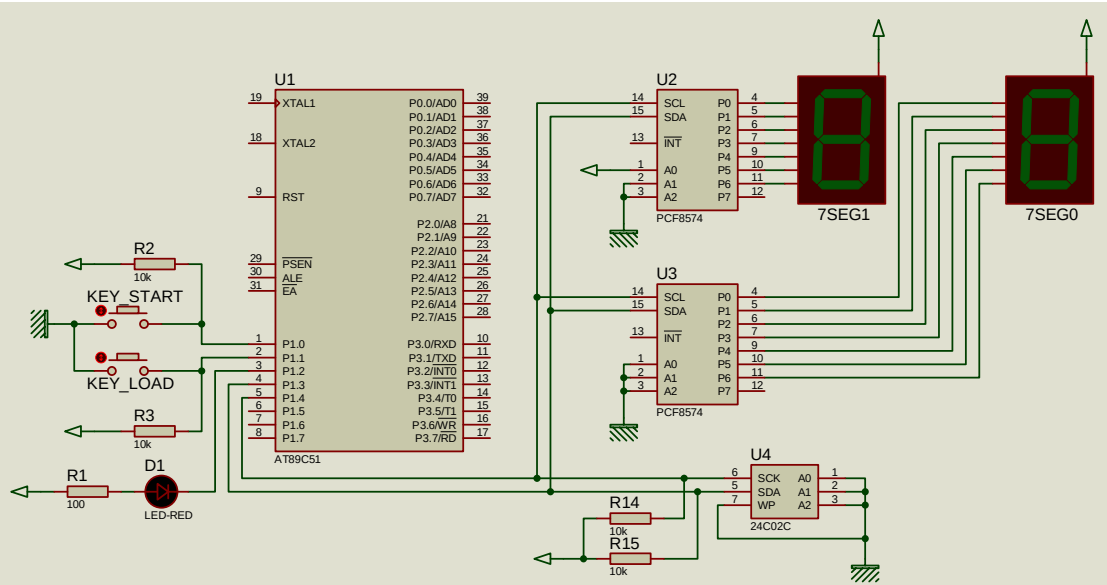


图 3.51 I2C 总线扩展示例实验

其中所用元器件如下表所示：

选件编号	元件名称	参数	所在元件库类名	子类名	生产厂家
U1	AT89C51		Microprocessor ICs 微处理器芯片	8051 Family 8051 家族	ATMEL
U2, U3	PCF8574		Microprocessor ICs 微处理器芯片	Peripherals 外设	Philips
U4	24C02C		Memory ICs 存储器芯片	I2C Memories I2C 存储器	Microchip
7SEG1, 7SEGO	7SEG-COM-AN-GRN		Optoelectronics 光电子器件	7-Segment Displays 7 段显示	

D1	LED-RED		Optoelectronics 光电子器件	LEDs 发光二极管	
R1	RES	100	Resistors 电阻	Generic 一般的	
R2, R3	RES	10k	Resistors 电阻	Generic 一般的	
KEY_STAR T, KEY_LOAD	BUTTON		Switches & Relays 开关与继电器	Switches 开关	

(2) 功能要求

要求：本实验与“3.3.2 自我完成实验”要求的功能类似，实现一个倒计时器。每过1s 钟倒计时器减1，当前时间用两个数码管显示。该倒计时器可以通过按键控制其起停与初值重载。

本实验与“3.3.2 自我完成实验”不同之处是数码管的驱动方式不同。本实验利用具有 I2C 接口的 PCF8574 来驱动数码管，而“3.3.2 自我完成实验”中利用了 74LS373 锁存器驱动。另外，本实验增加了一个具有 I2C 接口的 EEPROM 24c02 来记录当前的运行状态，包括当前时间和当前的起停状态。这样，当重新运行程序时，系统会自动恢复上次程序运行停止前的状态，并在此状态基础上继续运行。

(3) 示例程序

```

/*****必要的变量定义*****/
#include <reg51.h>
#include <intrins.h> //包含_nop_()函数

#define uint unsigned int
#define uchar unsigned char

#define FiveNOP(); _nop();_nop();_nop();_nop();_nop(); //定义空指令

sbit SCL = P1^4; //模拟 I2C 时钟控制位
sbit SDA = P1^3; //模拟 I2C 数据传输位
sbit Light=P1^2; //指示灯
sbit Load=P1^1; //装载初值键
sbit Start_Stop=P1^0; //起停键

bit bdata I2C_Ack; //应答标志位
bit bdata rsuccess;
bit run;

```

```

uchar rdata,wdata,time;

uchar code
LED_code[]={0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82, 0xF8, 0x80, 0x90};
//共阴极 LED 数码管显示码表

uchar begintime=99;

uchar data w2402buffer[3];
uchar data r2402buffer[3];

uchar SA8574_0=0x40;
uchar SA8574_1=0x42;
uchar SA2402=0xA0;

/*****短延时子程序，作键盘去抖动功能*****/
void delays10ms()
{
    uchar k,ms;
    ms=10;
    while(ms-->0) for(k=0;k<120;k++);
}

/***** I2C_Start *****/
功能描述：启动 I2C 总线，即发送 I2C 初始条件
解释：在 I2C 总线协议中规定的起始位格式是：在 SCL 高电平期间，SDA 发生
        从高到低的电平跳变。
*****/
void I2C_Start()
{
    SDA = 1; //发送起始条件的数据信号
    _nop_();
    SCL = 1;
    FiveNOP(); //起始条件建立时间大于 4.7us, 延时
    SDA = 0; //发送起始信号
    FiveNOP(); //起始条件建立时间大于 4us, 延时

```

```

    SCL = 0; //钳住 I2C 总线准备发送或接收数据
    _nop_();
    _nop_();
}
/***** I2C_Stop *****/

```

功能描述: 结束 I2C 总线, 即发送 I2C 结束条件

解释: 结束条件的格式是在 SCL 高电平期间, SDA 由低电平向高电平跳变.

*****/

```

void I2C_Stop()
{
    SDA = 0; //发送结束条件的数据信号
    _nop_();
    SCL = 1; //发送结束条件的时钟信号
    FiveNOP(); //结束条件建立时间大于 4us, 延时
    SDA = 1; //发送 I2C 总线结束信号
    FiveNOP();
}

```

*****/

出口: 0 (无应答), 1 (有应答)

功能描述: 检验 I2C 总线应答信号, 有应答则返回 1, 否则返回 0, 超时值取 255.

解释: I2C 总线协议中规定传输的每个字节之后必须跟一个应答位, 所以从器件在接收到每个字节之后必须反馈一个应答信号给主控制器, 而主控制器就需要检测从器件回传的应答信号, 根据其信息做出相应的处理. 另外, 主从之别是相对的, 接收数据的即为从, 发送数据的及为主.

应答信号的格式: 在由发送器产生的时钟响应周期里, 发送器先释放 SDA (置高), 然后由接受器将 SDA 拉低, 并在这个时钟脉冲周期的高电平期间保持稳定的低电平. 即表示从器件做出了应答.

调用函数: void I2C_Stop()

*****/

```

bit I2C_CheckAck(void)
{
    uchar errtime = 255; // 因故障接收方无 Ack, 超时值为 255
    SDA = 1; //发送器先释放 SDA
    FiveNOP();
    SCL = 1;
    FiveNOP(); //时钟电平周期大于 4 us
    while(SDA) //判断 SDA 是否被拉低

```

```

{
    errtime--;
    if(errtime==0)
    {
        I2C_Stop();
        return(0);
    }
    SDA=1;
}
SCL = 0;
_nop_();
return(1);
}

/***** I2C_SendB *****/

```

入口: uchar 型数据

功能描述: 字节数据传送函数, 将数据 c 发送出去, 可以是地址, 也可以是数据, 发完后等待应答, 并对此状态位进行操作

注意: 在传送数据时, 数据(SDA)的改变只能发生在 SCL 的低电平期间, 在 SCL 的高电平期间保持不变

调用函数: bit I2C_CheckAck()

全局变量: I2C_Ack

```

*****/
void I2C_SendB(uchar c)
{
    uchar BitCnt;
    for (BitCnt=0; BitCnt<8; BitCnt++) //要传送的数据长度为 8 位
    {
        if((c<<BitCnt)&0x80) //判断发送位(从高位起发送)
        {
            SDA = 1;
        }
        else
        {
            SDA = 0;
        }
        _nop_();
        _nop_();
    }
}

```

```

    SCL = 1; //置时钟线为高通知被控器开始接收数据位
    FiveNOP(); //保证时钟高电平周期大于 4us
    SCL = 0;
}
_nop_();
_nop_();
I2C_Ack = I2C_CheckAck(); //检验应答信号, 作为发送方, 所以要检测
                        //接收器反馈的应答信号.

_nop_();
_nop_();
}

/***** I2C_RcvB *****/
出口: uchar 型数据
功能描述: 接收从器件传来的数据, 并判断总线错误 (不发应答信号), 收完
          后需要调用应答函数。
*****/
uchar I2C_RcvB()
{
    uchar retc;
    uchar BitCnt; //位
    retc = 0;
    SDA = 1; //置数据总线为输入方式, 作为接收方要释放 SDA.
    for (BitCnt=0; BitCnt<8; BitCnt++)
    {
        _nop_();
        SCL = 0; //置时钟线为低准备接收数据位
        FiveNOP(); //时钟低电平周期大于 4.7us
        SCL = 1; //置时钟线为高使数据有效
        _nop_();
        _nop_();
        retc = retc<<1;
        if (SDA==1)
        {
            retc = retc + 1; //读数据位, 接收的数据放入 retc 中
        }
        _nop_();
        _nop_();
    }
}

```

```

    }
    SCL = 0;
    _nop_();
    _nop_();
    return(retc);
}

/***** I2C_Ackn *****/

```

入口：0 或 1

功能描述：主控制器进行应答信号（可以是应答或非应答信号）

说明：作为接收方的时候，必须根据当前自己的状态向发送器反馈应答信号

```

*****/

```

```

void I2C_Ackn(bit a)
{
    if(a==0) //在此发送应答或非应答信号
    {
        SDA = 0;
    }
    else
    {
        SDA = 1;
    }
    FiveNOP();
    SCL = 1;
    FiveNOP(); //时钟电平周期大于 4 us
    SCL = 0; //清时钟线钳住 I2C 总线以便继续接收
    _nop_();
    _nop_();
}

```

```

/***** W1Bto8574 *****/

```

入口：从器件地址 sla，发送字节 c

出口：0（操作有误），1（操作成功）

功能描述：往 PCF8574 写 1 个字节的数据从启动总线到发送地址、数据，结束总线的全过程，如果返回 1，表示操作成功，否则操作有误。

调用函数：I2C_Start(), I2C_SendB(uchar c), I2C_Stop()

全局变量：I2C_Ack

```

*****/

```



```

bit W1Bto8574(uchar sla, uchar c)
{
    I2C_Start(); //启动总线
    I2C_SendB(sla); //发送器件地址
    if(!I2C_Ack)
    {
        return(0);
    }
    I2C_SendB(c); //发送数据
    if(!I2C_Ack)
    {
        return(0);
    }
    I2C_Stop(); //结束总线
    return(1);
}

```

/***** W1Bto2402 *****/

入口：从器件地址 sla，子地址 suba（要存放数据的单元地址），发送单元 c

出口：0（操作有误），1（操作成功）

功能描述：向 AT24C02 写一个字节，从启动总线到发送地址、数据，结束总线
 的全过程，如果返回 1，表示操作成功，否则操作有误。

调用函数：I2C_Start(), I2C_SendB(uchar c), I2C_Stop()

全局变量：I2C_Ack

*****/

```

bit W1Bto2402(uchar sla, uchar suba, uchar c)
{
    I2C_Start(); //启动总线
    I2C_SendB(sla); //发送器件地址
    if(!I2C_Ack)
    {
        return(0);
    }
    I2C_SendB(suba); //发送器件子地址
    if(!I2C_Ack)
    {
        return(0);
    }

```

```

    }

    I2C_SendB(c); //发送数据
    if(!I2C_Ack)
    {
        return(0);
    }

    I2C_Stop(); //结束总线
    return(1);
}

/*****R1Bfrom2402*****/
入口：从器件地址 sla, 子地址 suba(即：单元起始地址), 收到的数据在 c
出口：1（操作成功），0（操作有误）
功能描述：从 AT24C02 读一个字节，从启动总线到发送地址、读数据，结束总线
          的全过程。
调用函数： I2CS_tart(),
I2C_SendB(uchar c),
I2C_RcvB(),
I2C_Ackn(bit a),
I2C_Stop()
全局变量： I2C_Ack
*****/
bit R1Bfrom2402(uchar sla, uchar suba, uchar *c)
{
    I2C_Start(); //启动总线
    I2C_SendB(sla);
    if(!I2C_Ack)
    {
        return(0);
    }
    I2C_SendB(suba); //发送器件子地址
    if(!I2C_Ack)
    {
        return(0);
    }
}

```

```

I2C_Start(); //重复起始条件

I2C_SendB(sla+1); //发送读操作的地址

if(!I2C_Ack)
{
    return(0);
}

*c = I2C_RcvB(); //读取数据

I2C_Ackn(1); //发送非应答位

I2C_Stop(); //结束总线
return(1);
}

void ds_time()
{
    uchar temp,i,out;

    temp=time%10;
    out=LED_code[temp];
    W1Bto8574(SA8574_0,out);

    temp=time/10;
    out=LED_code[temp];
    W1Bto8574(SA8574_1,out);

    w2402buffer[0]=1;
    w2402buffer[1]=time;

    for (i=0;i<2;i++)
    {
        rsuccess=0;

        wdata=w2402buffer[i];

```

```

        while(!rsuccess)
        {
            rsuccess=W1Bto2402(SA2402, i,wdata); //周期到达时保存当前状态
        }
    }
}

/*****主程序*****/
void main()
{
    uchar temp,i,t,out;
    bit bdata oldstate;
    uchar rdata,wdata;

    SP=0x60;
    t= 100; //100*10ms=1s

    oldstate=0;

    for (i=0;i<3;i++)
    {
        rsuccess=0;
        while(!rsuccess)
        {
            rsuccess=R1Bfrom2402(SA2402, i,&rdata); //读保存状态
        }
        r2402buffer[i]=rdata;
    }

    oldstate=r2402buffer[0]==0x1; //如果不是初次运行则
    r2402buffer[0]=0x1

    if(oldstate)
    {
        time=r2402buffer[1];
        run= r2402buffer[2];
    }
}

```

```

}
else
{
    time=beginTime;
    run=0;
}

Light=~run;

temp=time%10;
out=LED_code[temp];
I2C_W1Bto8574(SA8574_0, out);

temp=time/10;
out=LED_code[temp];
I2C_W1Bto8574(SA8574_1, out);

while(1)
{
    Start_Stop=1;
    if(!Start_Stop)
    {
        delays10ms();    //延时 10ms 去抖动
        Start_Stop=1;
        if(!Start_Stop)
        {
            run=~run;
            Light=~run;

            w2402buffer[2]=run;

            rsuccess=0;

            wdata=(uchar)run;

            while(!rsuccess)
            {

```

```

        rsuccess=W1Bto2402(SA2402, 2,wdata);
                                //起停按键按下，则保存起停状态
    }
}
Start_Stop=1;
while(!Start_Stop) //等待按键释放
    Start_Stop=1;
}

Load=1;
if(!Load)
{
    delays10ms();    //延时 10ms 去抖动
    Load=1;
    if(!Load)
    {
        time=beginTime;
        ds_time();//显示并保存当前时间
    }
    Load=1;
    while(!Load) //等待按键释放
        Load=1;
}

if(run)
{
    if(t--)
    {
        delays10ms();
    }
    else
    {
        t= 100;    //周期到达时重新赋值延时控制变量
        if(time)
        {
            time--;

```

```

        ds_time();//显示并保存当前时间
    }
}
}
}
}
}

```

(4) 实验步骤

- ① 根据上述实验内容，参考 1.2.2，在 Proteus 环境下建立图 3.51 所示原理图，并将其保存为 I2C.DSN 文件。
- ② 将上面 (3) 中控制源程序保存为 I2C.c。
- ③ 运行 Keil uVision2 开发环境，按照 1.1.3 节介绍的方法建立工程 I2C.uv2，CPU 为 AT89C51，包含启动文件 STARTUP.A51。
- ④ 按照 1.2.2 第 (6) 节介绍的方法将 C 语言源程序 I2C.c 加入工程 I2C.uv2，并设置工程 I2C.uv2 属性，将其晶振频率设置为 12MHz，选择输出可执行文件，仿真方式为选择硬仿真，并选择其中的“PROTEUS VSM MONITOR 51 DRIVER”仿真器。
- ⑤ 构造 (Build) 工程 I2C.uv2。如果输入有误进行修改，直至构造正确，生成可执行程序 I2C.hex 为止。
- ⑥ 为 AT89C51 设置可执行程序 I2C.hex。
- ⑦ 运行程序，点击图 2.51 中的各按键，程序运行情况。
- ⑧ 点击主窗口左下方的  按钮，停止仿真过程。再点击  按钮重新运行程序观察程序运行是否能够继续上一次运行时的状态继续运行。

(5) 实验作业

- 分析源程序并仔细理解源程序每个函数的功能。
- 记录示例程序运行结果截图。
- 总结 c51 仿真 I2C 的方法，以及 PCF8574 与 24C02 的 I2C 控制方法，理解有关控制时序。

3.10.5 自我完成实验

(1) 实验内容

在 Proteus 环境下搭建如下图 3.52 所示的电路图：

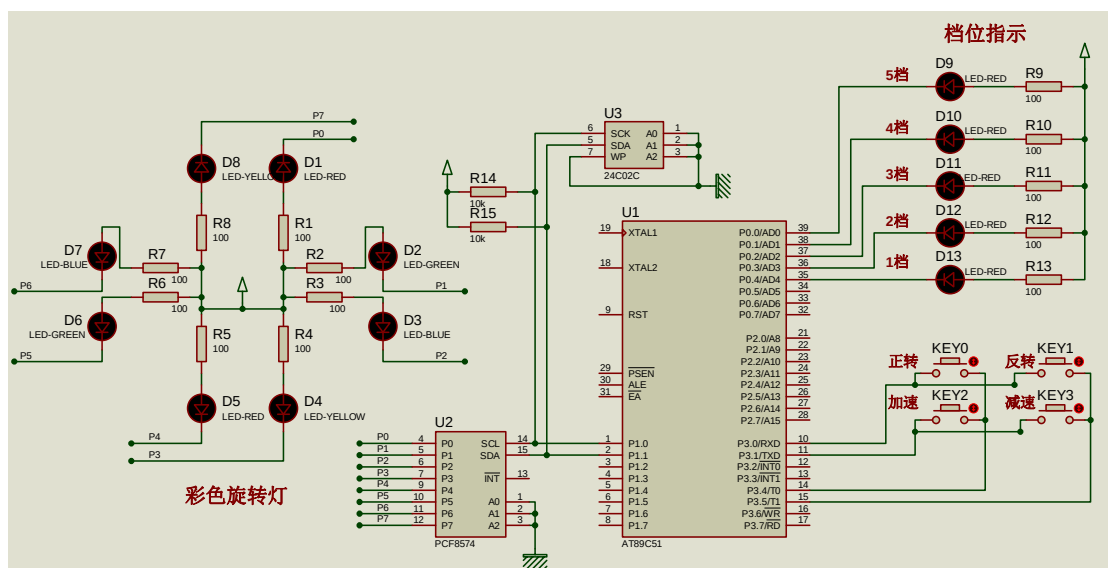


图 3.52 I2C 总线扩展示例自我完成实验

其中所用元器件如下表所示：

选件编号	元件名称	参数	所在元件库类名	子类名	生产厂家
U1	AT89C51		Microprocessor ICs 微处理器	8051 Family 8051 家族	ATMEL
U2	PCF8574		Microprocessor ICs 微处理器芯片	Peripherals 外设	Philips
U3	24C02C		Memory ICs 存储器芯片	I2C Memories I2C 存储器	Microchip
D1, D5, D9-D13	LED-RED		Optoelectronics 光电子器件	LEDs 发光二极管	
D2, D6	LED-GREEN		Optoelectronics 光电子器件	LEDs 发光二极管	
D3, D7	LED-BLUE		Optoelectronics 光电子器件	LEDs 发光二极管	
D4, D8	LED-YELLOW		Optoelectronics 光电子器件	LEDs 发光二极管	
R1-R13	RES	100	Resistors 电阻	Generic 一般的	
R14, R14	RES	10k	Resistors 电阻	Generic 一般的	
KEY0-KEY3	BUTTON		Switches & Relays 开关与继电器	Switches 开关	

(2) 控制要求

本实验控制要求与“3.4.2 自我完成实验”一致，D1~D8 八个发光二极管构成彩色旋转灯，D9~D13 为档位指示灯。四个按键 KEY0~KEY1 用于设定旋转方向为顺时针旋转或者逆时针旋转，KEY2~KEY3 用于增快或则减慢旋转速度。这里增加了 PCF8574 作为发光二极管的驱动器，同时增加了 24C02 存储当前工作状态包括：正转/反转，旋转速度，当前正点亮的 LED 位置。要求程序停止并重新运行后系统能够按照上次停止时的状态继续运行。

(3) 编程思路

参考“3.4.2 自我完成实验”节的控制思路，以及 3.10.4 节的 PCF8574 与 24C02 的数据读写方法实现。

(4) 实验步骤

- ① 根据上述实验内容，参考 1.2.2，在 Proteus 环境下建立图 3.52 所示原理图，并将其保存为 I2C_self.DSN 文件。
- ② 根据 (2) 和 (3) 画出流程图，并编写源程序，将其保存为 I2C_self.c。
- ③ 运行 Keil uVision2 开发环境，按照 1.1.3 节介绍的方法建立工程 I2C_self.uv2，CPU 为 AT89C51，包含启动文件 STARTUP.A51。
- ④ 按照 1.2.2 第 (6) 节介绍的方法将 C 语言源程序 I2C_self.c 加入工程 I2C_self.uv2，并设置工程 I2C_self.uv2 属性，将其晶振频率设置为 12MHz，选择输出可执行文件，仿真方式为选择硬仿真，并选择其中的“PROTEUS VSM MONITOR 51 DRIVER”仿真器。
- ⑤ 构造 (Build) 工程 I2C_self.uv2。如果输入有误进行修改，直至构造正确，生成可执行程序 I2C_self.hex 为止。
- ⑥ 为 AT89C51 设置可执行程序 I2C_self.hex。
- ⑦ 运行程序，观察运行结果，并验证其是否正确。

(5) 实验作业

- 编写源程序并进行注释
- 实验过程记录
- 记录程序运行结果截图

3.11 实验 11：SPI 扩展实验

3.11.1 SPI 总线简介

SPI 总线是 Motorola 公司提出的一个同步串行外设接口。利用 SPI 总线 CPU 可以与各种支持该接口的外设(包括模/数转换器、数/模转换器、RTC 等)以串行方式进行通信。

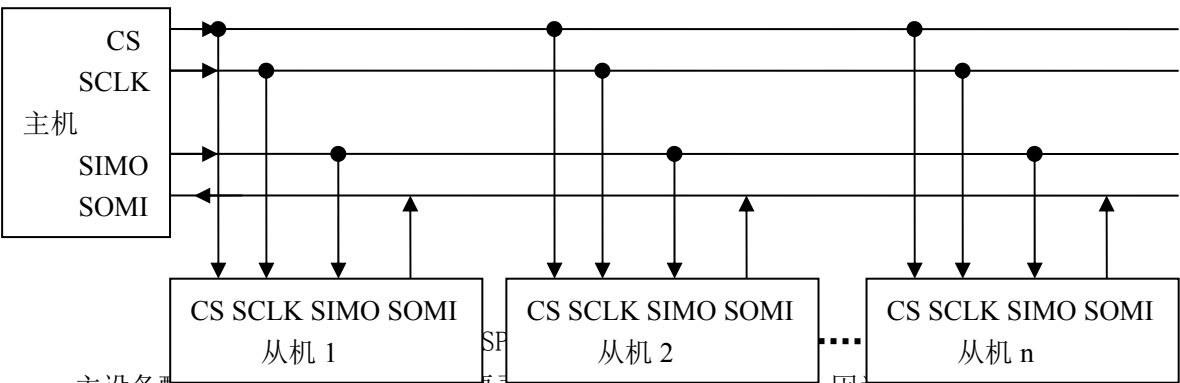
SPI 总线具有以下特点：

- (1) 连线较少, 简化电路设计, 仅需 4 根信号线即可完成扩展功能。
- (2) 器件统一编址, 并与系统地址无关, 操作 SPI 独立性好。
- (3) 器件操作遵循统一的规范, 使系统软硬件具有良好的通用性。

SPI 总线采用主从 (Master Slaver) 模式。一个 SPI 通信系统中, 一般包含 1 个主设备和多个从设备。主设备选择与之通信的从设备, 并管理控制整个通信过程。

SPI 一般包括 4 条信号线: 串行时钟线 (SCK)、主输入/从输出线 (MISO) 或简称为输入信号线 (SDI)、主输出/从输入线 (MOSI 或简称为输出信号线 (SDO)、低电平有效的片选信号线 (CS)。当同一条 SPI 总线上有多个从器件时, 主器件需要通过片选信号 CS 使能与之通信的从器件。如果只有一个从器件, 则可以不使用片选信号线, 这时从器件的片选信号线直接接地, 这时的 SPI 为三线总线。如果通信方式是确定的单向输出, 即只有主到从方向的通信, 没有从到主的数据传递 (如 DA 转换), 这时两根信号线: 串行时钟线 (SCK)、主输出/从输入线 (MOSI) 也可以完成通信, 这时候的 SPI 为双线总线。MISO 与 MOSI 是在 SCK 的同步下一位一位的进行串行数据传送。SPI 的通信速度可达几 Mbps。

多从结构的 SPI 总线连接图如下图 3.53 所示：



主设备配置 SPI 时钟的时候一定要弄清楚从设备的时钟要求, 因为主设备的时钟极性和相位都是以从设备为基准的。在时钟极性的配置上一定要搞清楚从设备是在时钟的上升沿还是下降沿接收数据, 以及是在时钟的下降沿还是上升沿输出数据。由于主设备的 SDO 连接从设备的 SDI, 从设备的 SDO 连接主设备的 SDI, 从设备 SDI 接收的数据是主设备的 SDO 发送过来的, 主设备 SDI 接收的数据是从设备 SDO 发送过来的, 所以主设备这边 SPI 时钟极性的配置 (即 SDO 的配置), 跟从设备 SDO 发送数据的极性是相同的, 跟从设备的 SDI 接收数据的极性是相反的。

如采用 SPI 总线接口的 RTC DS1302 其控制字总是从最低位开始输出，在控制字指令输入后的下一个 SCLK 时钟的上升沿时，数据被写入 DS1302，数据输入从最低位（0 位）开始。同样，在紧跟 8 位的控制字指令后的下一个 SCLK 脉冲的下降沿，读出 DS1302 的数据，读出的数据也是从最低位到最高位。即一个时钟完成两个操作，如下图 3.54 所示：

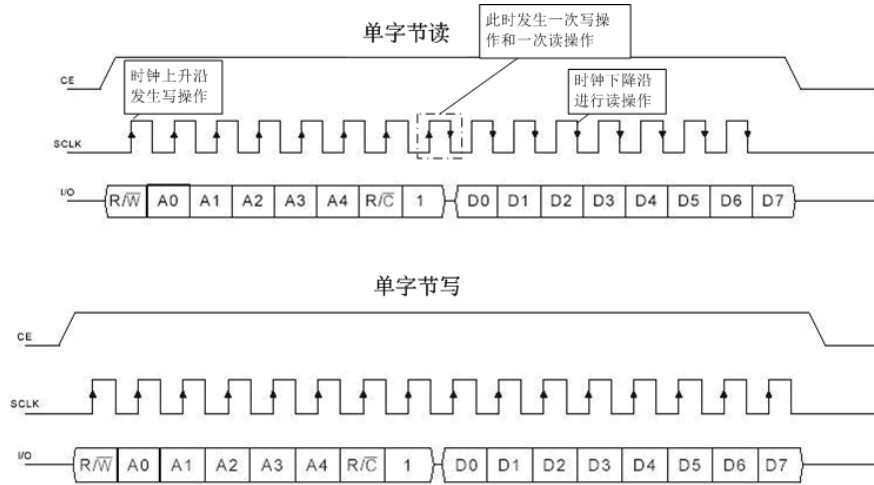


图 3.54 DS1302 读写时序

3.11.2 DS1302 简介

1、功能与引脚

DS1302 是美国 DALLAS 公司推出的一种高性能、低功耗的实时时钟芯片，附加 31 字节静态 RAM，采用 SPI 三线接口与 CPU 进行同步通信，并可采用突发方式一次传送多个字节的时钟信号和 RAM 数据。实时时钟可提供秒、分、时、日、星期、月和年，一个月小于 31 天时可以自动调整，且具有闰年补偿功能。工作电压宽达 2.5~5.5V。采用双电源供电（主电源和备用电源），可设置备用电源充电方式，提供了对后背电源进行涓细电流充电的能力。DS1302 的外部引脚分配如图 3.55 所示及内部结构如图 3.56 所示。DS1302 用于数据记录，特别是对某些具有特殊意义的数据点的记录上，能实现数据与出现该数据的时间同时记录，因此广泛应用于测量系统中。

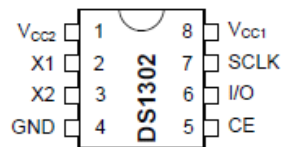


图 3.55 DS1302 引脚分布图

其中：

- X1, X2: 32.768KHz 晶振管脚
- Vcc1: 主电源；Vcc2: 备份电源。当 $V_{cc2} > V_{cc1} + 0.2V$ 时，由 V_{cc2} 向 DS1302 供电，当 $V_{cc2} < V_{cc1}$ 时，由 V_{cc1} 向 DS1302 供电。
- SCLK: 串行时钟，输入，控制数据的输入与输出；
- I/O: 三线接口时的双向数据线；

➤ CE：输入信号，在读、写数据期间，必须为高

2、寄存器和 RAM

①内部寄存器

对 DS1302 的操作就是对其内部寄存器的操作，DS1302 内部共有 12 个寄存器，其中有 7 个寄存器与日历、时钟相关，存放的数据位为 BCD 码形式。此外，DS1302 还有年份寄存器、控制寄存器、充电寄存器、时钟突发寄存器及与 RAM 相关的寄存器等。时钟突发寄存器可一次性顺序读写除充电寄存器以外的寄存器。DS1302 内部寄存器分配如下表所示：

READ	WRITE	BIT 7	BIT 6	BIT 5	BIT 4	BIT 3	BIT 2	BIT 1	BIT 0	RANGE
81h	80h	CH		10 Seconds			Seconds			00–59
83h	82h			10 Minutes			Minutes			00–59
85h	84h	12/24	0	10 AM/PM	Hour		Hour			1–12/0–23
87h	86h	0	0	10 Date			Date			1–31
89h	88h	0	0	0	10 Month		Month			1–12
8Bh	8Ah	0	0	0	0	0	Day			1–7
8Dh	8Ch			10 Year			Year			00–99
8Fh	8Eh	WP	0	0	0	0	0	0	0	—
91h	90h	TCS	TCS	TCS	TCS	DS	DS	RS	RS	—

小时寄存器（85h、84h）的位 7 用于定义 DS1302 是运行于 12 小时模式还是 24 小时模式。当为高时，选择 12 小时模式。在 12 小时模式时，位 5 是 ， 当为 1 时，表示 PM。在 24 小时模式时，位 5 是第二个 10 小时位。

秒寄存器（81h、80h）的 BIT7 定义为时钟暂停标志（CH），当 CH 为 1 时，时钟振荡器停止工作，DS1302 进入低功耗模式，电源消耗小于 100 微安，当 CH 为 0 时，时钟振荡器启动，DS1302 正常工作。BIT6–BIT4 为十秒位，BIT3–BIT0 为秒位。

写保护寄存器（8Fh、8Eh）的 BIT7：WP 是写保护位。在任何的对时钟和 RAM 的写操作之前，WP 位必须为 0。当 WP 位为 1 时，写保护位防止对任一寄存器的写操作。

②内部 RAM

DS1302 中附加 31 字节静态 RAM，其地址分布如下表所示：

读地址	写地址	数据范围
C1h	C0h	00–FFh
C3h	C2h	00–FFh
C5h	C4h	00–FFh
.	.	.
.	.	.
.	.	.
FDh	FCh	00–FFh

3、控制字

DS1302 不仅要向寄存器写入控制字，还需要读取相应寄存器的数据。要想与 DS1302 通信，首先要了解 DS1302 的控制字。DS1302 的控制字如下所示：

7	6	5	4	3	2	1	0
1	RAM	A4	A3	A2	A1	A0	RD
	$\overline{CK}$						$\overline{WR}$

控制字的最高有效位 (BIT7) 必须是逻辑 1，如果它为 0，则不能把数据写入到 DS1302 中。

BIT6 为日历时钟和 RAM 数据选择位：如果为 0，则表示存取日历时钟数据，为 1 表示存取 RAM 数据；

BIT5 至 BIT1 (A4~A0)：指示操作单元的地址；

位 0 (最低有效位)：如为 0，表示要进行写操作，为 1 表示进行读操作。

4、突发模式

所谓突发模式是指一次传送多个字节的时钟信号和 RAM 数据。突发模式可以指定任何的时钟/日历或者 RAM 寄存器为突发模式，和以前一样，第 6 位指定时钟或 RAM 而 0 位指定读或写。突发模式的实质是指一次传送多个字节的时钟信号和 RAM 数据。

DS1302 的工作模式寄存器：

工作模式寄存器		读寄存器	写寄存器
时钟突发模式寄存器	CLOCK BURST	BFh	BEh
RAM 突发模式寄存器	RAM BURST	FFh	FEh

在时钟/日历寄存器中的 9 至 31 和在 RAM 寄存器的地址 31 不能存储数据。突发模式的读取或写入从地址的位 0 开始。

3. 11. 3 基于 SPI 器件 DS1302 的万年历实验

(1) 实验内容

在 Proteus 环境下搭建万年历电路图如下图 3. 56 所示的：

(2) 控制要求

本实验控制要求利用 DS1302 实现万年历功能，SPI 接口由 AT89C51 单片机的 P1.0, P1.1 和 P1.2 三条并行 I/O 口仿真实现。该万年历具有时间设定（即校准）功能，设定的方法是电击 KEY12（“时间设置”按钮）进入时间设置模式，如下图 3.57 所示：

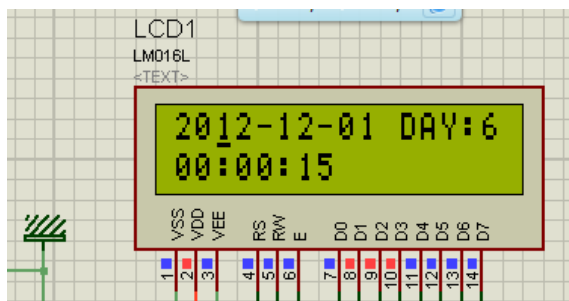


图 3.57 时间设置模式

光标处于年的下方，这时可以通过 KEY15（“右移”按钮）移动光标，使其处于月、日、星期、小时、分钟和秒的下方，再输入相应的数字或者通过 KEY10（“+”按钮）、KEY11（“-”按钮）使其数值增加或者减少。设定成自己想要设定的时间，然后再次点击 KEY12（“时间设置”按钮）退出时间设置模式，万年历进入正常运行模式，并每过一秒钟更新一次显示。如下图 3.58 所示：

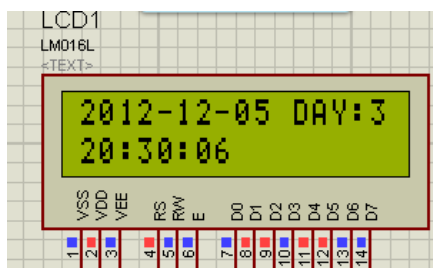


图 3.58 正常运行模式

DS1302 本身没有闹钟设置功能，可以利用软件来实现。本实验利用点击 KEY13（“闹钟设置”按钮），使其进入“闹钟设置”模式，如下图 3.59 所示：

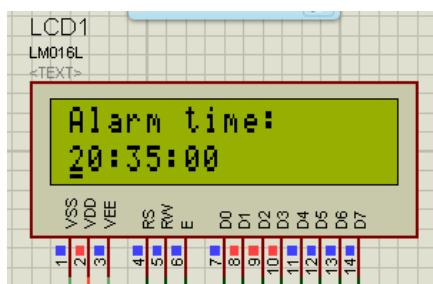


图 3.59 闹钟设置模式

通过和时钟设置相同的方式设定闹钟时间。当闹钟时间到时，P1.3 会输出高电平，蜂鸣器会发出声响，D1 会点亮，如下图 3.60 所示。P1.3 的电平每过 1s 反转一次，该过程持续 1 分钟结束。

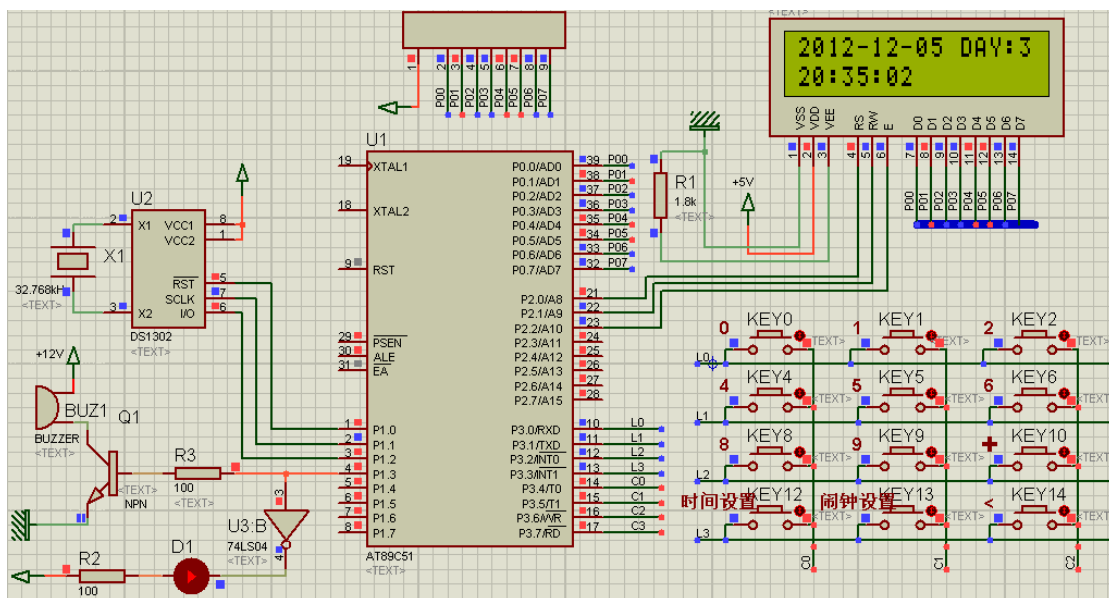


图 3.60 闹钟声光报警输出

(3) 控制思想及关键代码

LCD1602 的控制方法在前面的实验中已描述，这里不再重复。按键识别仍然可以采用线反转法来实现。DS1302 的读写方法在 3.11.2 节已作描述，下面给出其关键读写代码：

```

/*****

函数名: WritelByteToRT()
功    能: 实时时钟写入一字节
说    明: 往 DS1302 写入 1Byte 数据 (内部函数)
入口参数: d 写入的数据

*****/

void WritelByteToRT(uchar d)
{
    uchar i;
    ACC = d;
    for(i=8; i>0; i--)
    {
        T_I0 = ACC0;          /*相当于汇编中的 RRC */
        T_CLK = 1;
        T_CLK = 0;
        ACC = ACC >> 1;
    }
}

/*****

函数名: Read1ByteFromRT()

```


功 能：实时时钟读取一字节

说 明：从 DS1302 读取 1Byte 数据（内部函数）

入口参数：无

返 回 值：ACC

```
uchar Read1ByteFromRT(void)
```

```
{
    uchar i;
    for(i=8; i>0; i--)
    {
        ACC = ACC >>1;          /*相当于汇编中的 RRC */
        ACC7 = T_I0;
        T_CLK = 1;
        T_CLK = 0;
    }
    return(ACC);
}
```

函 数 名：W1302()

功 能：往 DS1302 写入数据

说 明：先写地址，后写命令/数据（内部函数）

调 用：Write1ByteToRT() , Read1ByteFromRT()

入口参数：ucAddr: DS1302 地址, ucData: 要写的的数据

/

```
void W1302(uchar ucAddr, uchar ucDa)
```

```
{
    T_RST = 0;
    T_CLK = 0;
    T_RST = 1;
    Write1ByteToRT(ucAddr);    /* 地址, 命令 */
    Write1ByteToRT(ucDa);     /* 写 1Byte 数据*/
    T_CLK = 1;
    T_RST = 0;
}
```

```

/*****
函 数 名: R1302()
功    能: 读取 DS1302 某地址的数据
说    明: 先写地址, 后读命令/数据 (内部函数)
调    用: Write1ByteToRT() , Read1ByteFromRT()
入口参数: ucAddr: DS1302 地址
返 回 值: ucData :读取的数据
*****/
uchar R1302(uchar ucAddr)
{
    uchar ucData;
    T_RST = 0;
    T_CLK = 0;
    T_RST = 1;
    Write1ByteToRT(ucAddr);          // 地址, 命令
    ucData = Read1ByteFromRT();      // 读 1Byte 数据
    T_CLK = 1;
    T_RST = 0;
    return(ucData);
}

/*****
函 数 名: Set1302()
功    能: 设置初始时间
说    明: 先写地址, 后读命令/数据(寄存器多字节方式)
调    用: W1302()
入口参数: pClock: 设置时钟数据地址 格式为: 秒 分 时 日 月 星期 年
              7Byte (BCD 码) 1B 1B 1B 1B 1B 1B 1B
*****/
void Set1302(uchar *pClock)
{
    uchar i;
    uchar ucAddr = 0x80;
    W1302(0x8e, 0x00);              // 控制命令, WP=0, 写操作
    for(i = 7; i > 0; i--)
    {
        W1302(ucAddr, *pClock);    // 秒 分 时 日 月 星期 年
    }
}

```

```

        pClock++;
        ucAddr +=2;
    }
    W1302(0x8e, 0x80);          // 控制命令, WP=1, 写保护
}

```

/******

函数名: Get1302()

功 能: 读取 DS1302 当前时间

说 明:

调 用: R1302()

入口参数: ucCurtime: 保存当前时间地址。

当前时间格式为: 秒 分 时 日 月 星期 年

7Byte (BCD 码) 1B 1B 1B 1B 1B 1B 1B

返回值: 无

/

```

void Get1302(uchar ucCurtime[])
{
    uchar i;
    uchar ucAddr = 0x81;
    for (i=0; i<7; i++)
    {
        ucCurtime[i] = R1302(ucAddr); //格式为: 秒 分 时 日 月 星期 年/
        ucAddr += 2;
    }
}

```

(4) 实验步骤

- ① 根据上述实验内容, 参考 1.2.2, 在 Proteus 环境下建立图 3.60 所示原理图, 并将其保存为 SPI.DSN 文件。
- ② 根据 (2) 和 (3) 画出流程图, 并编写源程序, 将其保存为 SPI.c。
- ③ 运行 Keil uVision2 开发环境, 按照 1.1.3 节介绍的方法建立工程 SPI.uv2, CPU 为 AT89C51, 包含启动文件 STARTUP.A51。
- ④ 按照 1.2.2 第 (6) 节介绍的方法将 C 语言源程序 SPI.c 加入工程 SPI.uv2, 并设置工程 SPI.uv2 属性, 将其晶振频率设置为 12MHz, 选择输出可执行文件, 仿真方式为选择硬仿真, 并选择其中的 “PROTEUS VSM MONITOR 51 DRIVER” 仿真器。

- ⑤ 构造 (Build) 工程 SPI.uv2。如果输入有误进行修改，直至构造正确，生成可执行程序 SPI.hex 为止。
- ⑥ 为 AT89C51 设置可执行程序 SPI.hex。
- ⑦ 运行程序，观察运行结果，并验证其是否正确。

(5) 实验作业

- 编写源程序并进行注释
- 实验过程记录
- 记录程序运行结果截图

3.12 实验 12：LCD12864 显示实验

3.12.1 点阵型 LCD 12864 简介

1、模块简介

LCD12864 图形点阵液晶显示模块，即可显示 ASCII 字符，又可显示汉字及图形。LCD12864 内置 8192 个 16*16 点汉字，和 128 个 16*8 点 ASCII 字符集。利用该模块灵活的接口方式和简单、方便的操作指令，可构成全中文人机交互图形界面，也可完成图形显示。该器件的外观如下：

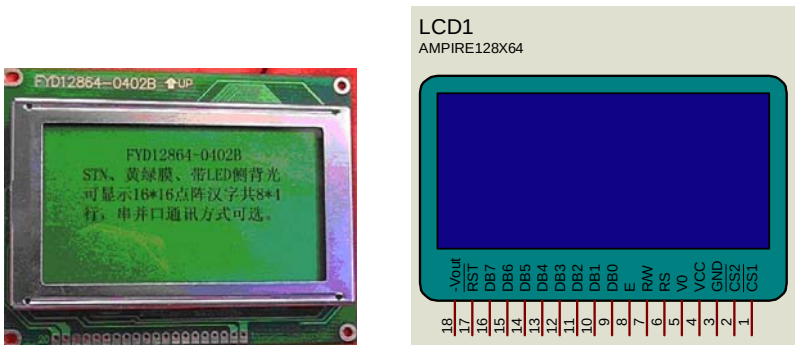


图 3.61 LCD12864 外观图

该器件的引脚功能描述如下：

实物模块 管脚号	仿真模型 管脚号	管脚名称	电平	管脚功能描述
1	3	VSS (GND)	0V	电源地
2	4	VCC	+5V	电源正
3	5	V0	-	亮度调整
4	6	RS (D/I)	H/L	RS= “H”，表示 DB7—DB0 为显示数据 RS= “L”，表示 DB7—DB0 为显示指令数据
5	7	R/W	H/L	R/W= “H”，E= “H”，数据被读到 DB7——DB0 R/W= “L”，E= “H→L”，DB7——DB0 的数据被 写到 IR 或 DR
6	8	E	H/L	使能信号

7	9	DB0	H/L	三态数据线
8	10	DB1	H/L	三态数据线
9	11	DB2	H/L	三态数据线
10	12	DB3	H/L	三态数据线
11	13	DB4	H/L	三态数据线
12	14	DB5	H/L	三态数据线
13	15	DB6	H/L	三态数据线
14	16	DB7	H/L	三态数据线
15		PSB	H/L	H: 8 位或 4 位并口方式, L: 串口方式 (仅实物模块有)
16		NC	-	空脚 (仅实物模块有)
17	17	/RST	H/L	复位端, 低电平有效
18	18	VOUT	-	LCD 驱动电压输出端
19		A	VDD	背光源正端 (+5V) (仅实物模块有)
20		K	VSS	背光源负端 (仅实物模块有)
	1	/CS1		片选 1 (仅仿真模型有)
	2	/CS2		片选 2 (仅仿真模型有)

2、模块的内部结构

12864 模块内部结构如下图所示:

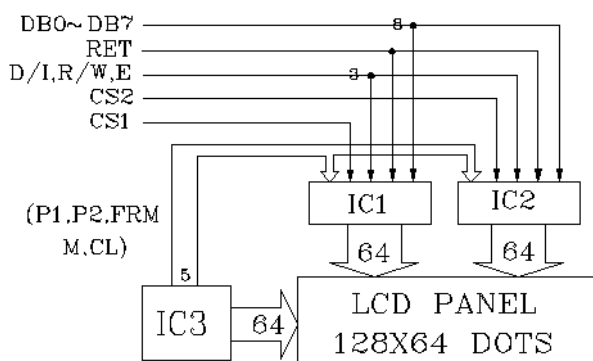


图 3.62 LCD12864 内部结构图

IC1, IC2 为列驱动器, IC1 控制模块的右半屏, IC2 控制模块的左半屏。IC3 为行驱动器。IC1, IC2, IC3 含有如下主要功能器件。

(1) 指令寄存器 (IR)

IR 是用来寄存指令码, 与数据寄存器寄存数据相对应。当 D/I=1 时, 在 E 信号下降沿的作用下, 指令码写入 IR。

(2) 数据寄存器 (DR)

DR 是用来寄存数据的, 与指令寄存器寄存指令相对应。当 D/I=1 时, 在 E 信号的下降沿作用下, 图形显示数据写入 DR, 或在 E 信号高电平作用下由 DR 读到 DB7~DB0 数据总线。DR 和 DDRAM 之间的数据传输是模块内部自动执行的。

(3) 忙标志:BF

BF 标志提供内部工作情况。BF=1 表示模块在进行内部操作，此时模块不接受外部指令和数据。BF=0 时，模块为准备状态，随时可接受外部指令和数据。

利用 STATUS READ 指令，可以将 BF 读到 DB7 总线，从而检验模块之工作状态。

(4) 显示控制触发器 DFF

此触发器是用于模块屏幕显示开和关的控制。DFF=1 为开显示 (DISPLAY ON)，DDRAM 的内容就显示在屏幕上，DDF=0 为关显示 (DISPLAY OFF)。

DDF 的状态是指令 DISPLAY ON/OFF 和 RST 信号控制的。

(5) XY 地址计数器

XY 地址计数器是一个 9 位计数器。高三位是 X 地址计数器，低 6 位为 Y 地址计数器，XY 地址计数器实际上是作为 DDRAM 的地址指针，X 地址计数器为 DDRAM 的页指针，Y 地址计数器为 DDRAM 的 Y 地址指针。

X 地址计数器是没有记数功能的，只能用指令设置。

Y 地址计数器具有循环记数功能，各显示数据写入后，Y 地址自动加 1，Y 地址指针从 0 到 63。

(6) 显示数据 RAM (DDRAM)

DDRAM 是存贮图形显示数据的。数据为 1 表示显示选择，数据为 0 表示显示非选择。

DDRAM 与地址和显示位置的关系见下面 DDRAM 地址表。

CS1=1						CS2=1					
Y=	0	1		62	63	0	1		62	63	行号
X=0	DB0	DB0	DB0	DB0	DB0	DB0	DB0	DB0	DB0	DB0	0
	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
	DB7	DB7	DB7	DB7	DB7	DB7	DB7	DB7	DB7	DB7	7
↓	DB0	DB0	DB0	DB0	DB0	DB0	DB0	DB0	DB0	DB0	8
	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
	DB7	DB7	DB7	DB7	DB7	DB7	DB7	DB7	DB7	DB7	55
X=7	DB0	DB0	DB0	DB0	DB0	DB0	DB0	DB0	DB0	DB0	56
	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
	DB7	DB7	DB7	DB7	DB7	DB7	DB7	DB7	DB7	DB7	63

(7) Z 地址计数器

Z 地址计数器是一个 6 位计数器，此计数器具备循环记数功能，它是用于显示行扫描同步。当一行扫描完成，此地址计数器自动加 1，指向下一行扫描数据，RST 复位后 Z 地址计数器为 0。

Z 地址计数器可以用指令 DISPLAY START LINE 预置。因此，显示屏幕的起始行就由此指令控制，即 DDRAM 的数据从哪一行开始显示在屏幕的第一行。此模块的 DDRAM 共 64 行，屏幕可以循环滚动显示 64 行。

3、模块原理说明

(1)、RS, R/W 的配合选择决定控制界面的 4 种模式：

RS	R/W	功能说明
L	L	MPU 写指令到指令暂存器 (IR)
L	H	读出忙标志 (BF) 及地址计数器 (AC) 的状态
H	L	MPU 写入数据到数据暂存器 (DR)
H	H	MPU 从数据暂存器 (DR) 中读出数据

(2)、E 信号

E 状态	执行动作	结果
高——>低	I/O 缓冲——>DR	配合 /W 进行写数据或指令
高	DR——>I/O 缓冲	配合 R 进行读数据或指令
低/低——>高	无动作	

(3) 显示数据 RAM (DDRAM)

模块内部显示数据 RAM 提供 64×2 个位元组的空间，最多可控制 4 行 16 字 (64 个字) 的中文字型显示，当写入显示数据 RAM 时，可分别显示 CGROM (字型产生 ROM 提供 8192 个字模) 与 CGRAM (自定义字型存储区) 的字型；此模块可显示三种字型，分别是半角英数字型 (16*8)、CGRAM 字型及 CGROM 的中文字型，三种字型的选择，由在 DDRAM 中写入的编码选择，在 0000H—0006H 的编码中 (其代码分别是 0000、0002、0004、0006 共 4 个) 将选择 CGRAM 的自定义字型，02H—7FH 的编码中将选择半角英数字的字型，至于 A1 以上的编码将自动的结合下一个位元组，组成两个位元组的编码形成中文字型的编码 BIG5 (A140—D75F)，GB (A1A0—F7FFH)。

4、具体指令

(1) 显示开关控制 (DISPLAY ON/OFF)

代码	R/W	D/I	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
形式	0	0	0	0	1	1	1	1	1	D

D=1: 开显示 (DISPLAY ON) 意即显示器可以进行各种显示操作

D=0: 关显示 (DISPLAY OFF) 意即不能对显示器进行各种显示操作

(2) 设置显示起始行 (DISPLAY START LINE)

代码	R/W	D/I	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

形式	0	0	1	1	A5	A4	A3	A2	A1	A0
----	---	---	---	---	----	----	----	----	----	----

前面在 Z 地址计数器一节已经描述了显示起始行是由 Z 地址计数器控制的。A5~A0，6 位地址自动送入 Z 地址计数器，起始行的地址可以是 0~63 的任意一行。

例如：

选择 A5~A0 是 62, 则起始行与 DDRAM 行的对应关系如下：

DDRAM 行：62 63 0 1 2 3 28 29

屏幕显示行：1 2 3 4 5 6 31 32

(3) 设置页地址 (SET PAGE “X ADDRESS”)

代码	R/W	D/I	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
形式	0	0	1	0	1	1	1	A2	A1	A0

所谓页地址就是 DDRAM 的行地址，8 行为一页，模块共 64 行即 8 页（见前面表），A2~A0 表示 0~7 页。读写数据对地址没有影响，页地址由本指令或 RST 信号改变复位后页地址为 0。页地址与 DDRAM 的对应关系见 DDRAM 地址表。

(4) 设置 Y 地址 (SET Y ADDRESS)

代码	R/W	D/I	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
形式	0	0	0	1	A5	A4	A3	A2	A1	A0

此指令的作用是将 A5~A0 送入 Y 地址计数器，作为 DDRAM 的 Y 地址指针。在对 DDRA M 进行读写操作后，Y 地址指针自动加 1，指向下一个 DDRAM 单元。

(5) 读状态 (STATUS READ)

代码	R/W	D/I	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
形式	0	1	BUSY	0	ON/OFF	RET	0	0	0	0

当 R/W=1 D/I=0 时，在 E 信号为“H”的作用下，状态分别输出到数据总线 (DB7~DB0) 的相应位。

BF：前面已叙述过（见 BF 标志位一节）。

ON/OFF：表示 DFF 触发器的状态（见 DFF 触发器一节）。

RST：RST=1 表示内部正在初始化，此时组件不接受任何指令和数据。

(6) 写显示数据 (WRITE DISPLAY DATE)

代码	R/W	D/I	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
形式	0	1	D7	D6	D5	D4	D3	D2	D1	D0

D7~D0 为显示数据，此指令把 D7~D0 写入相应的 DDRAM 单元，Y 地址指针自动加 1。

(7) 读显示数据 (READ DISPLAY DATE)

代码	R/W	D/I	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0
形式	1	1	D7	D6	D5	D4	D3	D2	D1	D0

此指令把 DDRAM 的内容 D7~D0 读到数据总线 DB7~DB0，Y 地址指针自动加 1。

5、读写操作时序

(1) 写操作时序

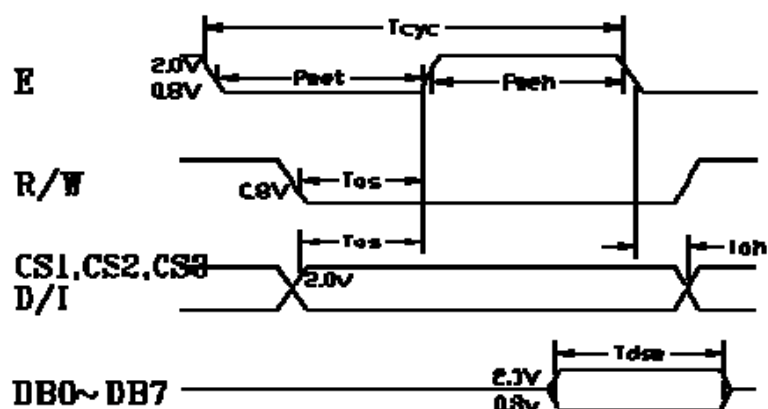


图 3.62 12864 写操作时序

(2) 读操作时序

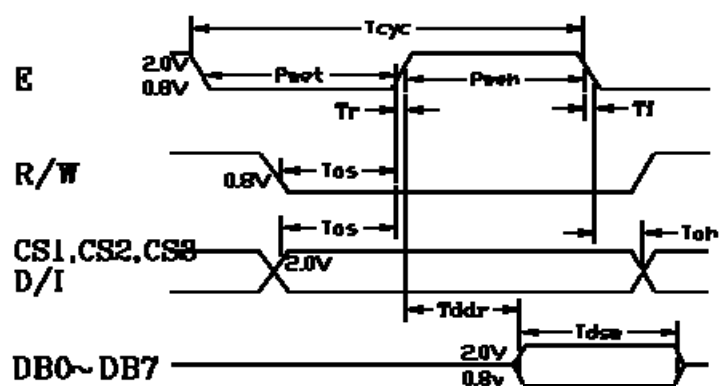


图 3.63 12864 读操作时序

(3) 读写时序参数表

名 称	符 号	最小值	典型值	最大值	单位
E 周期时间	Tcyc	1000	---	---	ns
E 高电平宽度	Pweh	450	---	---	ns
E 低电平宽度	Pwel	450	---	---	ns
E 上升时间	Tr	---	---	25	ns
E 下降时间	Tf	---	---	25	ns
地址建立时间	Tas	140	---	---	ns
地址保持时间	Tah	10	---	---	ns

数据建立时间	Tdsw	200	---	---	ns
数据延迟时间	Tddr	---	---	320	ns
写数据保持时间	Tdhw	10	---	---	ns
读数据保持时间	Tdhw	20	---	---	ns

3. 12. 2 LCD 12864 显示实验

(1) 实验内容

在 Proteus 环境下搭建万年历电路图如下图 3. 63 所示的：

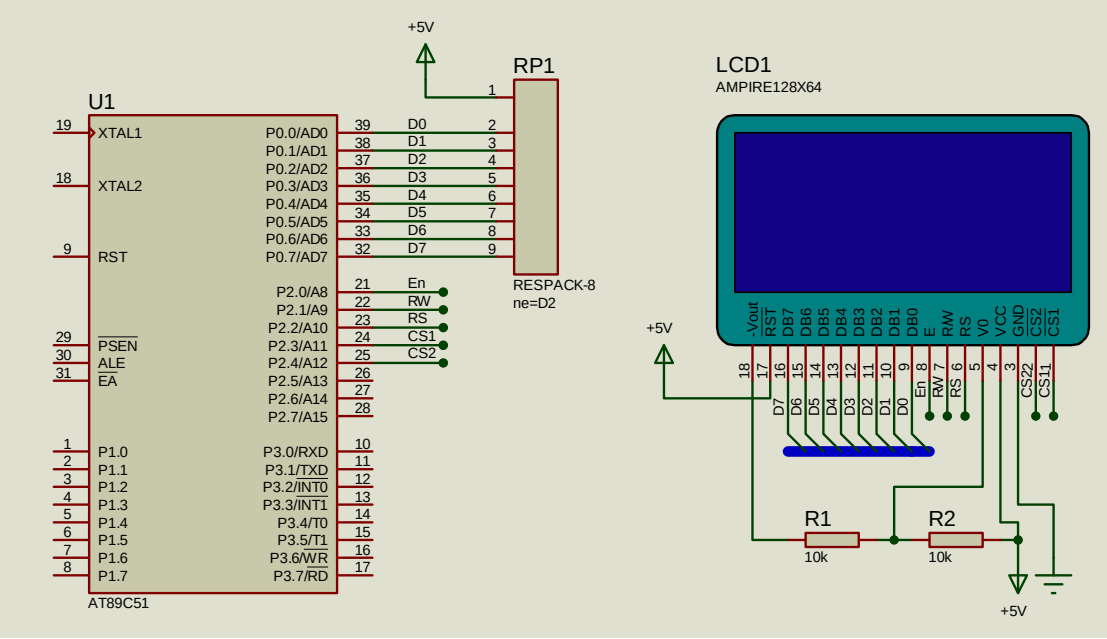


图 3. 63 LCD12864 显示实验

其中所用元器件如下表所示：

选件编号	元件名称	参数	所在元件库类名	子类名	生产厂家
U1	AT89C51		Microprocessor ICs 微处理器	8051 Family 8051 家族	ATMEL
LCD1	AMPIRE 128*64		Optoelectronic 光电器件	Graphical LCDs 图形 LCD	
RP1	RESPACK- 8		Resistors Packs 电阻		
R1, R2	RES	10k	Resistors 电阻	Generic 一般的	

(2) 控制要求

本实验控制程序运行时，能够用 LCD12864 显示“山大电工电子中心”几个汉字，并实现从下向上滚屏移动所显示内容，如下图 3.64 所示：

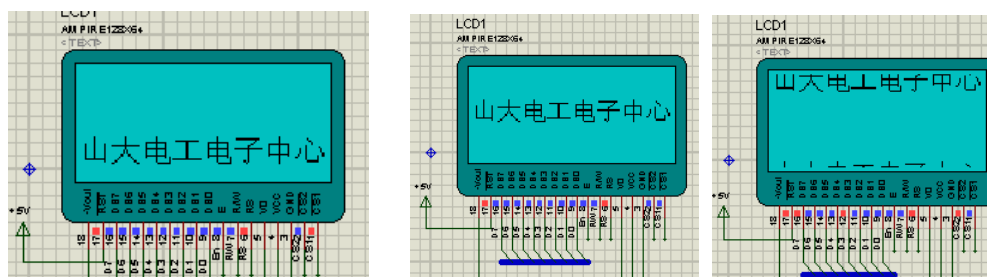


图 3.64 LCD12864 滚屏显示效果

(3) 控制程序

```

/*****必要的变量定义*****/
#include <reg51.h>
#include <intrins.h>
#define uint unsigned int
#define uchar unsigned char
#define DATA P0

sbit RS=P2^2;
sbit RW=P2^1;
sbit EN=P2^0;
sbit cs1=P2^3;
sbit cs2=P2^4;

/*****定义字库*****/
uchar code Hzk[]={

0x00,0x00,0xF0,0x00,0x00,0x00,0x00,0xFF,0x00,0x00,0x00,0x00,0xF0,0x00,0x00,0x
00,0x00,0x00,0x3F,0x20,0x20,0x20,0x20,0x3F,0x20,0x20,0x20,0x20,0x7F,0x00,0x0
0,0x00,/*"山",0*/

0x20,0x20,0x20,0x20,0x20,0x20,0x20,0xFF,0x20,0x20,0x20,0x20,0x20,0x20,0x
00,0x80,0x80,0x40,0x20,0x10,0x0C,0x03,0x00,0x03,0x0C,0x10,0x20,0x40,0x80,0x8
0,0x00,/*"大",1*/

0x00,0x00,0xF8,0x88,0x88,0x88,0x88,0xFF,0x88,0x88,0x88,0x88,0xF8,0x00,0x00,0x
00,0x00,0x00,0x1F,0x08,0x08,0x08,0x08,0x7F,0x88,0x88,0x88,0x88,0x9F,0x80,0xF
0,0x00,/*"电",2*/

```



```

do {
    DATA=0x00;
    EN=1;
    _nop_();
    dat=DATA;
    EN=0;
    dat=0x80&dat; //检查忙信号
}while(!(dat==0x00)); //当忙信号为 0 时才可继续操作
}

/*****写命令子程序*****/
void SendCommandToLCD(uchar com)
{
    CheckState();
    RS=0; //写命令
    RW=0;
    DATA=com;
    EN=1; //利用 EN 下降沿完成命令写操作
    _nop_();
    _nop_();
    EN=0;
}

void SetLine(uchar page) //设置页码, 页码为 0~7
{
    page=0xb8|page;
    SendCommandToLCD(page);
}

void SetStartLine(uchar startline) //设置起始行, 行号为 0~63
{
    startline=0xc0|startline;
    SendCommandToLCD(startline);
}

void SetColumn(uchar column) //设置列, 列号为 0~63
{

```

```

    column=column&0x3f;
    column=0x40|column;
    SendCommandToLCD(column);
}

void SetOnOff(uchar onoff)    //开关显示屏, onoff 只能为 0 或 1
{
    onoff=0x3e|onoff;
    SendCommandToLCD(onoff);
}

void WriteByte(uchar dat)    //写数据
{
    CheckState();
    RS=1;
    RW=0;
    DATA=dat;
    EN=1;
    _nop_();
    _nop_();
    EN=0;
}

void SelectScreen(uchar screen)    //选屏, screen=0,1,2
{
    switch(screen)
    {
        case 0: cs1=0;    //全屏显示
            _nop_();
            _nop_();
            _nop_();
            cs2=0;
            _nop_();
            _nop_();
            _nop_();
            break;
        case 1: cs1=0;    //左屏显示

```

```

        _nop_();
        _nop_();
        _nop_();
        cs2=1;
        _nop_();
        _nop_();
        _nop_();
        break;
    case 2: cs1=1;    //右屏显示
        _nop_();
        _nop_();
        _nop_();
        cs2=0;
        _nop_();
        _nop_();
        _nop_();
        break;
}
}

```

```

void ClearScreen(uchar screen) //清屏, screen=0,1,2
{
    uchar i, j;
    SelectScreen(screen);
    for (i=0; i<8; i++)
    {
        SetLine(i);
        SetColumn(0);
        for (j=0; j<64; j++)
        {
            WriteByte(0x00); //写数据列地址将自动加 1
        }
    }
}

```

```

/*****初始化子程序*****/
void InitLCD()

```

```

{
    CheckState();
    SelectScreen(0);
    SetOnOff(0);    //关屏
    SelectScreen(0);
    SetOnOff(1);    //开屏
    SelectScreen(0);
    ClearScreen(0); //清屏
    SetStartLine(0); //开始行为 0
}

```

/******显示全角汉字******/

```
void Display(uchar ss,uchar page,uchar column,uchar number)
```

```

{
    int i;
    SelectScreen(ss); //ss 为屏号
    column=column&0x3f; //column 为列号
    SetLine(page); //page 为页号，显示第 number 个汉字的上半部分，
                    //page 可理解为要显示的汉字位于屏幕的第 page 行
    SetColumn(column);
    for(i=0;i<16;i++) //i 为一个字里面的各个列
    {
        WriteByte(Hzk[i+32*number]); //number 为字号，
                                     //取第 number 个汉字的第 i 列数据编码
    }

    SetLine(page+1); //显示第 number 个汉字的下半部分
    SetColumn(column);
    for(i=0;i<16;i++)
    {
        WriteByte(Hzk[i+32*number+16]); //取第 number 个汉字的下半部分
                                     //第 i 列数据编码值
    }
}

```

值

/******主程序******/

```
void main()
```



```

{
    uint i;
    InitLCD();
    ClearScreen(0);
    while(1)
    {
        for (i=0;i<128;i++)
        {
            SetStartLine(i);
            Display(1,0,0*16,0); //显示第 0 号字,“山”
            Display(1,0,1*16,1); //显示第 1 号字,“大”
            Display(1,0,2*16,2); //显示第 2 号字,“电”
            Display(1,0,3*16,3); //显示第 3 号字,“工”
            Display(2,0,4*16,4); //显示第 4 号字,“电”
            Display(2,0,5*16,5); //显示第 5 号字,“子”
            Display(2,0,6*16,6); //显示第 6 号字,“中”
            Display(2,0,7*16,7); //显示第 7 号字,“心”
            SelectScreen(0);
            delay(50);
        }
    }
}

```

(4) 实验步骤

- ① 根据上述实验内容,参考 1.2.2,在 Proteus 环境下建立图 3.63 所示原理图,并将其保存为 LCD12864.DSN 文件。
- ② 根据 (2) 和 (3) 画出流程图,并编写源程序,将其保存为 LCD12864.c。
- ③ 运行 Keil uVision2 开发环境,按照 1.1.3 节介绍的方法建立工程 LCD12864.uv2, CPU 为 AT89C51,包含启动文件 STARTUP.A51。
- ④ 按照 1.2.2 第(6)节介绍的方法将 C 语言源程序 LCD12864.c 加入工程 LCD12864.uv2,并设置工程 LCD12864.uv2 属性,将其晶振频率设置为 12MHz,选择输出可执行文件,仿真方式为选择硬仿真,并选择其中的“PROTEUS VSM MONITOR 51 DRIVER”仿真器。
- ⑤ 构造 (Build) 工程 LCD12864.uv2。如果输入有误进行修改,直至构造正确,生成可执行程序 LCD12864.hex 为止。
- ⑥ 为 AT89C51 设置可执行程序 LCD12864.hex。
- ⑦ 运行程序,观察运行结果,并验证其是否正确。

(5) 实验作业

- 分析源程序并进行注释

- 实验过程记录
- 记录程序运行结果截图
- 尝试实现从下向上滚屏。

第四章 硬件接口实验

本章硬件接口实验基于“SDU_MCU 系列单片机教学试验箱”设计和编写的。该试验箱是山东大学电工电子实验教学示范中心组织了有多年教学和实际应用经验的教师团队，为满足新时期电子、通信、电气及工业自动化等相关专业本科生完成 51 单片机实验教学任务，提高本科生单片机实际应用能力，以及为工科电子类本科生参加全国电子设计大赛而进行的技能实训等需要而设计的。

该试验箱选用的器件除了传统的必备原理性接口器件（如 ADC0809，DAC0832，8255 等）外还大量采用在实际单片机应用系统设计中常用的器件（如串行接口扩展器件串行 ADC 芯片 AD7810，串行 DAC 芯片 TLC5615， 串行 EEPROM 芯片 AT24C01， 串行 I/O 扩展芯片 PCF8574，串行 RTC 芯片 PCF8563 等），缩短学生学习与实际应用的差距。

该试验箱共配置了 8 块子板，基本涵盖了各种功能模块。子板包括：MCU 板，PIO 板，KEY&LED 板，LCD 板，AD&DA 板，Memory&RTC 板，Sensor 板，Driver 板等。有关各模块的相关芯片和电路图，请阅读“SDU_MCU 系列单片机教学试验箱”的使用手册。

4.1 实验 1：按键声光报警实验

（1）实验内容

本实验联系静态按键识别，发光二极管驱动，以及外部中断的使用方法，实验原理图如下图 4.1.1 所示（注：图中元件编号中括号里面的内容表示该元件所在的板子名称，如 CPU 表示该元器件位于 CPU 板，以下类同）。

要求利用外部硬件中断，按键按下一次产生一次外部中断，在中断服务程序中计数器加 1，同时，通过发光二极管的闪烁和蜂鸣器响的次数，指示计数器的当前值。当计数到 10 时，再次按键将重新从 1 开始计。

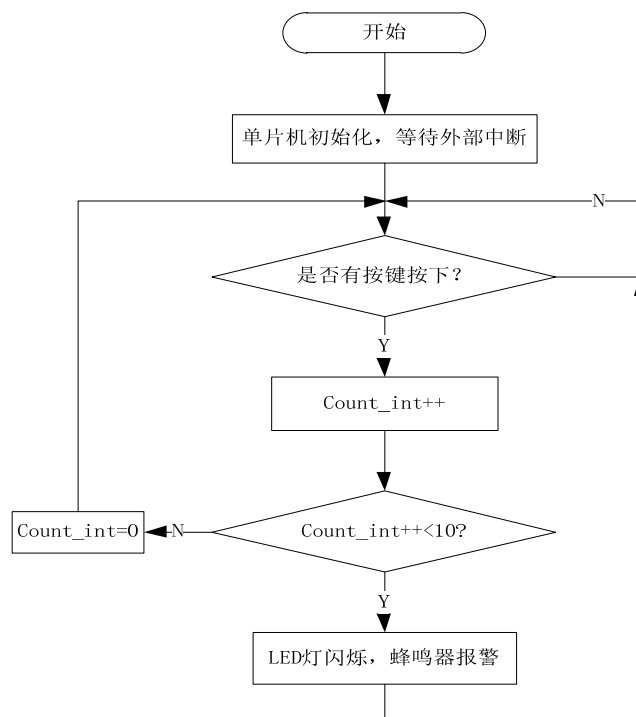


图 4.1.2 按键声光报警实验流程图

(4) 实验步骤

1. 关掉实验箱电源。将 MCU 板插接在母板上，按照前面连接关系表将硬件连接好。
2. 在仿真器断电情况下将仿真器的仿真头插在 MCU 板的 MCU 插座上。将仿真器与开发 PC 机的通信口连接好，打开实验箱及仿真器的电源。
3. 运行 Keil uVision2 开发环境，建立工程 int0_c.uv2，MCU 为 AT89S51，包含启动文件 STARTUP.A51。
4. 按照实验功能要求创建源程序 int0.c 并加入到工程 int0_c.uv2，并设置工程 int0_c.uv2 属性，将其晶振频率设置为 11.0592MHz，选择输出可执行文件，DEBUG 方式选择硬件 DEBUG，并选择其中的“WAVE V series MCS51 Driver”仿真器。
5. 构造（Build）工程 int0_c.uv2。如果编程有误进行修改，直至构造正确为止。
6. 运行程序，按下 MCU 板上的 KEY0 按键，观察每次按键按下时主板上的发光二极管的闪烁和蜂鸣器响的次数，是否符合程序要求，若不符合要求，分析出错原因，继续重复第 4、5 步的步骤，直至结果正确。

(5) 实验作业

- 总结 C 语言实现中断控制及中断服务的方法
- 尝试利用汇编语言编程实现程序中的相同功能

4.2 实验 2：373 并口扩展实验

(1) 实验内容

本实验利用 74LS373 锁存器实现并行 IO 扩展功能，实验原理图如下图 4.2.1 所示。

要求利用 373 扩展输入 IO 电路读取 DIP 开关状态，然后再利用 373 扩展输出 IO 电路将其利用发光二极管指示出来。DIP 某一位上的开关打在 ON 状态，对应位上的发光二极管亮，DIP 某一位上的开关打在 OFF 状态，对应位上的发光二极管灭。

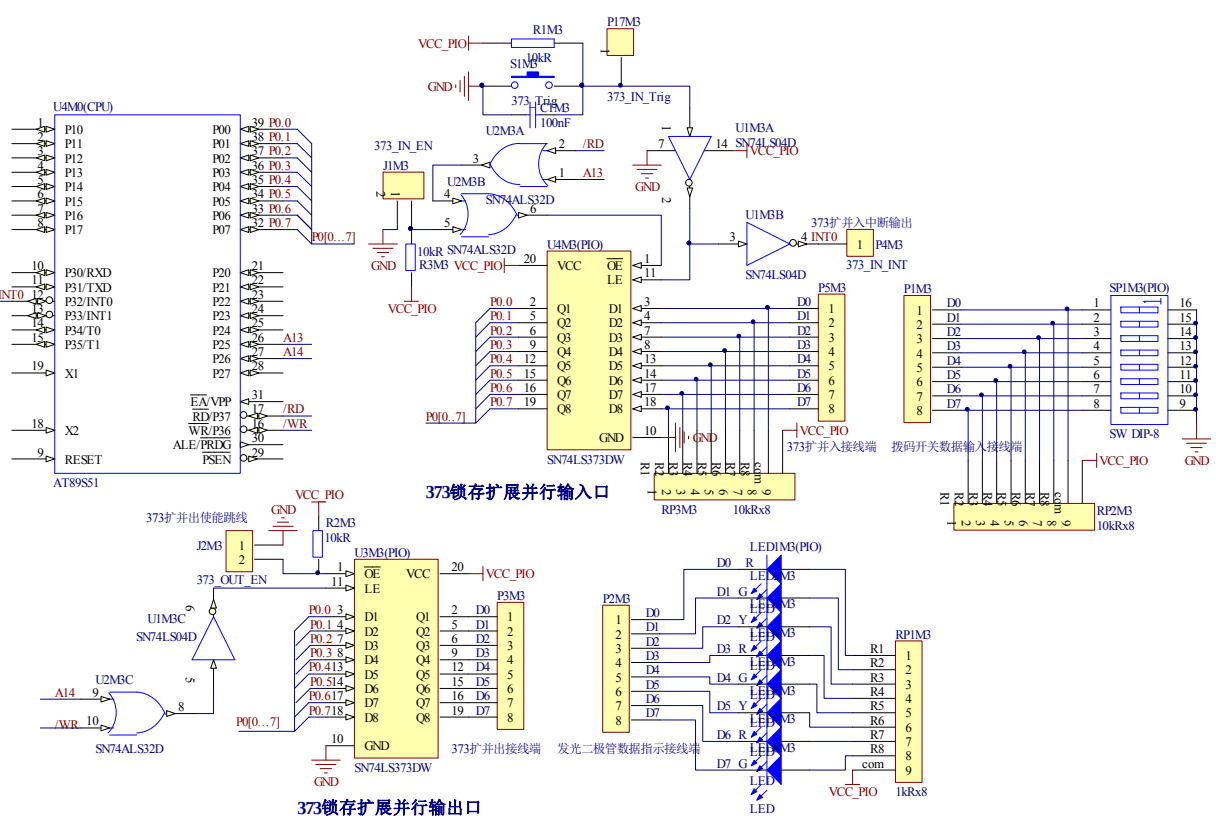


图 4.2.1 373 扩展 IO 实验原理图

(2) 连线关系

实验中端子连接关系如下：

杜邦线连线关系				
线序号	连接端子 1		连接端子 2	
	开发板	端子	开发板	端子
S1	PIO	P4M3:373_IN_INT	PIO	J5M3:INT0
S2	PIO	P5M3:[D0……D7]	PIO	P1M3:[D0……D7]
S3	PIO	P3M3:[D0……D7]	PIO	P2M3:[D0……D7]
S4	PIO	J1M3: (用短路帽短接)		
S5	PIO	J2M3: (用短路帽短接)		

(3) 程序流程图

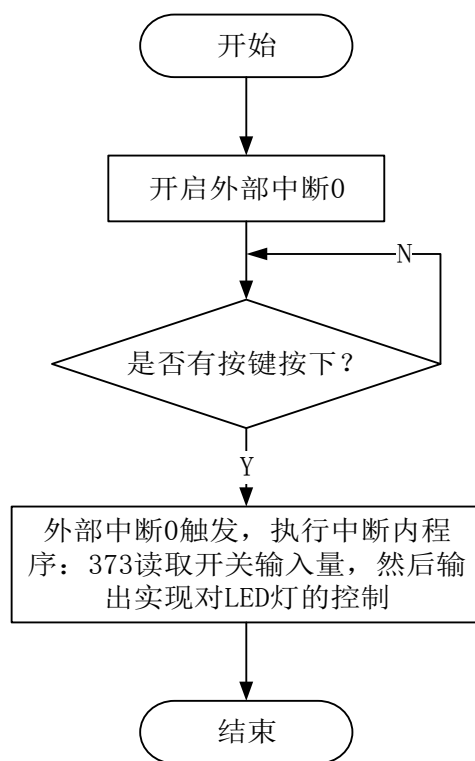


图 4.2.2 373 扩展 IO 实验流程图

(4) 编程注意事项

373 输入扩展的 DIP 状态利用中端方式读取。PIO 扩展板上的 S1M3 按键用于引起 CPU 中断，并将 DIP 开关状态装入 74LS373。

输入扩展 373 的地址连接到 A13 上，相应的地址为 DFFFH, 输出扩展 373 的地址连接到地址线 A14 上，相应的地址 BFFFH。CPU 对该地址的读操作读取的是输入 373 锁存的值，对该地址的写操作时将数据写入输出 373，并通过输出发光二极管 LED1-LED8 指示出来。

(5) 实验步骤

1. 关掉实验箱电源。将 MCU 板、PIO 版插接母版上。按照前面连接关系表将硬件连接好。将 PIO 子板 J1M3 和 J2M3 用跳线短路，使能 373 输入输出。
2. 在仿真器断电情况下将仿真器的仿真头插在 MCU 板的 CPU 插座上。将仿真器与开发 PC 机的通信口连接好，打开实验箱及仿真器的电源。
3. 运行 Keil uVision2 开发环境，建立工程 PIO373_c.uv2，CPU 为 AT89S51，包含启动文件 STARTUP.A51。
4. 按照实验功能要求创建源程序 PIO373.c 并加入到工程 PIO373_c.uv2，并设置工程 PIO373_c.uv2 属性，将其晶振频率设置为 11.0592MHz，选择输出可执行文件，DEBUG 方式选择硬件 DEBUG，并选择其中的“WAVE V series MCS51 Driver”仿真器。
5. 构造 (Build) 工程 PIO373_c.uv2。如果编程有误进行修改，直至构造正确为止。
6. 运行程序，按下 PIO 板上的 S1M3 按键，观察每次按键按下时主板上的发光二极管 LED1-LED8 指示的状态是和主板上的 DIP 开关状态一致，若不一致，分析出错原因，继续重复第 4、5 步的步骤，直至结果正确。

(6) 实验作业

- 总结 74LS373 锁存方式扩展并行 IO 的实现方法
- 分析本程序中 IO 扩展器件地址分配方法
- 尝试利用汇编语言编程实现程序中的相同功能

4.3 实验 3：74LS165 并/串行 IO 口转换实验

(1) 实验内容

本实验利用 74LS165 并串转换器件，以及 MCS-51 单片机实现并行 IO 转串行功能。要求将 DIP 开关状态通过 74LS165 读入单片机，然后经 74LS373 输出至 LED 灯。通过拨码开关控制 LED 灯的亮灭。实验原理图如下图 4.3.1 所示：

禁止。CLK 和 CLKINK 在功能上是等价的，可以交换使用。当 CLK 和 CLK INK 有一个为低电平并且 SH/LD 为高电平时，另一个时钟可以输入。当 CLK 和 CLK INK 有一个为高电平时，另一个时钟被禁止。只有在 CLK 为高电平时 CLK INK 才可变为高电平。

(3) 连线关系

实验中端子连接关系如下：

杜邦线连线关系				
线序号	线端 A 插接位置		线端 B 插接位置	
	开发板	端子	开发板	端子
S1	PIO	JK2M3: P3.0	PIO	J10M3: RXD
S2	PIO	JK2M3: P3.1	PIO	J10M3: TXD
S3	PIO	JK2M3: P1.1	PIO	J10M3: 164/165
S4-S11	PIO	P6M3: D[0...7]	PIO	P1M3: D[0...7]
S12-S19	PIO	P3M3: D[0...7]	PIO	P2M3: D[0...7]
J2M3: 373_OUT_EN 使用短路帽短接				

(4) 程序流程图

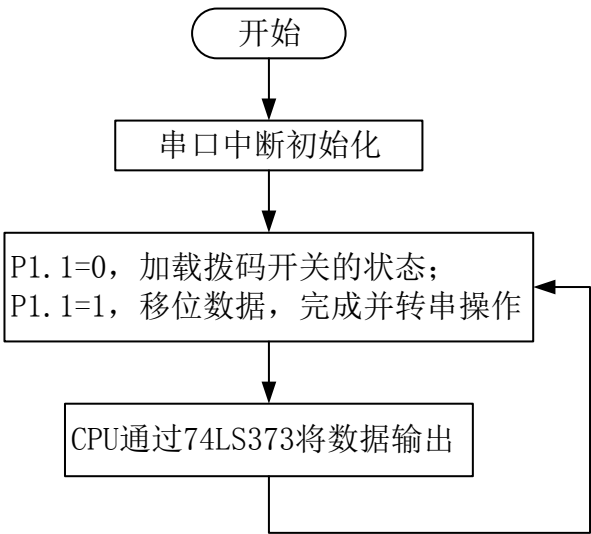


图 4.3.3 74LS165 并串 I/O 转换实验流程图

(5) 实验步骤

1. 关掉实验箱电源。将 MCU 板，PIO 板插接在母板上。按照前面连接关系表将硬件连接好。

2. 在仿真器断电情况下将仿真器的仿真头插在 MCU 板的 CPU 插座上。将仿真器与开发 PC 机的通信口连接好，打开实验箱及仿真器的电源。
3. 运行 Keil uVision2 开发环境，建立工程 PIO165_c.uv2，CPU 为 AT89S51，包含启动文件 STARTUP.A51。
4. 按照实验功能要求创建源程序 PIO_165.c 并加入到工程 PIO_165_c.uv2，并设置工程 PIO_165_c.uv2 属性，将其晶振频率设置为 11.0592MHz，选择输出可执行文件，DEBUG 方式选择硬件 DEBUG，并选择其中的“WAVE V series MCS51 Driver”仿真器。
5. 构造 (Build) 工程 PIO_165_c.uv2。如果编程有误进行修改，直至构造正确为止。
6. 运行程序，更改 DIP 开关 SP1M3 (PIO) 的开关状态，观察 LED 上的显示状态和 DIP 开关的状态是否一致。若不一致，分析出错原因，继续重复第 4、5 步的步骤，直至结果正确。

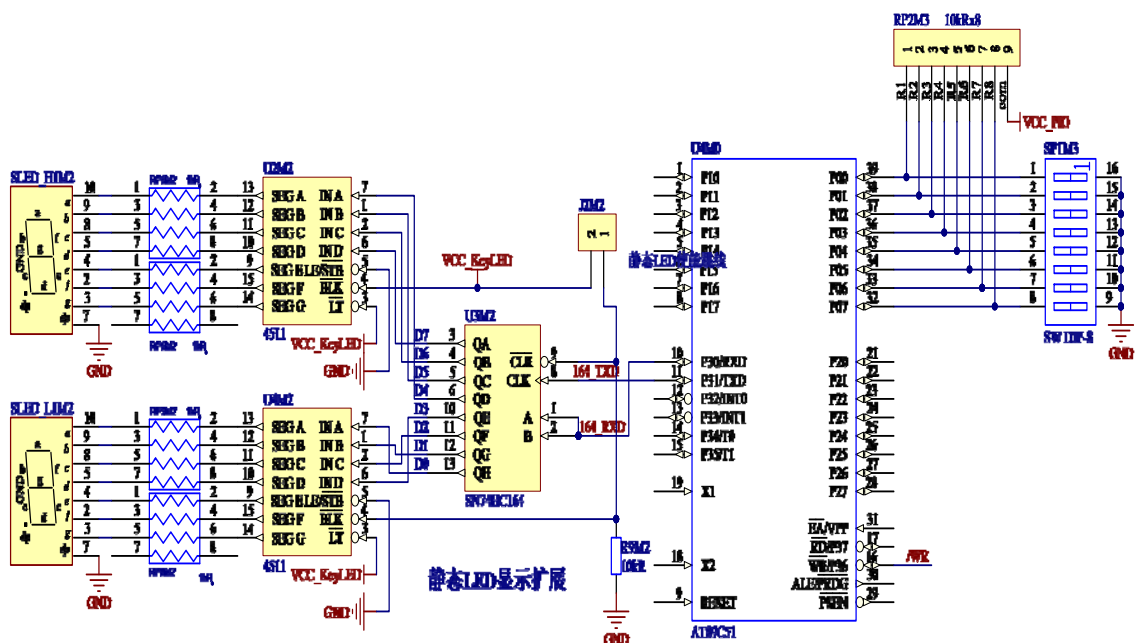
(6) 实验作业

- 总结 51 单片机串口方式 0 的工作原理。
- 总结 74LS165 芯片的使用方法。
- 尝试利用汇编语言编程实现程序中的相同功能

4.4 实验 4：74LS164 串/并行 IO 口转换实验

(1) 实验内容

本实验利用 74LS164 串并转换器件，以及 MCS-51 单片机实现串行行 IO 转并行功能。要求将 DIP 开关状态通过 P0 读入单片机，然后经 74LS164 输出至静态数码管灯。通过拨码开关控制静态数码管灯的显示值。实验原理图如下图 4.4.1 所示：



(2) 实验原理

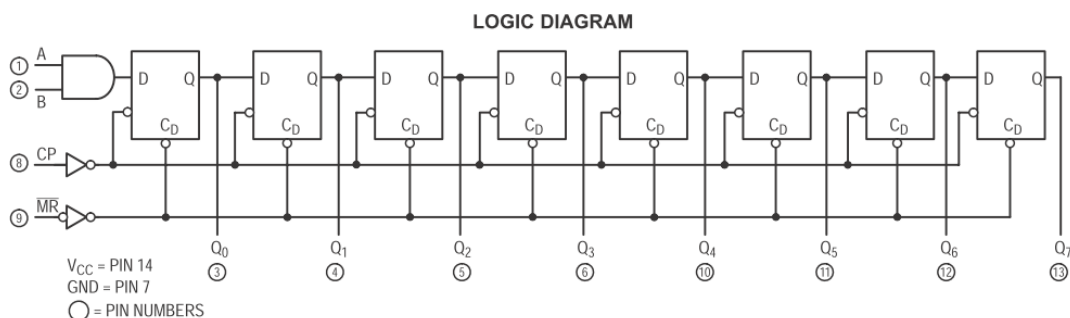


图 4.4.2 74LS164 逻辑图

74HC164、74HCT164 是 8 位边沿触发式移位寄存器，串行输入数据，然后并行输出。数据通过两个输入端（DSA 或 DSB）之一串行输入；任一输入端可以用作高电平使能端，控制另一输入端的数据输入。两个输入端或者连接在一起，或者把不用的输入端接高电平，一定不要悬空。

时钟 (CP) 每次由低变高时, 数据右移一位, 输入到 Q0, Q0 是两个数据输入端 (DSA 和 DSB) 的逻辑与, 它将上升时钟沿之前保持一个建立时间的长度。

主复位（MR）输入端上的一个低电平将使其它所有输入端都无效，非同步地清除寄存器，强制所有的输出为低电平。

(3) 连线关系

实验中端子连接关系如下：

杜邦线连线关系				
线序号	线端 A 插接位置		线端 B 插接位置	
	开发板	端子	开发板	端子
S1-S8	PIO	JK2M3: P0.0…P0.7	PIO	P1M3: D0…D7
S9	KEY&LED	JK2M2: RXD	KEY&LED	J8M2: RXD
S10	KEY&LED	JK2M3: TXD	KEY&LED	J8M2: TXD
J7M2: Static_LED_EN 使用短路帽短接				

(4) 程序流程图

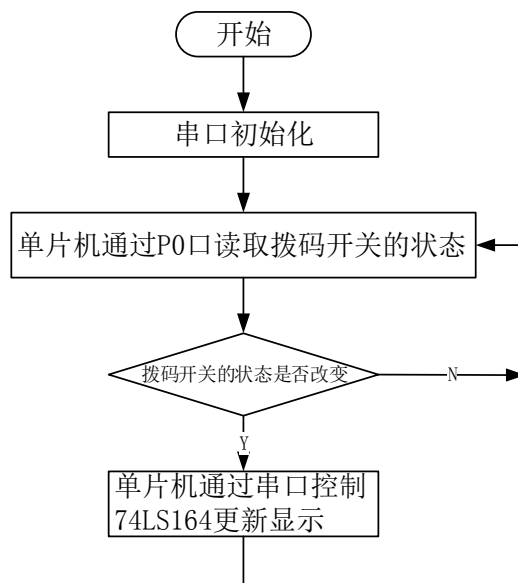


图 4.4.3 74LS164 串并 I/O 转换实验流程图

(5) 实验步骤

1. 关掉实验箱电源。将 MCU 板，PIO 板，KEY&LED 板插接在母板上。按照前面连接关系表将硬件连接好。
2. 在仿真器断电情况下将仿真器的仿真头插在 MCU 板的 CPU 插座上。将仿真器与开发 PC 机的通信口连接好，打开实验箱及仿真器的电源。
3. 运行 Keil uVision2 开发环境，建立工程 PIO164_c.uv2，CPU 为 AT89S51，包含启动文件 STARTUP.A51。
4. 按照实验功能要求创建源程序 PIO_164.c 并加入到工程 PIO_164_c.uv2，并设置工程 PIO_164_c.uv2 属性，将其晶振频率设置为 11.0592MHz，选择输出可执行

文件,DEBUG 方式选择硬件 DEBUG,并选择其中的“WAVE V series MCS51 Driver”仿真器。

5. 构造 (Build) 工程 PIO_164_c.uv2。如果编程有误进行修改,直至构造正确为止。
6. 运行程序,更改 DIP 开关 SP1M3 (PI0) 的开关状态,观察静态数码管显示的数值和 DIP 开关对应的十进制数是否一致。若不一致,分析出错原因,继续重复第 4、5 步的步骤,直至结果正确。

(6) 实验作业

- 总结 74LS164 芯片的工作原理。
- 尝试利用汇编语言编程实现程序中的相同功能

4.5 实验 5: 8255 并行 I/O 扩展及交通信号灯控制实验

(1) 实验内容

本实验利用 8255 实现可编程的并行 I/O 扩展功能,并利用其完成交通灯控制。实验要求红灯常亮 30S,绿灯常亮 25S 后闪烁 5S。使用静态数码管显示绿灯常亮倒计时。实验原理图如下图 4.5.1 所示:

实验原理图如下:

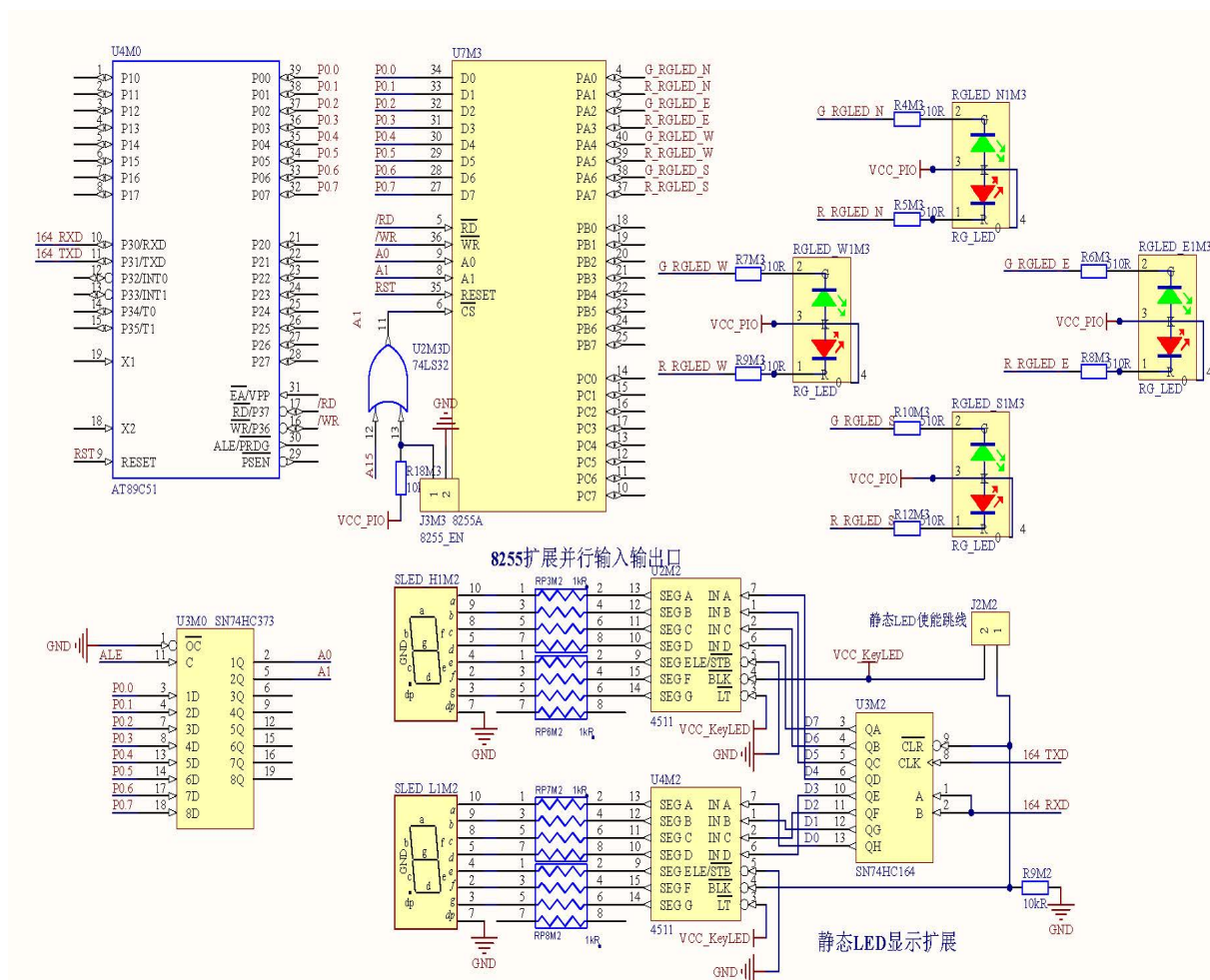


图 4.5.1 8255 并行 I/O 扩展及交通信号灯控制实验原理图

(2) 实验原理

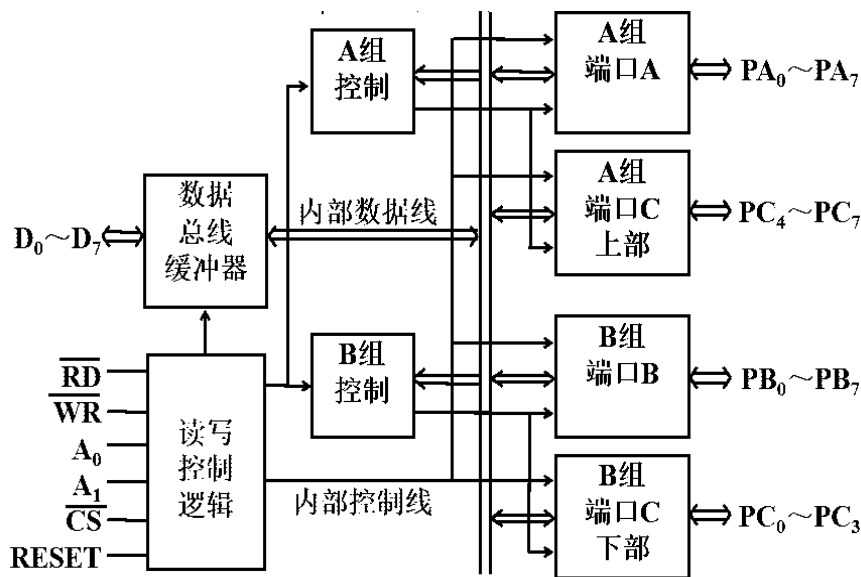
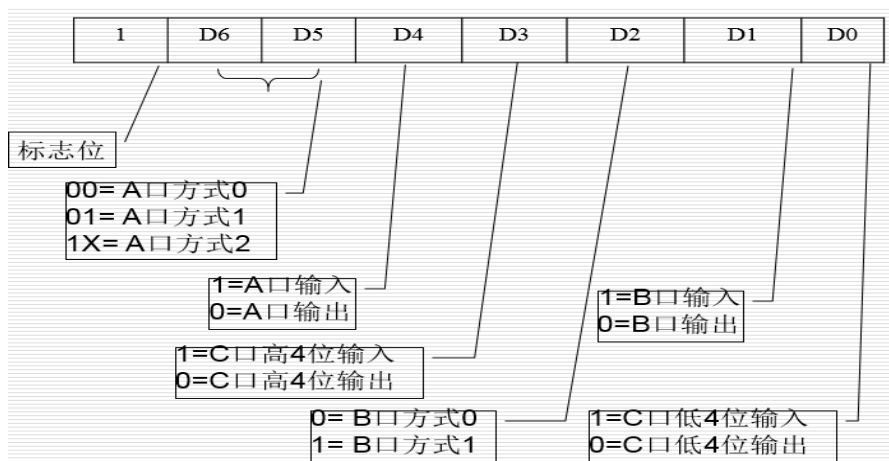


图 4.5.2 8255A 的内部结构

8255A 的基本操作

A1	A0	RD	WR	CS	操作	
0	0	0	1	0	A口内容读至数据总线	输入
0	1	0	1	0	B口内容读至数据总线	
1	0	0	1	0	C口内容读至数据总线	
0	0	1	0	0	数据总线内容写至A口	输出
0	1	1	0	0	数据总线内容写至B口	
1	0	1	0	0	数据总线内容写至C口	
1	1	1	0	0	数据总线内容写至控制寄存器	
×	×	×	×	1	端口输出为高阻	禁止
1	1	0	1	0	非法	
×	×	1	1	0	端口输出为高阻	

8255A 的控制字



(3) 连线关系

接插件连接关系如下：

杜邦线连线关系				
线序号	线端 A 插接位置		线端 B 插接位置	
	开发板	端子	开发板	端子
S1-S8	PIO	P12M3: PA[0...7]	PIO	P9M3: NG.....SR
S9	KEY&LED	JK2M2: RXD	KEY&LED	J8M2: RXD
S10	KEY&LED	JK2M2: TXD	KEY&LED	J8M2: TXD
S11	PIO	J3M3: 使用短路帽短接		
S12	KEY&LED	J7M2: 使用短路帽短接		

(4) 程序流程图

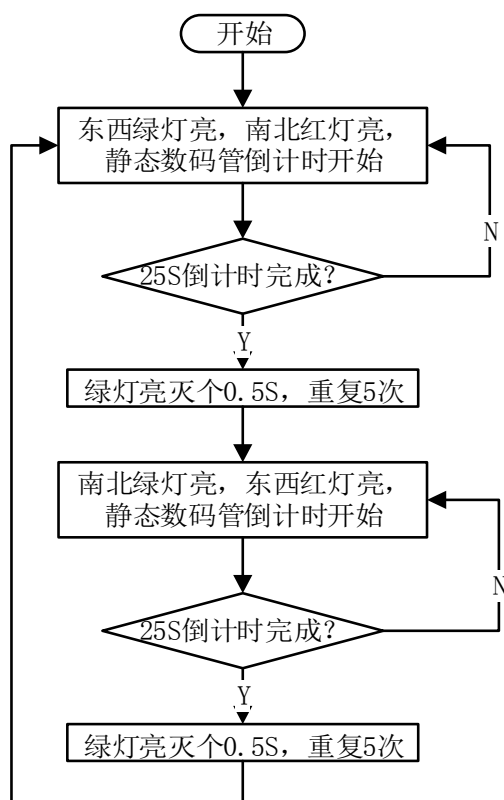


图 4.5.3 8255 并行 I/O 扩展及交通信号灯控制流程图

(5) 注意事项

本实验中的东西方向信号灯同步控制，南北方向信号灯同步控制，即：东西方向上同种颜色的灯同时亮或灭，南北方向上同种颜色的灯同时亮或灭。

程序中的延时 1 秒钟，可以利用纯软件延时实现，也可以结合定时器硬件中断加软件计数方式实现。

图中的 CD4511 是 BCD 码到 LED 数码管的显示码转换芯片，因此要显示某个数字，只需要在相应的端口送出该数字的 BCD 码即可，不用进行软件译码。

(6) 实验步骤

1. 关掉实验箱电源。将 MCU 板，PIO 版，KEY&LED 板插接在母板上。按照前面连接关系表将硬件连接好。
2. 在仿真器断电情况下将仿真器的仿真头插在 MCU 板的 CPU 插座上。将仿真器与开发 PC 机的通信口连接好，打开实验箱及仿真器的电源。

3. 运行 Keil uVision2 开发环境, 建立工程 PI08255_traffic_c.uv2, CPU 为 AT89S51, 包含启动文件 STARTUP.A51。
4. 按照实验功能要求创建源程序 PI08255_traffic.c 并加入到工程 PI08255_traffic_c.uv2, 并设置工程 PI08255_traffic_c.uv2 属性, 将其晶振频率设置为 11.0592MHz, 选择输出可执行文件, DEBUG 方式选择硬件 DEBUG, 并选择其中的“WAVE V series MCS51 Driver”仿真器。
5. 构造 (Build) 工程 PI08255_traffic_c.uv2。如果编程有误进行修改, 直至构造正确为止。
6. 运行程序, 交通灯状态切换以及倒计时器的显示是否符合程序要求, 若不符合, 分析出错原因, 继续重复第 4、5 步的步骤, 直至结果正确。

(7) 实验作业

- 总结 51 单片机利用可编程 I/O 扩展芯片 8255 的使用方法。
- 尝试使用定时器实现更准确的延时。
- 尝试利用汇编语言编程实现程序中的相同功能

4.6 实验 6: 8255 动态 LED 显示实验

(1) 实验内容

本实验利用 8255 实现可编程的并行 I/O 扩展功能, 并利用其完成对动态 LED 数码管显示控制。实验原理图如下图所示:

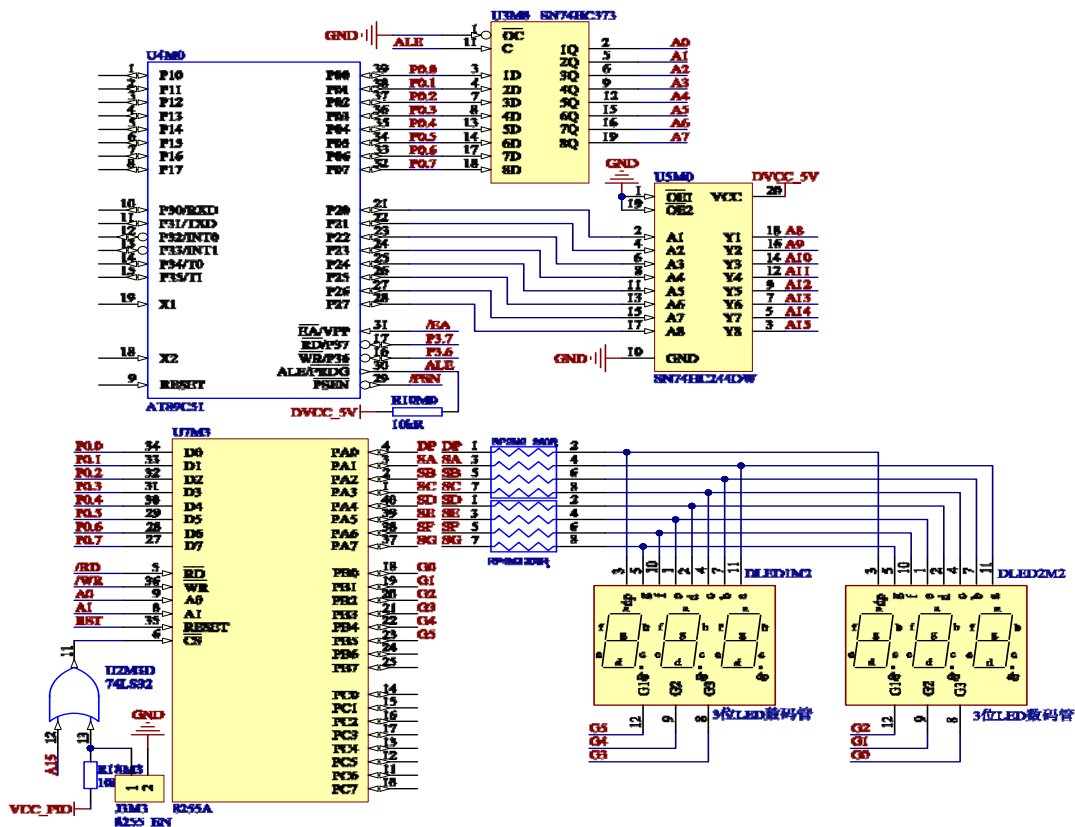


图 4.6.1 8255 动态 LED 显示实验原理图

利用 8255 的 PA 口设置为输出, 用于输出 LED 数码管段码（共阴极方式）。PB 口设置为输出, PB[0..5]口用于驱动 LED 数码管位选信号（低电平有效）。

1. 依次在动态数码管上显示 0-F。满屏后向左滚动。
2. F 左移出动态数码管后字符改变移动方向右移。
3. 0 右移出动态数码管后字符改变移动方向左移。
4. 重复 2.3 现象

(2) 连线关系

实验中端子连接关系如下：

杜邦线连线关系				
线序号	线端 A 插接位置		线端 B 插接位置	
	开发板	端子	开发板	端子
S1-S6	KEY&LED	J5M2: Dig_0-Dig_5	PIO	P15M3:PB0-PB5
S7-S12	KEY&LED	J1M2:DP, SA-SG	PIO	P12M3:PA0-PA7
PIO: J3M3 使用短路帽短接				

(3) 程序流程图

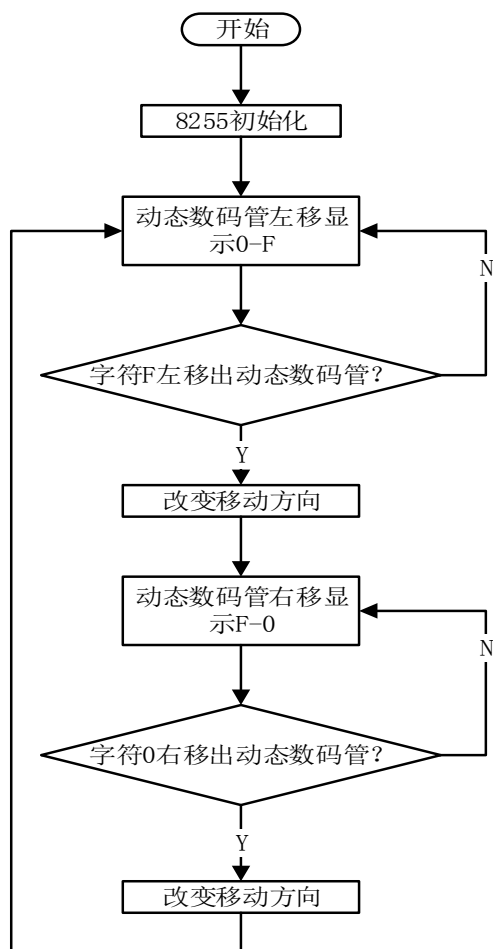


图 4.6.2 8255 动态 LED 显示实验流程图

(4) 实验步骤

1. 关掉实验箱电源。将 MCU 板、PIO 板、KEY&LED 板插接在母板插槽上。按照前面连接关系表将硬件连接好。
2. 在仿真器断电情况下将仿真器的仿真头插在 MCU 板的 CPU 插座上。将仿真器与开发 PC 机的通信口连接好，打开实验箱及仿真器的电源。
3. 运行 Keil uVision2 开发环境，建立工程 PI08255_LED_c.uv2，CPU 为 AT89S51，包含启动文件 STARTUP.A51。
4. 按照实验功能要求创建源程序 PI08255_LED.c 并加入到工程 PI08255_LED_c.uv2，并设置工程 PI08255_LED_c.uv2 属性，将其晶振频率设置为 11.0592MHz，选择输出可执行文件，DEBUG 方式选择硬件 DEBUG，并选择其中的“WAVE V series MCS51 Driver”仿真器。

- 5. 构造（Build）工程 PI08255_LED_c.uv2。如果编程有误进行修改，直至构造正确为止。
- 6. 运行程序，观察结果是否符合程序要求，若不符合，分析出错原因，继续重复第 4、5 步的步骤，直至结果正确。

（5）实验作业

- 总结利用 8255 实现行列式键盘扫描和动态 LED 显示控制的实现方法。
- 将按键键盘识别方法改为线反转法再次进行实验。
- 将每位的显示时间分别设定为 0.1S 和 1ms，再次运行并观察实验结果。
- 尝试利用汇编语言编程实现程序中的相同功能

4.7 实验 7：7279 键盘扫描及动态 LED 显示实验

（1）实验内容

本实验利用 7279 进行键盘扫描及动态 LED 数码管显示控制。实验原理图如下图 4.7.1 所示。

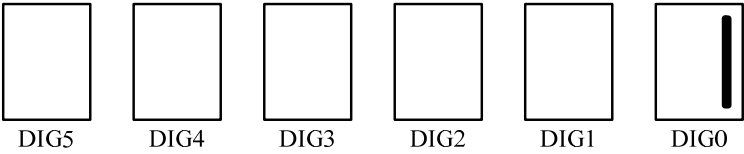
实验功能要求：

（1） 按键编码如下：

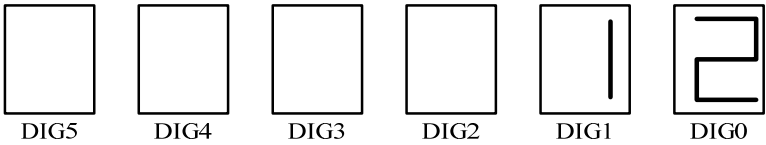
按键名称	按键编码	显示字符
KEY16M2	1bH	‘0’
KEY15M2	13H	‘1’
KEY14M2	0bH	‘2’
KEY13M2	03H	‘3’
KEY12M2	1aH	‘4’
KEY11M2	12H	‘5’
KEY10M2	0aH	‘6’
KEY9M2	02H	‘7’
KEY8M2	19H	‘8’
KEY7M2	11H	‘9’
KEY6M2	09H	‘A’

KEY5M2	01H	‘B’
KEY4M2	18H	‘C’
KEY3M2	10H	‘D’
KEY2M2	08H	‘E’
KEY1M2	00H	‘F’

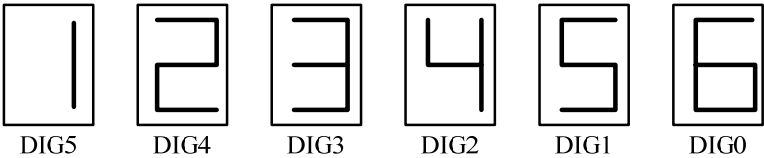
(2) 当按下某个按键时所按按键对应的字符显示在最右端 LED 数码管上，如果是第一次按键，假如是按下的按键’ 1’，将显示为：



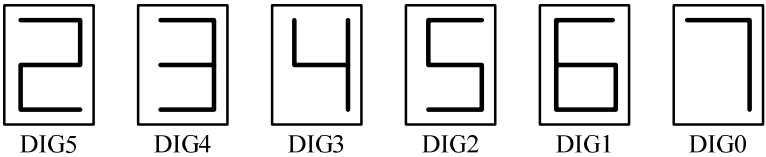
如果，再次按下一个按键’ 2’，则原来显示的内容往左移 1 位，将新按下的按键’ 2’ 的字符显示在最右端，如下图所示，



当 6 位 LED 均显示满（如 123456），如下图所示：



这时如再次按下新的按键（假设为字符 7），则原来显示的内容同样都左移一位，最后一位显示新按按键的字符，如下图所示：



实验原理图如下：

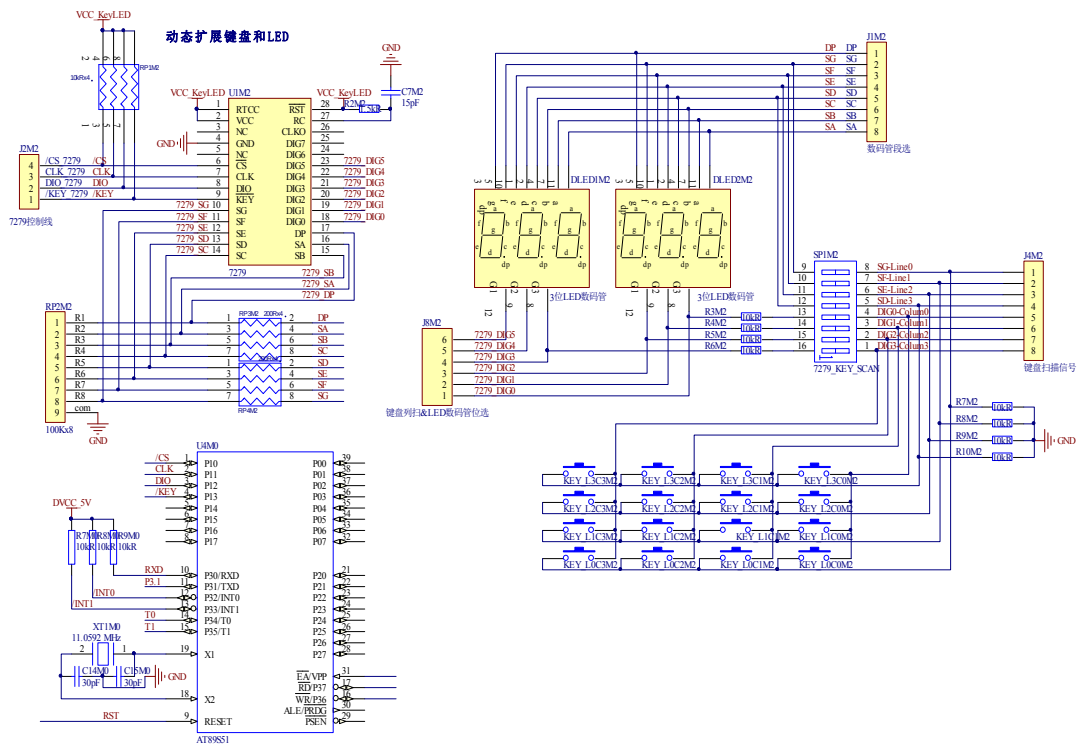


图 4.7.1 7279 键盘扫描及动态 LED 显示实验原理图

(2) 实验原理

HD7279A 是一片具有串行接口的，可驱动 8 位共阴式数码管（或 64 只独立 LED）的智能显示驱动芯片，该芯片同时还可以连接多达 64 键的键盘矩阵，单片即可完成 LED 显示、键盘接口的全部功能。有关 HD7279 的指令和工作原理请查阅实验例程所在目录下的 HD7279 的数据手册。

(3) 连线关系

接插件连接关系如下：

杜邦线连线关系				
线序号	线端 A 插接位置		线端 B 插接位置	
	开发板	端子	开发板	端子
S1	KEY & LED	J1M2: /KEY	KEY & LED	J2M2: P1.3
S2	KEY & LED	J1M2: DIO	KEY & LED	J2M2: P1.2
S3	KEY & LED	J1M2: CLK	KEY & LED	J2M2: P1.1
S4	KEY & LED	J1M2: /CS	KEY & LED	J2M2: P1.0
S5-S12	KEY & LED	J6M2: DP-SG	KEY & LED	J6M2: DP-SG
S13-S18	KEY & LED	J3M2:DIG0-DIG5	KEY & LED	J5M2:DIG0-DIG5

注:连线之前，KEY&LED 板上的 DIP 开关 SP1M2 所有的位都要置于 ON 状态。

(4) 程序流程图

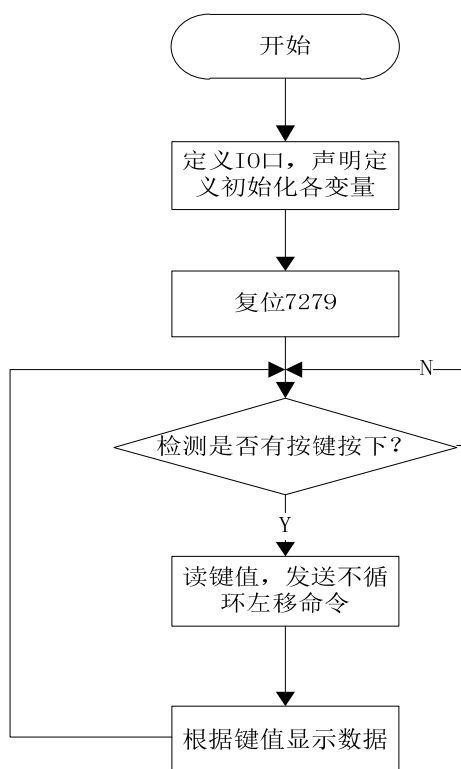


图 4.7.2 7279 键盘扫描及动态 LED 显示实验流程图

(5) 实验步骤

1. 关掉实验箱电源。将 MCU 板，KEY&LED 板插接在母板上。按照前面连接关系表将硬件连接好。
2. 在仿真器断电情况下将仿真器的仿真头插在 MCU 板的 CPU 插座上。将仿真器与开发 PC 机的通信口连接好，打开实验箱及仿真器的电源。
3. 运行 Keil uVision2 开发环境，按照建立工程 HD7279_c.uv2，CPU 为 AT89S51，包含启动文件 STARTUP.A51。
4. 按照实验功能要求创建源程序 HD7279.c 并加入到工程 HD7279_c.uv2，并设置工程 HD7279_c.uv2 属性，将其晶振频率设置为 11.0592MHz，选择输出可执行文件，DEBUG 方式选择硬件 DEBUG，并选择其中的“WAVE V series MCS51 Driver”仿真器。
5. 构造 (Build) 工程 HD7279_c.uv2。如果编程有误进行修改，直至构造正确为止。

- 运行程序，观察结果是否符合程序要求，若不符合，分析出错原因，继续重复第 4、5 步的步骤，直至结果正确。

(6) 实验作业

- 总结利用 7279 实现行列式键盘扫描和动态 LED 显示控制的实现方法。
- 研究 7279 实现 LED 数码管的移位，消隐等控制。
- 尝试利用汇编语言编程实现程序中的相同功能

4.8 实验 8：1602 LCD 显示实验

(1) 实验内容

实验原理图如下：

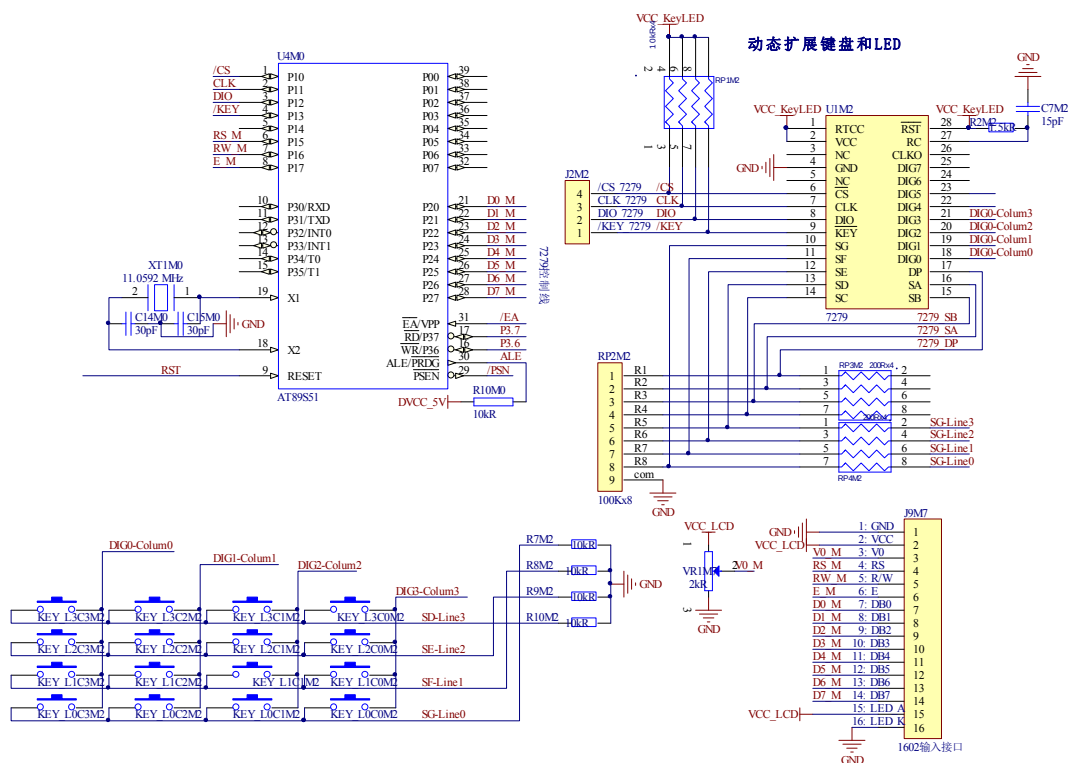


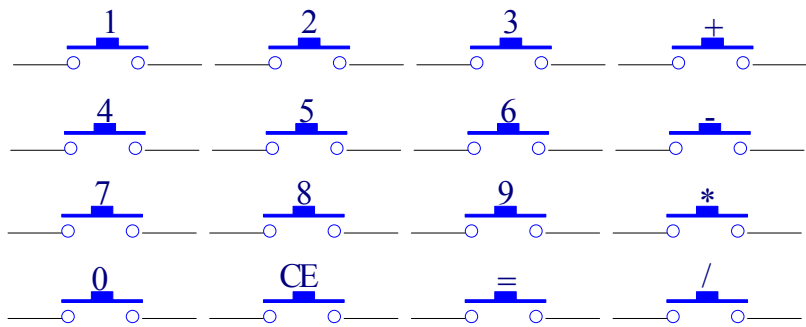
图 4.8.1 7279 键盘扫描及 1602 LCD 显示实验原理图

我们采用 Key&LED 板子的左侧四列键盘组成 4*4 矩阵键盘。

4*4 按键的编码及其代表的功能如下表所示：

按键名称	按键编码	代表的功能
KEY1M2	1bH	数字 1
KEY2M2	13H	数字 2
KEY3M2	0bH	数字 3
KEY4M2	03H	运算符+
KEY5M2	1aH	数字 4
KEY6M2	12H	数字 5
KEY7M2	0aH	数字 6
KEY8M2	02H	运算符-
KEY9M2	19H	数字 7
KEY10M2	11H	数字 8
KEY11M2	09H	数字 9
KEY12M2	01H	运算符*
KEY13M2	18H	数字 0
KEY14M2	10H	CE
KEY15M2	08H	运算符=
KEY16M2	00H	运算符/

编码后按键功能如下图所示：



功能要求：

1. 利用 7279 实现键盘扫描。

2. 当按下某个按键（除 CE 按键外）时，读出该按键的键码，转换成相应的显示字符，并在 LCD1602 上显示出该字符（LCD1602 的使用说明请阅相关资料）。
3. 实现简单的 1 位十进制数的整数四则运算计算器功能，包括除数为 0 报错功能。

扩展功能要求：

1. 实现多位（5 位以下）的数值录入与显示功能。
2. 实现 5 位以下数字的整数四则运算计算器功能，包括除数为 0 报错功能。
3. 将上述 5 位以下数字的四则运算中的除法结果保留两位小数位并显示。

（2）连线关系

接插件连接关系如下：

杜邦线连线关系				
线序号	线端 A 插接位置		线端 B 插接位置	
	开发板	端子	开发板	端子
S1	LCD	J4M7: D[0...7]	LCD	JK2M7: A8-A15 (P2[0...7])
S2	LCD	J6M7: E	LCD	JK2M7: P1.7
S3	LCD	J6M7: RW	LCD	JK2M7: P1.6
S4	LCD	J6M7: RS	LCD	JK2M7: P1.5
S5	KEY & LED	J2M2: /KEY	KEY & LED	JK2M2: P1.3
S6	KEY & LED	J2M2: DIO	KEY & LED	JK2M2: P1.2
S7	KEY & LED	J2M2: CLK	KEY & LED	JK2M2: P1.1
S8	KEY & LED	J2M2: /CS	KEY & LED	JK2M2: P1.0
S9-S16	KEY & LED	J6M2: DP-SA	KEY & LED	J1M2: DP-SA
S17-S24	KEY & LED	J3M2: DIG0-DIG5	KEY & LED	J5M2: DIG0-DIG5

（3）程序流程图

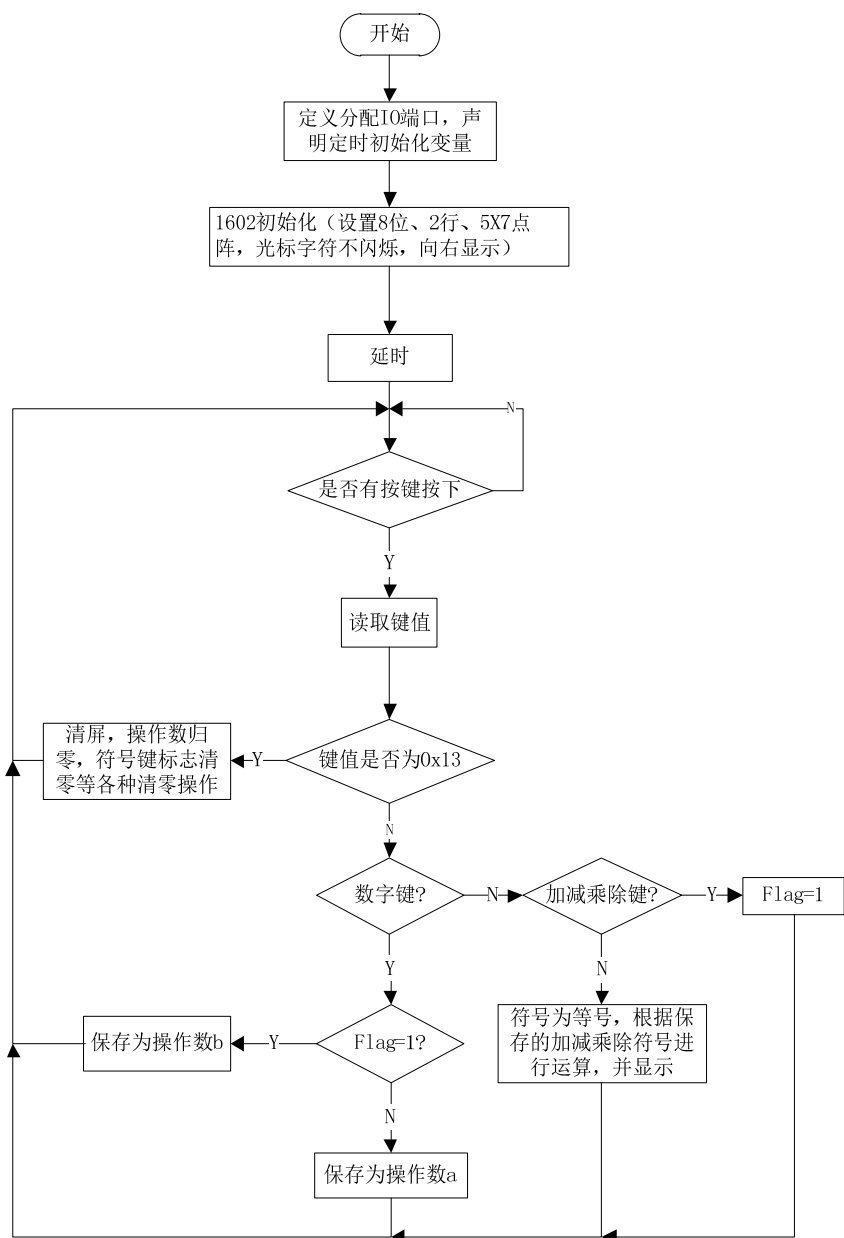


图 4.8.2 7279 键盘扫描及 1602 LCD 显示实验流程图

(4) 实验步骤

1. 关掉实验箱电源。将 MCU 板，KEY&LED 板，LCD 板插接在母板上。LCD1602 插接在 LCD 板的 J5M7（LCD1602 Slot）上，注意插接方向。按照前面连接关系表将硬件连接好。
2. 在仿真器断电情况下将仿真器的仿真头插在 MCU 板的 CPU 插座上。将仿真器与开发 PC 机的通信口连接好，打开实验箱及仿真器的电源。

- 3. 运行 Keil uVision2 开发环境，按照 1.2.3 节介绍的方法建立工程 LCD1602_c.uv2，CPU 为 AT89S51，包含启动文件 STARTUP.A51。
- 4. 按照 1.2.3 节介绍的方法及实验功能要求创建源程序 LCD1602.c 并加入到工程 LCD1602_c.uv2，并设置工程 LCD1602_c.uv2 属性，将其晶振频率设置为 11.0592MHz，选择输出可执行文件，DEBUG 方式选择硬件 DEBUG，并选择其中的“WAVE V series MCS51 Driver”仿真器。
- 5. 构造（Build）工程 LCD1602_c.uv2。如果编程有误进行修改，直至构造正确为止。
- 6. 运行程序，观察结果是否符合程序要求，若不符合，分析出错原因，继续重复第 4、5 步的步骤，直至结果正确。

(5) 实验作业

- 总结利用 LCD1602 显示控制的实现方法。
- 尝试利用汇编语言编程实现程序中的相同功能

4.9 实验 9：12864 LCD 显示实验

(1) 实验内容

本实验利用单片机并行口实现 LCD12864 的显示控制。实验原理图如下图 4.9.1 所示。

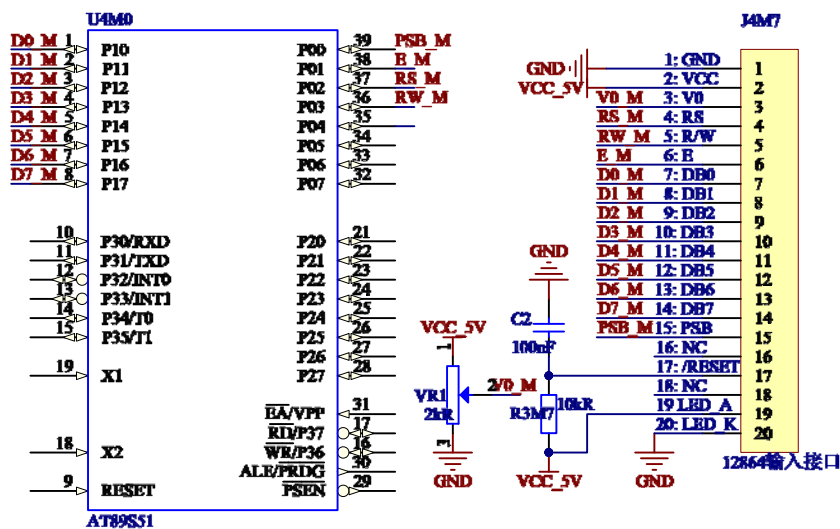


图 4.9.1 12864LCD 显示实验原理图

(2) 连线关系

接插件连接关系如下：

线序号	线端 A 插接位置		线端 B 插接位置	
	开发板	连接端子 1	开发板	连接端子 2
S1-S7	LCD	J4M7: D[0...7]	LCD	JK2M7: (P1[0...7])
S8	LCD	J6M7: PSB	LCD	JK2M7: P0.0
S9	LCD	J6M7: E	LCD	JK2M7: P0.1
S10	LCD	J6M7: RW	LCD	JK2M7: P0.2
S11	LCD	J6M7: RS	LCD	JK2M7: P0.3

(3) 程序流程图

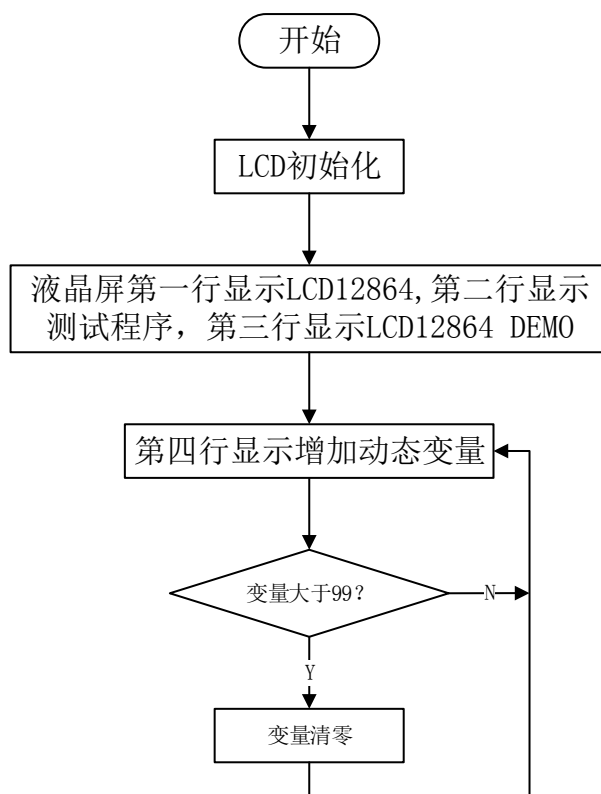


图 4.9.2 12864 LCD 显示实验程序图

(4) 实验步骤

1. 关掉实验箱电源。将 MCU 板，LCD 板插接在母板上。将 LCD12864 插接在 LCD 板的 J4M7 上，按照前面连接关系表将硬件连接好。
2. 在仿真器断电情况下将仿真器的仿真头插在 MCU 板的 CPU 插座上。将仿真器与开发 PC 机的通信口连接好，打开实验箱及仿真器的电源。

其中 16*16LED 点阵电路原理图为：

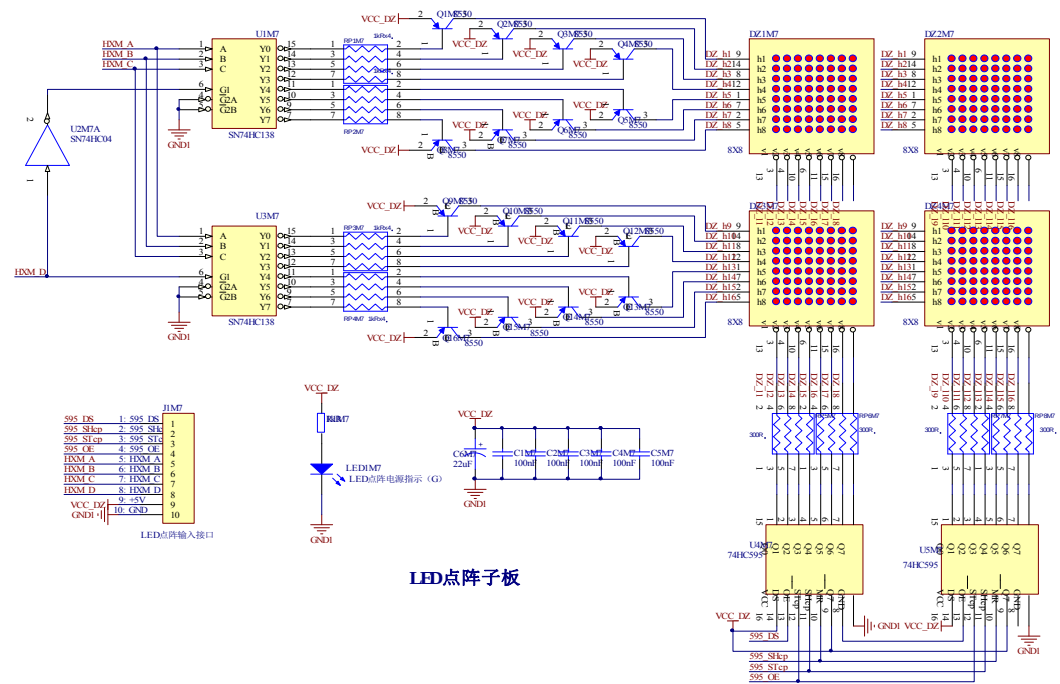


图 4.10.2 16*16 LED 点阵子板原理图

其中行驱动是由两片 74LS138 与三极管电路构成的，HXM_D、HXM_C、HXM_B、HXM_A 为行选择信号，选择关系为：

HXM_D	HXM_C	HXM_B	HXM_A	
0	0	0	0	第 1 行(最上端) 使能
0	0	0	1	第 2 行使能
0	0	1	0	第 3 行使能
0	0	1	1	第 4 行使能
0	1	0	0	第 5 行使能
0	1	0	1	第 6 行使能
0	1	1	0	第 7 行使能
0	1	1	1	第 8 行使能
1	0	0	0	第 9 行使能
1	0	0	1	第 10 行使能
1	0	1	0	第 11 行使能
1	0	1	1	第 12 行使能
1	1	0	0	第 13 行使能
1	1	0	1	第 14 行使能
1	1	1	0	第 15 行使能
1	1	1	1	第 16 行(最下端) 使能

16*16LED 点阵的列驱动是由 74HC595 来提供的。74HC595 内具有一个 8 位移位寄存器和一个存储器和一个三态输出控制。移位寄存器和存储器是采用分开的时钟。数据在 SHcp 的上升沿输入到移位寄存器中，在 STcp 的上升沿输入到存储寄存器中去。如果两个时钟连在一起，则移位寄存器总是比存储寄存器早一个脉冲。移位寄存器有一个串行移位输入（Ds），和一个串行输出（Q7'），和一个异步的低电平复位，存储寄存器有一个 8 位并行的，具备三态的总线输出，当使能 OE 时（为低电平），存储寄存器的数据输出到总线。

按键功能如下表所示：

按键名称	代表的功能
KEY_LOC0M2	UP ↑
KEY_LOC1M2	Left ←
KEY_LOC2M2	Down ↓
KEY_LOC3M2	Right →
KEY_L1COM2	闪烁
KEY_L1C1M2	亮度+
KEY_L1C2M2	亮度-
KEY_L1C3M2	ON/OFF

功能要求：

1. 上电时默认显示一向上的箭头“↑”，亮度最大。按 KEY_LOC1M2 时，显示的内容为“←”；按 KEY_LOC3M2 时显示的内容为“→”；按 KEY_LOC2M2 时显示的内容为“↓”；按 KEY_LOC0M2 时显示的内容是“↑”。按 KEY_L1COM2 时，显示的内容闪烁 3 次（亮 0.5S，灭 0.5S）。
2. 按下 KEY_L1C1M2 时显示亮度增加，按下 KEY_L1C2M2 时显示的亮度减少，按下 KEY_L1C3M2 时，LED 点阵在亮或灭之间切换。

（2）连线关系

实验中连接关系如下：

杜邦线连线关系				
线序号	线端 A 所在位置		线端 B 所在位置	
	开发板	连接端子 1	开发板	连接端子 2
S1	LCD	J2M7: DS	LCD	JK2M7: A8
S2	LCD	J2M7: SH	LCD	JK2M7: A9
S3	LCD	J2M7: ST	LCD	JK2M7: A10
S4	LCD	J2M7: OE	LCD	JK2M7: A11
S5	LCD	J2M7: A	LCD	JK2M7: A12
S6	LCD	J2M7: B	LCD	JK2M7: A13
S7	LCD	J2M7: C	LCD	JK2M7: A14
S8	LCD	J2M7: D	LCD	JK2M7: A15
S9	KEY&LED	J2M2: /KEY	KEY&LED	JK2M2: INT0
S10	KEY&LED	J2M2: DIO	KEY&LED	JK2M2: P1.2
S11	KEY&LED	J2M2: CLK	KEY&LED	JK2M2: P1.1
S12	KEY&LED	J2M2: /CS	KEY&LED	JK2M2: P1.0
S13-S16	KEY&LED	J6M2: SG-SD	KEY&LED	J4M2: Row_3-Row_0
S17-S20	KEY&LED	J3M2: DIG0-DIG3	KEY&LED	J4M2: Col_3-Col_0

(3) 程序流程图

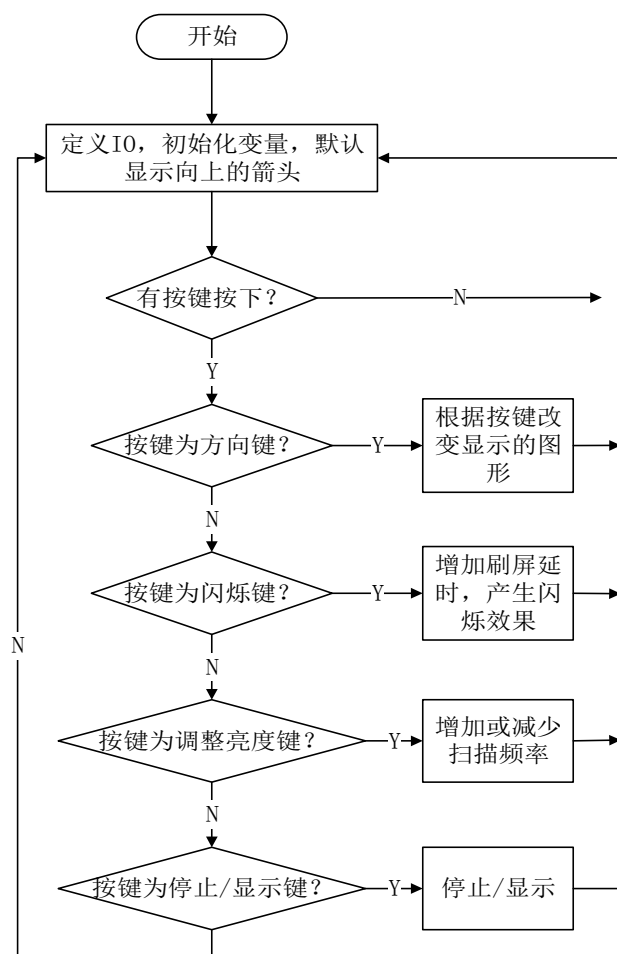
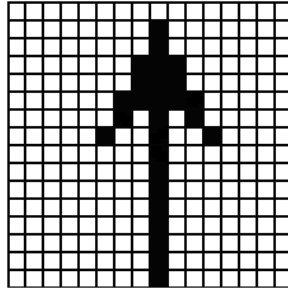


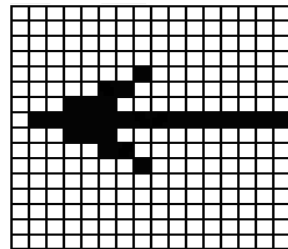
图 4.9.3 16*16 LED 点阵子板实验流程图

(4) 编程思路

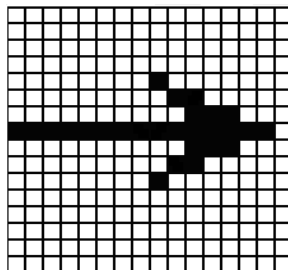
箭头“↑”，“↓”，“←”，“→”的16*16点阵编码（从左到右低位在前，从上到下的编码）如下：



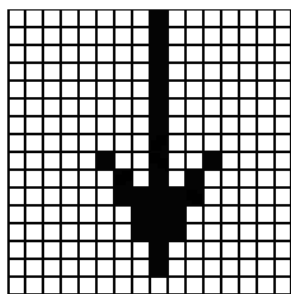
```
{0x00, 0x00, 0x80, 0x00, 0x80, 0x00, 0xC0, 0x01, 0xC0, 0x01, 0xE0, 0x03, 0xA0, 0x02, 0x90, 0x04};
{0x80, 0x00, 0x80, 0x00, 0x80, 0x00, 0x80, 0x00, 0x80, 0x00, 0x80, 0x00, 0x80, 0x00, 0x80, 0x00}; // "↑"
```



```
{0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x01, 0x00, 0x06, 0x00, 0x1C, 0xFF, 0x7F};
{0x00, 0x1C, 0x00, 0x06, 0x00, 0x01, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00}; // "←"
```

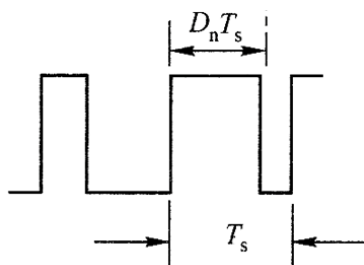


```
{0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x80, 0x00, 0x60, 0x00, 0x38, 0x00, 0xFE, 0xFF};
{0x38, 0x00, 0x60, 0x00, 0x80, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00}; // "→"
```



```
{0x80, 0x00, 0x80, 0x00, 0x80, 0x00, 0x80, 0x00, 0x80, 0x00, 0x80, 0x00, 0x80, 0x00, 0x80, 0x00};
{0x90, 0x04, 0xA0, 0x02, 0xE0, 0x03, 0xC0, 0x01, 0xC0, 0x01, 0x80, 0x00, 0x80, 0x00, 0x00, 0x00}; // "↓"
```

亮度调节使用 PWM 技术调节占空比来实现。



T_s 为某个 LED 的最大点亮时间，高电平持续期为 $D_n T_s$ ，表示该 LED 的实际点亮时间为 $D_n T_s$ 。 D_n 叫作占空比（这里分成 5 个等级级：20%，40%，60%，80%，100%）。剩余的 $(1-D_n) T_s$ 期间该 LED 是灭的。这样从平均的角度来看，当 $D_n=20\%$ 时，该发光二极管的亮度最低，当 $D_n=100\%$ 时该发光二极管的亮度最高。 T_s 实际上一行的显示时间，为了防止出现频闪现象， T_s 不宜选的太大。这里选用 1ms 即可。

(5) 实验步骤

1. 关掉实验箱电源。将 MCU 板，LCD 板，16*16LED 点阵子板插接在 LCD 板的 J2M7 上，注意插接方向。按照前面连接关系表将硬件连接好。
2. 在仿真器断电情况下将仿真器的仿真头插在 MCU 板的 MCU 插座上。将仿真器与开发 PC 机的通信口连接好，打开实验箱及仿真器的电源。
3. 运行 Keil uVision2 开发环境，建立工程 dianzhen_c.uv2，CPU 为 AT89S51，包含启动文件 STARTUP.A51。
4. 按照实验功能要求创建源程序 dianzhen.c 并加入到工程 dianzhen_c.uv2，并设置工程 dianzhen_c.uv2 属性，将其晶振频率设置为 11.0592MHz，选择输出可执行文

- 构造 (Build) 工程 dianzhen_c.uv2。如果编程有误进行修改，直至构造正确为止。
- 运行程序，观察结果是否符合程序要求，若不符合，分析出错原因，继续重复第 4、5 步的步骤，直至结果正确。

- 总结利用 16*16LED 点阵的显示控制实现方法。
- 尝试利用汇编语言编程实现程序中的相同功能。
- 尝试利用 16*16LED 点阵显示英文字母和中文汉字并进行滚动显示，英文字母和中文汉字使用字模提取软件提取其显示码表。

(1) 实验内容

243

(2) 实验原理

ALE 为地址锁存允许输入线，高电平有效。当 ALE 线为高电平时，地址锁存与译码器将 A, B, C 三条地址线的地址信号进行锁存，经译码后被选中的通道的模拟量进入转换器进行转换。A, B 和 C 为地址输入线，用于选通 IN0—IN7 上的一路模拟量输入。

ST 为转换启动信号。当 ST 上跳沿时，所有内部寄存器清零；下跳沿时，开始进行 A/D 转换；在转换期间，ST 应保持低电平。EOC 为转换结束信号。当 EOC 为高电平时，表明转换结束；否则，表明正在进行 A/D 转换。OE 为输出允许信号，用于控制三条输出锁存器向单片机输出转换得到的数据。OE=1，输出转换得到的数据；OE=0，输出数据线呈高阻状态。D7—D0 为数字量输出线。

CLK 为时钟输入信号线。因 ADC0809 的内部没有时钟电路，所需时钟信号必须由外界提供。

初始化时，使 ST 和 OE 信号全为低电平。

送要转换的哪一通道的地址到 A, B, C 端口上。

在 ST 端给出一个至少有 100ns 宽的正脉冲信号。

是否转换完毕，我们根据 EOC 信号来判断。

当 EOC 变为高电平时，这时给 OE 为高电平，转换的数据就输出给单片机了。

(3) 连线关系

接插件连接关系如下：

杜邦线连线关系				
线序号	连接端子 1		连接端子 2	
	开发板	接口 1	开发板	接口 1
S1	ADDA	P1M1: EOC	ADDA	JK2M1: INT0
S2	ADDA	J2M1: ADD_A	ADDA	JK2M1: A0
S3	ADDA	J2M1: ADD_B	ADDA	JK2M1: A1
S4	ADDA	J2M1: ADD_C	ADDA	JK2M1: A2
S5	ADDA	J1M1: 0	ADDA	P2M1: 0~+5V
S6	ADDA	J1M1: 1	ADDA	P2M1: 5V
S7	ADDA	J1M1: 2	ADDA	P2M1: 0V
S8	PI0	P3M3: D[0-7]	LCD	J4M7: D[0-7]
S9	LCD	J6M7: E	LCD	JK2M7: P1. 7
S10	LCD	J6M7: RW	LCD	JK2M7: P1. 6
S11	LCD	J6M7: RS	LCD	JK2M7: P1. 5
S12	PI0	J2M3: 用短路帽短接		
S13	ADDA	JP1M1: 用短路帽短接		

(4) 程序流程图

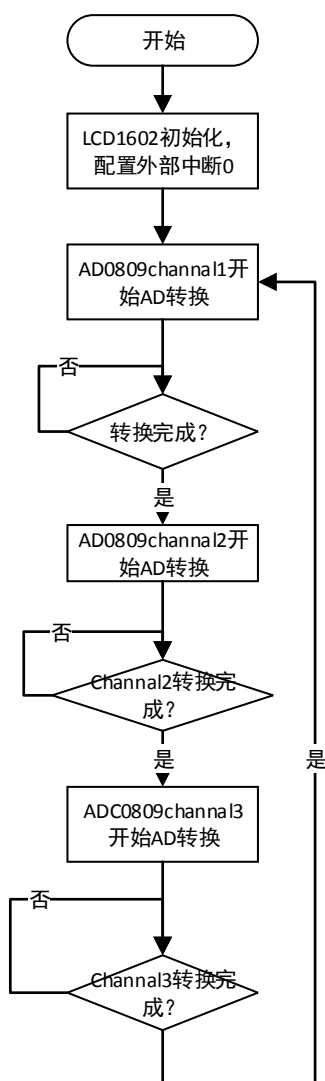


图 4. 10. 2 ADC0809 并行接口 AD 转换实验流程图

(5) 编程思路

ADC0809 的地址由 P2. 7 (A15) 来决定, P2. 7 为低电平时, 选通 ADC0809, 对 ADC0809 的写操作将启动其 AD 转换过程, 转换结束后, EOC 将输出低电平, 引起 CPU 的外部中断 0 中断, 在中断服务程序中对 ADC0809 进行读操作即可读到转换结果。输入通道 0 输入的是可变模拟量, 通过调节可调电阻 VR2 可以改变其大小, 变换的范围为 0-5V。输入通道 1 输入的是 5V 信号, 输入通道 2 输入的是 0V 信号。将这三个通道的采集结果显示在 1602 显示器上。1602 的数据信号是通过 373 锁存提供的, 该 373 的地址由 P2. 6 决定。

(6) 实验步骤

1. 关掉实验箱电源。将 MCU 板, PIO 板, LCD 板, ADDA 板插接母版上, 注意 MCU 板的插接方向。将 LCD1602 子板插接在 LCD 板上, 将注意插接方向。按照前面连接关系表将硬件连接好。
2. 在仿真器断电情况下将仿真器的仿真头插在 MCU 板的 CPU 插座上。将仿真器与开发 PC 机的通信口连接好, 打开实验箱及仿真器的电源。
3. 运行 Keil uVision2 开发环境, 按照 1.2.3 节介绍的方法建立工程 ADC0809_c.uv2, CPU 为 AT89S51, 包含启动文件 STARTUP.A51。
4. 按照 1.2.3 节介绍的方法及实验功能要求创建源程序 ADC0809.c 并加入到工程 ADC0809_c.uv2, 并设置工程 ADC0809_c.uv2 属性, 将其晶振频率设置为 11.0592MHz, 选择输出可执行文件, DEBUG 方式选择硬件 DEBUG, 并选择其中的“WAVE V series MCS51 Driver”仿真器。
5. 构造 (Build) 工程 ADC0809_c.uv2。如果编程有误进行修改, 直至构造正确为止。
6. 运行程序, 观察结果是否符合程序要求, 若不符合, 分析出错原因, 继续重复第 4、5 步的步骤, 直至结果正确。

(7) 实验作业

- 调节 ADDA 板上 VR2, 使 IN0 通道上的输入信号电平发生改变, 利用数字万用表测量其电压值, 并将其与 LCD 上显示的采样值进行对比。
- 分析 AD 过程产生误差的原因。

4.12 实验 12: DAC0832 并行接口 DA 转换实验

(1) 实验内容

实验原理图如下:

由上式可见，输出的模拟量与输入的数字量（ $D_{n-1} \cdot 2^{n-1} + A + D_0 \cdot 2^0$ ）成正比，这就实现了从数字量到模拟量的转换。

(3) 连线关系

实验中连接关系如下：

杜邦线连线关系				
线序号	连接端子 1		连接端子 2	
	开发板	接口 1	开发板	接口 1
S1	MCU	P2M0: KEY0	MCU	JK2M0: INT0
S2	MCU	P2M1: KEY1	MCU	JK2M0: INT1
S3	ADDA	J8M1: CS	ADDA	JK2M1: A15
S4	PIO	P3M3: D[0-7]	LCD	J4M7: D[0-7]
S5	LCD	J6M7: E	LCD	JK2M7: P1.2
S6	LCD	J6M7: RW	LCD	JK2M7: P1.1
S7	LCD	J6M7: RS	LCD	JK2M7: P1.0
	PIO	J2M3:用短路帽短接		
	ADDA	JP3M1:用短路帽短接		
	ADDA	J6M1: OUT	示波器探头	
	ADDA	J6M1: GND	示波器探头接地	

(4) 程序流程图

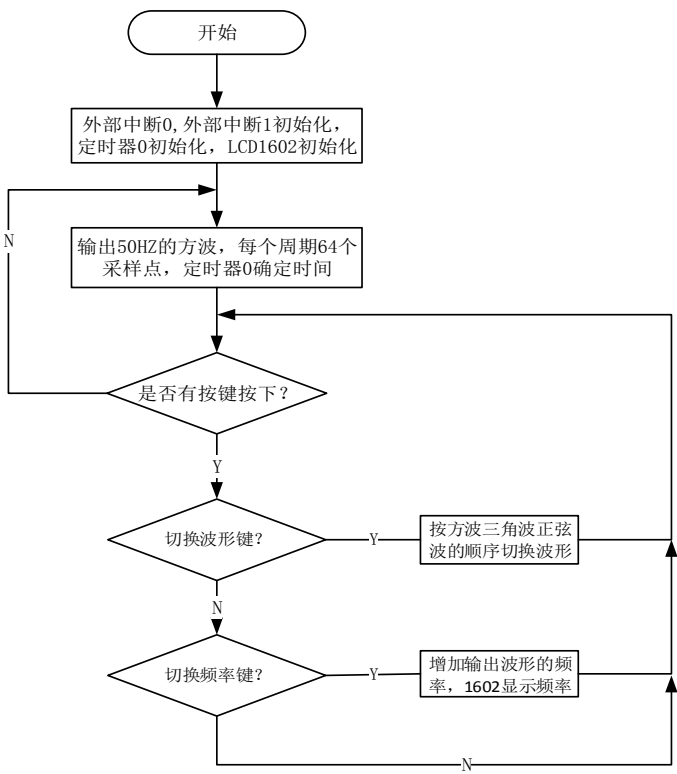
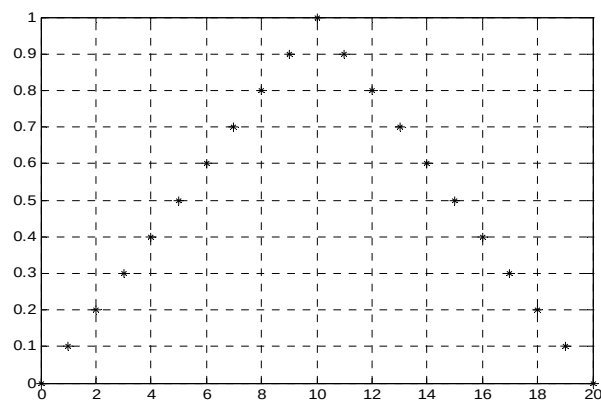


图 4. 12. 3 DAC0832 并行接口 DA 转换实验流程图

(5) 编程思路

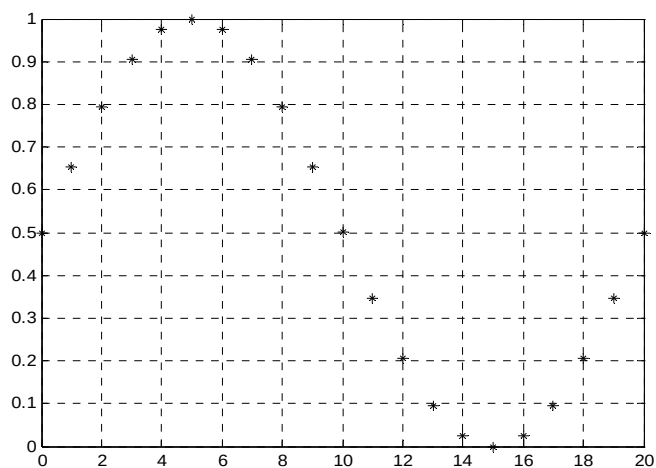
方波的占空比为 50%，即一半周期内为高电平（0xff），一半周期内为低电平（0）。

三角波的函数曲线每个周期分成两部分：上升阶段和下降阶段。上升阶段从 0 变到 0xFF，下降阶段从 0xFF 变到 0。每个阶段可以输出 10 个点。三角波一个周期内的信号变化如下图所示：



图中为规一化的信号幅值，1 对应 5V，0.1 对应 0.5V，依次类推。实际送给 DAC0832 的值 1 对应 0xff。

正弦波一个周期内的信号变化如下图所示：

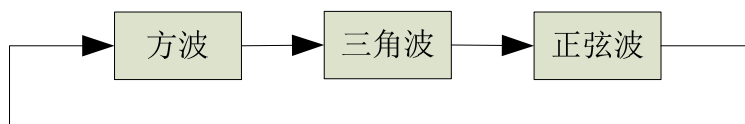


这里规一化正弦函数值为：

{0.5000, 0.6545, 0.7939, 0.9045, 0.9755, 1.0000, 0.9755, 0.9045, 0.7939, 0.6545,
0.5000, 0.3455, 0.2061, 0.0955, 0.0245, 0.0000, 0.0245, 0.0955, 0.2061, 0.3455, 0.5000}

一个周期的时间长度根据选择的信号频率计算得出。选用恰当的定时方式，以便产生较为精确的定时时长。

模式切换按照：



开机默认情况下为方波。

(6) 实验步骤

1. 关掉实验箱电源。将 MCU 板，PIO 板，ADDA 板，LCD 板插接在母板上。将 LCD1602 子板插接在 LCD 板上，将注意插接方向。按照前面连接关系表将硬件连接好。
2. 在仿真器断电情况下将仿真器的仿真头插在 MCU 板的 CPU 插座上。将仿真器与开发 PC 机的通信口连接好，打开实验箱及仿真器的电源。
3. 运行 Keil uVision2 开发环境，按照 1.2.3 节介绍的方法建立工程 ADC0832_c.uv2，CPU 为 AT89S51，包含启动文件 STARTUP.A51。
4. 按照 1.2.3 节介绍的方法及实验功能要求创建源程序 ADC0832.c 并加入到工程 ADC0832_c.uv2，并设置工程 ADC0832_c.uv2 属性，将其晶振频率设置为 11.0592MHz，选择输出可执行文件，DEBUG 方式选择硬件 DEBUG，并选择其中的“WAVE V series MCS51 Driver”仿真器。
5. 构造 (Build) 工程 ADC0832_c.uv2。如果编程有误进行修改，直至构造正确为止。
6. 运行程序，观察结果是否符合程序要求，若不符合，分析出错原因，继续重复第 4、5 步的步骤，直至结果正确。

(7) 实验作业

- 总结 DAC0832 的数据读写方法。
- 尝试使用汇编语言实现。

4.13 实验 13: DS12887 并行接口 RTC 实验

(1) 实验内容

实验原理图如下:

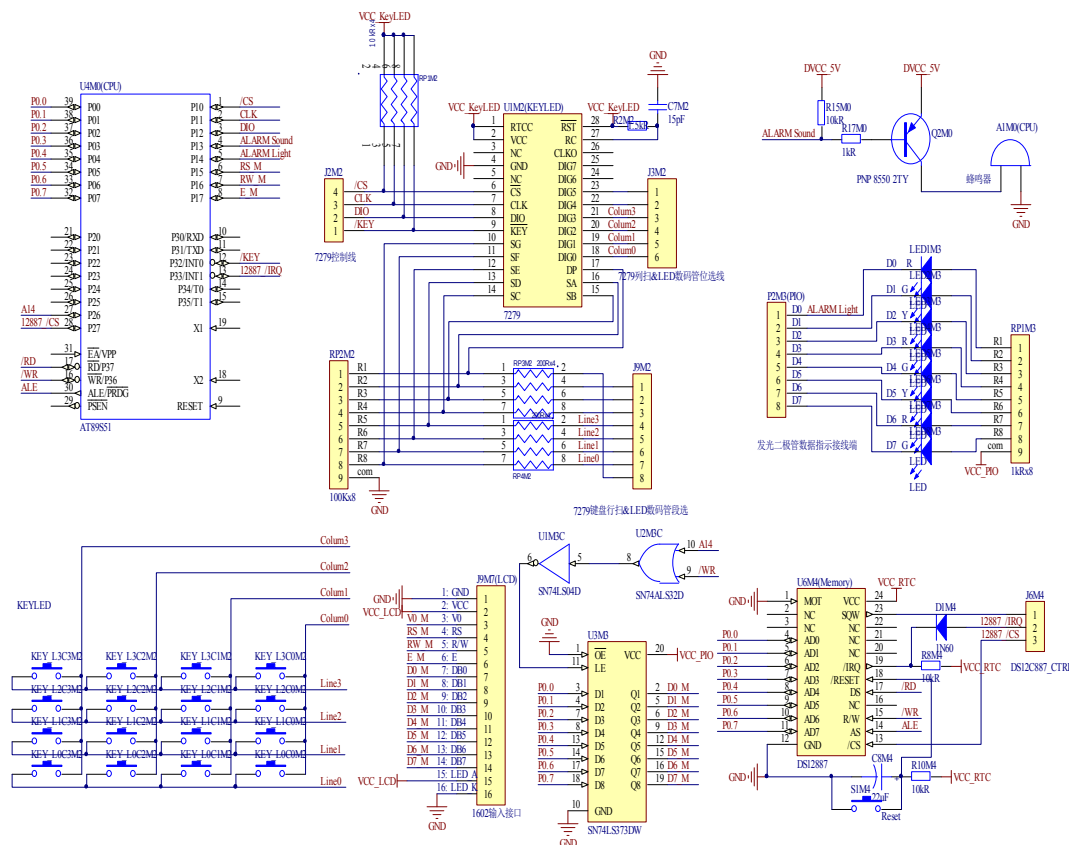


图 4.13.1 DS12887 并行接口 RTC 实验原理图

本实验利用 7279 进行键盘扫描, 利用 LCD1602 的显示, 利用 12887 实现实时时钟, 从而实现简单的电子日历功能。

(2) 实验原理

DS12887 内部 RAM 和各专用寄存器地址

地址单元	用途	地址单元	用途
地址 00H	秒	地址 01H	秒闹
地址 02H	分	地址 03H	分闹
地址 04H	时	地址 05H	时闹
地址 06H	星期	地址 07H	日（两位数）
地址 08H	月（两位数）	地址 09H	年（两位数）
地址 0AH	寄存器 A	地址 0BH	寄存器 B
地址 0CH	寄存器 C	地址 0CH	寄存器 D
0EH~7EH	不掉电 RAM 区，共 114 字节		

寄存器 A 各位不受复位的影响，UIP 位为只读位，其它各位均可读写，寄存器的控制字的格式如下表所列。

BIT7	BIT6	BIT5	BIT4	BIT3	BIT2	BIT1	BIT0
UIP	DV2	DV1	DV0	RS3	RS2	RS1	RS0

1. UIP 位：更新周期标志位。该位为“1”时，表示芯片正处于或将开始更新周期，此时程序不准读写时标寄存器；该位为“0”时，表示至少在 244 μ s 后才开始更新周期，此时程序可读芯片内时标寄存器。该位是只读位。
2. DV0、DV1、DV2：芯片内部振荡器 RTC 控制位。当芯片解除复位状态，并将 010 写入 DV0、DV1、DV2 后，另一个更新周期将在 500ms 后开始。因此，在程序初始化时可用这三位精确地使芯片在设定的时间开始工作。这与 MC146818 不同的是，DS12887 固定使用 32 768Hz 的内部晶体，所以，DV0=“0”，DV1=“1”，DV2=“0”，即只有一种 010 的组合选择即可启动 RTC。
3. RS3、RS2、RS1、RS0：周期中断可编程方波输出速率选择位。各种不同的组合可以产生不同的输出。程序可以通过设置寄存器 B 的 SQWF 和 PIE 位控制是否允许周期中断方波输出。

寄存器 B 允许读写，主要用于控制芯片的工作状态。寄存器 B 的控制字的格式如下表所列。

BIT7	BIT6	BIT5	BIT4	BIT3	BIT2	BIT1	BIT0
SET	PIE	AIE	UIE	SQWE	DM	24/12	DSF

1. SET 位：当该位为“0”时，芯片处于正常工作状态，每秒产生一个更新周期来更新时标寄存器。为“1”时，芯片停止工作，程序在此期间可初始化芯片的各个时标寄存器。
2. PIE、AIE、UIE 位：分别为周期中断、报警中断、更新周期结束中断允许位。各位为“1”时，允许芯片发相应的中断。
3. SQWE 位：方波输出允许位。SQWE=“1”，按寄存器 A 输出速率选择位所确定的频率输出方波；SQWE=“0”，脚 SQW 保持低电平。
4. DM 位：时标寄存器用于十进制 BCD 码表示或用二进制表示格式选择位。DM=“0”时，为十进制 BCD 码；DM=“1”时，为二进制码。
5. 24/12 位：24/12 小时模式设置位。24/12 位=“1”时，为 24 小时工作模式；24/12=“0”时，为 12 小时工作模式。
6. DSE 位：夏令时服务位。DSE=“1”时，夏令时设置有效，夏令时结束可自动刷新恢复时间；DSE=“0”，无效。

(3) 连线关系

实验中连接关系如下：

杜邦线连线关系				
线序号	线端 A 插接位置		线端 B 插接位置	
	开发板	端子	开发板	端子
S1	KEY&LED	JK2M2: P1.0	KEY&LED	J2M2: CS
S2	KEY&LED	JK2M2: P1.1	KEY&LED	J2M2: CLK
S3	KEY&LED	JK2M2: P1.2	KEY&LED	J2M2: DIO
S4	KEY&LED	JK2M2: P3.2	KEY&LED	J2M2: KEY
S5-S8	KEY&LED	J5M2: DIG_0...3	KEY&LED	J4M2: Col_3...Col_0 (注意顺序)
S9-S12	KEY&LED	J6M2: SG...SD	KEY&LED	J1M2: Row_3...Row_0 (注意顺序)
S13	MCU	P2M0: BELL	MCU	JK2M0: P1.3
S14	LCD	JK2M7: P1.5	LCD	J6M7: RS
S15	LCD	JK2M7: P1.6	LCD	J6M7: RW
S16	LCD	JK2M7: P1.7	LCD	J6M7: E
S17	PIO	P2M3: D0	PIO	JK2M3: P1.4
S18	PIO	J2M3: 用短路帽短接		
S19-S26	PIO	P3M3: D[0...7]	LCD	J4M7: D[0...7]
S27	Memory&RTC	JK2M4: INT1	Memory&RTC	J3M4: /IRQ
S28	Memory&RTC	JK2M4: A15	Memory&RTC	J3M4: /CS

(4) 实验流程图

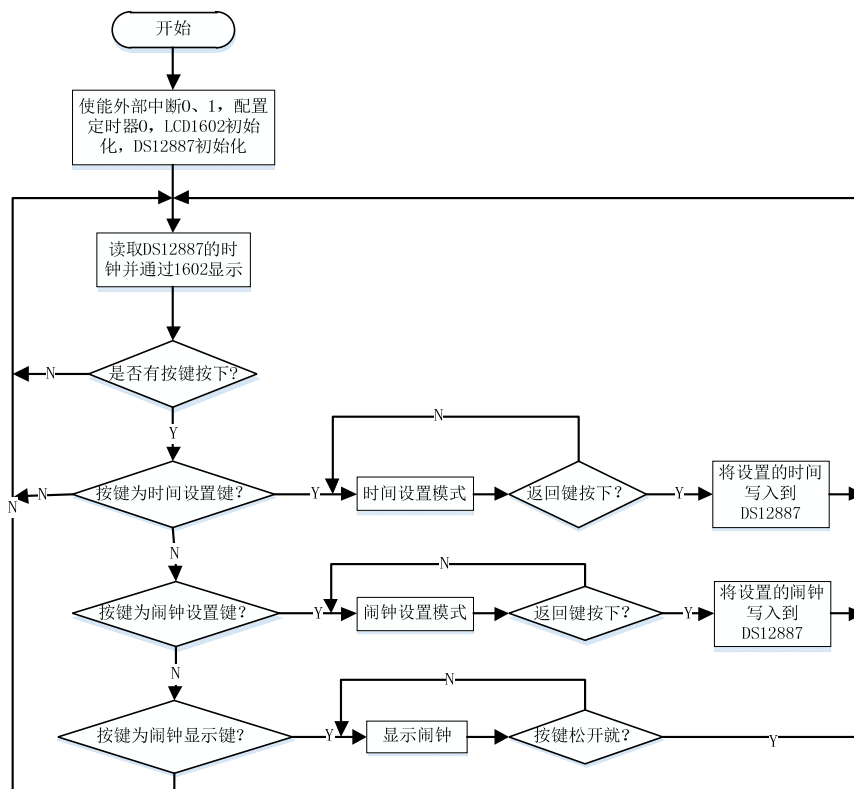


图 4.13.2 DS12887 并行接口 RTC 实验流程图

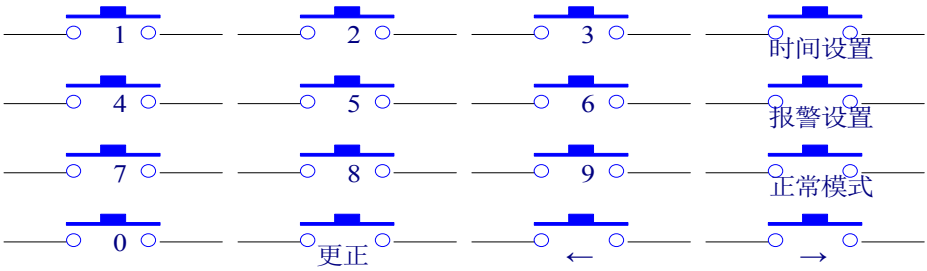
(5) 编程思路

要求查看 DS12887 的数据手册，能写相应的内存字节设置时间、日历与闹钟，访问相应的内存字节来获取时间和日历信息并使用 LCD 模块进行实时时钟输出。

4*4 按键的编码及其代表的功能如下表所示：

按键名称	按键编码	代表的功能
DKEY1M2	00H	数字 1
DKEY2M2	01H	数字 2
DKEY3M2	02H	数字 3
DKEY4M2	03H	时间设置键（闹钟设置模式下为闹钟开关键）
DKEY5M2	04H	数字 4
DKEY6M2	05H	数字 5
DKEY7M2	06H	数字 6
DKEY8M2	07H	闹钟设置键
DKEY9M2	08H	数字 7
DKEY10M2	09H	数字 8
DKEY11M2	0AH	数字 9
DKEY12M2	0BH	设置返回键（正常模式下显示闹钟时间）
DKEY13M2	0CH	数字 0
DKEY14M2	0DH	更正键（按此键后才能更正光标所在位）
DKEY15M2	0EH	光标左移
DKEY16M2	0FH	光标右移

编码后按键功能如下图所示：



其中，时间设置用于将 RTC 工作与时间校准模式，这时通过按前进按键 “→” 或后退按键 “←”，更换设置内容：年 “Year”，月 “Month”，日 “DATE”，星期 “DAY”，时

“H”，分“M”，秒“S”。注意输入各项参数时要有输错提示功能，如月参数输入0和13以上肯定是错的，要有提示。同样Date参数输入31以上也是错的。

报警设置雷同时间设置，只不过这里设置的报警时间而不是系统时间。

正常模式是从时间设置模式或者报警设置模式回到正常模式。

更正按键的作用就是更改当前的数值。

前进按键“→”或后退按键“←”仅仅用于更改设置项目。“→”按照Year→Month→DATE→DAY→H→M→S的顺序更改设置项目，而“←”按照相反的顺序更改设置项目。

报警时间到达时，按照1Hz的频率使声光报警器闪烁（亮0.5s，灭0.5s），闪烁一分钟后自动停止。

（6）实验步骤

1. 关掉实验箱电源。将MCU板，Memory&RTC板，KEYLED板，LCD板，PIO板插接在母板上。将LCD1602子板插接在LCD板的上，将注意插接方向。按照前面连接关系表将硬件连接好。
2. 在仿真器断电情况下将仿真器的仿真头插在MCU板的CPU插座上。将仿真器与开发PC机的通信口连接好，打开实验箱及仿真器的电源。
3. 运行Keil uVision2开发环境，建立工程DS12887_c.uv2，CPU为AT89S51，包含启动文件STARTUP.A51。
4. 按照实验功能要求创建源程序DS12887.c并加入到工程DS12887_c.uv2，并设置工程DS12887_c.uv2属性，将其晶振频率设置为11.0592MHz，选择输出可执行文件，DEBUG方式选择硬件DEBUG，并选择其中的“WAVE V series MCS51 Driver”仿真器。
5. 构造（Build）工程DS12887_c.uv2。如果编程有误进行修改，直至构造正确为止。
6. 运行程序，观察结果是否符合程序要求，若不符合，分析出错原因，继续重复第4、5步的步骤，直至结果正确。

（7）实验作业

- #### 4.14 实验 14: I2C 串行 E2PROM 24C02 读写实验

(1) 实验内容

The schematic diagram illustrates the static LED display control circuit. At the top, power supply connections for +5V and GND are shown. The circuit consists of several integrated circuits:

- U3M4**: A 74HC164 shift register used for address decoding.
- U3M2**: A 74HC164 counter used for data output.
- U4M2**: A 74HC164 decoder used for segment selection.
- U4M1**: A 74HC164 decoder used for segment selection.
- U7M2**: A 74HC164 decoder used for key input.
- U7M1**: A 74HC164 decoder used for key input.
- U8M2**: A 74HC164 decoder used for status LEDs.
- U8M1**: A 74HC164 decoder used for status LEDs.
- U9M2**: A 74HC164 decoder used for status LEDs.
- U9M1**: A 74HC164 decoder used for status LEDs.

The circuit also includes various passive components such as resistors (R12M0, R12M1, R12M2, R12M3, R12M4, R12M5, R12M6, R12M7, R12M8, R12M9, R12M10, R12M11, R12M12, R12M13, R12M14, R12M15, R12M16, R12M17, R12M18, R12M19, R12M20, R12M21, R12M22, R12M23, R12M24, R12M25, R12M26, R12M27, R12M28, R12M29, R12M30, R12M31, R12M32, R12M33, R12M34, R12M35, R12M36, R12M37, R12M38, R12M39, R12M40, R12M41, R12M42, R12M43, R12M44, R12M45, R12M46, R12M47, R12M48, R12M49, R12M50, R12M51, R12M52, R12M53, R12M54, R12M55, R12M56, R12M57, R12M58, R12M59, R12M60, R12M61, R12M62, R12M63, R12M64, R12M65, R12M66, R12M67, R12M68, R12M69, R12M70, R12M71, R12M72, R12M73, R12M74, R12M75, R12M76, R12M77, R12M78, R12M79, R12M80, R12M81, R12M82, R12M83, R12M84, R12M85, R12M86, R12M87, R12M88, R12M89, R12M90, R12M91, R12M92, R12M93, R12M94, R12M95, R12M96, R12M97, R12M98, R12M99, R12M100, R12M101, R12M102, R12M103, R12M104, R12M105, R12M106, R12M107, R12M108, R12M109, R12M110, R12M111, R12M112, R12M113, R12M114, R12M115, R12M116, R12M117, R12M118, R12M119, R12M120, R12M121, R12M122, R12M123, R12M124, R12M125, R12M126, R12M127, R12M128, R12M129, R12M130, R12M131, R12M132, R12M133, R12M134, R12M135, R12M136, R12M137, R12M138, R12M139, R12M140, R12M141, R12M142, R12M143, R12M144, R12M145, R12M146, R12M147, R12M148, R12M149, R12M150, R12M151, R12M152, R12M153, R12M154, R12M155, R12M156, R12M157, R12M158, R12M159, R12M160, R12M161, R12M162, R12M163, R12M164, R12M165, R12M166, R12M167, R12M168, R12M169, R12M170, R12M171, R12M172, R12M173, R12M174, R12M175, R12M176, R12M177, R12M178, R12M179, R12M180, R12M181, R12M182, R12M183, R12M184, R12M185, R12M186, R12M187, R12M188, R12M189, R12M190, R12M191, R12M192, R12M193, R12M194, R12M195, R12M196, R12M197, R12M198, R12M199, R12M200, R12M201, R12M202, R12M203, R12M204, R12M205, R12M206, R12M207, R12M208, R12M209, R12M210, R12M211, R12M212, R12M213, R12M214, R12M215, R12M216, R12M217, R12M218, R12M219, R12M220, R12M221, R12M222, R12M223, R12M224, R12M225, R12M226, R12M227, R12M228, R12M229, R12M230, R12M231, R12M232, R12M233, R12M234, R12M235, R12M236, R12M237, R12M238, R12M239, R12M240, R12M241, R12M242, R12M243, R12M244, R12M245, R12M246, R12M247, R12M248, R12M249, R12M250, R12M251, R12M252, R12M253, R12M254, R12M255, R12M256, R12M257, R12M258, R12M259, R12M260, R12M261, R12M262, R12M263, R12M264, R12M265, R12M266, R12M267, R12M268, R12M269, R12M270, R12M271, R12M272, R12M273, R12M274, R12M275, R12M276, R12M277, R12M278, R12M279, R12M280, R12M281, R12M282, R12M283, R12M284, R12M285, R12M286, R12M287, R12M288, R12M289, R12M290, R12M291, R12M292, R12M293, R12M294, R12M295, R12M296, R12M297, R12M298, R12M299, R12M300, R12M301, R12M302, R12M303, R12M304, R12M305, R12M306, R12M307, R12M308, R12M309, R12M310, R12M311, R12M312, R12M313, R12M314, R12M315, R12M316, R12M317, R12M318, R12M319, R12M320, R12M321, R12M322, R12M323, R12M324, R12M325, R12M326, R12M327, R12M328, R12M329, R12M330, R12M331, R12M332, R12M333, R12M334, R12M335, R12M336, R12M337, R12M338, R12M339, R12M340, R12M341, R12M342, R12M343, R12M344, R12M345, R12M346, R12M347, R12M348, R12M349, R12M350, R12M351, R12M352, R12M353, R12M354, R12M355, R12M356, R12M357, R12M358, R12M359, R12M360, R12M361, R12M362, R12M363, R12M364, R12M365, R12M366, R12M367, R12M368, R12M369, R12M370, R12M371, R12M372, R12M373, R12M374, R12M375, R12M376, R12M377, R12M378, R12M379, R12M380, R12M381, R12M382, R12M383, R12M384, R12M385, R12M386, R12M387, R12M388, R12M389, R12M390, R12M391, R12M392, R12M393, R12M394, R12M395, R12M396, R12M397, R12M398, R12M399, R12M400, R12M401, R12M402, R12M403, R12M404, R12M405, R12M406, R12M407, R12M408, R12M409, R12M410, R12M411, R12M412, R12M413, R12M414, R12M415, R12M416, R12M417, R12M418, R12M419, R12M420, R12M421, R12M422, R12M423, R12M424, R12M425, R12M426, R12M427, R12M428, R12M429, R12M430, R12M431, R12M432, R12M433, R12M434, R12M435, R12M436, R12M437, R12M438, R12M439, R12M440, R12M441, R12M442, R12M443, R12M444, R12M445, R12M446, R12M447, R12M448, R12M449, R12M450, R12M451, R12M452, R12M453, R12M454, R12M455, R12M456, R12M457, R12M458, R12M459, R12M460, R12M461, R12M46

图 4.14.1 I2C 串行 E2PROM 24C02 读写实验原理图

本实验进行 I2C 接口的 24C02 读写验证。

24C02 储存 1 个单字节 BCD 码数据, 使用静态 LED 数码管显示该 BCD 码。

256

态也要存入 24C02，默认情况下是停止，增加，并且初始数据为 00。每次上电之后都要自动读取 24C02 中所储存的状态数据。数据增加或减少都是按照 BCD 码变化。

运行过程中，硬件复位 CPU，使其从头运行，数据变换的状态和速度应该和复位之前一致。

在运行过程中，关掉单机电源，重新上电，数据变换的状态和速度应该和关电之前一致。

(2) 实验原理

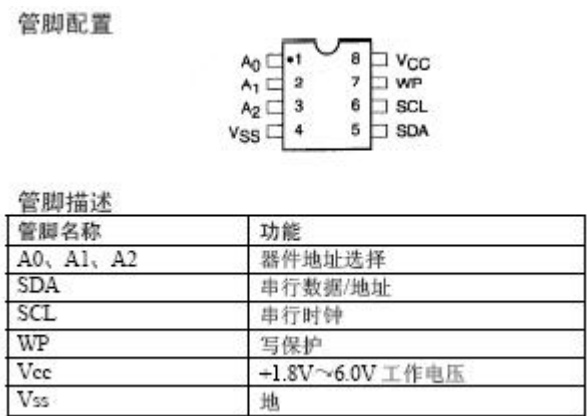


图 4.14.2 24C02 管脚配置和使用说明

24c02 是一个 2K 位串行 CMOS E2PROM，内部含有 256 个 8 位字节存储空间。器件通过 I2C 总线接口进行操作。有关 IIC 总线的使用方法请阅读附录部分 IIC 总线简介。

(3) 连线关系

实验中连接关系如下：

杜邦线连线关系				
线序号	线端 A 插接位置		线端 B 插接位置	
	开发板	端子	开发板	端子
S1	MCU	P2M0：KEY0	MCU	JK2M0：INT0
S2	MCU	P3M0：KEY1	MCU	JK2M0：INT1
S3	KEY&LED	J8M2：RXD	KEY&LED	JK2M2：RXD
S4	KEY&LED	J8M2：TXD	KEY&LED	JK2M2：TXD
S5	Memory&RTC	JK2M4：P1.0	Memory&RTC	J2M4：I2C_CLK
S6	Memory&RTC	JK2M4：P1.1	Memory&RTC	J2M4：I2C_DATA
S7	J7M2：使用短路帽短接			

(4) 程序流程图

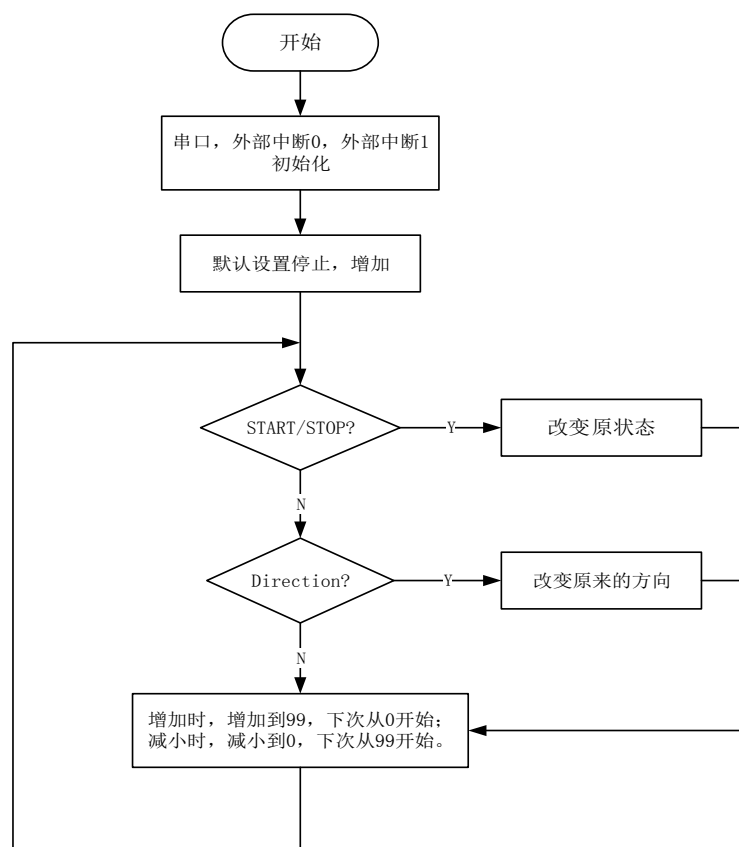


图 4.14.3 I2C 串行 E2PROM 24C02 读写实验流程图

(5) 编程思路

由于 51 单片机自身没有 I2C 电路，这里要采用 P1.0 和 P1.1 仿真来实现，P1.0 仿真 I2C_CLK，P1.1 仿真 I2C_DATA。24C02 的读写命令格式要参考其手册。I2C 的操作时序请参考附录 1，将 I2C 的操作函数要设计成标准函数，以便其他 I2C 器件实验时使用。

静态按键按键功能如下表所示：

按键名称	代表的功能
KEY1	Direction
KEY0	START/STOP

(6) 实验步骤

1. 关掉实验箱电源。将 MCU 板，Memory\$RTC 板，KEYLED 板插接在母板上。按照前面连接关系表将硬件连接好。

2. 在仿真器断电情况下将仿真器的仿真头插在 MCU 板的 CPU 插座上。将仿真器与开发 PC 机的通信口连接好，打开实验箱及仿真器的电源。
3. 运行 Keil uVision2 开发环境，建立工程 I2C24C02_c.uv2，CPU 为 AT89S51，包含启动文件 STARTUP.A51。
4. 按照实验功能要求创建源程序 I2C24C02.c 并加入到工程 I2C24C02_c.uv2，并设置工程 I2C24C02_c.uv2 属性，将其晶振频率设置为 11.0592MHz，选择输出可执行文件，DEBUG 方式选择硬件 DEBUG，并选择其中的“WAVE V series MCS51 Driver”仿真器。
5. 构造 (Build) 工程 I2C24C02_c.uv2。如果编程有误进行修改，直至构造正确为止。
6. 运行程序，观察结果是否符合程序要求，若不符合，分析出错原因，继续重复第 4、5 步的步骤，直至结果正确。

(7) 实验作业

- 总结 I2C 总线时序的单片机仿真实现方法
- 总结 24C02 内部的数据读写方法。
- 尝试使用汇编语言实现。

4.15 实验 15：I2C 接口芯片 PCF8574 扩展并口实验

(1) 实验内容

本实验利用单片机并行口仿真实现 I2C 总线接口，并利用 I2C 扩展并行 IO 器件 PCF8574 来实现并口扩展。其中高 4 位扩展为输出口，低 4 位扩展为输入口。输入口连接 DIP 开关实现的开关量生成电路，输出口连接开关量 LED 指示电路。当相应位 (IN_D0—OUT_D0, IN_D1—OUT_D1, IN_D2—OUT_D3, IN_D3—OUT_D3) 的 DIP 处于 ON 状态时，相应的输出位的发光二极管亮，反之，处于 OFF 状态时，相应的输出位的发光二极管灭。

该实验的电路原理图如下：

(3) 连接关系

实验中连接关系如下：

杜邦线连线关系				
线序号	线端 A 插接位置		线端 B 插接位置	
	开发板	端子	开发板	端子
S1	PI0	JK2M3: P1. 6	PI0	P14M3: SDA
S2	PI0	JK2M3: P1. 7	PI0	P14M3: SCL
S3	PI0	JK2M3: INT0	PI0	P14M3: /INT
S4-S11	PI0	P1M3:D[0...3]	PI0	P13M3: D[0...3]
S12-S19	PI0	P2M3:D[0...3]	PI0	P13M3: D[4...7]

(4) 编程思路

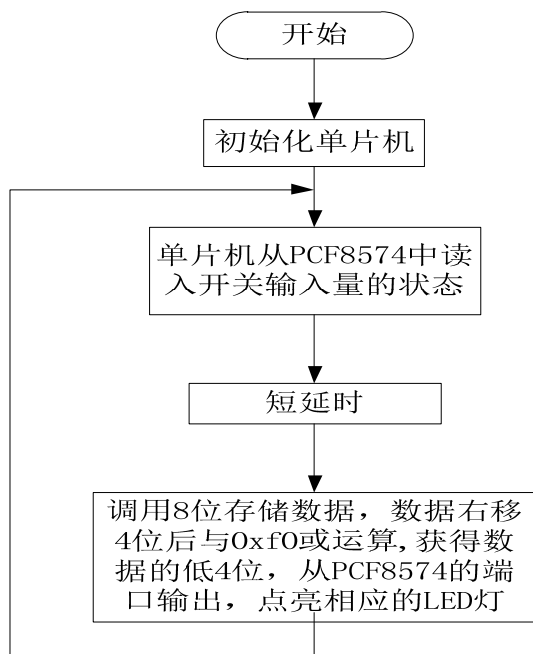


图 4.15.4 I2C 扩展 PCF8574 实验流程图

(5) 编程思路

由于 51 单片机自身没有 I2C 电路,这里要采用 P1.6 和 P1.7 模拟实现,P1.7 模拟 SCL, P1.6 模拟 SDA。PCF8574 的读写命令格式要参考其手册。I2C 的操作时序请参考附录 1。

(6) 实验步骤

1. 关掉实验箱电源。将 MCU 板，PIO 板插接在母板上，注意各板的插接方向。按照前面连接关系表将硬件连接好。
2. 在仿真器断电情况下将仿真器的仿真头插在 MCU 板的 CPU 插座上。将仿真器与开发 PC 机的通信口连接好，打开实验箱及仿真器的电源。
3. 运行 Keil uVision2 开发环境，建立工程 I2C8574_c.uv2，CPU 为 AT89S51，包含启动文件 STARTUP.A51。
4. 按照实验功能要求创建源程序 I2C8574.c 并加入到工程 I2C8574_c.uv2，并设置工程 I2C8574_c.uv2 属性，将其晶振频率设置为 11.0592MHz，选择输出可执行文件，DEBUG 方式选择硬件 DEBUG，并选择其中的“WAVE V series MCS51 Driver”仿真器。
5. 构造（Build）工程 I2C8574_c.uv2。如果编程有误进行修改，直至构造正确为止。
6. 运行程序，观察结果是否符合程序要求，若不符合，分析出错原因，继续重复第 4、5 步的步骤，直至结果正确。

（7）实验作业

- 总结 PCF8574 的数据读写方法。
- 尝试使用汇编语言实现。

4.16 实验 16：I2C 接口芯片 PCF8563 扩展 RTC 实验

（1）实验内容

实验原理图如下：

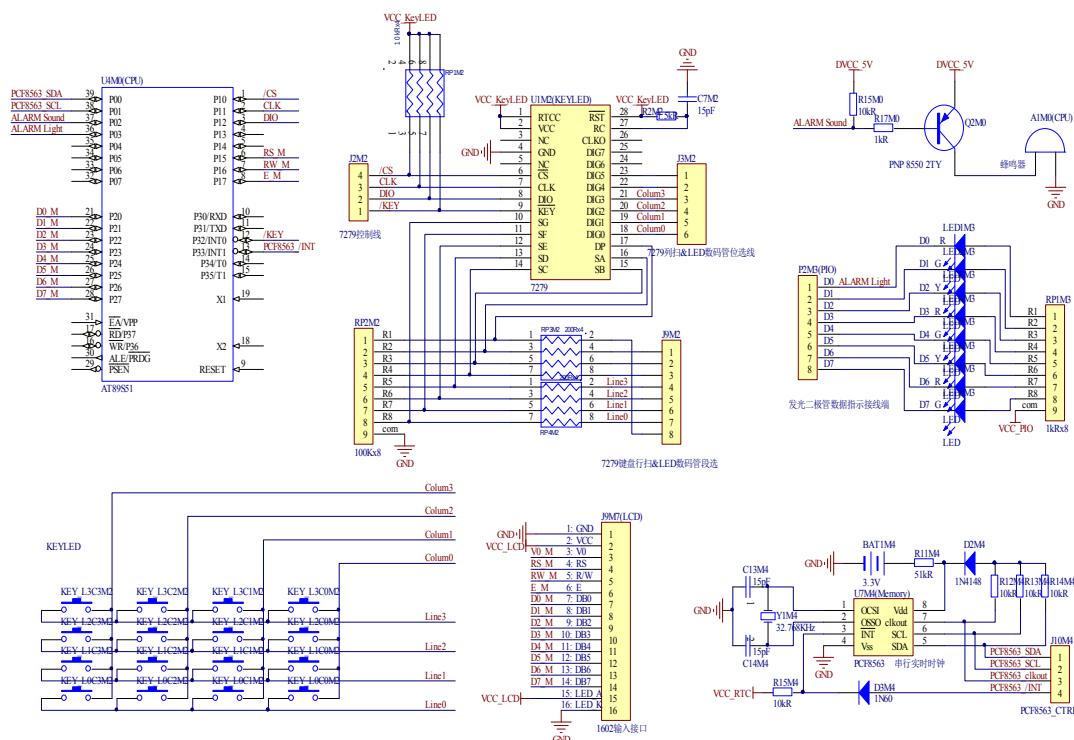


图 4.16.1 I2C 扩展 PCF8563 实验原理图

本实验利用 7279 进行键盘扫描，利用 LCD1602 的显示，利用 PCF8563 实现实时时钟，从而实现简单的电子日历功能。

(2) 实验原理

PCF8563 是 PHILIPS 公司推出的一款带 I2C 总线，具有极低功耗的多功能时钟/日历芯片，PCF8563 有多种报警功能、定时器功能、时钟输出功能以及中断输出功能。

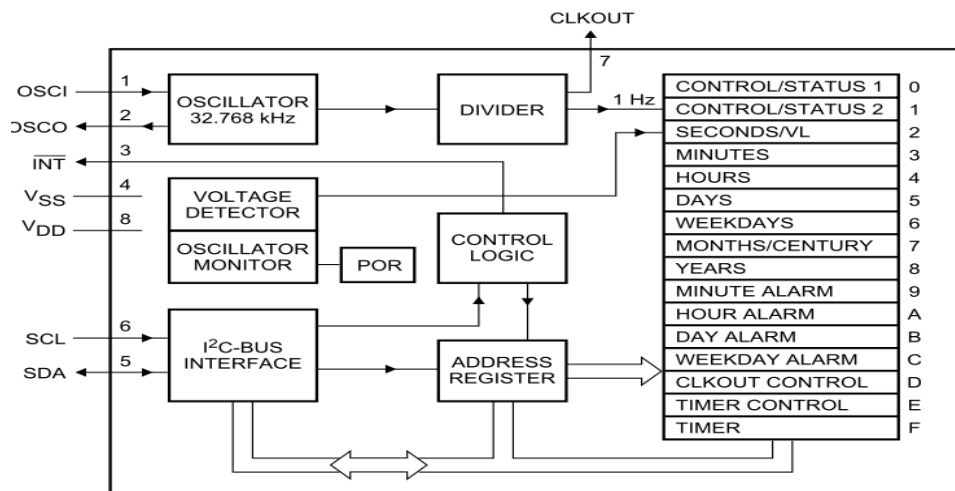


图 4.16.2 PCF8563 内部结构图

PCF8563 共有 16 个寄存器其中 00H01H 为控制方式寄存器 09H0CH 为报警功能寄存器 0DH 为时钟输出寄存器 0EH 和 0FH 为定时器功能寄存器 02H08H 为秒年时间寄存器各寄存器的位描述综合于下表。

Address	Register name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
00H	Control/Status 1	TEST1	0	STOP	0	TESTC	0	0	0
01H	Control/Status 2	0	0	0	TI/TP	AF	TF	AIE	TIE
0DH	CLKOUT frequency	FE	–	–	–	–	–	FD1	FD0
0EH	Timer control	TE	–	–	–	–	–	TD1	TD0
0FH	Timer countdown value	<timer countdown value>							

Address	Register name	BCD format tens nibble				BCD format units nibble			
		Bit 7 2 ³	Bit 6 2 ²	Bit 5 2 ¹	Bit 4 2 ⁰	Bit 3 2 ³	Bit 2 2 ²	Bit 1 2 ¹	Bit 0 2 ⁰
02H	Seconds	VL	<seconds 00 to 59 coded in BCD>						
03H	Minutes	–	<minutes 00 to 59 coded in BCD>						
04H	Hours	–	–	<hours 00 to 23 coded in BCD>					
05H	Days	–	–	<days 01 to 31 coded in BCD>					
06H	Weekdays	–	–	–	–	<weekdays 0 to 6 > ^[1]			
07H	Months/Century	C	–	–	<months 01 to 12 coded in BCD>				
08H	Years	<years 00 to 99 coded in BCD>							
09H	Minute alarm	AE	<minute alarm 00 to 59 coded in BCD>						
0AH	Hour alarm	AE	–	<hour alarm 00 to 23 coded in BCD>					
0BH	Day alarm	AE	–	<day alarm 01 to 31 coded in BCD>					
0CH	Weekday alarm	AE	–	–	–	<weekday alarm 0 to 6 > ^[1]			

[1] Not coded in BCD.

图 4.16.3 PCF8563 内部寄存器

(3) 连线关系

实验中连接关系如下：

杜邦线连线关系				
线序号	线端 A 插接位置		线端 B 插接位置	
	开发板	端子	开发板	端子
S1	KEYLED	JK2M2: P1.0	KEYLED	J2M2: CS
S2	KEYLED	JK2M2: P1.1	KEYLED	J2M2: CLK
S3	KEYLED	JK2M2: P1.2	KEYLED	J2M2: DIO
S4	KEYLED	JK22: INT0	KEYLED	J2M2: KEY
S5	KEYLED	J4M2: Row_3	KEYLED	J6M2: SG
S6	KEYLED	J4M2: Row_2	KEYLED	J6M2: SF
S7	KEYLED	J4M2: Row_1	KEYLED	J6M2: SE
S8	KEYLED	J4M2: Row_0	KEYLED	J6M2: SD
S9	KEYLED	J4M2: Col_3	KEYLED	J3M2: DIG0
S10	KEYLED	J4M2: Col_2	KEYLED	J3M2: DIG1
S11	KEYLED	J4M2: Col_1	KEYLED	J3M2: DIG2
S12	KEYLED	J4M2: Col_0	KEYLED	J3M2: DIG3
S13	PI0	P2M3: D0	PI0	JK2M3: P0.3
S14	MCU	P2M0: BELL	MCU	J1M0: P0.2
S15	LCD	JK2M7: P1.5	LCD	J6M7: RS
S16	LCD	JK2M7: P1.6	LCD	J6M7: RW
S17	LCD	JK2M7: P1.7	LCD	J6M7: E
S18-S25	LCD	J4M7: D0-D7	LCD	JK2M7: A8-A15
S26	Memory&RTC	JK2M4: P0.0	Memory&RTC	J4M4: SDA
S27	Memory&RTC	JK2M4: P0.1	Memory&RTC	J4M4: SCL
S28	Memory&RTC	JK2M4: INT1	Memory&RTC	J4M4: /INT

(4) 程序流程图

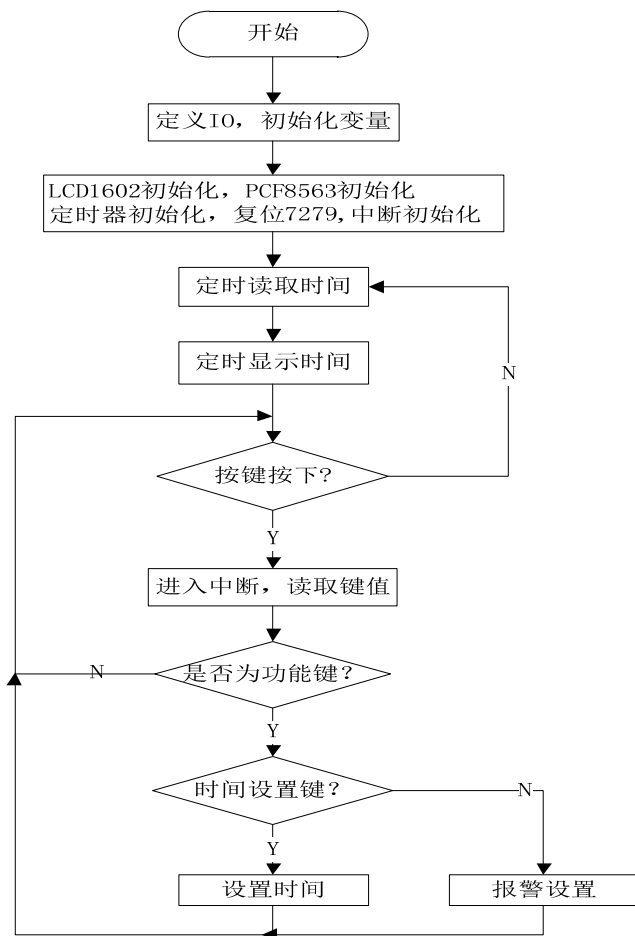


图 4.16.4 I2C 扩展 PCF8563 实验流程图

(5) 编程思路

按键功能分配、显示方法及编程思路等同与前面的“实验 4.12 DS12887 并行接口 RTC 实验”。参考 PCF8563 的用户使用手册对其进行读写控制。通信接口仍然为 IIC 接口。该 IIC 接口使用 P0.0 和 P0.1 来仿真实现。I2C 的操作时序请参考附录 1。另外，IIC 子板上 J7 有个 CLKOUT 端，可以利用该端子输出方波信号，假设输出信号的频率为 1Hz，利用程序设置好 PCF8563，然后观察该端信号在示波器上的波形。

(5) 实验步骤

1. 关掉实验箱电源。将 MCU 板，Memory&RTC 板，KEYLED 板，PIO 板，LCD 板插接在母板上。按照前面连接关系表将硬件连接好。

2. 在仿真器断电情况下将仿真器的仿真头插在 MCU 板的 CPU 插座上。将仿真器与开发 PC 机的通信口连接好，打开实验箱及仿真器的电源。
3. 运行 Keil uVision2 开发环境，建立工程 I2C8563_c.uv2，CPU 为 AT89S51，包含启动文件 STARTUP.A51。
4. 按照实验功能要求创建源程序 I2C8563.c 并加入到工程 I2C8563_c.uv2，并设置工程 I2C8574_c.uv2 属性，将其晶振频率设置为 11.0592MHz，选择输出可执行文件，DEBUG 方式选择硬件 DEBUG，并选择其中的“WAVE V series MCS51 Driver”仿真器。
5. 构造（Build）工程 I2C8563_c.uv2。如果编程有误进行修改，直至构造正确为止。
6. 运行程序，观察结果是否符合程序要求，若不符合，分析出错原因，继续重复第 4、5 步的步骤，直至结果正确。

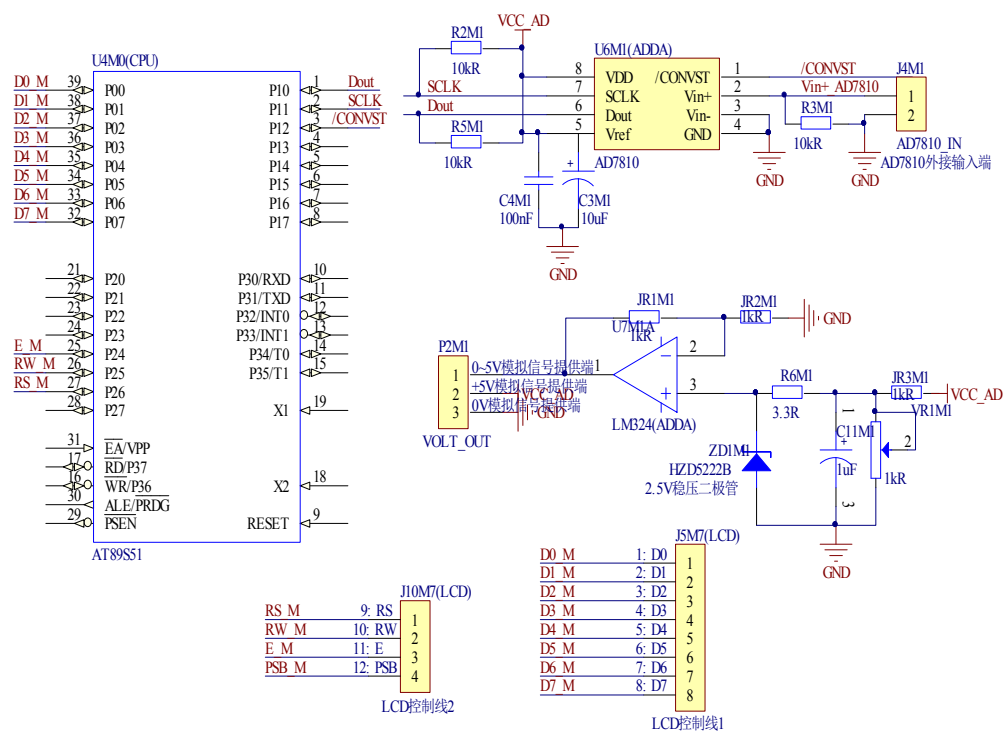
（7）实验作业

- 总结 PCF8563 的数据读写方法。
- 对比 PCF8563 与 DS12887 实现 RTC 的方法，分析各自的特点。

4.17 实验 17：I2C 接口芯片 AD7810 扩展 AD 实验

（1）实验内容

实验原理图如下：



(2) 实验原理

下图为高速模式的时序图

图 4.17.2 AD7810 工作在高速模式时的时序图

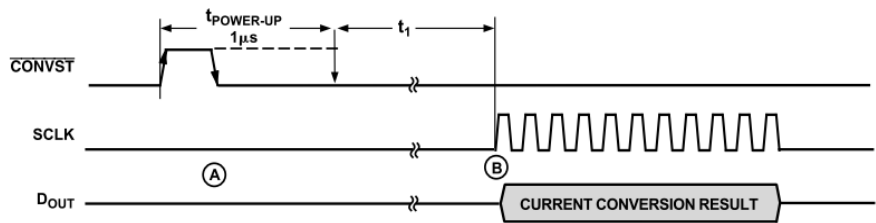


图 4.17.3 AD7810 工作在自动低功耗模式时的时序图

在低功耗模式下,启动信号 CONVST 为低电平时,器件处于低功耗休眠状态。当在 CONVST 端输入一个正脉冲时,可在其上升沿将器件从休眠状态唤醒,唤醒过程需要 $1\mu\text{s}$ 的时间 (图中 t_2)。当器件被唤醒后,系统将自动启动一次转换,转换时间也是 $2\mu\text{s}$ (图中 t_1)。转换结束时,AD7810 将转换结果锁存到输出移位寄存器中,同时自动将器件再一次置于低拉耗状态。

(3) 连线关系

接插件连接关系如下:

杜邦线连线关系				
线序号	线端 A 插接位置		线端 B 插接位置	
	开发板	端子	开发板	端子
S1	ADDA	J3M1: SCLK	ADDA	JK2M1: P1.1
S2	ADDA	J3M1: SDA	ADDA	JK2M1: P1.0
S3	ADDA	J3M1: CST	ADDA	JK2M1: P1.2
S4	ADDA	J4M1: GND	ADDA	P2M1: GND
S5	ADDA	J4M1: Volt_In	ADDA	P2M1: 0~+5V
S6	ADDA	JP2M1: 用短路帽短接		
S7	LCD	J6M7: E	LCD	JK2M7: A12
S8	LCD	J6M7: RW	LCD	JK2M7: A13
S9	LCD	J6M7: RS	LCD	JK2M7: A14
S10-S17	LCD	J4M7: D[0-7]	LCD	JK2M7: P0[0-7]

(4) 程序流程图

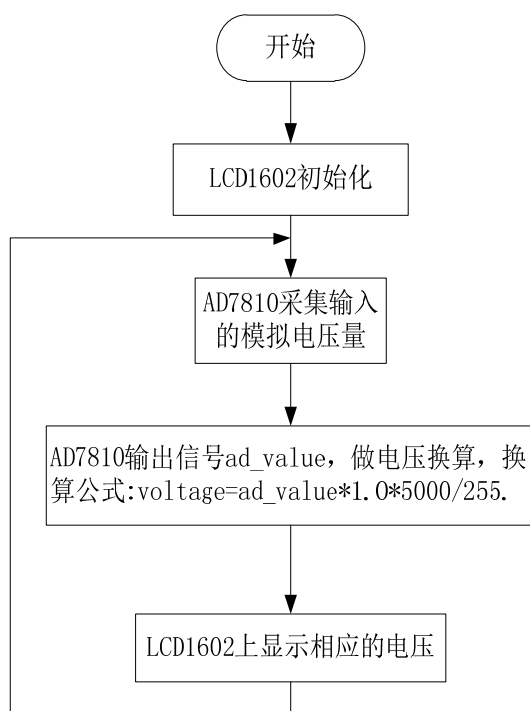


图 4.17.4 AD7810 串行接口 AD 转换实验原理图

(5) 编程思路

把 LM324 输出的电压信号作为 AD7810 的输入。读出 AD7810 输出的数据后用 LCD1602 输出。

(6) 实验步骤

1. 关掉实验箱电源。将 MCU 板，ADDA 板，LCD 板插接在母板上。将 LCD1602 子板插接在 LCD 板的 J9M7 上，注意插接方向。按照前面连接关系表将硬件连接好。
2. 在仿真器断电情况下将仿真器的仿真头插在 MCU 板的 CPU 插座上。将仿真器与开发 PC 机的通信口连接好，打开实验箱及仿真器的电源。
3. 运行 Keil uVision2 开发环境，建立工程 AD7810_c.uv2，CPU 为 AT89S51，包含启动文件 STARTUP.A51。
4. 按照实验功能要求创建源程序 AD7810.c 并加入到工程 TAD7810_c.uv2，并设置工程 AD7810_c.uv2 属性，将其晶振频率设置为 11.0592MHz，选择输出可执行文件，DEBUG 方式选择硬件 DEBUG，并选择其中的“WAVE V series MCS51 Driver”仿真器。

能够实现的信号类型包括：方波，三角波，正弦波，频率有：10HZ，20HZ，30HZ，40HZ 几种。

(2) 实验原理

TLC5615 的内部功能框图如下图所示,它主要由以下几部分组成:

- 1、10 位 DAC 电路;
- 2、一个 16 位移位寄存器,接受串行移入的二进制数,并且有一个级联的数据输出端 DOUT ;
- 3、并行输入输出的 10 位 DAC 寄存器,为 10 位 DAC 电路提供待转换的二进制数据;
- 4、电压跟随器为参考电压端 REFIN 提供很高的输入阻抗,大约 $10M\Omega$;
- 5、 $\times 2$ 电路提供最大值为 2 倍于 REFIN 的输出;
- 6、上电复位电路和控制电路。

TLC5615 有两种工作方式:

- 1) 12 位数据序列模式: 6 位移位寄存器分为高 4 位虚拟位、低两位填充位以及 10 位有效位。在单片 TLC5615 工作时,只需要向 16 位移位寄存器按先后输入 10 位有效位和低 2 位填充位,2 位填充位数据任意。
- 2) 二种方式为级联方式,即 16 位数据列,可以将本片的 DOUT 接到下一片的 DIN,需要向 16 位移位寄存器按先后输入高 4 位虚拟位、0 位有效位和低 2 位填充位,由于增加了高 4 位虚拟位,所以需要 16 个时钟脉冲。

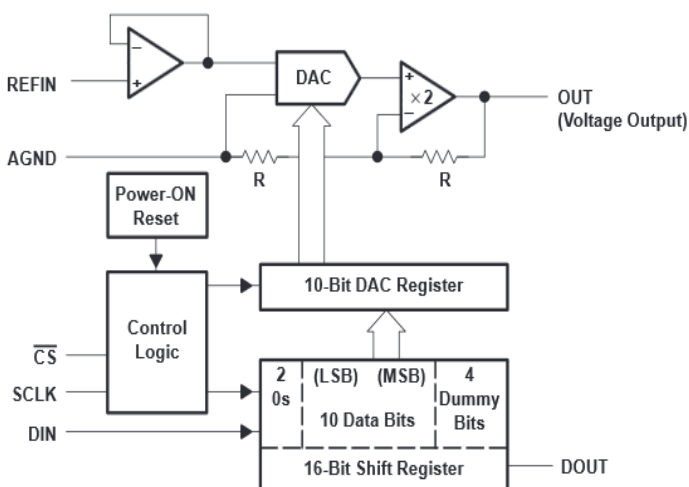


图 4. 18. 2 TLC5615 内部结构图

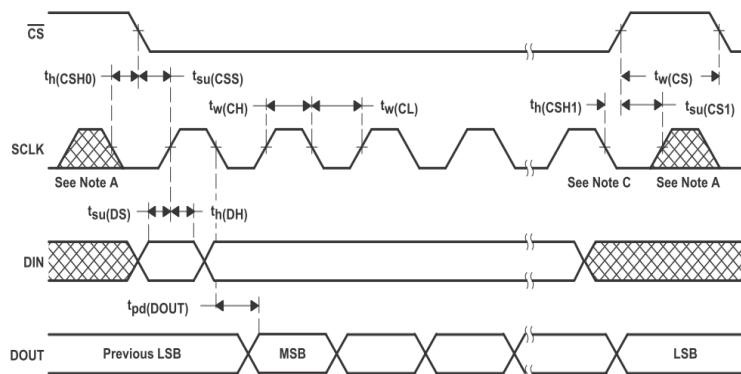


图 4.18.3 LC5615 时序图

当 CS 为低电平时，每一个 SCLK 时钟的上升沿将 DIN 的一位数据移入 16 位移寄存器。接着 CS 的上升沿将 16 位移位寄存器的 10 位有效数据锁存于 10 位 DAC 寄存器。DAC 电路进行转换。

(3) 连线关系

实验中连接关系如下：

杜邦线连线关系				
线序号	线端 A 插接位置		线端 B 插接位置	
	开发板	端子	开发板	端子
S1	LCD	JK2M7: P0.7	LCD	J6M7: RS
S2	LCD	JK2M7: P0.6	LCD	J6M7: RW
S3	LCD	JK2M7: P0.5	LCD	J6M7: E
S4-S11	LCD	JK2M7: A8-A15	LCD	J4M7: D0-D7
S12	MCU	JK2M0: INT0	MCU	P2M0: KEY0
S13	MCU	JK2M0: INT1	MCU	P2M0: KEY2
S14	ADDA	JK2M1: A8	ADDA	J5M1: DI
S15	ADDA	JK2M1: A9	ADDA	J5M1: SCL
S16	ADDA	J6M1: A10	ADDA	J5M1: CS
	ADDA	J10M1: OUT		示波器探头
	ADDA	J10M1: GND		示波器探头接地

(4) 程序流程图

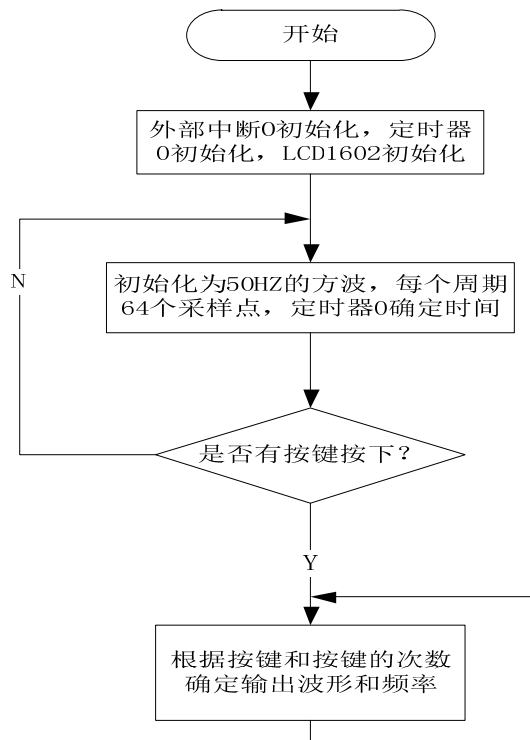


图 4.18.4 I2C 扩展 TLC5615 实验流程图

(5) 编程思路

见前面的“4.12 实验 12: DAC0832 并行接口 DA 转换实验”。

(6) 实验步骤

1. 关掉实验箱电源。将 MCU 板, ADDA 板, KEYLED 板, LCD 板插接在母版上。将 LCD1602 子板插接在 LCD 板上, 将注意插接方向。按照前面连接关系表将硬件连接好。
2. 在仿真器断电情况下将仿真器的仿真头插在 MCU 板的 CPU 插座上。将仿真器与开发 PC 机的通信口连接好, 打开实验箱及仿真器的电源。
3. 运行 Keil uVision2 开发环境, 建立工程 TLC5615_c.uv2, CPU 为 AT89S51, 包含启动文件 STARTUP.A51。
4. 按照实验功能要求创建源程序 TLC5615.c 并加入到工程 TLC5615_c.uv2, 并设置工程 TLC5615_c.uv2 属性, 将其晶振频率设置为 11.0592MHz, 选择输出可

- 执行文件，DEBUG 方式选择硬件 DEBUG，并选择其中的“WAVE V series MCS51 Driver” 仿真器。
- 构造 (Build) 工程 TLC5615_c.uV2。如果编程有误进行修改，直至构造正确为止。
 - 运行程序，观察结果是否符合程序要求，若不符合，分析出错原因，继续重复第 4、5 步的步骤，直至结果正确。

(7) 实验作业

- 总结 TLC5615 串行 AD 的使用方法。
- 比较利用 TLC5615 实现 DA 与采用 ADC0832 实现相同功能在软硬件上的异同。

4.19 实验 19：18B20 温度测量实验

(1) 实验内容

本实验研究温度传感器 DS18B20 的原理和使用，学习单总线通信方式使用方法。

实验原理图如下：

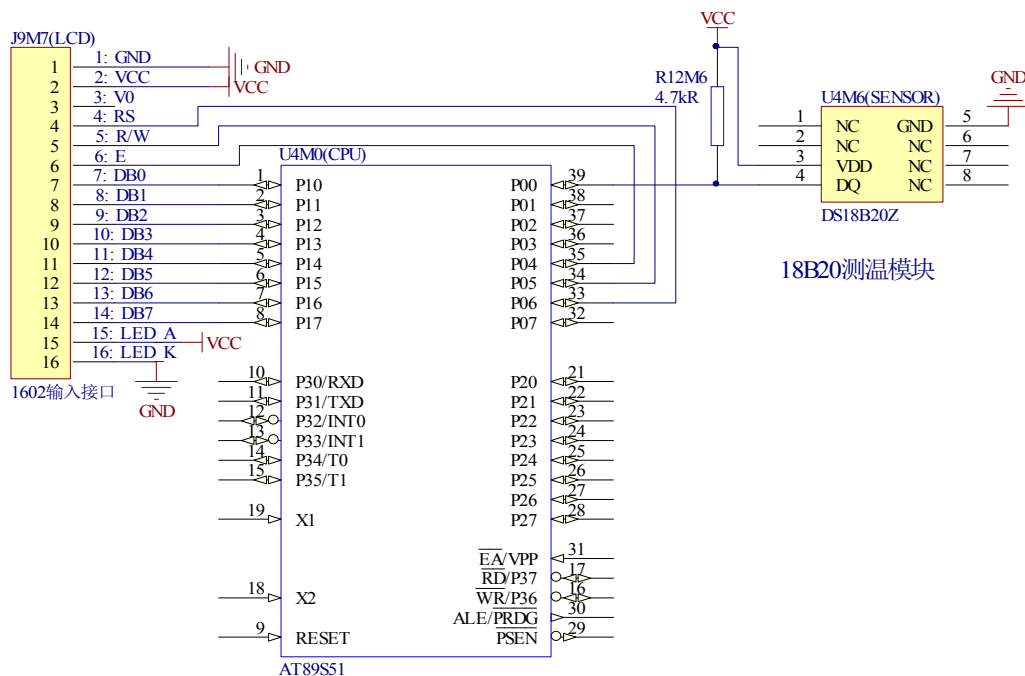


图 4.19.1 18B20 温度测量实验原理图

(2) 实验原理

DS18B20 数字式温度传感器，与传统的热敏电阻有所不同的是，使用集成芯片，采用单总线技术，其能够有效的减小外界的干扰，提高测量的精度，同时，它可以直接将被测温度转化成串行数字信号供微机处理，接口简单，使数据传输和处理简单化。部分功能电路的集成，使总体硬件设计更简洁，能有效地降低成本，搭建电路和焊接电路时更快，调试也更方便简单化，这也就缩短了开发的周期。

DS18B20 单线数字温度传感器，即“一线器件”，其具有独特的优点：

1. 采用单总线的接口方式与微处理器连接时仅需要一条口线即可实现微处理器与 DS18B20 的双向通讯。单总线具有经济性好，抗干扰能力强，适合于恶劣环境的现场温度测量，使用方便等优点，使用户可轻松地组建传感器网络，为测量系统的构建引入全新概念。
2. 测量温度范围宽，测量精度高 DS18B20 的测量范围为 $-55^{\circ}\text{C} \sim +125^{\circ}\text{C}$ ；在 $-10^{\circ}\text{C} \sim +85^{\circ}\text{C}$ 范围内，精度为 $\pm 0.5^{\circ}\text{C}$ 。
3. 在使用中不需要任何外围元件。
4. 持多点组网功能多个 DS18B20 可以并联在惟一的三线上，实现多点测温。
5. 供电方式灵活 DS18B20 可以通过内部寄生电路从数据线上获取电源。因此，当数据线上的时序满足一定的要求时，可以不接外部电源，从而使系统结构更趋简单，可靠性更高。
6. 测量参数可配置 DS18B20 的测量分辨率可通过程序设定 9~12 位。
7. 负压特性电源极性接反时，温度计不会因发热而烧毁，但不能正常工作。
8. 掉电保护功能 DS18B20 内部含有 EEPROM，在系统掉电以后，它仍可保存分辨率及报警温度的设定值。

DS18B20 内部结构主要由四部分组成： 64 位光刻 ROM 、温度传感器、非挥发的温度报警触发器 TH 和 TL 、配置寄存器。

DS18B20 中的温度传感器用于完成对温度的测量，它的测量精度可以配置成 9 位，10 位， 11 位或 12 位 4 种状态。温度传感器在测量完成后将测量的结果存储在 DS18B20 的两个 8BIT 的 RAM 中，单片机可通过单线接口读到该数据，读取时低位在前，高位在后数据的存储格式如下表（以 12 位转化为例）：

MSB	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}	LSB
	S	S	S	S	S	2^6	2^5	2^4	MSB

有关 DS18B20 的详细资料请阅读 DS18B20 的数据手册。

(3) 连线关系

接插件连接关系如下：

杜邦线连线关系				
线序号	线端 A 插接位置		线端 B 插接位置	
	开发板	端子	开发板	端子
S1	SENSOR	J2M6: 18B20_OUT	SENSOR	JK2M6: P0.0
S2	LCD	J6M7: E	LCD	JK2M7: P0.4
S3	LCD	J6M7: RW	LCD	JK2M7: P0.5
S4	LCD	J6M7: RS	LCD	JK2M7: P0.6
S5-S12	LCD	J4M7: D[0...7]	LCD	JK2M7: P1[0...7]

(4) 程序流程图

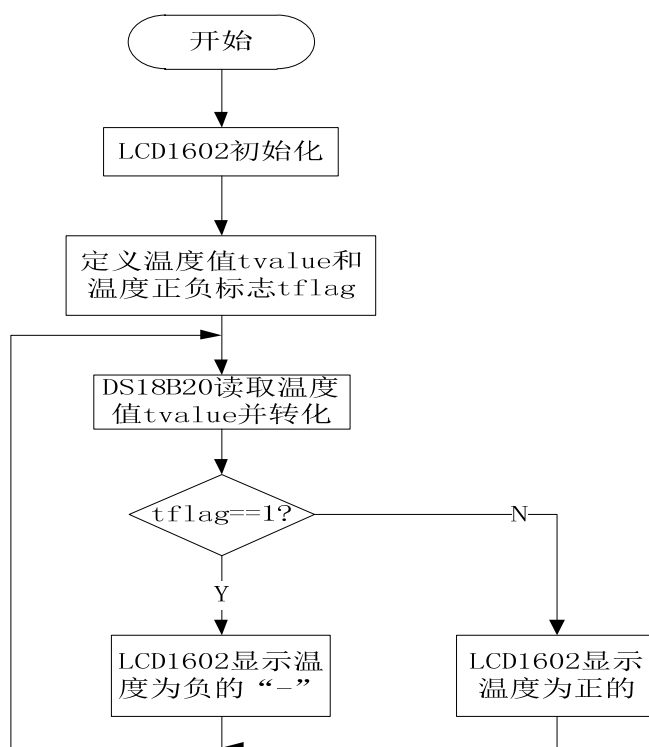


图 4.19.2 18B20 温度测量实验原理图

(5) 编程思路

DS18B20 有一个串行输入输出端，进行命令的读入和温度数据的读出。阅读 DS18B20 的数据手册，从 DS18B20 中读出数据并用 LCD 模块显示出来。

(6) 实验步骤

1. 关掉实验箱电源。将 MCU 板，SENSOR 板，LCD 板插接在母版上，按照前面连接关系表将硬件连接好。
2. 在仿真器断电情况下将仿真器的仿真头插在 MCU 板的 CPU 插座上。将仿真器与开发 PC 机的通信口连接好，打开实验箱及仿真器的电源。
3. 运行 Keil uVision2 开发环境，建立工程 DS18B20_c.uv2，CPU 为 AT89S51，包含启动文件 STARTUP.A51。
4. 按照实验功能要求创建源程序 DS18B20.c 并加入到工程 DS18B20_c.uv2，并设置工程 DS18B20_c.uv2 属性，将其晶振频率设置为 11.0592MHz，选择输出可执行文件，DEBUG 方式选择硬件 DEBUG，并选择其中的“WAVE V series MCS51 Driver”仿真器。
5. 构造 (Build) 工程 DS18B20_c.uv2。如果编程有误进行修改，直至构造正确为止。
6. 运行程序，观察 LCD 是否能正确显示当前温度。用手触摸 18b20 并观察 LCD 中显示的温度是否随之改变。若不符合要求，分析出错原因，直至结果正确。

(7) 实验作业

- 总结 DS18B20 单总线通信方式的特点。
- 尝试使用 DS18B20 读取不同精度的温度。
- 尝试模拟 DS18B20 在现实生活中的应用。

4.20 实验 20：DHT11 温湿度测量实验

(1) 实验内容

本实验研究另一个单总线通信的传感器 DHT11 温湿度传感器，学习 DHT11 单总线通信方式的特点，学会应用 DHT11 温湿度传感器。实验原理图如下：

DHT11 的管脚排列如图 4. 20. 2 所示：

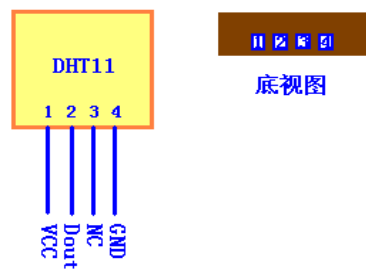


图 4. 20. 2 DHT11 管脚排列图

虽然 DHT11 与 DS18B20 类似，都是单总线访问，但是 DHT11 的访问，相对 DS18B20 来说要简单很多。下面我们先来看看 DHT11 的数据结构。

DHT11 数字湿温度传感器采用单总线数据格式。即，单个数据引脚端口完成输入输出双向传输。其数据包由 5Byte（40Bit）组成。数据分小数部分和整数部分，一次完整的数据传输为 40bit，高位先出。DHT11 的数据格式为：8bit 湿度整数数据+8bit 湿度小数数据+8bit 温度整数数据+8bit 温度小数数据+8bit 校验和。其中校验和数据为前四个字节相加。

传感器数据输出的是未编码的二进制数据。数据(湿度、温度、整数、小数)之间应该分开处理。例如，某次从 DHT11 读到的数据如图 4. 20. 3 所示：

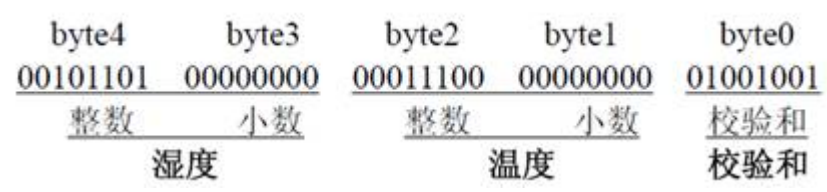


图 4. 20. 3 某次读取到 DHT11 的数据

由以上数据就可得到湿度和温度的值，计算方法：

湿度= byte4 . byte3=45.0 (%RH)

温度= byte2 . byte1=28.0 (°C)

校验= byte4+ byte3+ byte2+ byte1=73(=湿度+温度) (校验正确)

可以看出，DHT11 的数据格式是十分简单的，DHT11 和 MCU 的一次通信最大为 3ms 左右，建议主机连续读取时间间隔不要小于 100ms。

下面，我们介绍一下 DHT11 的传输时序。DHT11 的数据发送流程如图 4. 20. 4 所示：

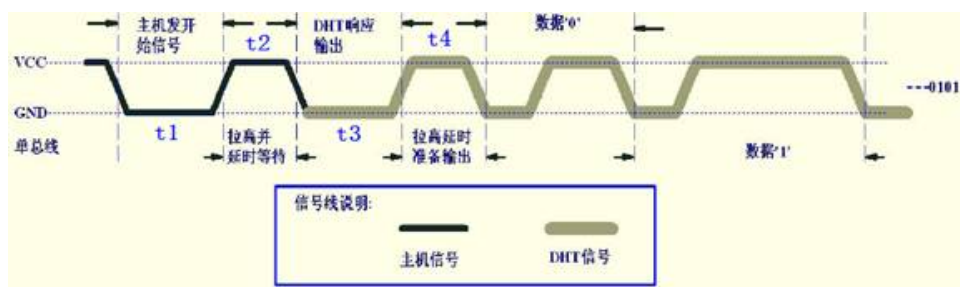


图 4.20.4 DHT11 数据发送流程

首先主机发送开始信号，即：拉低数据线，保持 t_1 （至少 18ms）时间，然后拉高数据线 t_2 （20~40us）时间，然后读取 DHT11 的相应，正常的话，DHT11 会拉低数据线，保持 t_3 （40~50us）时间，作为响应信号，然后 DHT11 拉高数据线，保持 t_4 （40~50us）时间后，开始输出数据。

DHT11 输出数字 ‘0’ 的时序如图 4.20.5 所示：

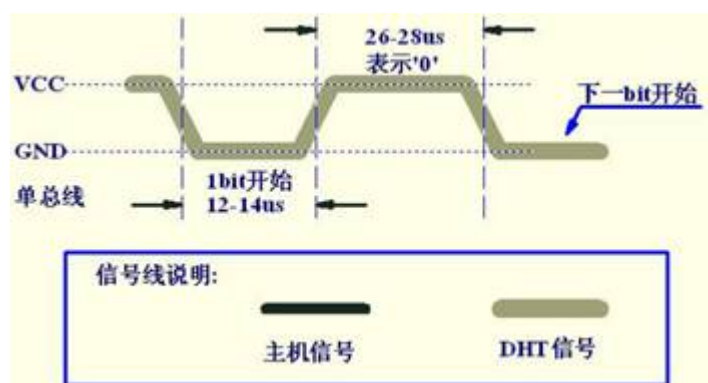


图 4.20.5 DHT11 数字 ‘0’ 时序

DHT11 输出数字 ‘1’ 的时序如图 4.20.6 所示：

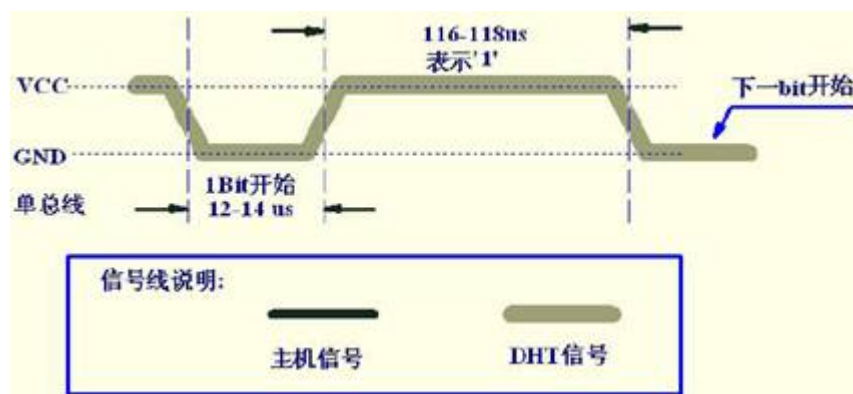


图 4.20.6 DHT11 数字 ‘1’ 时序

通过以上了解，我们就可以通过处理器来实现对 DHT11 的读取了。DHT11 的介绍就到这里，更详细的介绍，请参考 DHT11 的数据手册。

(3) 连线关系

接插件连接关系如下：

杜邦线连线关系				
线序号	线端 A 插接位置		线端 B 插接位置	
	开发板	端子	开发板	端子
S1	SENSOR	J9M6 :DHT11_OUT	SENSOR	JK2M6: P0.0
S2	KEYLED	J2M2: /CS	KEYLED	JK2M2: P1.7
S3	KEYLED	J2M2: CLK	KEYLED	JK2M2: P1.6
S4	KEYLED	J2M2: DIO	KEYLED	JK2M2: P1.5
S5	KEYLED	J2M2: /KEY	KEYLED	JK2M2: P1.4
S6-S11	KEYLED	J3M2: DIG[0-5]	KEYLED	J5M2: DIG[0-5]
S12-S	KEYLED	J6M2: DP-SG	KEYLED	J1M2: DP-SG

(4) 编程思路

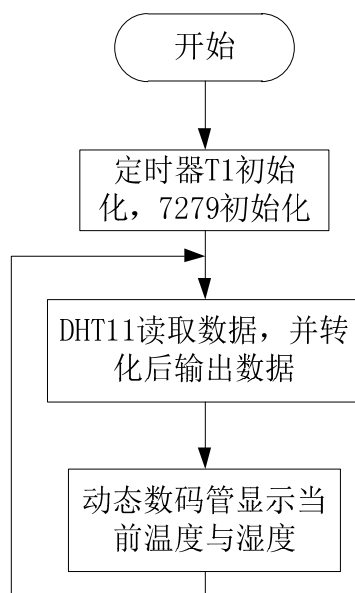


图 4.20.7 DHT11 温湿度测量模块程序

(5) 编程思路

DHT11 有一个串行输入/输出端，进行命令的读入和温湿度数据的读出。阅读 DHT11 的数据手册，从 DHT11 中读出数据并用 KEY_LED 模块或 LCD 模块显示出来。

(6) 实验步骤

1. 关掉实验箱电源。将 MCU 板，SENSOR 板，KEYLED 板插接在母版上。按照前面连接关系表将硬件连接好。
2. 在仿真器断电情况下将仿真器的仿真头插在 MCU 板的 CPU 插座上。将仿真器与开发 PC 机的通信口连接好，打开实验箱及仿真器的电源。
3. 运行 Keil uVision2 开发环境，建立工程 Humidity_c.uv2，CPU 为 AT89S51，包含启动文件 STARTUP.A51。
4. 按照实验功能要求创建源程序 Humidity.c 并加入到工程 Humidity_c.uv2，并设置工程 Humidity_c.uv2 属性，将其晶振频率设置为 11.0592MHz，选择输出可执行文件，DEBUG 方式选择硬件 DEBUG，并选择其中的“WAVE V series MCS51 Driver”仿真器。
5. 构造（Build）工程 Humidity_c.uv2。如果编程有误进行修改，直至构造正确为止。
6. 运行程序，观察是否能正确显示当前温湿度。用手触摸 DHT11 并观察数码管中显示的温湿度是否随之改变。若不符合要求，分析出错原因，直至结果正确。

（7）实验作业

- 比较 DHT11 和 DS18B20 通信通信方式的异同。
- 尝试使用其他方式（比如 LCD1602）显示温湿度。
- 模拟实现 DHT11 在现实生活中的应用。

4.21 实验 21：红外对管障碍物检测实验

（1）实验内容

本实验研究红外发射和接收的原理，通过 LED 的亮灭变化反映是否接收到红外信号。

红外对管模块原理图如下：

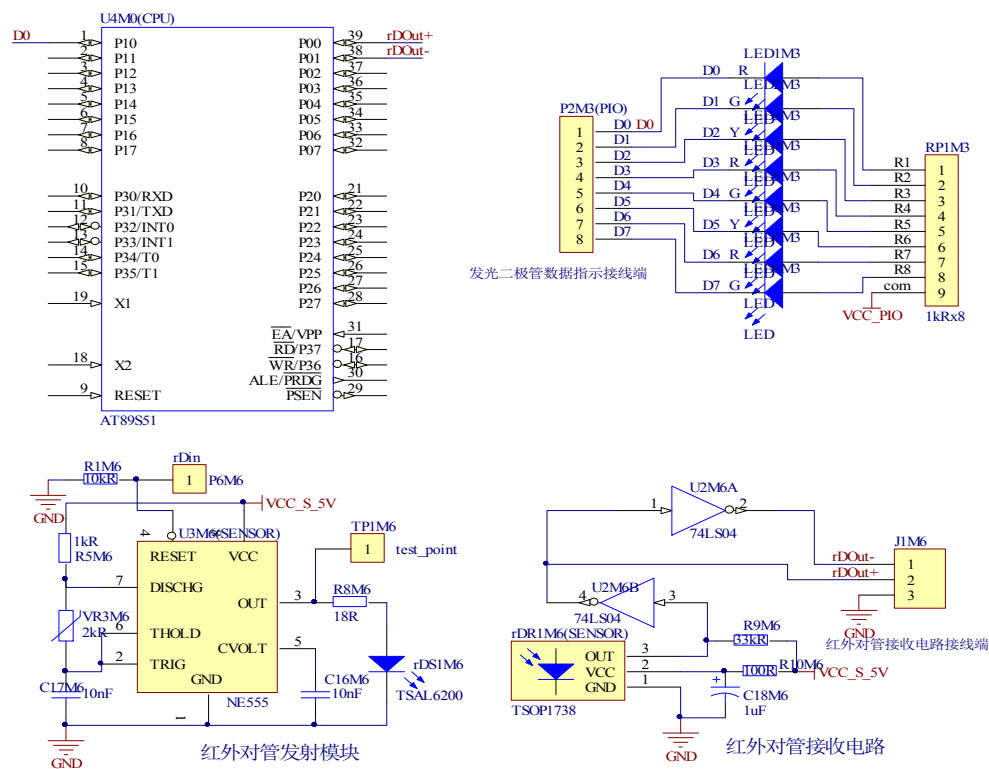


图 4.21.1 红外对管障碍物检测实验原理图

(2) 实验原理

本单片机教学实验箱采用的是 TSOP1738 一体式红外接收头，可以对收到的遥控信号进行放大、检波、整形、解调，滤除 38kHz 的载波信号，并将接收到的信号输出，直接得到原发射器发出的数字编码信号，使用方便，性能可靠。

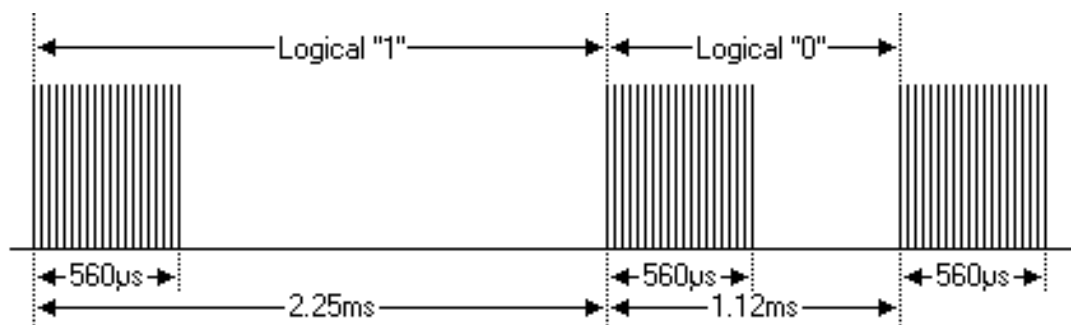
目前市面上常用的红外遥控编码主要是 NEC 标准和 PHILIPS 标准，其它都是这两类的变种。

1、NEC 编码标准：编码芯片有 PT2221/PT2222、HT6221/HT6222 等，一般家庭用的 DVD、VCD、音响都使用这种编码方式。

2、PHILIPS 的 RC-5 编码标准：编码芯片有 SAA3010、PT2210/PT2211/PT1215、HT6230 等。

下面以常用电器的 NEC 标准为例，介绍一下红外编码：

如图所示，NEC 标准使用 0.56ms 的高电平+0.565ms 的低电平代表数据中的 0，用 0.565ms 的高电平+1.685ms 的低电平代表数据中的 1。



一段 NEC 标准编码由引导码+用户码+用户反码+操作码+操作反码组成，引导码为 9ms 的高电平+4.5ms 的低电平组成。用户码和操作码均用 8 位 16 进制发送。（此处低电平和 高电平不是实际的电平概念，只是个代表 0 和 1 的标志，即低电平期间不发射 38KHZ 的红外波，此时发射管可亮可灭，高电平期间发射 38KHZ 的红外波。）为了稳定性的要求，用户码和操作码都发了两遍，其中第二遍是第一遍按位取反的结果，可以用来验证第一遍发送的准确性，而且因为‘0’、‘1’信号的长度不同，这样发送使信号的总长度恒定。



图 4.21.3 遥控信号编码波形图

即使遥控器的按键一直按下，遥控器只发送一遍控制信号，但是每隔 110ms 会发一遍包括 9msAGC 脉冲和 2.25ms 的低电平的重复码，如图 4.20.4 所示。



图 4.21.4 重复码 (左) 按键一直按下时遥控器发出的编码 (右)

(3) 连线关系

接插件连接关系如下:

杜邦线连线关系				
线序号	线端 A 插接位置		线端 B 插接位置	
	开发板	端子	开发板	端子
S1	SENSOR	J1M6: rDout+	SENSOR	JK2M6: P0.0
S2	SENSOR	J1M6: rDout-	SENSOR	JK2M6: P0.1
S3	PIO	P2M3 : D0	PIO	JK2M3: P1.0
使能开关 S1M6 拨到 ON 的位置				

(4) 程序流程图

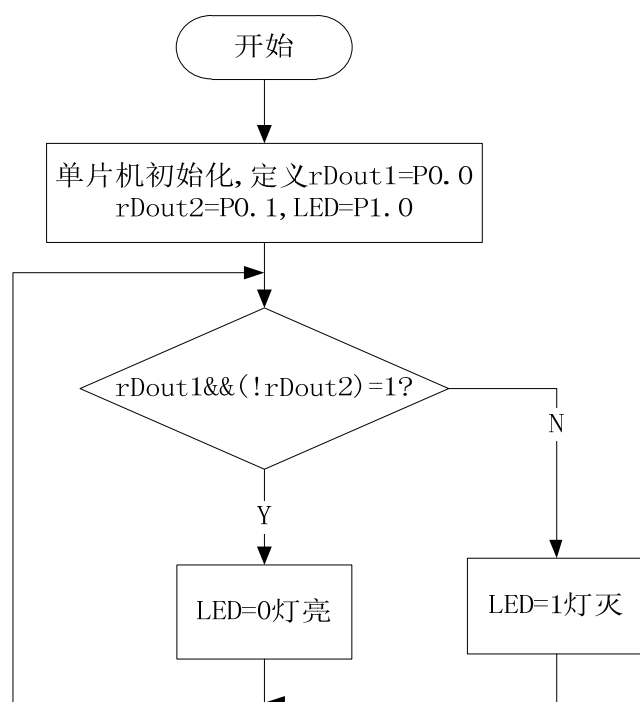


图 4.21.5 红外对管障碍物检测实验流程图

(5) 编程思路

输入端加高电平，红外发射管发出 38kHz 红外信号，加低电平时红外发射管不工作。接收端当 TSOP1738 没有接收到红外信号时，rDout+输出高电平，rDout-输出低电平；当 TSOP1738 接收到 38kHz 的红外信号时，rDout+输出低电平，rDout-输出高电平。给输入端加恒定的高电平，通过判断输出电平高低就可以实现红外对管障碍物检测功能。使用 LED 灯指示结果。

(6) 实验步骤

1. 关掉实验箱电源。将 MCU 板，SENSOR 板，PIO 板插接在母版上。按照前面连接关系表将硬件连接好。
2. 在仿真器断电情况下将仿真器的仿真头插在 MCU 板的 CPU 插座上。将仿真器与
3. 发 PC 机的通信口连接好，打开实验箱及仿真器的电源。
4. 运行 Keil uVision2 开发环境，建立工程 Infra_Red_c.uv2，CPU 为 AT89S51，包含启动文件 STARTUP.A51。
5. 按照实验功能要求创建源程序 Infra_Red.c 并加入到工程 Infra_Red_c.uv2，并设置工程 Infra_Red_c.uv2 属性，将其晶振频率设置为 11.0592MHz，选择输出可执行文件，DEBUG 方式选择硬件 DEBUG，并选择其中的“WAVE V series MCS51 Driver”仿真器。
6. 构造（Build）工程 Infra_Red_c.uv2。如果编程有误进行修改，直至构造正确为止。
7. 运行程序，用物体遮挡住发射和接收之间的区域，观察发光二极管的变化是否符合要求，若否分析原因，重复步骤 4-5。

（7）实验作业

- 查阅资料了解遥控器的原理。
- 了解红外对管在现实中的应用。
- 模拟实现红外对管的简单应用。

4.22 实验 22：红外热释电实验

（1）实验内容

本实验研究红外热释电传感器的原理和应用。红外热释电传感器感应到红外信号，经过专用芯片的处理后传递到微处理器，微处理器通过控制 LED 灯的亮灭直观的反应出来。实验原理图如下：

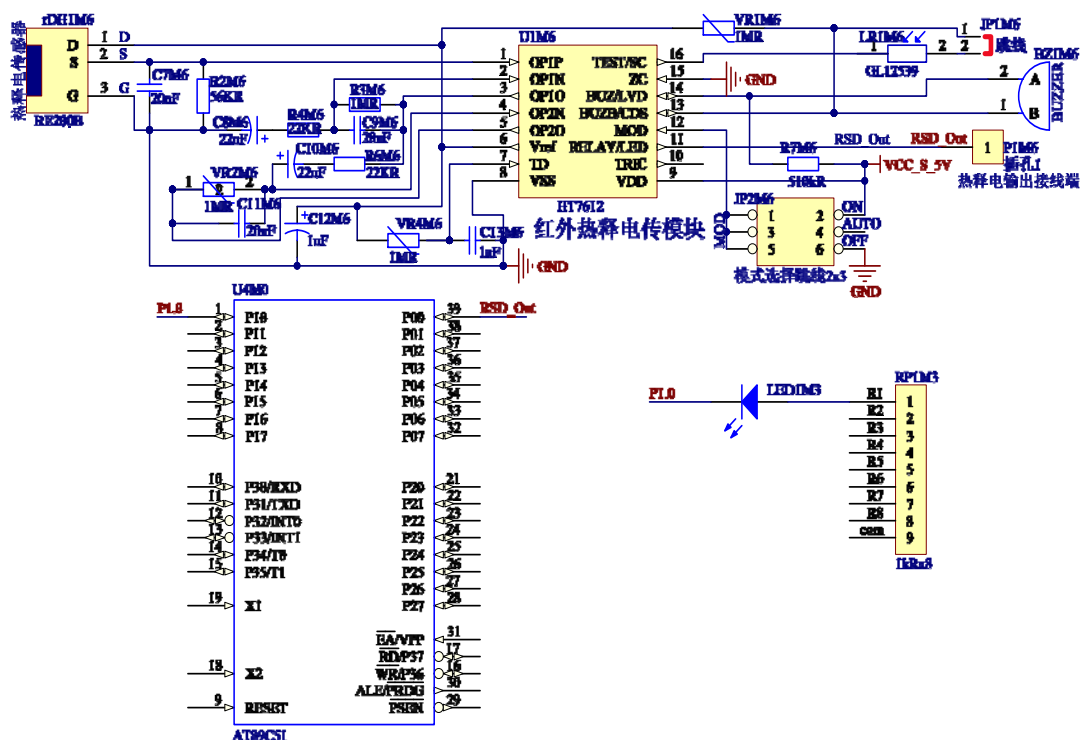


图 4.22.1 红外热释电模块原理图

(2) 实验原理

人体会发出波长 10 μm 左右的红外线，通过菲涅尔透镜增强后聚集到热释电元件上，使之失去电荷平衡，向外释放电荷，后续电路经检测处理就能产生输出信号，这就是红外热释电传感器的工作原理。

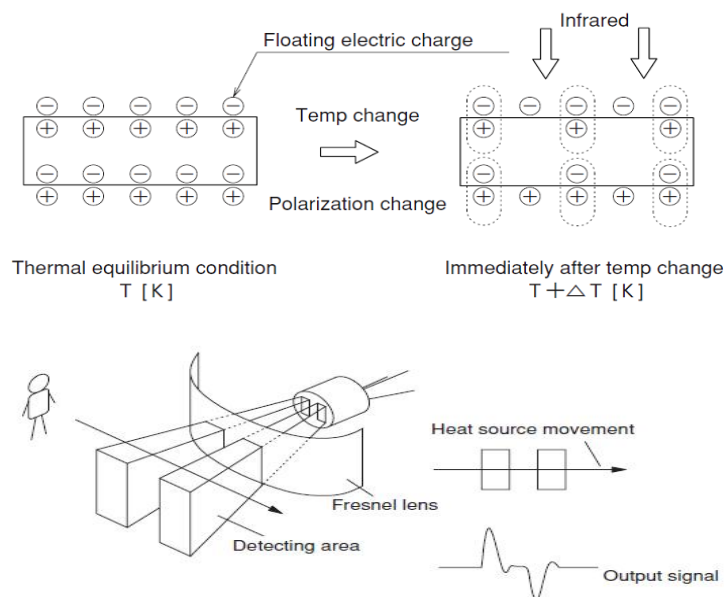


图 4.22.2 红外热释电传感器原理

热释电红外传感器和热电偶都是基于热电效应原理的热电型红外传感器。不同的是热释电红外传感器的热电系数远远高于热电偶，为了抑制因自身温度变化而产生的干扰，该传感器在工艺上将两个特征一致的热电元反向串联或接成差动平衡电路方式，因而能以非接触式检测出物体放出的红外线能量变化 并将其转换为电信号输出。当有移动红外辐射源进入探测区域内时，其产生的红外辐射通过部分镜面聚焦并被热释电元接受，但是两片热释电元接收到的热量不同，热释电也不同，不能抵消，从而使传感器输出电压信号。

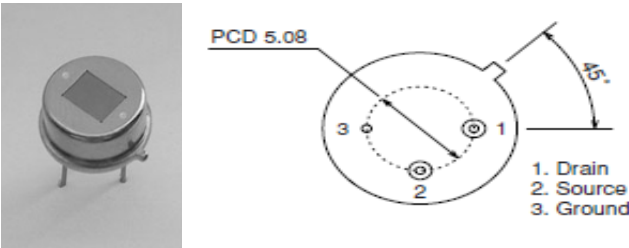


图 4.22.3 RE200B 红外热释电传感器

RE200B 是一款常用的红外热释电传感器，其既有较高的灵敏度，能满足一般报警器或自动灯控装置应用的要求，又有较低的价格，因此被广泛采用。

(3) 连线关系

实验连线如下：

杜邦线连线关系				
线序号	线端 A 插接位置		线端 B 插接位置	
	开发板	端子	开发板	端子
S1	SENSOR	P1M6:RSD_OUT	SENSOR	JK2M6: P0.0
S2	PIO	P2M3: D0	PIO	JK2M3:P1.0
	SENSOR	JP2M6: 用短路帽短接 ON		
	SENSOR	JP1M6: 用短路帽短接		

(4) 实验步骤

1. 关掉实验箱电源。将 MCU 板，SENSOR 板，PIO 板插接在母版上。按照前面连接关系表将硬件连接好。
2. 在仿真器断电情况下将仿真器的仿真头插在 MCU 板的 CPU 插座上。将仿真器与
3. 运行 Keil uVision2 开发环境，建立工程 Pyroelectric.uv2，CPU 为 AT89S51，包含启动文件 STARTUP.A51。
4. 按照实验功能要求创建源程序 Pyroelectric.c 并加入到工程 Pyroelectric.uv2，并设置工程 Pyroelectric.uv2 属性，将其晶振频率设置为

11.0592MHz，选择输出可执行文件，DEBUG 方式选择硬件 DEBUG，并选择其中的“WAVE V series MCS51 Driver”仿真器。

5. 构造 (Build) 工程 Pyroelectric.uv2。如果编程有误进行修改，直至构造正确为止。
6. 运行程序，用手遮挡住发射和接收之间的区域，观察发光二极管的变化是否符合要求，若否分析原因，重复步骤 4-5。

(5) 实验作业

- 查阅资料了解热释电传感器在现实生活中的应用。
- 尝试搭建电路模拟实现一项实际应用。

4.23 实验 23：超声测距实验

(1) 实验内容

本实验研究超声波测距的基本原理和超声测距模块的使用方法。实验原理图如下：

- (1) 采用 IO 口 TRIG 出发测距，给至少 10us 的高电平信号；
- (2) 模块自动发送 8 个 40kHz 的方波，自动检测是否有信号返回；
- (3) 有信号返回，通过 IO 口 ECHO 输出一个高电平，高电平持续的时间就是超声波从发射到返回的时间。测试距离=（高电平时间*声速）/2；

引脚定义：

- (1) VCC 为超声测距模块提供 5V 供电电源；
- (2) TRIG 触发控制信号输入，高电平有效；
- (3) ECHO 回声信号输出，输出一段时间的高电平；
- (4) GND 地线接口。

电气参数：

电气参数	HC-SR04 超声波模块
工作电压	DC 5V
工作电流	15mA
工作频率	40Hz
最远射程	4m
最近射程	2cm
测量角度	15 度
输入触发信号	10uS 的 TTL 脉冲
输出回声信号	输出 TTL 电平信号，与射程成比例
规格尺寸	45*20*15mm

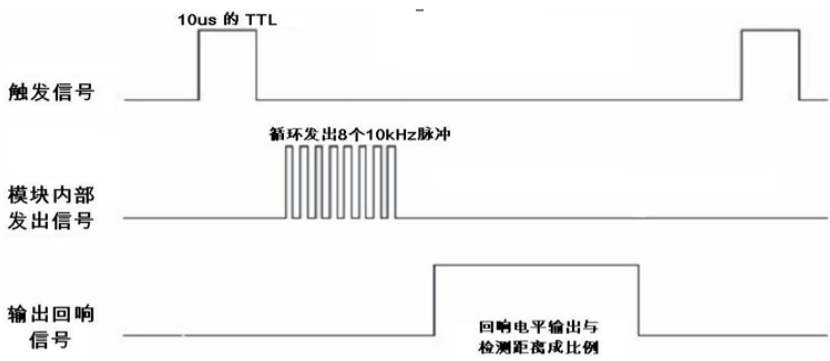


图 4.23. 超声波时序图

注意：进行实验时，不要带电连接，否则会影响模块的正常工作；测距时，被测物体的面积不少于 0.5 平方米且平面尽量要求平整，否则影响测量的结果。

(3) 连线关系

接插件连接关系如下：

杜邦线连线关系				
线序号	线端 A 插接位置		线端 B 插接位置	
	开发板	端子	开发板	端子
S1	超声模块	VCC	SENSOR	P3M6: VCC
S2	超声模块	Trig	SENSOR	P3M6:Trig
S3	超声模块	Echo	SENSOR	P3M6:Echo
S4	超声模块	GND	SENSOR	P3M6:GND
S5	SENSOR	JK2M6: P1. 0	SENSOR	P4M6:Trig
S6	SENSOR	JK2M6: P1. 1	SENSOR	P4M6:Echo
S7	KEYLED	J2M2:/CS	KEYLED	JK2M2:P1. 7
S8	KEYLED	J2M2:CLK	KEYLED	JK2M2:P1. 6
S9	KEYLED	J2M2:DIO	KEYLED	JK2M2:P1. 5
S10	KEYLED	J2M2:/KEY	KEYLED	JK2M2:P1. 4
S11-S18	KEYLED	J3M2:DIG[0-5]	KEYLED	J5M2:DIG[0-5]
S19-S26	KEYLED	J6M2:SG-DP	KEYLED	J1M2:LED_SG-DP

(4) 程序流程图

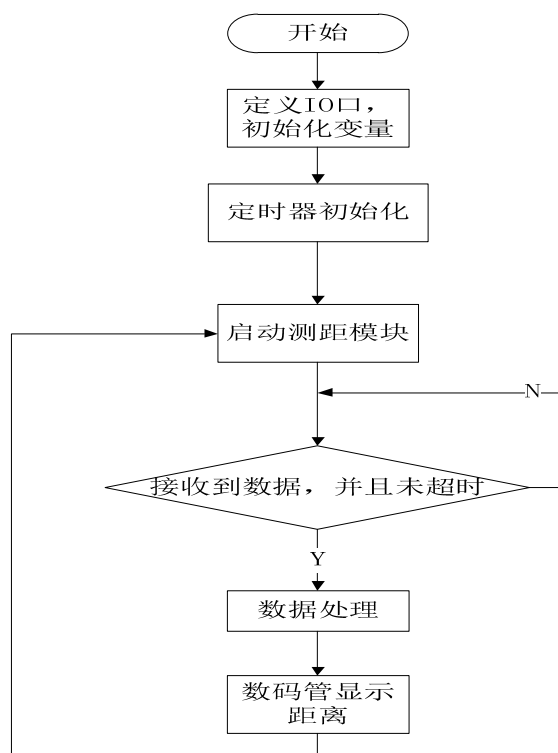


图 4. 23. 3 超声波测试实验程序图

(5) 编程思路

用一个定时计数器进行定时中断，以防超量程后程序卡死，用另一个定时计数器进行计数，当高电平到来时进行计数，高电平结束时关闭计数，用来计算返回的高电平的时间，用这个时间计算出障碍物的距离并用动态数码管显示出来。

(6) 实验步骤

1. 关掉实验箱电源。将 MCU 板，SENSOR 板，KEYLCD 板插接在母板上。将超声波测距模块插入到 SENSOR 板“传感器模块扩展区”，按照前面连接关系表将硬件连接好。
2. 在仿真器断电情况下将仿真器的仿真头插在 MCU 板的 CPU 插座上。将仿真器与开发 PC 机的通信口连接好，打开实验箱及仿真器的电源。
3. 运行 Keil uVision2 开发环境，建立工程 UltraSonic_c.uv2，CPU 为 AT89S51，包含启动文件 STARTUP.A51。
4. 按照实验功能要求创建源程序 UltraSonic.c 并加入到工程 UltraSonic_c.uv2，并设置工程 UltraSonic_c.uv2 属性，将其晶振频率设置为 11.0592MHz，选择输出可执行文件，DEBUG 方式选择硬件 DEBUG，并选择其中的“WAVE V series MCS51 Driver”仿真器。
5. 构造（Build）工程 UltraSonic_c.uv2。如果编程有误进行修改，直至构造正确为止。
6. 运行程序，观察结果是否符合程序要求，若不符合，分析出错原因，继续重复第 4、5 步的步骤，直至结果正确。

(7) 实验作业

- 尝试通过其他方式显示距离。
- 查阅资料了解超声测距的领用领域。
- 了解更多超声测距的电路

4.24 实验 24：步进电机驱动实验

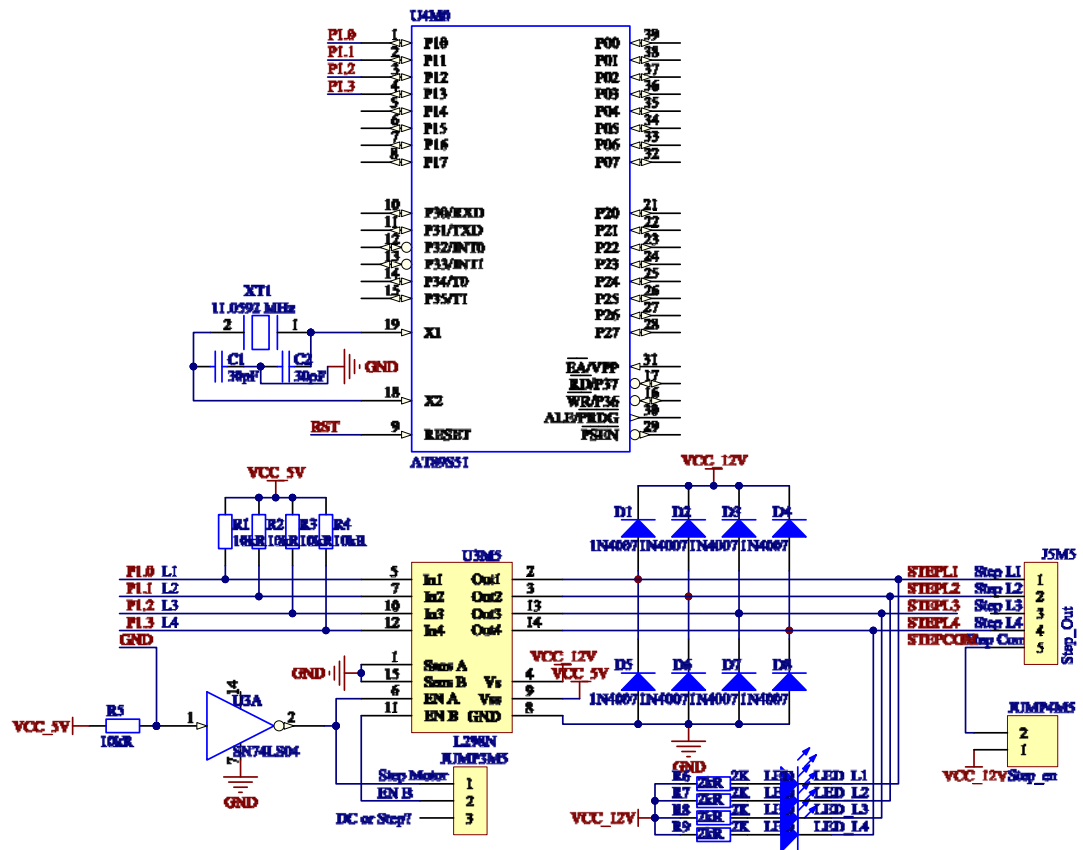
(1) 实验内容

本实验研究步进电机的原理和控制方法。

步进电机是将电脉冲信号转变为角位移或线位移的开环控制元步进电机件。在非超载的情况下，电机的转速、停止的位置只取决于脉冲信号的频率和脉冲数，而不受负载变化的影响，当步进驱动器接收到一个脉冲信号，它就驱动步进电机按设定的方向转动一个固定的角度，称为“步距角”，它的旋转是以固定的角度一步一步运行的。可以通过控制脉冲个数来控制角位移量，从而达到准确定位的目的；同时可以通过控制脉冲频率来控制电机转动的速度和加速度，从而达到调速的目的。

通常电机的转子为永磁体，当电流流过定子绕组时，定子绕组产生一矢量磁场。该磁场会带动转子旋转一角度，使得转子的一对磁场方向与定子的磁场方向一致。当定子的矢量磁场旋转一个角度。转子也随着该磁场转一个角度。每输入一个电脉冲，电动机转动一个角度前进一步。它输出的角位移与输入的脉冲数成正比、转速与脉冲频率成正比。改变绕组通电的顺序，电机就会反转。所以可用控制脉冲数量、频率及电动机各相绕组的通电顺序来控制步进电机的转动。

实验原理图如下：



杜邦线连线关系				
线序号	线端 A 插接位置		线端 B 插接位置	
	开发板	端子	开发板	端子
S1	DRIVERS	J5M5: L1	DRIVERS	JK2M5: P1.0
S2	DRIVERS	J5M5: L2	DRIVERS	JK2M5: P1.1
S3	DRIVERS	J5M5: L3	DRIVERS	JK2M5: P1.2
S4	DRIVERS	J5M5: L4	DRIVERS	JK2M5: P1.3
S5	KEYLED	J13M2: SKEY1	KEYLED	JK2M2: P3.5
S6	KEYLED	J13M2: SKEY2	KEYLED	JK2M2: P3.6
S7	KEYLED	J13M2: SKEY3	KEYLED	JK2M2: P3.7
	DRIVERS	JUMP3M5 (DC or Step?) 选择 Step		
	DRIVERS	JUMP4M5 (Step_en) 用跳线帽短接		

(4) 编程思路

本实验箱采用的是四相八拍步进电机，驱动方式为四相八拍方式，各线圈通电顺序如下表所示。

拍	L4	L3	L2	L1	编码
1	0	0	0	1	01H
2	0	0	1	1	03H
3	0	0	1	0	02H
4	0	1	1	0	06H
5	0	1	0	0	04H
6	1	1	0	0	0CH
7	1	0	0	0	08H
8	1	0	0	1	09H

将编码按 1-8 拍的顺序赋给 P1，则电机正转，将编码按 8-1 拍的顺序赋给 P1，则电机反转。

(5) 实验步骤

1. 关掉实验箱电源。将 MCU 板，DRIVERS 板，KEYLED 板插接在母板上。按照前面连接关系表将硬件连接好。
2. 在仿真器断电情况下将仿真器的仿真头插在 MCU 板的 CPU 插座上。将仿真器与开发 PC 机的通信口连接好，打开实验箱及仿真器的电源。

3. 运行 Keil uVision2 开发环境，建立工程 STEPMOTOR_c.uv2，CPU 为 AT89S51，包含启动文件 STARTUP.A51。
4. 按照实验功能要求创建源程序 STEPMOTOR.c 并加入到工程 STEPMOTOR_c.uv2，并设置工程 STEPMOTOR_c.uv2 属性，将其晶振频率设置为 11.0592MHz，选择输出可执行文件，DEBUG 方式选择硬件 DEBUG，并选择其中的“WAVE V series MCS51 Driver”仿真器。
5. 构造（Build）工程 STEPMOTOR_c.uv2。如果编程有误进行修改，直至构造正确为止。
6. 运行程序，观察结果是否符合程序要求，若不符合，分析出错原因，继续重复第 4、5 步的步骤，直至结果正确。

（6）实验作业

- 查阅资料了解步进电机的结构。
- 查阅资料了解步进电机的应用领域。

4.25 实验 25：直流电机驱动及转速测量实验

（1）实验内容

本实验研究电机驱动的基本原理和霍尔元件的使用。

实验原理图如下：

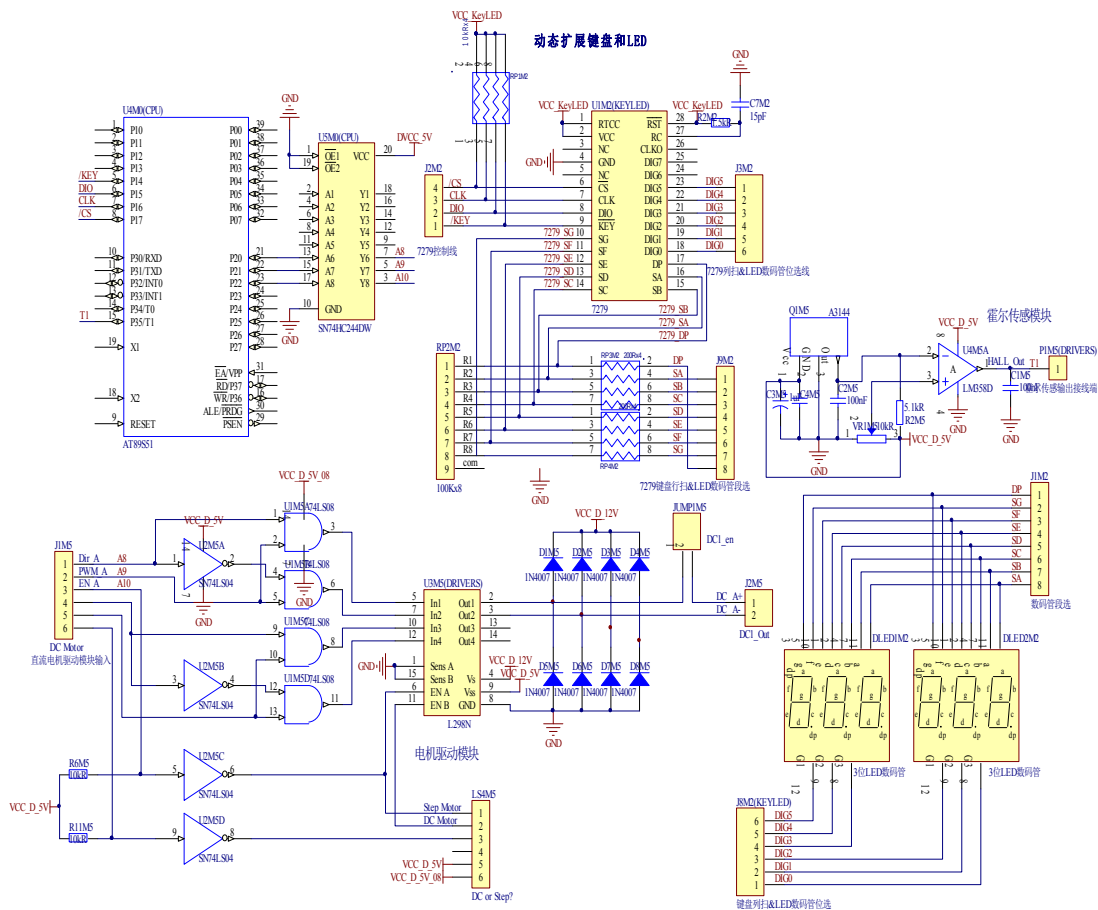


图 4.25.1 电机驱动模块原理图

(2) 连线关系

接插件连接关系如下：

杜邦线连线关系				
线序号	线端 A 插接位置		线端 B 插接位置	
	开发板	端子	开发板	端子
S1	DRIVERS	J1M5: Dir_A	DRIVERS	J10M5:A8
S2	DRIVERS	J2M5: PWM_A	DRIVERS	JK2M5:A9
S3	DRIVERS	J2M5: EN_A	DRIVERS	JK2M5:A10
S4	DRIVERS	JUMP3M5 (DC or Step?) 用短路帽选择 DC		
S5	DRIVERS	JUMP1M5 (DC1_en) 用跳线帽连接		
S6	DRIVERS	P2M5	DRIVERS	JK2M5:P3. 5
S7	KEYLED	J2M2:CS	KEYLED	JK2M2:P1. 7
S8	KEYLED	J2M2:CLK	KEYLED	JK2M2:P1. 6
S9	KEYLED	J2M2:DO	KEYLED	JK2M2:P1. 5
S10	KEYLED	J2M2:KEY	KEYLED	JK2M2:P1. 4
S11-S16	KEYLED	J3M2:DIG[0... 5]	KEYLED	J5M2:DIG[0... 5]
S17-S23	KEYLED	J6M2:SG... DP	KEYLED	J1M2:SG... DP

(3) 程序流程图

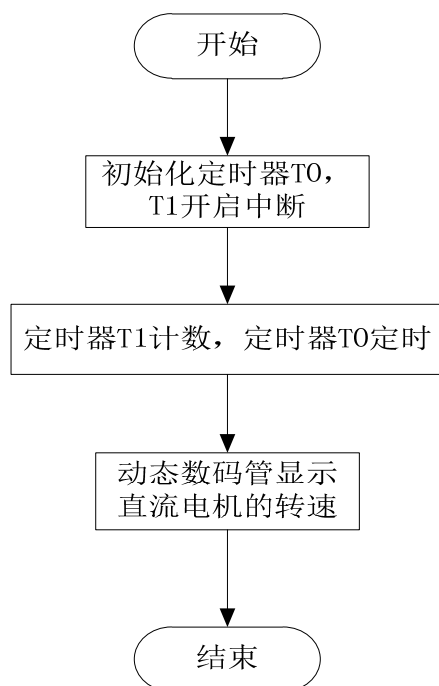


图 4.25.2 电机驱动模块流程图

(4) 编程思路

直流电机上安装的亚克力圆片上嵌有一个小磁铁，每当小磁铁经过霍尔传感器上方时，霍尔传感器模块会产生一个高电平，将其输出连接到 T1，定时器 1 为霍尔传感器传出的脉冲计数，定时器 0 为定时中断。每隔一定时间定时器 0 溢出产生中断，读取 TH1、TL1 的值并输出。主程序控制电机转动。调节电位器来调节霍尔传感器的灵敏度。利用数码管输出转速。

(5) 实验步骤

1. 关掉实验箱电源。将 MCU 板, DRIVERS 板, KEYLED 板插接在母板上。按照前面连接关系表将硬件连接好。
2. 在仿真器断电情况下将仿真器的仿真头插在 MCU 板的 CPU 插座上。将仿真器与开发 PC 机的通信口连接好，打开实验箱及仿真器的电源。
3. 运行 Keil uVision2 开发环境，建立工程 MOTOR_c.uv2，CPU 为 AT89S51，包含启动文件 STARTUP.A51。

- #### 4.26 实验 26: 电机手动调速与程控调速

本实验研究通过调节 PWM 脉冲占空比的方式改变直流电机的速度，处理器读取按键的状态，确定是增加还是降低频率。

图 4.26.1 电机驱动模块原理图

(2) 实验原理

脉冲宽度调制 (PWM) 是一种对模拟信号电平进行数字编码的方法。通过高分辨率计数器的使用, 方波的占空比被调制用来对一个具体模拟信号的电平进行编码。PWM 信号仍然是数字的, 因为在给定的任何时刻, 满幅值的直流供电要么完全有 (ON), 要么完全无 (OFF)。电压或电流源是以一种通 (ON) 或断 (OFF) 的重复脉冲序列被加到模拟负载上去的。通的时候即是直流供电被加到负载上的时候, 断的时候即是供电被断开的时候。只要带宽足够, 任何模拟值都可以使用 PWM 进行编码。

脉宽调制 (PWM) 基本原理: 控制方式就是对逆变电路开关器件的通断进行控制, 使输出端得到一系列幅值相等的脉冲, 用这些脉冲来代替正弦波或所需要的波形。也就是在输出波形的半个周期中产生多个脉冲, 使各脉冲的等值电压为正弦波形, 所获得的输出平滑且低次谐波少。按一定的规则对各脉冲的宽度进行调制, 即可改变逆变电路输出电压的大小, 也可改变输出频率。

脉宽调制 (PWM) 的理论依据: 冲量相等而形状不同的窄脉冲加在具有惯性的环节上时, 其效果基本相同。PWM 控制技术就是以该结论为理论基础, 对半导体开关器件的导通和关断进行控制, 使输出端得到一系列幅值相等而宽度不相等的脉冲, 用这些脉冲来代替正弦波或其他所需要的波形。按一定的规则对各脉冲的宽度进行调制, 既可改变逆变电路输出电压的大小。

(3) 连线关系

接插件连接关系如下:

杜邦线连线关系				
线序号	线端 A 插接位置		线端 B 插接位置	
	开发板	端子	开发板	端子
S1	DRIVERS	J2M5: Dir_A	DRIVERS	JK2M5: A8
S2	DRIVERS	J2M5: PWM_A	DRIVERS	JK2M5: A9
S3	DRIVERS	J2M5: EN_A	DRIVERS	JK2M5: A10
S4	DRIVERS	JUMP3M5 (DC or Step?) 打到 DC 档		
S5	DRIVERS	JUMP1M5 (DC1_en) 用跳线帽连接		
S6	MCU	P2M0:SKEY1	MCU	JKOM0: P1.7

(4) 程序流程图

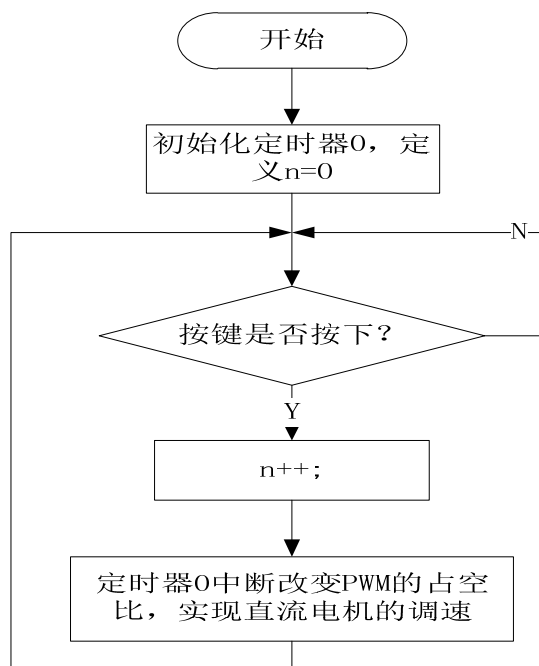


图 4. 26. 2 电机驱动模块流程图

(5) 编程思路

用单片机输出一个占空比与变量 n 有关的波形，通过 PWM 给 DC 电机供电，从而控制直流电机的转速。使用按键控制修改变量 n 的值，从而达到控制直流电机转速的目的。

(6) 实验步骤

1. 关掉实验箱电源。将 MCU 板，DRIVERS 板插接在母板上。按照前面连接关系表将硬件连接好。
2. 在仿真器断电情况下将仿真器的仿真头插在 MCU 板的 CPU 插座上。将仿真器与开发 PC 机的通信口连接好，打开实验箱及仿真器的电源。
3. 运行 Keil uVision2 开发环境，建立工程 MOTOR_SPEED_c.uv2，CPU 为 AT89S51，包含启动文件 STARTUP.A51。
4. 按照实验功能要求创建源程序 PI08255_traffic.c 并加入到工程 MOTOR_SPEED_c.uv2，并设置工程 MOTOR_SPEED_c.uv2 属性，将其晶振频率设置为 11.0592MHz，选择输出可执行文件，DEBUG 方式选择硬件 DEBUG，并选择其中的“WAVE V series MCS51 Driver”仿真器。

5. 构造 (Build) 工程 MOTOR_SPEED_c.uv2。如果编程有误进行修改，直至构造正确为止。
6. 运行程序，观察直流电机的转动和数码管的显示是否符合程序要求，若不符合，分析出错原因，继续重复第 4、5 步的步骤，直至结果正确。

(7) 实验作业

- 查阅资料了解更多脉宽调制 (PWM) 的知识。
- 总结脉宽调制 (PWM) 的特点。
- 总结脉宽调制 (PWM) 调速相比其它调速方式的优点

第五章 系统综合实验

5.1 单片机应用系统的设计流程简介

MCS-51 单片机应用系统的开发一般包含：系统需求调查、可行性分析、系统总体方案设计、系统硬件设计、系统软件设计、系统抗干扰设计、仿真调试—固化程序—脱机调试等七个步骤。具体的开发流程如下图 5.1 所示。

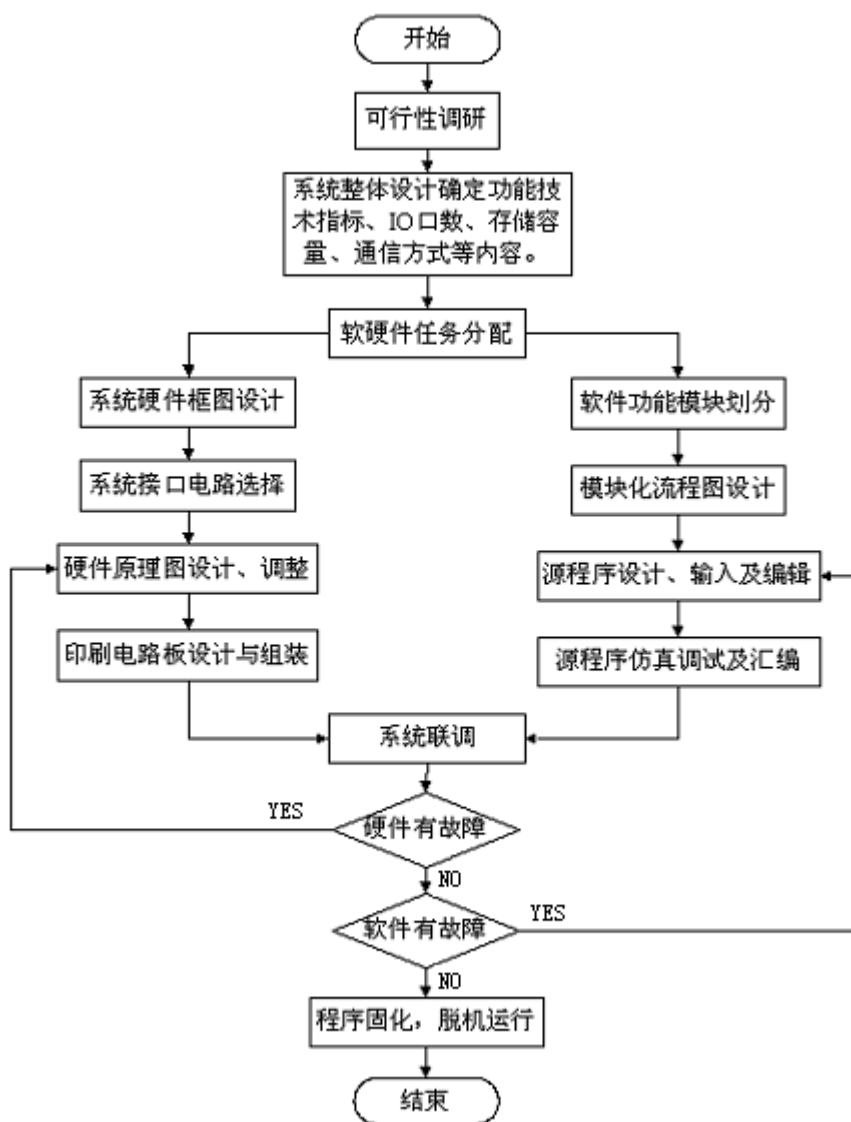


图 5.1 MCS-51 单片机应用系统的开发流程

5.1.1 需求调查

这一步的任务就是了解用户的需求，包括用户目前遇到的困难、希望新系统完成哪些功能、新系统的外观要求，接口要求，工作环境、用户可接受的设备价格等因素，将这些用户需求都记录下来，写出需求调查报告，作为可行性调研的重要依据之一。

5.1.2 可行性分析

可行性分析包括市场可行性分析和技术可行性分析。

市场可行性分析，也就是市场调研的目的是了解市场上有没有同类产品，其市场占有情况怎样，将来的市场发展情况。新产品的市场价值如何？应该采取什么策略，才能使新产品具有更强的市场竞争力。

技术可行性分析，也就是技术调研的目的是了解市场上有没有同类产品，如果有其技术路线是怎样的，找出其中的可以借鉴的地方，以及可以改进的地方。如果没有，则进一步分析将来实现该系统时所牵扯的各个技术环节，从理论上探讨其实现过程中的重点环节有没有大的障碍，客观上是否具备开发该系统的必备条件（如开发环境、仪器设备、资金等），估计系统的开发成本。看成本能否控制在用户可以接受的价格之内，并留有合理的开发回报。即系统是否值得开发。

根据前面的市场调研和技术调研的情况写出可行性分析报告，决定项目是否立项。

5.1.3 总体方案设计

经过可行性分析认为技术和市场都是可行的产品将进入总体方案设计阶段。在对应用系统进行总体设计时，应根据应用系统提出的各项技术性能指标，拟订出性价比最高的一套方案。首先，应根据任务的繁杂程度和技术指标要求选择机型。选定机型后，再选择系统中要用到的其它外围元器件，如传感器、执行器件等。

这一阶段的工作包含：总体框架规划、人员搭配和开发环境准备三个步骤。

框架规划：确定系统的整体框架结构，所采用的主要技术路线，确定 I/O 口数、存储容量、通信方式，明确系统的功能指标，分析估计系统开发各个环节的工作量及开发进度计划。确定开发条件（包括硬件、软件设备仪器、开发工作环境）要求。

人员搭配：根据整个系统的开发工作量，以及开发进度计划，确定软硬件开发队伍的人员构成，组成项目开发小组，并对组成人员进行分工，提出具体的工作任务书和目标责任书。

开发环境准备：准备开发工作场所，购买开发所用硬件设备仪器和有关软件，开始开发。

5.1.4 硬件设计

总体方案设计之后便要进行软硬件功能分配。确定哪些功能由硬件实现，哪些功能由软件实现。有些功能必须由硬件实现，如：键盘接口、显示器接口、A/D、D/A 等；有些功能只能由软件实现，如点阵液晶显示器的驱动、大部分的通信编码，还有一些信号处理的算法等。而有些功能可能即可以用软件实现又可以用硬件实现，如 A/D 信号的滤波即可以在采样之前通过模拟滤波器预先经过滤波，将采集信号的带宽限制在有用范围之内；也可以不采用模拟滤波器，而是采用高速 A/D 先进行采样，变成数字信号之后，再利用软件方法设计数字信号滤波器进行滤波处理。前者的优势是后期软件处理简单，实现周期短，对处理器的要求也较低，但是硬件成本较高，比较适合于少量定制性产品的开发，而不适合于批量生产产品的开发。后者的优势是硬件成本较低，适合于批量生产的产品开发，但是软件开发复杂，对处理器的要求较高。因此，在具体的系统设计时，基本的原则是 CPU 处理能力许可，开发周期也足够长的情况下，能用软件实现的就用软件实现。

软硬件功能划分之后就可以进行硬件详细设计和软件详细设计了。

硬件详细设计包含以下几个步骤：

（1）硬件模块划分：根据系统整体要求，将系统划分成多个功能相对独立的模块（如：中央处理模块、系统扩展模块、信号测量模块、信号控制模块、人机接口模块、通信模块等，分别确定各自的功能框架结构、模块之间的接口约定。

(2) 原理图设计：根据前面的功能划分情况，分别设计各个模块的具体硬件实现，包括器件的选择，原理电路图的设计，以及原理图的仿真测试。

(3) 电路板设计：根据各个模块原理电路图的设计情况，以及各个功能模块的性质和接口连接情况决定硬件电路板的分布情况，并设计系统电路板。

(4) 电路板装配：根据电路板的设计情况，结合原理图的设计，列出所用元件列表，购买有关元件，等电路板印制完成后，装配有关元件。

(5) 模块功能测试：电路板装配焊接好后，就可以采用标准测试引号，在有可能的情况下测试各个功能模块的功能实现情况，必要时进行调整，各个模块基本测试通过后等待软件开发完成后，进行系统联调。

硬件设计时需注意以下几个问题：

(1) 尽可能选用标准化、模块化、集成度高的典型电路，提高设计成功的可能性，集成度高的电路能够减少外围器件，提高系统的可靠性。

(2) 系统设计时，在满足当前的要求的前提下，要留有适当的扩展余地。包括存储空间留有余地，电路板设计也不要太拥挤，留有适当的过线孔，一般将来调整电路时，插接元器件方便，连接可靠。对于测试完全通过的系统，在系统定型时，可以在结构上稍微紧凑些。

(3) 在技术成熟的前提下，尽可能地选用一些技术上更新，集成度更高、功能更强的芯片，而不要选用过时的器件，一方面方便系统设计，另一方面也能节省成本。

(4) 在设计电路时，还要考虑系统各部分的驱动能力，输入输出阻抗是否匹配，接地，安装、维修是否方便，以及抗干扰性能等有关细节。

5.1.5 软件设计

软件详细设计包含以下几个步骤：

(1) 软件模块划分：根据系统整体功能要求，将系统软件划分成多个功能相对独立的模块（如：中央处理模块、信号测量模块、信号控制模块、人机接口模块、通信模块等，

分别确定各自的功能框架结构根据硬件连接情况，确定各扩展期间的地址空间，合理分配系统的内存资源，约定模块之间的软件接口。

(2) 流程图设计：根据前面的功能划分情况，分别设计各个模块的具体软件流程图。

(3) 软件的输入、编辑和调试：根据前面的各个模块的流程图，分别设计各个模块的软件代码，输入、编辑并仿真测试各个模块代码的功能，若有问题则及时调整，直到各个软件模块都能测试通过。

软件设计时需注意以下几个问题：

- (1) 尽可能选用标准化、成熟的软件代码，提高设计成功的可能性。
- (2) 模块划分时，各个模块要尽量独立，单个模块的功能也要尽量单一，即提高程序的结构化程度。
- (3) 模块间的接口定义在整个系统内要尽量唯一，接口占用的资源（RAM 单元）在整个系统内要尽量不被其他单元使用，减少模块间的相互干扰。
- (4) 软件模块内部所使用的公共寄存器（如：A，B，PSW，R0，R1 等）在使用前应该先加以保护，使用后再进行恢复，以免影响其它模块使用。
- (5) 软件模块代码前应该有该模块的功能描述，接口描述，甚至作者、修改时间等记录。代码中，关键语句的功能也要有描述，所用变量的含义要有注释，以便其它人员阅读，也便于作者修改代码时参考。
- (6) 软件设计时，也要考虑软件抗干扰设计。它是提高程序可靠性的有力保障。（比如，软件陷阱及看门狗技术，数据采集时的多次采样技术等）。

5.1.6 仿真调试

单片机应用系统开发仿真环境如下图 5.2 所示：

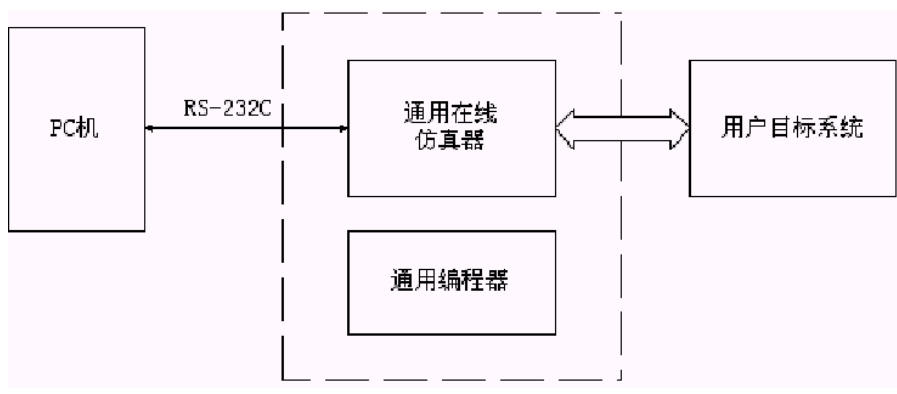


图 5.2 MCS-51 单片机应用系统开发环境

PC 机上运行单片机的仿真调试软件，如 uVersion2 IDE，PC 机通过平行口、串行口或者 USB 口与仿真器相连，仿真器与用户目标系统板（简称目标板）通过专用并行口或者 JTAG 接口相连。这样就构成了一个 MCS-51 单片机的硬件仿真调试环境，可以通过仿真软件对用户目标系统进行在线调试。

单片机系统的调试分为硬件调试、软件调试、系统联调和现场调试四个过程，四个过程的执行顺序如下图 4.3 所示：

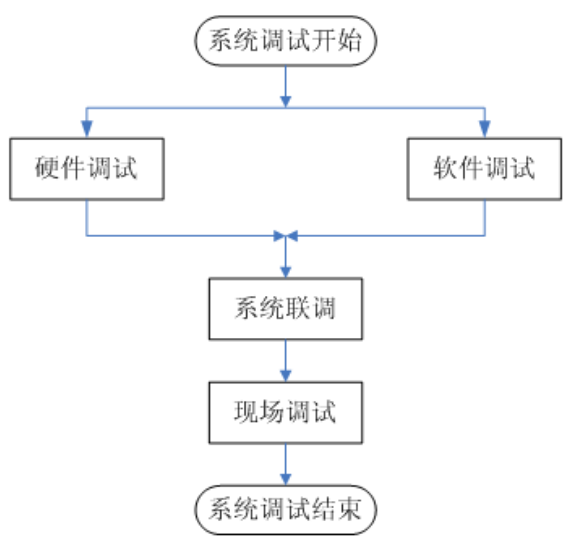


图 4.3 MCS-51 单片机应用系统调试步骤

(1) 硬件调试

硬件调试的任务是排除系统的硬件电路故障，包括设计性错误和工艺性故障。单片机应用系统的硬件调试可以按以下几个步骤进行：

- 静态调试：用户系统未工作时的测试，分为不加电状态下的目测、万用表测试，加电后的电压测试、典型信号测试。
- 模块调试：编写专用的模块功能测试子程序，分别测试各个硬件模块的功能，若有问题各个模块分别排除，各模块都测试通过后，再编写综合测试程序，测试所有模块的功能。
- 联机调试：将系统软件加载，进行联机调试，建成系统功能是否正常。
- 抗干扰测试：在系统联调通过后，模拟系统的实际工作环境，分别施加各种干扰信号，测试系统的抗干扰能力。

(2) 软件调试

软件调试是利用开发工具进行在线仿真调试，除发现和解决程序错误外，也可以发现硬件故障。程序调试一般是一个模块一个模块地进行，一个子程序一个子程序地调试，最后联起来统调。因此，软件调试可以按以下几个步骤进行：

- 分块调试：编写各个分块的程序后，分别进行调试，调试时可以使用单步、设置断点等技术，逐步观察仿真环境下各个寄存器及程序状态字、地址指针等是否符合程序运行逻辑。若有问题及时调整。
- 组合调试：根据模块间的关系，分别组合各个模块联合工作时的参数传递是否正常、寄存器状态是否正常、程序运行是否无误。
- 联机调试：将仿真调试无误后的程序加载到硬件板上，测试有关参数是否正常，联机调试时也可以分模块逐步加载，以便测试各个模块软件在系统中的运行情况。各个模块都正常后再将完整系统加载测试。
- 抗干扰测试：模仿系统实际工作环境，进行抗干扰测试。若抗干扰性能不是很好，则尽量用软件抗干扰措施补救，实在不行时才考虑修改硬件。

5.2 电梯控制系统实验

◆ 实验内容

利用本实验箱 MCU 子板, Key&LED 子板, PIO 子板, Driver 子板共同实现电梯控制功能。要求通过, Key&LED 子板中的静态或动态 LED 指示当前楼层（楼层高度选定 6 层）和上升下降指示。用 PIO 子板中的 LED 发光二极管和 DIP 开关作为楼层设定。用 Key&LED 子板中的按键作为开门和关门按键。用 Driver 子板中的步进电机模拟电梯上升或下降动作。

◆ 实验功能要求

1、基本功能要求：

- (1) 能够设定楼层，并根据当前楼层状态自动决定电梯的上升与下降。
- (2) 当电梯到达指定楼层时，能够停止运行，并模拟开门指示灯亮。2 秒钟之后，开门指示灯灭，电梯根据设定继续运行。当没有其他楼层设定时，电梯停止运行。
- (3) LED 能够自动显示当前所处的楼层。
- (4) 开门按键可以在电梯暂停期间延长停止与开门时间，关门按键可以立即关门。

2、扩展功能要求：

- (1) 电梯运行期间防止开门动作。
- (2) 模拟真实电梯运行期间，上升期间先把更高层的运行请求执行完毕，再执行下降动作。同样，下降期间，先把更低层的请求执行完毕，再执行上升。
- (3) 增加 Audio 子板，实现语音提示功能。
- (4) 增加红外传感器功能，防止电梯关门期间的夹人。

◆ 实验步骤

- (1) 根据 4.1 节单片机系统设计流程，进行需求分析，编写功能实现基本方案，设计出端口分配表，片外地址分配表。
- (2) 进行端口连线。
- (3) 编写程序。
- (4) 调试程序
- (5) 记录系统运行结果。

5.3 程控函数发生器系统实验

◆ 实验内容

利用本实验箱 MCU 子板, Key&LED 子板, AD&DA 子板, LCD 子板加 12864 模块实现程控函数发生器系统实验。要求通过 AD&DA 子板中的并行或串行 AD 实现方波, 脉冲波, 正弦波, 三角波, 锯齿波, 梯形波等波形信号输出。用 Key&LED 子板中键盘选定输出波形, 设定波形参数, 用 LCD 12864 作为输入提示与输入结果指示。

◆ 实验功能要求

1、基本功能要求:

- (1) 各种波形频率在 20Hz 到 20KHz 之间可任意设定。
- (2) 脉冲波形占空比在 1%至 99%之间可以任意设定。
- (3) 锯齿波、梯形波的上升沿和下降沿都可以在单脉冲周期的 1%至 99%之间设定。
- (4) 单周期内信号样值不少于 10 个。

2、扩展功能要求:

- (1) 双通道同步输出两个信号的功能。
- (2) 利用波形发生器 WAVE 子板产生某一频率的正弦波, 利用 AD&DA 子板的 AD 功能完成信号采集, 完成同步方波转换, 将该正弦波所对应的方波通过单片机的某 I/O 口输出, 完成正弦波的过零检测功能, 通过另一 I/O 口输出过零脉冲。
- (3) 分析 (2) 中所输入信号的频率及相位, 对其进行倍频输出。
- (4) 对 (2) 中所输入信号进行 AM 调制, 要求调制信号的频率为输入信号频率的 1/10, 调制深度为 0.2。

◆ 实验步骤

(1) 根据 4.1 节单片机系统设计流程, 进行需求分析, 编写功能实现基本方案, 设计出端口分配表, 片外地址分配表。

(2) 进行端口连线。

(3) 编写程序。

(4) 调试程序

(5) 记录系统运行结果。

5.4 自动国旗升降系统实验

◆ 实验内容

利用本实验箱 MCU 子板, Driver 子板, Audio 子板, LCD 子板加 12864 模块实现自动国旗升降系统实验。要求通过 Driver 子板中的步进电机实现国旗升降电机, 用 MCU 子板的 S1M0 按键作为国旗上升开始按键, S3M0 按键作为国旗下降按键, LED4M0 作为国旗上升过程指示灯(在国旗上升过程中点亮, 上升结束后以及下降过程中, 该指示灯都灭)。用 Audio 子板中的 MP3 播放器播放国歌。LCD12864 显示上升或下降过程, 并用滑动条块指示升降行程。

◆ 实验功能要求

1、基本功能要求:

- (1) 按动升旗按键, 电机正转, 国歌奏响, 国歌奏毕, 电机停止, 升旗结束。
- (2) 按动降旗, 电机反转, 反转行程和正转行程相同时, 电机停止, 降旗结束。

2、扩展功能要求:

- (1) 增加 KeyLED 子板, 实现手动输入或手动测量国旗行程, 并根据行程结果以及国歌时间长度, 自动计算电机转速, 实现任意指定高度国旗均可做到曲终旗到顶。下降电机速度可以指定, 但行程和上升期间相同。
- (2) 利用 LCD 显示当前行程, 用数字百分比和条块同时指示。

◆ 实验步骤

(1) 根据 5.1 节单片机系统设计流程, 进行需求分析, 编写功能实现基本方案, 设计出端口分配表, 片外地址分配表。

(2) 进行端口连线。

(3) 编写程序。

(4) 调试程序

(5) 记录系统运行结果。

5.5 自动远程抄表系统实验

◆ 实验内容

利用本实验箱 MCU 子板, LCD 子板加 12864 模块, Sensor 子板, Communication 子板实现自动远程抄表系统实验。要求通过 Sensor 子板的温度传感器 18B20, 湿度传感器 DHT11 测量室内温湿度。两台实验箱通过 Communication 子板的 485 总线相互连接, 利用 LCD 子板加 12864 模块显示本机和对方的温湿度值。

◆ 实验功能要求

1、基本功能要求:

- (1) 正常测量本地温湿度, 并进行显示。
- (2) 利用 485 通信, 设计通信协议, 采集对方温湿度, 并进行显示。

2、扩展功能要求:

- (1) 利用 PC 机 232 口, 另外增加 232 转 485 模块, 实现 PC 机采集多路实验箱的温湿度值。编写多机通信程序, 利用 VB 编写上位机, 实现多机数据实时显示。并通过图形方式显示各机温度曲线。
- (2) 采用 nRF2401 实现无线通信替代 485, 重新进行实验, 实现基本功能。
- (3) 利用 W5100 实现网络通信, 实现基于网络的温湿度采集系统。

◆ 实验步骤

- (1) 根据 4.1 节单片机系统设计流程, 进行需求分析, 编写功能实现基本方案, 设计出端口分配表, 片外地址分配表。
- (2) 进行端口连线。

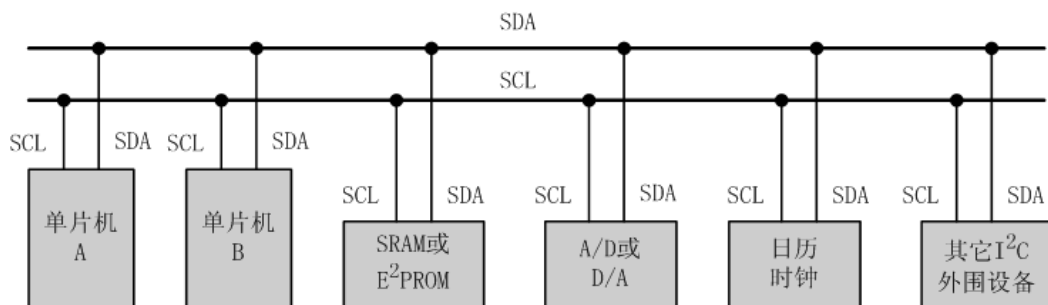
- (3) 编写程序。
- (4) 调试程序
- (5) 记录系统运行结果。

附录

I2C 总线简介

1、I2C 总线结构

I2C 总线是 PHILIPS 公司推出的一种串行总线。I2C 总线可以具备多个主机与多个从机，是一种高性能的同步串行总线。 I2C 总线有两根双向信号线，分别是：数据线 SDA 和时钟信号线 SCL。其通信系统结构如下图所示：



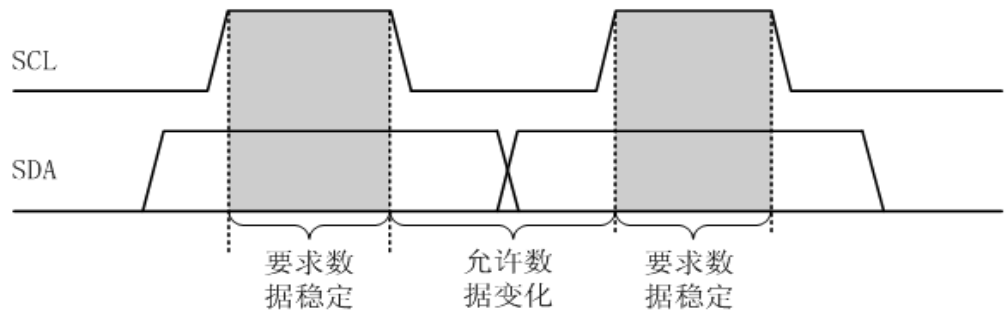
在 MCS-51 单片机应用系统 I2C 总线扩展中，经常以单片机为主机，其它接口器件为从机。

每个接接到 I2C 总线上的器件都有自己的地址。主机与这些器件之间进行数据通信时，将先发送需要与之通信的器件地址，所有器件收到地址帧之后进行判断，只有地址相符的器件才会做出响应。

2、I2C 总线时序

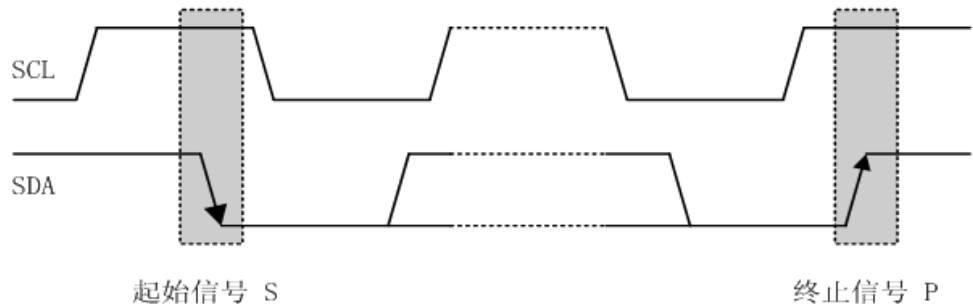
(1) 数据有效性规定

I2C 总线进行数据传送时，在时钟信号高电平期间，数据线上的数据必须保持稳定。只有在时钟线上的信号为低电平期间，数据线上的电平状态才允许变化，这就是 I2C 总线的数据有效性规定。如图下图所示：



(2) 起始和终止信号规定

SCL 线为高电平期间，SDA 线由高电平向低电平的变化表示起始信号；SCL 线为高电平期间，SDA 线由低电平向高电平的变化表示终止信号。如图下图所示：



起始和终止信号只能由主机发出。在起始信号产生之后，总线处于被占用状态；在终止信号产生后，总线处于空闲状态。

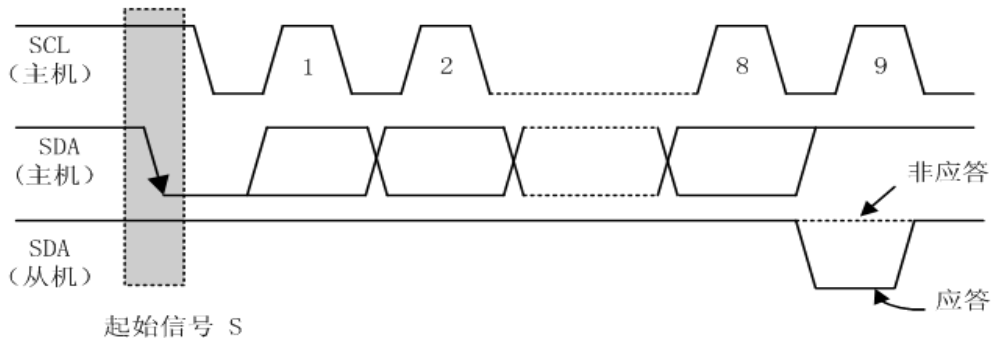
具有 I2C 总线硬件接口的器件很容易检测到起始和终止信号。不具备 I2C 总线硬件接口器件（如单片机）仿真实现 I2C 总线时，为了检测起始和终止信号，必须在每个 SCL 时钟周期内对数据线 SDA 采样两次。

接收器件收到一个完整的数据字节后，有可能需要完成一些其它工作（如保存所接收的数据并做出适当处理等），无法立刻接收下一个字节，这时接收器件可以将 SCL 线拉成低电平，从而使主机处于等待状态。直到接收器件准备好接收下一个字节时，再释放 SCL 线使之成为高电平，从而使数据传送可以继续进行。

(3) 数据传送格式

① 字节传送与应答

每一个字节必须保证是 8 位长度。数据传送时，先传送最高位（MSB），每一个被传送的字节后面都必须跟随一位应答位（即一帧共有 9 位）。



由于某种原因从机不对主机寻址信号应答时（如从机正在进行实时处理工作而无法接收总线上的数据），从机必须将数据线置于高电平，从而使主机产生一个终止信号以结束总线数据传送。

如果从机对主机进行了应答，但在数据传送一段时间后无法继续接收更多的数据时，从机可以通过对无法接收的第一个数据字节的“非应答”通知主机，主机则应发出终止信号以结束数据的继续传送。

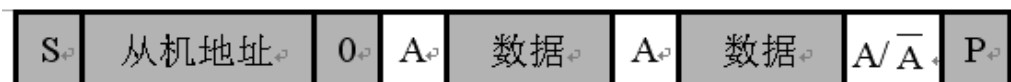
当主机接收数据时，它收到最后一个数据字节后，必须向从机发出一个结束传送的信号。这个信号是由对从机的“非应答”来实现的。然后，从机释放 SDA 线，以允许主机产生终止信号。

② 数据帧格式

I2C 总线上在起始信号后必须传送一个从机的地址（7 位），第 8 位是数据的传送方向位（R/W），用“0”表示主机发送数据（W），“1”表示主机接收数据（R）。每次数据传送总是由主机产生的终止信号结束。但是，若主机希望继续占用总线进行新的数据传送，则可以不产生终止信号，马上再次发出起始信号对另一从机进行寻址。

在总线的一次数据传送过程中，可以有以下几种组合方式：

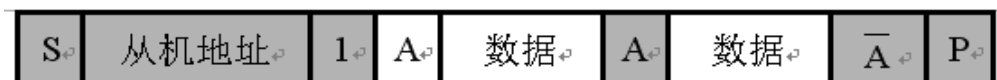
（a）主机向从机发送数据（主机→从机）



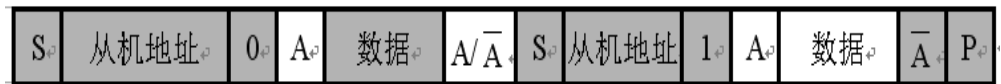
注：有阴影部分表示数据由主机向从机传送，无阴影部分则表示数据由从机向主机传送。

A 表示应答， $\overline{A}$ 表示非应答（高电平），S 表示起始信号，P 表示终止信号。

（b）主机在第一个字节后，立即由从机发送数据，主机接收数据（从机→主机）



(c)、在传送过程中，当需要改变传送方向时（先主机→从机，再从机→主机）



④ 总线的寻址

I2C 总线明确规定：采用 7 位的寻址字节（寻址字节是起始信号后的第一个字节）。

(a) 寻址字节的位定义

位：	7	6	5	4	3	2	1	0
	从机地址							R/ $\overline{W}$

D7~D1 位组成从机的地址。D0 位是数据传送方向位，为“0”时表示主机向从机写数据，为“1”时表示主机由从机读数据。

主机发送地址时，总线上的每个从机都将这 7 位地址码与自己的地址进行比较，如果相同，则认为自己正被主机寻址，根据 R/W 位将自己确定为发送器或接收器。

从机的地址由 4 位固定部分和 3 位可编程部分组成。固定地址由硬件生产厂家出厂时直接固定。可编程部分决定了可接入总线的该类器件的最大数目。3 位是可编程位，决定了可以有 8 个同样的器件接入到同一 I2C 总线系统中。

(b) 特殊地址

固定地址编号 0000 和 1111 已被保留作为特殊用途。

地 址 位							R/ $\overline{W}$	意 义
0	0	0	0	0	0	0	0	通用呼叫地址
0	0	0	0	0	0	0	1	起始字节
0	0	0	0	0	0	1	×	CBUS 地址
0	0	0	0	0	1	0	×	保留
0	0	0	0	0	1	1	×	
0	0	0	0	1	×	×	×	
1	1	1	1	1	×	×	×	
1	1	1	1	0	×	×	×	十位从机地址

起始信号后的第一字节的 8 位为“0000 0000”时，称为通用呼叫地址。通用呼叫地址的用意在第二字节中加以说明。格式为

第一字节（通用呼叫地址）									第二字节							LSB	
0	0	0	0	0	0	0	0	A	×	×	×	×	×	×	×	B	A

第二字节为 06H 时，所有能响应通用呼叫地址的从机器件复位，并由硬件装入从机地址的可编程部分。

第二字节为 04H 时，所有能响应通用呼叫地址并通过硬件来定义其可编程地址的从机器件将锁定地址中的可编程位，但不进行复位。

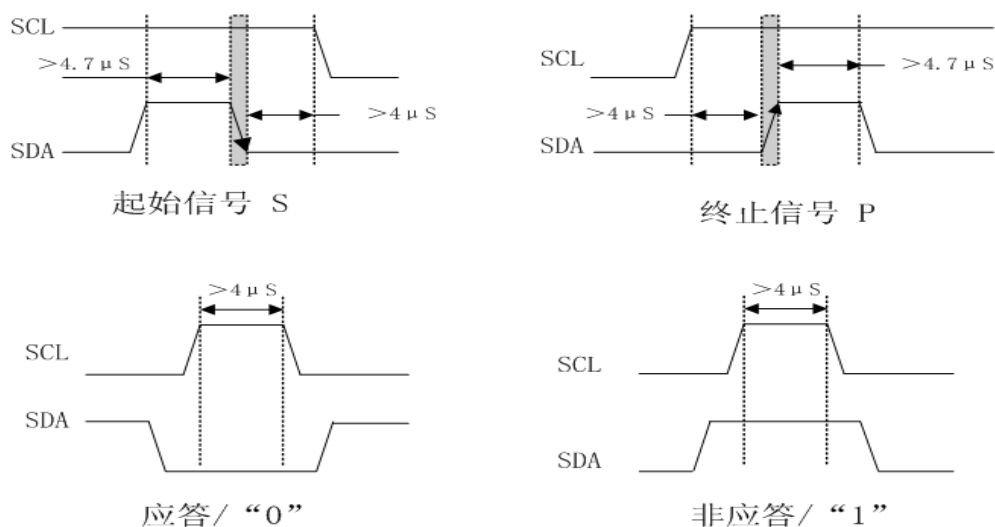
主机可以利用通用呼叫地址中的第二字节高 7 位说明自己的地址。这样当其它设备作主机时可以与其通信。

S	0000 0000	A	主机地址	1	A	数据	A	数据	A	P
---	-----------	---	------	---	---	----	---	----	---	---

3、MCS-51 单片机的 I2C 总线仿真

不带 I2C 总线接口的单片机，如 MCS-51 单片机，可以利用软件结合 PIO 口仿真实现 I2C 总线。

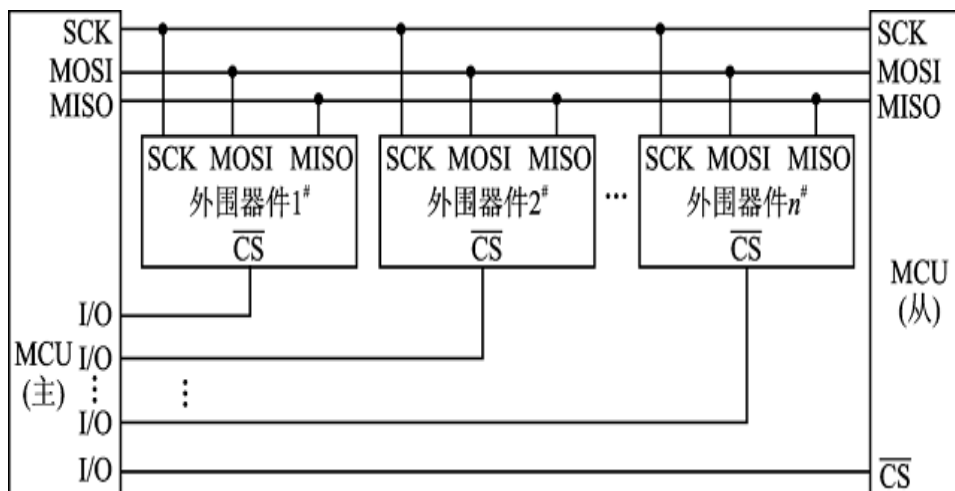
为了保证数据传送的可靠性，标准的 I2C 总线的数据传送有严格的时序要求。I2C 总线的起始信号、终止信号、发送“0”及发送“1”的模拟时序，如下图所示：



SPI 总线简介

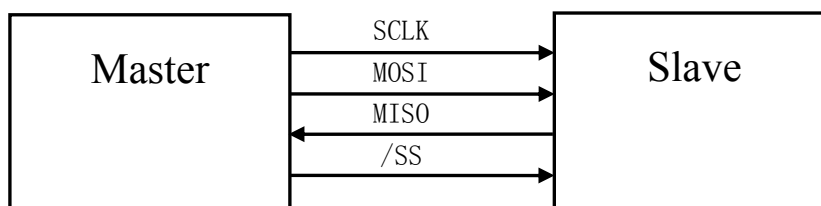
1、技术性能

SPI 接口是 Motorola 首先提出的全双工三线同步串行外围接口，采用主从模式（Master Slave）架构；支持多 slave 模式应用，一般仅支持单 Master。时钟由 Master 控制，在时钟移位脉冲下，数据按位传输，高位在前，低位在后（MSB first）；SPI 接口有 2 根单向数据线，为全双工通信，目前应用中的数据速率可达几 Mbps 的水平。总线结构如下图所示。



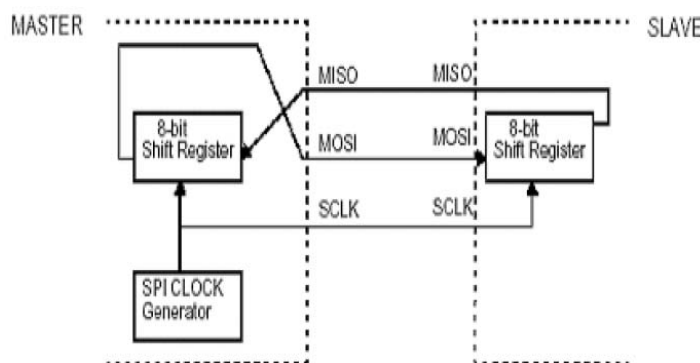
2、接口定义

SPI 接口共有 4 根信号线，分别是：设备选择线、时钟线、串行输出数据线、串行输入数据线。



- (1) MOSI: 主器件数据输出，从器件数据输入
- (2) MISO: 主器件数据输入，从器件数据输出
- (3) SCLK : 时钟信号，由主器件产生
- (4) /SS: 从器件使能信号，由主器件控制

3、内部结构



4、时钟极性和时钟相位

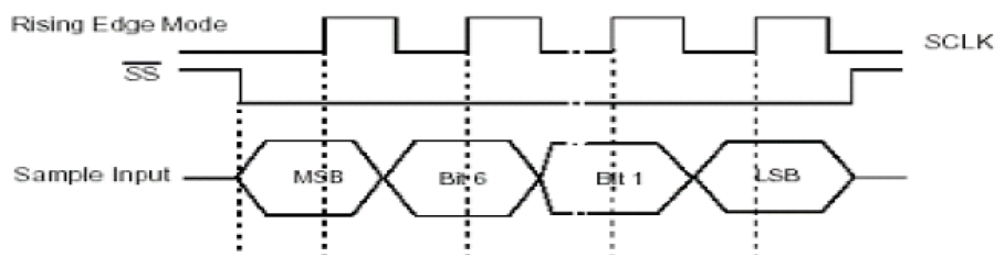
在 SPI 操作中，最重要的两项设置就是时钟极性（CPOL 或 UCCKPL）和时钟相位（CPHA 或 UCCKPH）。时钟极性设置时钟空闲时的电平，时钟相位设置读取数据和发送数据的时钟沿。

主机和从机的发送数据是同时完成的，两者的接收数据也是同时完成的。所以为了保证主机从机正确通信，应使得它们的 SPI 具有相同的时钟极性和时钟相位。

SPI 接口时钟配置心得：在主设备这边配置 SPI 接口时钟的时候一定要弄清楚从设备的时钟要求，因为主设备这边的时钟极性和相位都是以从设备为基准的。因此在时钟极性的配置上一定要搞清楚从设备是在时钟的上升沿还是下降沿接收数据，是在时钟的下降沿还是上升沿输出数据。

五、传输时序

SPI 接口在内部硬件实际上是两个简单的移位寄存器, 传输的数据为 8 位，在主器件产生的从器件使能信号和移位脉冲下，按位传输，高位在前，低位在后。如下图所示，在 SCLK 的下降沿上数据改变，上升沿一位数据被存入移位寄存器。

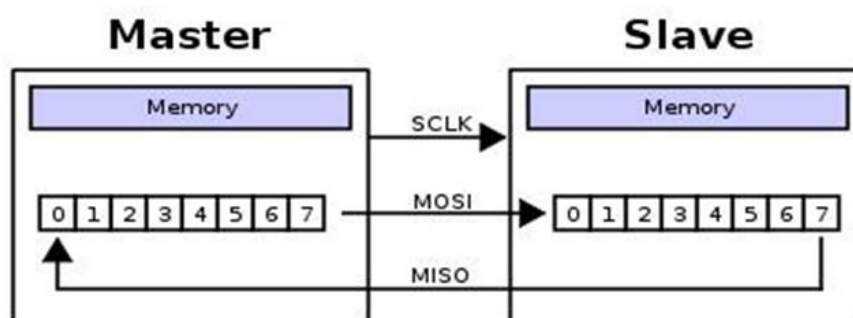


五、数据传输

在一个 SPI 时钟周期内，会完成如下操作：

- 1) 主机通过 MOSI 线发送 1 位数据，从机通过该线读取这 1 位数据；
- 2) 从机通过 MISO 线发送 1 位数据，主机通过该线读取这 1 位数据。

这是通过移位寄存器来实现的。如下图所示，主机和从机各有一个移位寄存器，且二者连接成环。随着时钟脉冲，数据按照从高位到低位的方式依次移出主机寄存器和从机寄存器，并且依次移入从机寄存器和主机寄存器。当寄存器中的内容全部移出时，相当于完成了两个寄存器内容的交换。



六、优缺点

SPI 接口具有如下优点：

- 1) 支持全双工操作；
- 2) 操作简单；
- 3) 数据传输速率较高。

同时，它也具有如下缺点：

- 1) 需要占用主机较多的口线（每个从机都需要一根片选线）；
- 2) 只支持单个主机。
- 3) 没有指定的流控制，没有应答机制确认是否接收到数据。

FAT 文件系统简介

1、 简介：

FAT32 文件系统您一定不会陌生，最多看到它是在 windows 操作系统里，但在一些嵌入式产品（如手机、MP3、MP4 等）中也能看到它的身影。FAT32 文件系统可以使我们可以脱离底层储存设备驱动，更为方便高效地组织数据。给单片机系统中的大容量存储器（如 SD 卡、CF 卡、硬盘等）配以 FAT32 文件系统，将是非常有意义的（如创建的数据文件可以在 windows 等操作系统中直接读取等）。

（一）FAT32 文件系统将逻辑盘的空间划分为三部分，依次是引导区（BOOT 区）、文件分配表区（FAT 区）、数据区（DATA 区）。引导区和文件分配表区又合称为系统区。

（二）引导区从第一扇区开始，使用了三个扇区，保存了该逻辑盘每扇区字节数，每簇对应的扇区数等重要参数和引导记录。之后还留有若干保留扇区。而 FAT16 文件系统的引导区只占用一个扇区，没有保留扇区。

（三）文件分配表区共保存了两个相同的文件分配表，因为文件所占用的存储空间（簇链）及空闲空间的管理都是通过 FAT 实现的，FAT 如此重要，保存两个以便第一个损坏时，还有第二个可用。文件系统对数据区的存储空间是按簇进行划分和管理的，簇是空间分配和回收的基本单位，即一个文件总是占用若干个整簇，文件所使用的最后一簇剩余的空间就不再使用，而是浪费掉了。

从统计学上讲，平均每个文件浪费 0.5 簇的空间，簇越大，存储文件时空间浪费越多，利用率越低。因此，簇的大小决定了该盘数据区的利用率。FAT16 系统簇号用 16 位二进制数表示，从 0002H 到 FFEFH 个可用簇号（FFF0H 到 FFFFH 另有定义，用来表示坏簇，文件结束簇等），允许每一逻辑盘的数据区最多不超过 FFEDH(65518)个簇。FAT32 系统簇号改用 32 位二进制数表示，大致从 00000002H 到 FFFFFFFFH 个可用簇号。FAT 表按顺序依次记录了该盘各簇的使用情况，是一种位示图法。

每簇的使用情况用 32 位二进制填写，未被分配的簇相应位置写零；坏簇相应位置填入特定值；已分配的簇相应位置填入非零值，具体为：如果该簇是文件的最后一簇，填入的值为 FFFFFFFFH，如果该簇不是文件的最后一簇，填入的值为该文件占用的下一个簇的簇号，这样，正好将文件占用的各簇构成一个簇链，保存在 FAT 表中。0000000H、00000001H 两簇号不使用，其对应的两个 DWORD 位置（FAT 表开头的 8 个字节）用来存放该盘介质类型编号。FAT 表的大小就由该逻辑盘数据区共有多少簇所决定，取整数个扇区。

文件名称 ▼	扩展	修改时间	访问时间	记录更新	删除	链接	肤色图片比率值SC%
(根目录)		2.0 KB					8,192
空余空间		237 MB					
引导扇区		3.1 MB					0
卷残留空间		1.5 KB					493,464
FAT 2		474 KB					7,244
FAT 1		474 KB					6,296

2、 FAT32 文件系统的格式化



3、 DBR (DOS BOOT RECORD 操作系统引导记录区)

DBR 是我们进军 FAT32 的首道防线。其实 DBR 中的 BPB 部分才是这一区域的核心部分（第 12~90 字节为 BPB），只有深入详实的理解了 BPB 的意义，才能够更好的实现和操控 FAT32。

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	
00000000	EB	58	90	4D	53	44	4F	53	35	2E	30	00	02	04	98	18	èX MSDOS5.0 I
00000010	02	00	00	00	00	F8	00	00	3F	00	FF	00	65	00	00	00	ø ? ý e
00000020	9B	87	07	00	B4	03	00	00	00	00	00	00	02	00	00	00	II
00000030	01	00	06	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000040	80	00	29	4D	D1	7E	BC	4E	4F	20	4E	41	4D	45	20	20	I)MÑ~4NO NAME
00000050	20	20	46	41	54	33	32	20	20	20	33	C9	8E	D1	BC	F4	FAT32 3EIN46
00000060	7B	8E	C1	8E	D9	BD	00	7C	88	4E	02	8A	56	40	B4	41	{IÄIÜ½ IN IV@'A
00000070	BB	AA	55	CD	13	72	10	81	FB	55	AA	75	0A	F6	C1	01	»aUI r ûUau öÄ
00000080	74	05	FE	46	02	EB	2D	8A	56	40	B4	08	CD	13	73	05	t pF è-IV@' í s
00000090	B9	FF	FF	8A	F1	66	0F	B6	C6	40	66	0F	B6	D1	80	E2	'ÿÿlñf 4E@f 4Nlâ
000000A0	3F	F7	E2	86	CD	C0	ED	06	41	66	0F	B7	C9	66	F7	E1	?÷âIÄI Af ·Éf÷â
000000B0	66	89	46	F8	83	7E	16	00	75	38	83	7E	2A	00	77	32	fIFøI~ u8I~* w2
000000C0	66	8B	46	1C	66	83	C0	0C	BB	00	80	B9	01	00	E8	2B	fIF fIÄ » I' è+
000000D0	00	E9	2C	03	A0	FA	7D	B4	7D	8B	F0	AC	84	C0	74	17	é, ú}'Iä-IÄt
000000E0	3C	FF	74	09	B4	0E	BB	07	00	CD	10	EB	EE	A0	FB	7D	<ÿt ' » Í èi ù}
000000F0	EB	E5	A0	F9	7D	EB	E0	98	CD	16	CD	19	66	60	80	7E	èä ù}èäIÍ í f' I~
00000100	02	00	0F	84	20	00	66	6A	00	66	50	06	53	66	68	10	I fJ fP Sfh
00000110	00	01	00	B4	42	8A	56	40	8B	F4	CD	13	66	58	66	58	'BIV@IóI fXfX
00000120	66	58	66	58	EB	33	66	3B	46	F8	72	03	F9	EB	2A	66	fXfXè3f;Før ûè*f
00000130	33	D2	66	0F	B7	4E	18	66	F7	F1	FE	C2	8A	CA	66	8B	30f ·N f÷ñpÄIÉfI
00000140	D0	66	C1	EA	10	F7	76	1A	86	D6	8A	56	40	8A	E8	C0	ðfÄè ÷v IÖIV@IèÄ
00000150	E4	06	0A	CC	B8	01	02	CD	13	66	61	0F	82	75	FF	81	ä I, Í fa luÿ
00000160	C3	00	02	66	40	49	75	94	C3	42	4F	4F	54	4D	47	52	Ä f@IuIÄBOOTMGR
00000170	20	20	20	20	00	00	00	00	00	00	00	00	00	00	00	00	
00000180	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000190	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000001A0	00	00	00	00	00	00	00	00	00	00	00	00	0D	0A	52	65	Re
000001B0	6D	6F	76	65	20	64	69	73	6B	73	20	6F	72	20	6F	74	move disks or ot
000001C0	68	65	72	20	6D	65	64	69	61	2E	FF	0D	0A	44	69	73	her media.ÿ Dis
000001D0	6B	20	65	72	72	6F	72	FF	0D	0A	50	72	65	73	73	20	k errorÿ Press
000001E0	61	6E	79	20	6B	65	79	20	74	6F	20	72	65	73	74	61	any key to resta
000001F0	72	74	0D	0A	00	00	00	00	00	AC	CB	D8	00	00	55	AA	rt -E0 Uä

字段名称	长度	含义	偏移量
jmpBoot	3	跳转指令	0
OEMName	8	这是一个字符串，标识了格式化该分区的操作系统的名称和版本号	3
BytesPerSec	2	每扇区字节数	11
SecPerClus	1	每簇扇区数	13
RsvdSecCnt	2	保留扇区数目	14
NumFATs	1	此卷中 FAT 表数	16
RootEntCnt	2	FAT32 为 0	17
TotSec16	2	FAT32 为 0	19
Media	1	存储介质	21
FATSz16	2	FAT32 为 0	22
SecPerTrk	2	磁道扇区数	24
NumHeads	2	磁头数	26
HiddSec	4	FAT 区前隐扇区数	28
TotSec32	4	该卷总扇区数	32
FATSz32	4	FAT 表扇区数	36
ExtFlags	2	FAT32 特有	40

FSVer	2	FAT32 特有	42
RootClus	4	根目录簇号	44
FSInfo	2	文件系统信息	48
BkBootSec	2	通常为 6	50
Reserved	12	扩展用	52
DrvNum	1	—	64
Reserved1	1	—	65
BootSig	1	—	66
VolID	4	—	67
FilSysType	11	—	71
FilSysType1	8	—	82

DBR 实现代码：

```
//FAT32中对DBR的定义如下 一共占用90个字节
struct DBR
{
    UINT8 BS_jumpBoot[3]; //跳转指令 offset: 0
    UINT8 BS_OEMName[8]; //OEM名称 offset: 3

    UINT8 BPB_BytesPerSec[2]; //每扇区字节数 offset:11
    UINT8 BPB_SecPerClus; //每簇扇区数 offset:13
    UINT8 BPB_RsvdSecCnt[2]; //保留扇区数目 offset:14
    UINT8 BPB_NumFATs; //此卷中FAT表数 offset:16
    UINT8 BPB_RootEntCnt[2]; //FAT32为0 offset:17
    UINT8 BPB_TotSec16[2]; //FAT32为0 offset:19
    UINT8 BPB_Media; //存储介质 offset:21
    UINT8 BPB_FATSz16[2]; //FAT32为0 offset:22
    UINT8 BPB_SecPerTrk[2]; //磁道扇区数 offset:24
    UINT8 BPB_NumHeads[2]; //磁头数 offset:26
    UINT8 BPB_HiddSec[4]; //FAT区前隐扇区数 offset:28
    UINT8 BPB_TotSec32[4]; //该卷总扇区数 offset:32
    UINT8 BPB_FATSz32[4]; //一个FAT表扇区数 offset:36
    UINT8 BPB_ExtFlags[2]; //FAT32特有 offset:40
    UINT8 BPB_FSVer[2]; //FAT32特有 offset:42
    UINT8 BPB_RootClus[4]; //根目录簇号 offset:44
    UINT8 FSInfo[2]; //保留扇区FSINFO扇区数 offset:48
    UINT8 BPB_BkBootSec[2]; //通常为6 offset:50
    UINT8 BPB_Reserved[12]; //扩展用 offset:52
    UINT8 BS_DrvNum; // offset:64
    UINT8 BS_Reserved1; // offset:65
    UINT8 BS_BootSig; // offset:66
    UINT8 BS_VolID[4]; // offset:67
    UINT8 BS_FilSysType[11]; // offset:71
    UINT8 BS_FilSysType1[8]; //"FAT32 " offset:82
};
```

4、 对 FAT32 文件系统各部分的定位

解析全局参数的目的在于对 FAT32 各部分进行定位，比如前面讲的 MBR、DBR，再如后面将要讲到的 FAT 表、首目录簇等。对各部分在 SD 卡上的位置进行准确定位是非常重要的。首先，我们要对 FAT32 文件系统的整体结构有一个了解，FAT32 的文件系统如图所示

前隐扇区	MBR
DBR	
保留区	
第 1 个 FAT 表	
第 2 个 FAT 表	
第 2 簇	
第 3 簇	
第 4 簇	
·	
·	
·	
第 n 簇	
第 n+1 簇	
第 n+2 簇	
第 n+3 簇	

从结构图中我们可以看到，FAT32 的结构中，首先是 MBR 和 DBR 区，DBR 后面是保留区，DBR 占用 63 个扇区，保留扇区位 32。所以第一个 FAT 表的开始扇区为 95。再后面就是第一个 FAT 表，他占用 1936 个扇区，所以第二个 FAT 表的开始扇区位 2031。

第一个 FAT 表开始扇区=DBR 扇区+保留扇区

第二个 FAT 表开始扇区= 第一个 FAT 表开始扇区+FAT 表所占扇区数

这样，我们就已经知道了这张 SD 卡的 FAT32 文件系统的各个部分的准确为了，再加上前面解析出来的一些参数，就可以构造出一个结构体，用来装载这些信息。

有文件名，根目录的名称就是磁盘的卷标。如笔者的 SD 卡在格式会时设置卷标为 X，则根目录的名称就为 X，如下图：

每一个文件都对应一个描述它属性的结构，定义如下

FAT32 文件目录项 32 个字节的定义		
字节偏移量	字数量	定义
0~7	8	文件名
8~10	3	扩展名
11	1	属性字节
		0x00 (读写)
		0x01 (只读)
		0x02 (隐藏)
		0x04 (系统)
		0x08 (卷标)
		0x10 (子目录)
		0x20 (归档)
12	1	系统保留
13	1	创建时间的 10 毫秒位
14~15	2	文件创建时间
16~17	2	文件创建日期
18~19	2	文件最后访问日期
20~21	2	文件起始簇号的高 16 位
22~23	2	文件的最近修改时间
24~25	2	文件的最近修改日期
26~27	2	文件起始簇号的低 16 位
28~31	4	表示文件的长度

其数据结构如下图所示

```

struct FDI
{
    UINT8 Name[8];           // 文件名，不足部分以空格补充
    UINT8 Extension[3];      // 扩展名，不足部分以空格补充
    UINT8 Attributes;        // 文件属性
    UINT8 LowerCase;         // 0
    UINT8 CTime10ms;         // 创建时间的10毫秒位
    UINT8 CTime[2];          // 创建时间
    UINT8 CDate[2];          // 创建日期
    UINT8 ADate[2];          // 访问日期
    UINT8 HighClust[2];      // 开始簇的高字
    UINT8 MTime[2];          // 最近的修改时间
    UINT8 MDate[2];          // 最近的修改日期
    UINT8 LowClust[2];       // 开始簇的低字
    UINT8 FileSize[4];       // 文件大小
};

```